

defuqumat

what it computes, and how to ask for it

Plane-wave DFT in Python and JAX, validated against Quantum ESPRESSO

October 14, 2026

Abstract

User documentation for the feature set: every capability, the equation behind it, a snippet that runs it, what it was validated against, and *what it refuses*. Two conventions run through the whole document.

Everything is validated against Quantum ESPRESSO on the same input, and the agreement is quoted per feature rather than claimed in general. Where a number is missing it is because the comparison does not exist, and that is said rather than hidden.

Anything not implemented is refused by name, with an error saying what is missing — never silently approximated. The "Refuses" notes are as much of the documentation as the features: they are the promise that a run which starts is a run whose physics is there.

One equation is written down and the rest are derived from it. Almost everything below is a derivative of the total energy, taken by the compiler rather than by hand:

$$E[\{\psi\},\tau,\varepsilon] = T_s + E_{\mathrm{H}} + E_{xc} + E_{\mathrm{loc}} + E_{\mathrm{nl}} + E_{\mathrm{Ewald}}$$

quantity	what it is	how it is obtained
$v_{xc}(\mathbf{r})$	$\delta E_{xc}/\delta n$	`jax.grad` of the energy density
$\mathbf{F}_I$	$-\partial E/\partial \boldsymbol{\tau}_I$	`jax.grad` at frozen ψ
σ_{ij}	$-\Omega^{-1}\partial E/\partial \varepsilon_{ij}$	`jax.grad` along a strain
$C_{I\alpha,J\beta}$	$\partial^2 E/\partial\tau_{I\alpha}\partial\tau_{J\beta}$	`jvp` of the force
$Z^*_{I,\alpha\beta}$	$\partial F_{I\alpha}/\partial\mathcal{E}_\beta$	`jvp` of the force along the field response
$\partial\epsilon_{ij}/\partial\tau$	Raman	`jvp` of the second-order energy

The "frozen ψ" is what makes this exact rather than approximate: at self-consistency the energy is stationary with respect to the wavefunctions, so the terms from $\partial\psi/\partial\lambda$ vanish. That is the Hellmann–Feynman argument, and taking it one order up is the $2n{+}1$ theorem, which is why a *third* derivative needs only first-order wavefunctions.

Contents

1 Input and invocation **4**

2 Ground state **12**

2.1 An electric field across a slab, and the dipole correction 19

2.2 A finite electric field in a crystal, by the Berry phase 20

3 Bands, densities of states, projections **21**

4 Pseudopotentials and functionals **24**

5 Spin, magnetism and spin-orbit **25**

5.1 The source-free exchange-correlation field, and the torque 36

5.2 A gradient correction built from $\nabla\mathbf{m}$ 37

6 Forces, stress and relaxation **38**

7 Response: dielectric, phonons, Raman **40**

7.1 Phonons away from Γ 44

7.2 The piezoelectric tensor 50

7.3 Spin-polarized response 53

7.4 Magnons: the transverse spin susceptibility 54

8 Optical spectra and excitons: TDDFT **57**

8.1 The conductivity tensor, the Kerr angle and the anomalous Hall effect . . . 59

8.2 The shift current: the bulk photovoltaic effect 61

8.3 Second-harmonic generation 62

9 Real-time propagation: the current under a pulse, high harmonics and the third harmonic **64**

9.1 The dielectric function from a kick 67

9.2 The high-harmonic spectrum 67

9.3 The perturbative orders and the third harmonic 68

9.4 The spectrum of the orders: the frequency-domain hierarchy 70

9.5 The potential updated in time: local fields and the adiabatic kernel 72

9.6 Magnetic, spin-orbit, ultrasoft and PAW ground states 74

10 Topological invariants **76**

10.1 The curvature as a map: the Kubo route 77

10.2 The Berry-phase polarization 78

10.3 The magnetoelectric tensor 79

11 Effective masses, site angular momenta, nesting, structure factors and

STM images **80**
11.1 The effective mass tensor . . . 81
11.2 $\langle L \rangle$, $\langle S \rangle$ and $\langle J \rangle$ on each atom . . . 82
11.3 The orbital magnetization of the cell . . . 83
11.4 The Fermi-surface nesting function . . . 84
11.5 Magnetocrystalline anisotropy . . . 85
11.6 The relaxed anisotropy: total energies instead of band sums . . . 90
11.7 X-ray and magnetic structure factors . . . 91
11.8 Scanning-tunnelling microscopy images . . . 92
11.9 Tunnelling spectra: $\mathrm{d}I/\mathrm{d}V$ over an energy axis . . . 94
11.10 Vertical tunnelling transport through a two-dimensional material . . . 96
11.11 Which k-points the tunnelling current comes out of . . . 99

12 Things with no `pw.x` counterpart **101**
12.1 Spin-orbit coupling on a spiral, to first order . . . 104

13 A modulation over many unit cells: the ultracell **106**

14 What a modulation looks like to a tip **119**

15 Continuing an all-electron ground state **122**

16 Performance: dials, memory and GPUs **124**

17 Accuracy summary **135**

Where to go next **135**

1 Input and invocation

defuqumat reads **`pw.x` input files**. The same file that runs in Quantum ESPRESSO runs here, which is what makes every comparison in this document possible.

```
from defuqumat import Calculator

calc = Calculator.from_file("scf.in", pseudo_dir="pseudo", conv_thr=1.0e-8)
result = calc.get_scf()

print(result.total_energy, "Ry in", result.iterations, "iterations")
```

`conv_thr` means what it means in a `pw.x` input: it is compared against QE's `dr2`, the Hartree energy of the density residual.

It need not be given at all. `from_file` and `from_text` **adopt the input's own `&electrons` namelist** — `conv_thr`, `mixing_beta`, `mixing_mode`, `mixing_fixed_ns` and `electron_maxstep` — so an input file that states how to converge itself converges the same way here as under `pw.x`. A keyword argument given to the constructor still wins over the file, which is how the snippet above overrides it. One variable of that namelist is read by `pw.x` and deliberately *not* adopted: `diagonalization`, because this package offers one eigensolver, so honouring `diagonalization = 'cg'` would turn a run that works into an error, and mapping it silently onto Davidson would be the kind of substitution refused everywhere else here. Name a solver at construction instead.

An adopted default yields to a workflow that chose its own, and until 2026-09-20 it did not. A threshold means the same thing wherever it is named, which is why it is forwarded at all, but meaning the same thing is not wanting the same value: twenty entry points declare a `conv_thr` between 10^{-8} and 10^{-12} and say in their own documentation why — a magnetocrystalline anisotropy is a difference of band-energy sums in the fifth decimal of an eV, an effective mass is a second difference of eigenvalues, a Berry phase is a product of overlaps around a closed loop. An input file that states `pw.x`'s own 10^{-6} was replacing every one of them. So a value adopted from the file, or given at construction, reaches an entry point only where that entry point repeats `run_scf`'s own default, meaning it has no opinion; where it states one of its own, the workflow's number stands. Naming it at the call site — `get_anisotropy(soc, conv_thr=1.0e-6)` — still reaches it, which is the difference between a default and a refusal.

Tightening is always allowed, and only loosening is withheld: a threshold is a bound on an error, so a `conv_thr` of 10^{-12} given to the constructor still reaches a workflow whose own is 10^{-10}. That asymmetry belongs to thresholds alone — `max_iterations` of 40 against 100 is not better or worse, it is what the workflow wants — and it is what keeps an input stating 10^{-10} from being withheld from an ultracell run, whose own 10^{-8} is a second driver's default for the same quantity rather than a tightening.

On the committed tetragonal cobalt cell the substitution is worth little, and the reason is worth knowing: both legs of the force theorem diagonalise the *same* frozen density on the same k-points at the same threshold, so the eigenvalue error is common mode and the difference keeps only what does not cancel. Measured, 10^{-6} in place of 10^{-10} moves the in-plane leg by 2.06×10^{-5} meV and the out-of-plane leg by 1.97×10^{-5} meV, of which 9.0×10^{-7} meV survives into an anisotropy of 1.235 meV.

A band count does not cross between the two legs at all. The force theorem is two calculators, and `nbnd` is a property of the system whose bands are being counted: a spinor band holds one electron where a collinear one holds two, so the same cobalt atom needs

9 bands collinear and 16 as a spinor. A `nbnd` given to the scalar-relativistic calculator used to be forwarded into the noncollinear run, where it wins over the spinor input's own, and 9 spinor bands for 9 electrons is a filled-band insulator rather than a metal: the anisotropy reads **7.926 meV against 1.235**, and the free-energy anisotropy, which should differ from it by more than half, collapses onto the same number because there are no empty states for the Fermi level to sit among and the smearing entropy is zero. The spinor calculator's own `nbnd` crosses in its place, so stating it where it belongs still works.

The adopted namelist reaches a relaxation too, and did not until 2026-09-11: `electron_maxstep`, `diago_full_acc` and `mixing_fixed_ns` were read off the file and then dropped on the way into `get_relax`, so every SCF inside a relaxation ran at the default hundred iterations whatever the input asked for. On a DFT+U relaxation the dropped `mixing_fixed_ns` decides which minimum of the $+U$ functional the run lands in, so the relaxed *geometry* could differ with nothing saying the request had been ignored.

An input variable that changes the physics and is not implemented is refused by name, never read and ignored — a run that quietly drops one is wrong by far more than the agreement with `pw.x` this code is checked to. That covers a charged cell (`tot_charge`), the saw-tooth field and the dipole correction (`tefield`, `dipfield`), the Berry-phase finite field (`lelfield`), the isolated-system corrections (`assume_isolated`), two chemical potentials (`twochem`, in either namelist), per-orbital atomic occupations (`one_atom_occupations`), the constant-cutoff kinetic functional (`qcutz`), a hand-pinned FFT grid (`nr1`–`nr3s`), and the four **symmetry** switches `no_t_rev`, `force_symmorphic`, `use_all_frac` and `nosym_evc`.

Those last four were read by *nothing* until 2026-09-11. Each says that some part of the group is not a symmetry of the state being asked for, so ignoring one left the k-mesh reduced with operations the input had excluded and `sym_rho` averaging over them — a total energy that will not match a benchmark generated from the same file, with nothing saying why. Anything QE defaults to is silent, so an ordinary input never reaches the refusal.

Two more are refused because they are *redundant* rather than unimplemented, and both used to be dropped in silence. The first is the DFT+Hubbard spelling the `&system` namelist carried before QE 7.1 — `lda_plus_u`, `lda_plus_u_kind`, `Hubbard_U`, `Hubbard_U_back`, `Hubbard_J0`, `Hubbard_J`, `Hubbard_V`, `U_projection_type`, `Hubbard_parameters` and `backall`. DFT+U is implemented here and is read from the `HUBBARD` card, so the old names parsed and reached nothing: an input asking for a U converged as plain LSDA or PBE and reported success, and what was dropped is the whole U-on against U-off gap. `pw.x` 7.5 stops on the same set, and on the *value* rather than the name, so `lda_plus_u = .false.` and `Hubbard_U = 0.0` pass in both codes. The second is `ibrav` $\neq 0$ together with a `CELL_PARAMETERS` card: the card was never looked at, and because `ATOMIC_POSITIONS alat` is scaled by the `ibrav` cell's own `alat` the positions went with it, so pasting a relaxed cell back under an input that still carries its `ibrav` — the ordinary way a vc-relax is restarted by hand — ran the pre-relaxation geometry and reported a total energy, a force and a stress for it. Set `ibrav = 0` to use the card. Neither refusal is reached by any of the 1194 inputs in this repository, in `benchmarks/` or in QE's own vendored test-suite.

Two more say the lattice parameter twice, and `pw.x` stops on both. Giving `celldm` *and* `A` is one lattice constant written in bohr and in angstrom, which is what a half-finished conversion leaves behind, and taking `celldm` silently converges a ground state for whichever of the two was not meant. Giving a `CELL_PARAMETERS bohr` or `angstrom` card *and* `celldm(1)` is the same thing one step along: the card already carries the lattice

parameter, so `alat` became `celldm(1)` rather than the length the vectors set, and every `ATOMIC_POSITIONS alat` coordinate was scaled by a parameter the cell does not come from. Write the card in `alat` units to keep `celldm(1)`, or drop `celldm(1)` to keep the card. No `pw.x` input in this repository or in the vendored test-suite carries either pair.
One parsing trap sits under all of this and is worth stating once: Fortran's list-directed logical input accepts an optional `.`, then a `T` or an `F`, then anything, so `lberry = F` is legal and is false. The bare spellings were missing from the converter, so `F` survived as the *string* `'F'` and every truth test read it as true.

Note that `nosym` itself is not read exactly as `pw.x` reads it, in one place: on an automatic grid `pw.x` still pairs **k** with −**k** unless `noinv` is set as well, while here `nosym` gives the whole grid, so a shifted `2 2 1 1 1 0` mesh is four points here and two in `pw.x`. The energy is the same either way, since time reversal is an exact symmetry of a nonmagnetic run, and what changes is the cost and the count of **k**-points in a comparison of the two outputs.

One object, bound methods

A `Calculator` is a system together with its pseudopotentials, and every calculation in this document is a method on it. The pseudopotentials are read from the names the `ATOMIC_SPECIES` card gave, resolved against `pseudo_dir` — which `from_file` defaults to the input file's own directory — so a script names an input and then names physics:

```
calc = Calculator.from_file("scf.in", pseudo_dir="pseudo")

bands = calc.get_bands(kpoints=path)      # runs the SCF first if none is cached
dos = calc.get_dos(grid=(8, 8, 8))
forces = calc.get_forces()
epsilon = calc.get_dielectric_tensor()
phonons = calc.get_phonons()

bands.plot()                              # and each result draws itself
```

Three properties are worth knowing before relying on it.

The ground state is cached, and computed on demand. A method that needs one and finds no cache runs an SCF, printing a line to stderr saying so; `get_scf()` is the explicit path and is the same code. The cache is a *single slot* holding the result together with the options that produced it — calling `get_scf` again with different options reruns and replaces it — because what it holds is the wavefunctions, and a cache keyed by option sets would quietly hold several sets of them. `calc.scf_result` reads the slot without triggering anything, and a state that did not converge is refused rather than differentiated.

Nothing mutates. `with_positions`, `with_cell`, `with_spin` and `with_moments` return a *new* calculator with an empty cache; the converged state crosses as a starting guess where that is sound, which for `with_spin` is the spin-regime continuation described under *Spin, magnetism and spin-orbit* and converges in one iteration rather than twenty-five. A cached result under a moved atom would otherwise answer for the geometry it was converged at.

The refusals pass through untouched. Every restriction in this document — a metal's Raman tensor, a spiral with spin-orbit coupling, a phonon away from Γ on an augmented dataset — raises through the bound method exactly as through the function beneath it.

The functional entry points the rest of this document names — `run_scf`, `run_bands`, `dielectric_tensor`

and the others — are unchanged and remain the way to drive a calculation whose state is being managed by hand:

```
from defuqumat.io.pwin import read_pw_input
from defuqumat.pseudo import read_upf
from defuqumat.scf.driver import Calculation, run_scf
from defuqumat.system import build_system

system = build_system(read_pw_input("scf.in"))
pseudos = tuple(read_upf(f"pseudo/{s.pseudo_file}") for s in system.structure.species)
result = run_scf(system, pseudos, conv_thr=1.0e-8)
```

What such a call has to carry with the density is the rest of the converged state — `becsum` for a PAW dataset, `ns` under a Hubbard U, `tau` under a meta-GGA. None of the three can be rebuilt from the density, and the entry points refuse without them rather than computing something else. The calculator passes them.

Every method, in one place

The rest of this document introduces each quantity where its physics belongs, and names the functional entry point beneath it. This is the index the other way round — what to *call*, for a reader who knows the physics and wants the line. It is complete: a method absent from it is a documentation bug, and a test says so (`tests/unit/test_facade_documentation.py`).

The ground state	
`get_scf`	the self-consistent field loop, or the cached result
`get_elk_seed`	Elk's converged density on this run's grid, as a guess
Band structure and densities of states	
`get_bands`	diagonalise on a k-path at the converged density
`get_nscf`	diagonalise on a k-grid at the converged density, and occupy it
`get_dos`	the density of states, on a denser grid if asked
`get_pdos`	the projected density of states, with Löwdin charges
Derivatives of the energy	
`get_forces`	the forces on the atoms
`get_stress`	the stress tensor
`get_relax`	relax the geometry, and the whole optimisation with it
Response, vibrations and spectra	
`get_band_velocities`	$\mathrm{d}\varepsilon/\mathrm{d}\mathbf{k}$ for every band
`get_effective_mass`	$(1/m^*)_{ab}$ at one k-point, in $1/m_e$
`get_angular_momenta`	$\langle L\rangle$, $\langle S\rangle$, $\langle J\rangle$ on every atom
`get_dielectric_tensor`	ϵ^∞, with no empty states
`get_born_charges`	the Born effective charges Z^*
`get_phonons`	the dynamical matrix at Γ, and its modes
`get_phonons_at_q`	the dynamical matrix at one wavevector
`get_raman_tensors`	$\mathrm{d}\epsilon/\mathrm{d}\tau$, per atom
`get_vibrational_spectrum`	per-mode Raman and infrared activities
`get_strain_response`	the first-order response to a homogeneous strain
`get_elastic_constants`	C_{ijkl}
`get_piezoelectric_tensor`	$e_{(k)ij}$, clamped-ion, in C/m^2
`get_electrostriction`	$\mathrm{d}\chi/\mathrm{d}(\text{strain})$ and its four tensors
`get_absorption`	an optical spectrum, with excitons

`get_optical_conductivity`	$\sigma_{ab}(\omega)$, the Kerr angle, the anomalous Hall
`get_structure_factors`	$F(\mathbf{H})$, X-ray and magnetic
`get_stm`	a Tersoff–Hamann scanning-tunnelling image
`get_sts`	the same, over an energy axis: a tunnelling spectrum
`get_vertical_transport`	tunnelling *through* a 2D material
`get_momentum_transport`	which k-points that current comes out of
`get_nesting`	$N(\mathbf{q})$, the Fermi-surface nesting function
`get_spin_susceptibility`	$\chi^{+-}(\mathbf{q},\omega)$
`get_magnon_dispersion`	$\omega(\mathbf{q})$ along a path
`get_shift_current`	the bulk photovoltaic effect, in A/V^2
`get_shg`	$\chi^{(2)}(-2\omega;\omega,\omega)$, in pm/V

Real-time propagation and the nonlinear orders

`get_realtime`	the current under a pulse, frozen or updated potential
`get_realtime_dielectric`	$\epsilon(\omega+i\eta)$ from a kick
`get_hhg`	the high-harmonic spectrum $\lvert\omega\mathbf{J}(\omega)\rvert^2$
`get_harmonic_orders`	$\mathbf{J}_{(n,m)}$ at one frequency, by the propagation
`get_third_harmonic`	$\chi^{(3)}$ at one frequency, by the propagation
`get_nonlinear_spectrum`	$\mathbf{J}_{(n,m)}$, $\chi^{(2)}$ and $\chi^{(3)}$ over frequencies

Topology, polarization and magnetism

`get_berry_curvature`	$\Omega(\mathbf{k})$ on a plane mesh
`get_chern`	the Chern number of one plane
`get_z2`	the 2D Z_2 invariant, by Wilson loops or parity
`get_z2_3d`	the four Z_2 indices of a 3D crystal
`get_polarization`	the Berry-phase polarization
`get_orbital_magnetization`	M_{orb}, by the modern theory
`get_magnetoelectric_tensor`	$\alpha_{ij} = \mathrm{d}P_i/\mathrm{d}B_j$
`get_anisotropy`	the magnetocrystalline anisotropy
`get_torque`	the magnetic torque, and K from one angle
`get_orientation_torque`	the torque on a whole texture, three components
`get_relaxed_orientation`	the easy orientation of a texture, by BFGS in the rotation
`get_first_order_soc`	the spin-orbit expectation at coupling-free states
`get_force_theorem`	one direction of the anisotropy: its band energy alone

Spin spirals

`get_spiral_scan`	one SCF per spiral wavevector
`get_spiral_relaxation`	the spiral wavevector relaxed down $\mathrm{d}E/\mathrm{d}\mathbf{q}$
`get_spiral_spin_orbit_energy`	spin-orbit coupling to first order on the spiral, $\hat{\mathbf{n}}\cdot\mathbf{V}(\mathbf{q})$

Modulations over many cells

`get_ultracell`	a density or potential modulated over N unit cells
`get_ultracell_stm`	a Tersoff–Hamann image of that modulation
`get_ultracell_sts`	a tunnelling spectrum across that modulation
`get_ultracell_transport`	tunnelling through it, tip to substrate

A command line covers the common workflows:

```
python3 -m defuqumat.cli inspect <qe-output>   # summarise what the parser reads
python3 -m defuqumat.cli dos     <input>       # density of states
python3 -m defuqumat.cli pdos    <input>       # projected DOS
python3 -m defuqumat.cli relax   <input>       # structural relaxation
python3 -m defuqumat.cli stress  <input>       # stress tensor
python3 -m defuqumat.cli spiral  <input>       # spin-spiral scan
```

Standard `pw.x` variables — `ecutwfc`, `ecutrho`, `ibrav`, `nbnd`, `occupations`, `degauss` and the rest — mean what they mean there and are not re-documented here. A handful of knobs are worth knowing because they are specific to this code or to a feature below:

`mbj_c`	fixes Tran–Blaha's c instead of taking the cell average
`london_s6`, `london_rcut`	Grimme D2's scaling and real-space cutoff
`cell_dofree`	which cell degrees of freedom a vc-relax may move
`starting_ns_eigenvalue`	seeds the DFT+U occupation matrix
`spiral_q`	the spin-spiral wavevector, in lattice coordinates
`LOCAL_MAGNETIC_FIELDS`	a card with no `pw.x` counterpart

Units are Rydberg atomic units throughout (Ry, bohr), matching QE. The only layer that speaks anything else is `io/`: Hubbard U in the `HUBBARD` card is in eV and is converted at the boundary, as `pw.x` does it.

What a run will cost, before it is run

`Calculation` builds the G-vectors, the plane waves and the projectors on the device, so an input too large for the machine dies in setup without saying what was too large. `Calculator.estimate()` answers that from the cutoffs and the datasets alone, allocating nothing:

```
print(Calculator.from_file("scf.in").estimate().report())
```

or `python3 -m defuqumat.cli size scf.in`. **It sizes the run the input describes**, which means it reads that input's own `diago_david_ndim` and resolves `k_batch` the way the SCF would — an estimate that quietly assumed a library default would report a run that does not happen, and nobody relying on one is in a position to notice. The arguments and the CLI flags are *overrides* of the file, and the report's header states which assumptions it used. It reports n_{gm}, n_{gms}, n_{pwx}, n_{bnd}, n_{kb} and both FFT grids — *exactly*, from the same predicates the setup selects with, which is asserted against a real `Calculation` rather than trusted — and a floor on the bytes for the wavefunctions, the projectors, the eigensolver's subspace, the fields, the symmetry maps, and on an ultrasoft or PAW dataset the augmentation charge's table and phases and PAW's one-centre tables (in float64 whatever the precision policy, since that is what their build allocates). Below the device peak it lists, separately, what the run keeps in *host* RAM: the mixer's history, `mixing_ndim` input densities, as many residuals and as many fit vectors, which is 4.77 GB on a 157-atom slab and does not count against the card. The G-vector enumeration runs in slabs, so an input destined for a GPU can be sized on a laptop.

Two lines of that report are worth reading before believing a hand estimate. The Davidson subspace is $2\,n_{vecx}\,n_{dim}$ for ψ and $H\psi$ at the default $n_{vecx} = 4\,n_{bnd}$, plus the Ritz block, and it is usually the largest single array — on a 157-atom slab at $E_{\mathrm{cut}} = 60$ Ry it is 139 GB against 23 GB of wavefunctions. And the eigensolver is *not* doubled by spin, because the channels are solved one after another, where the wavefunctions are; the report separates the two.

The one lever that changes that floor without changing the physics is `diago_david_ndim` — QE's own name for it, read from `&electrons`, and settable per call or at construction:

```
calc = Calculator.from_file("scf.in", david=2)    # or diago_david_ndim = 2
```

It halves ψ and $H\psi$ exactly, for one more SCF iteration on silicon and the same total energy to 7.3e-11 Ry — the trade QE's documentation describes. The default here is

4 and `pw.x`'s is **2**: `input_parameters.f90` declares 4, but `read_namelists.f90`'s `electrons_defaults` sets 2 before the namelist is read, which is what `INPUT_PW.txt` says, so a step count against `pw.x` sets `diago_david_ndim` on both sides. The subspace is collapsed when it has no room for the bands still unconverged, which is `cegterg`'s test; at 2, where that room is smallest, this is what keeps the history between collapses (`PERFORMANCE.md`, "The collapse condition is `cegterg`'s since 2026-10-10"), and 1 is refused, as `cegterg` refuses it.

The subspace is capped at the space it lives in, which is $n_{pol}\ \min_{\mathbf{k}} n_{pw}$ — the *smallest* sphere on the k-set, not n_{pwx}, which is the padded maximum and sits above the k-points that go singular. A small cell asked for many bands is where this bites: silicon at $E_{\mathrm{cut}} = 12$ Ry folded to a 32 k-point ultracell holds between 169 and 192 plane waves, so $n_{bnd} = 48$ at the default asks for 192 vectors in a 169-dimensional space, and 25 of the 32 overlaps came back non-finite where the capped solve leaves none. A cap removes the case where the subspace is *larger* than the space and not the case where it *equals* it: at $n_{bnd} = 80$ on the same cell the cap is the whole space at the smallest k-point, one overlap still goes singular, and the canonical-orthogonalisation fallback carries it. Two cases no cap can rescue are refused by name instead — more bands than the space holds, which is QE's own `c_bands.f90:286`, and $2\,n_{bnd}$ not fitting, which leaves Davidson no room to expand (`cegterg.f90:125`) and wants a direct diagonalisation of the space that is not written here.

A converged state on disk

`result.save(path)` writes the state and `SCFResult.load(path, system=...)` reads it back for `run_scf(starting_from` so a job that hits its wall clock does not cost the run. The system is supplied rather than stored — a resume already has the input file — and is checked against a fingerprint. `run_relax(checkpoint_dir=...)` does the same every ionic step *including the optimizer's own history*, which is the half that is easy to omit and most of the value: BFGS earns its convergence rate from the inverse Hessian and the trust radius, so a resume from positions alone takes its next step as if it were the first. Stopping a six-step relaxation at step two and resuming costs $2 + 4$ steps, not $2 + 6$.

Gamma-only storage: half the plane waves

At $k = 0$ a Kohn–Sham state can be chosen real, so $c(-G) = c^*(G)$ and one plane wave of each $(G, -G)$ pair carries all of it. `K_POINTS gamma` stores that half, which halves n_{pwx} and with it every array a band lives in — the wavefunctions, `vkb`, the Davidson subspace. On a 157-atom slab at $E_{\mathrm{cut}} = 60$ Ry that is **189 GB against 96**. It is a memory feature, not a speed one: the two-bands-per-FFT packing of `vloc_psi_gamma` is deliberately not implemented.

Only the wavefunction sphere halves. The dense G set stays whole, where `pw.x` halves both. The memory is entirely in the plane-wave-sized arrays, so halving the dense set as well saves a fifth of a per cent and puts a conjugate fill inside every consumer of a real field. The consequence to know is that `ngm` here is not the one `pw.x` prints: the relation is $n_{gm} = 2n_{gm}^{\mathrm{QE}} - 1$, and the test suite asserts it.

Three things carry the trick: the field is rebuilt from both halves, every sum over plane waves becomes $2\,\mathrm{Re}(\sum) - (G{=}0)$ — that vector being its own conjugate partner rather than half a pair — and $\mathrm{Im}\,c(0)$ must stay zero. **The last is the silent one**: a complex $c(0)$ makes the rebuilt field complex and the run converges to a plausible wrong answer.

The validation needs no other code. An explicit $k = 0$ on the whole sphere is the same calculation in the other storage, so the two agree to round-off: totals to **1e-14** Ry,

eigenvalues to **3e-15** and forces to **3e-16** Ry/bohr, on LDA, PBE, LSDA and PBE+LSDA alike.

Refuses. `K_POINTS gamma` is *substituted* by an explicit $k = 0$ on the full sphere — the same physics at twice the storage, with a warning — for an ultrasoft or PAW dataset (`addusdens` and `newd` need their own `fact = 2`), for a run that uses symmetry (a rotation carries a stored G onto an unstored $-G'$, so the permutation would need a conjugation; `nosym = .true.` is the way out), for a spinor or spiral run, and for a **Hubbard** U. The last is the only one of them that would go wrong *during* the run rather than after it: n_σ is built from a plain $\langle\phi|\psi\rangle$ over the stored list, so the Hubbard potential and energy would follow occupations about a quarter of their true size and the SCF would converge silently to a different ground state. `estimate()` makes the same decision the driver makes, so it sizes the run either way.

Four things after the run refuse rather than substitute, because by then the states exist and the honest answer is to say the sum is not written: the projected density of states, the sum-over-states χ_0 (so an optical spectrum of a molecule in a box, which is the archetypal gamma case), an STM image and the vertical tunnelling transmission. Each is a plane-wave sum that would have to be $2\,\mathrm{Re}(\sum) - (G{=}0)$ and is not; the escape is the same explicit $k = 0$, which is an *exact* substitution rather than an approximate one.

The dynamical matrix works under gamma storage, including a partial one — the combination a molecule on a fixed slab needs. The Sternheimer solve is gamma-aware throughout (the projector, the CG's products, the preconditioner, the response density), and a displacement's source term is one jvp through `at_positions`, so it inherits the corrected `h_psi` with no branch of its own. Against the whole sphere: frequencies to **9.6e-10** cm^{-1} and the matrix to **1.5e-13** Ry/bohr^2, with the acoustic residue identical; a partial matrix to **5.9e-11**. And χ_0 against a central difference of the density — which shares no machinery with either storage — to **7.0e-6**.

A potential-only meta-GGA works under gamma storage and did not until 2026-09-18. τ is built from $i(\mathbf{k}{+}\mathbf{G})c_{\mathbf{G}}$, which is *odd* in $\mathbf{G}$ where $c_{\mathbf{G}}$ is Hermitian, so the density's conjugate fill is the wrong fill and the corrected form is not $2\,\mathrm{Re}(\sum) - (G{=}0)$ either. Pairing each $\mathbf{G}$ with $-\mathbf{G}$ gives $\nabla\psi = 2\,\mathrm{Re}\,h$ with $h = \sum_{\mathbf{G}\in\mathrm{half}} i\mathbf{G}\,c_{\mathbf{G}}e^{i\mathbf{G}\cdot\mathbf{r}}$, the $G = 0$ term dropping out on its own because it carries a factor $\mathbf{G}$, so the half-sphere τ is $4(\mathrm{Re}\,h)^2$ where the whole-sphere one is $|\nabla\psi|^2$. Taking $|h|^2$ there, which is what this did, **loses the oscillation pointwise** and is a clean factor of two only in the *integral*: on two-atom silicon under `tb09` the integrals were in the ratio **0.506**, the pointwise disagreement was **7.7e-2** on a τ whose maximum is 1.2e-1, and the two total energies were **16.9 mRy** apart — in a functional where τ enters v_x directly rather than through an energy. Corrected, the two storages agree to **1.3e-15** pointwise with the totals identical to every printed digit.

The velocity operator refuses, and it is the same oddness one step further out. $\partial\hat{H}/\partial\mathbf{k}$ is odd in $\mathbf{G}$, so the full-sphere sum is $2i\,\mathrm{Im}(\sum_{\mathrm{half}})$ and the diagonal of a real state is *exactly zero* — time reversal saying a real state at Γ does not move. The half-sum returns an $O(1)$ number instead: **0.863** Ry bohr against **1.8e-15** at an explicit $k = 0$, on the same cell. That is the whole error rather than half of it, so unlike a density it is not recoverable by the usual rule, which is why this refuses. `Calculation.at_kcart` is the one place it is said, because every consumer — the optical conductivity, the Kerr angle, second-harmonic generation, the shift current, the nesting function and `band_velocities` — reaches the operator through `VelocityOperator`, its only caller. A **topological invariant is unaffected**: `at_kpoints` rebuilds the sphere whole, so a Chern number or a Z_2 index on a gamma ground state runs, and agrees with the whole-

sphere curvature to **7.4e-12** on a curvature of 2.2e-4.
A **field** response is still refused, and the distinction is the reason: a field enters through the commutator $[H, \mathbf{r}]$, which builds plane-wave sums of its own. The dielectric tensor, the Raman tensor, the strain response and the Born charges all stand on it. It does not fail — with the guard lifted silicon's dielectric constant is $501.7/213.1/253.1$ against an isotropic 190.8 — which is why it is a refusal. A checkpoint refuses rather than drops a *driven* magnetic field (a fixed-spin-moment scheme's, which no input file determines); an applied field is saved normally, and so is a DFT+U state, whose manifold is rebuilt from the `HUBBARD` card on load. `stress`, `solver` and `history` are deliberately not stored, so what comes back is a state and not a report.

2 Ground state

The Kohn–Sham problem, generalised because ultrasoft and PAW make the overlap nontrivial:

$$\hat{H}\,|\psi_{n\mathbf{k}}\rangle = \epsilon_{n\mathbf{k}}\,\hat{S}\,|\psi_{n\mathbf{k}}\rangle, \qquad \hat{S} = 1 + \sum_{I,ij} q_{ij}\,|\beta_i^I\rangle\langle\beta_j^I|$$

$$\hat{H} = -\nabla^2 + v_{\mathrm{loc}}(\mathbf{r}) + v_{\mathrm{H}}[n] + v_{xc}[n] \qquad + \sum_{I,ij} D_{ij}^I\,|\beta_i^I\rangle\langle\beta_j^I|$$

with $n(\mathbf{r}) = \sum_{n\mathbf{k}} w_{\mathbf{k}} f_{n\mathbf{k}} \big(|\psi_{n\mathbf{k}}|^2 + \sum_{ij} Q_{ij}^I \langle\psi|\beta_i\rangle\langle\beta_j|\psi\rangle\big)$, the augmentation term being what ultrasoft adds. D_{ij} is rebuilt from the potential every iteration, which is why the SCF is a fixed point in (n, D) and not in n alone. Convergence is measured as QE measures it — the Hartree energy of the residual:

$$dr^2 = \iint \frac{\delta n(\mathbf{r})\,\delta n(\mathbf{r}')}{|\mathbf{r}-\mathbf{r}'|}\,d\mathbf{r}\,d\mathbf{r}' < \texttt{conv_thr}$$

```
result = run_scf(system, pseudos, conv_thr=1.0e-10, mixing_mode="anderson")
print(result.total_energy, result.energy_terms)   # QE's own term-by-term split
print(result.fermi_energy, result.homo, result.lumo)
```

Metals work with every smearing QE has (`gaussian`, `marzari-vanderbilt`, `methfessel-paxton`, `fermi-dirac`) and with the tetrahedron family — both linear and Bloechl-corrected — usable as an occupation scheme *inside* the SCF, not only for a density of states.

Continuation across a change of spin regime. Give `run_scf` a previous result as `starting_from` — `System.with_spin` underneath — and it promotes a converged state into a target's variables: non-magnetic into collinear, collinear into noncollinear, spin-orbit switched on. It reaches the *same* solution as a fresh run ($\leq$4e-8 Ry on six cases) in as little as one iteration. The magnetization is **seeded** from the target's `starting_magnetization` when the source has none, because nothing in the SCF breaks spin symmetry on its own and an unseeded promotion would converge back to the unpolarized solution and report success.

A `HUBBARD` card comes along, and the occupation matrix promotes the way the density does: it is decomposed into a charge and a Pauli vector, the vector is laid along the *target's* own axis — the `angle1`/`angle2` direction or the `STARTING_MOMENTS` card, exactly what the charge and `becsum` are turned onto — and the four spin pairs are rebuilt from the two. That closes the staged route into a hard magnet — converge the collinear antiferromagnet, promote it,

let the moments cant — and, because a spinor-to-spinor promotion is the same code path, the **checkpoint resume** of a noncollinear DFT+U run. Coming back the other way, the moment is projected onto the axis the density is collinear along and the part of it that lies off that axis is **refused** above 10^{-8} rather than dropped: the diagonal of a canted occupation matrix is a different state, not a coarser one.

The shell and the charge have to cross onto the same axis, and until 2026-09-20 only the charge did. The pair was written into the two diagonal spin blocks whatever was asked for, which is a moment along z, so a run whose density crossed onto x started with its correlated shell 90 degrees away from it. On fcc nickel at $U = 4.0$ eV the density arrived with 0.491 μ_B along x and the shell with 0.383 along z, worth 30.7 mRy of Hubbard splitting on the wrong axis. Left free the shell turns, because the carried density's own exchange field pulls it, and what the mismatch costs is iterations — a global spin rotation is free on a scalar-relativistic dataset, so the continuation must cost the same whatever axis is asked for, and it took 2, 7 and 8 iterations at `angle1` of 0, 45 and 90 against 2, 2 and 2 now. Under `mixing_fixed_ns` it is not a cost: the matrix is held at its starting value and the residual of that block is zero while it is held, so the run met `conv_thr` before the freeze was released and reported success with the shell on z, the density on x, and a total 4.11 mRy above the right answer on a cell whose anisotropy is exactly zero.

`magnetization` is how the moment crosses, on `run_scf` and on `Calculator.with_spin` and `with_moments`. `'auto'` is the default and carries the source's moment where it has one, seeding the target's `starting_magnetization` where it does not; `'carry'` insists on the first and raises rather than starting on a symmetric solution; `'seed'` keeps the converged *charge* and takes the moment from the seed regardless, which is how a **different** magnetic state is reached from the same charge; `'none'` starts unpolarized. `with_moments` defaults to `'seed'` where `with_spin` defaults to `'auto'`, because a new texture is a new `STARTING_MOMENTS` card and carrying a noncollinear source's moment applies no rotation, so the card would never be read.

Entering `nspin = 2` with no starting magnetization and no constraint is allowed, and means a starting magnetization of zero, which is a departure from `pw.x`: its `iosys` refuses an LSDA SCF with neither a `starting_magnetization` nor a `tot_magnetization` ("some starting_magnetization MUST be set", `input.f90:1506`). Left out, `with_spin` keeps the tuple the system has, which from an `nspin = 1` source is all zeros, so under `'auto'` the target is seeded with zero and converges to the unpolarized solution, reported as converged; pass `starting_magnetization` to `with_spin` to start polarized. It is not refused because the same call carries a converged moment — a spinor demoted to `nspin = 2` with nothing passed continues from the moment in its state — and `System` keeps only the starting tuple, so it can tell neither that route from a fresh one nor an explicit 0 from an absent value.

```
turning = 0.6 * np.array([[np.sin(a), 0.0, np.cos(a)]
                          for a in np.deg2rad([0, 90, 180, 270])])
cycloid = Calculator.from_file("h4-chain-ferro.in").with_moments(turning)
cycloid.get_scf()                       # 0.427 mu_B per site, a cycloid
afm = np.array([[0.0, 0.0, s] for s in (0.6, -0.6, 0.6, -0.6)])
flipped = cycloid.with_moments(afm).get_scf()   # the converged charge, a new texture
```

On that cell the default is worth 8.9 mRy. Carrying the cycloid's moment into the antiferromagnetic card gives a converged run at -3.792834 Ry whose site moments are $(0.402, 0, -0.402, 0)$, which is the cycloid's projection on z with two sites dead and is not what was asked for; seeding gives -3.801738 Ry at (± 0.589) alternating, the antiferromagnet, in one iteration fewer. Neither run says anything: both report `converged = True`.

Under a Hubbard U the occupation matrix follows the same decision, and until 2026-09-18 it did not: n_σ was the one member of the mixed triple crossing a continuation unchanged whatever `magnetization` said, while the density and `becsum` were reseeded or zeroed beside it. A DFT+U cell converged ferromagnetic and then continued with `magnetization='none'`, or handed a new texture by `with_moments`, entered iteration 1 with a Hubbard potential split by order U — $v = \alpha + \frac{U}{2}\delta - Un^\sigma$, so 0.3 Ry for a typical Ni U — on a density that was exactly spin-degenerate or the new texture's, and `mixing_fixed_ns` freezes it there for as many iterations as it is set to. The run can relax back onto the source's magnetic configuration and report its energy as the answer for the one that was asked for, which is the failure `with_moments`'s `'seed'` default exists to prevent for the density. Now: `'carry'` takes the matrix over, `'none'` takes its spin splitting out and keeps its converged charge, and `'seed'` drops it so that `init_ns` builds it from the *target's* own `starting_magnetization` — which is what a fresh run with that magnetization would do, and is better than any mapping of a density texture onto per-site occupations available here. A checkpoint resume is untouched: `magnetization` other than `'auto'` is refused outright on one.

A spin spiral is a change of frame and not only of regime. A spiral keeps its density in the *rotated* frame, the one that turns with **q**, so its charge and m_z mean the same thing as an ordinary noncollinear run's and its transverse pair (m_x, m_y) does not. A source that has a transverse pair to carry, meaning any source with `nspin_mag = 4`, is therefore **refused** in both directions across a spiral boundary. A collinear source is not: what crosses then is one scalar field, laid along the target's own `angle1`/`angle2`. Spiral to spiral is allowed at any **q**, which is what a sweep needs.

A moment on the spiral's own axis is refused as the ferromagnet it is, and the refusal is at the door. The spiral's spin rotation is about z, so a magnetization along z is invariant under it: a cone of zero opening angle, and a stationary point at every **q**. A run in that state converges, reports a moment, and has computed the ferromagnet with **q** playing no part, which is the *nothing breaks the symmetry on its own* rule of the paragraph above one axis further out. A spiral whose starting moments are all on the axis, or all zero, is therefore refused when the calculation is built, so a run started from scratch is caught as well as a continued one, and `angle1` is named as the way through: 90 degrees puts the moment in the plane, which is the proper screw. Including at $\mathbf{q} = 0$, where the collapsed state is the right answer and refusing it looks strict — a scan builds every other point from that same system, so the one place its moments can be looked at is there.

The direction a carried scalar is laid along comes from the atoms. A collinear source carries one number per point, and where it points in a spinor target is that target's own texture: `System.local_moments`, which is the `STARTING_MOMENTS` card where there is one and `angle1`/`angle2` where there is not. Reading the angles alone said *along* z for a target whose texture is stated only in the card, since the card leaves both angles at zero, and the card the run was built to honour was never read. A target whose atoms point along different axes is refused instead, because one scalar cannot follow a helix, with `magnetization='seed'` named as the way through.

The source density's own electron count is checked, by integrating it, rather than inferred from the atom list. What that catches is a **swapped dataset**: a file with a semicore shell carries a different `z_valence` on the same atoms, and a continuation carries a density rather than renormalising one.

A magnetization held by a field or a constraint says so when it is released. A warning and not a refusal, because hold-converge-release is a workflow: what must not happen is that the moment being carried is the field's answer rather than the

functional's without anyone noticing.
`magnetization` is refused rather than ignored where it has nothing to act on: with no `starting_from`, beside a `ContinuedState` that has already resolved the question, and on a **checkpoint resume**, where a resubmitted command line would otherwise throw the converged moment away after every wall-clock kill.

Convergence. `mixing_mode` selects `anderson` (default), `linear`, `broyden`, Kerker/Thomas–Fermi preconditioning (`TF`), `local-TF`, the local-density-of-states preconditioner (`ldos`), or `adaptive`; `scf_solver` offers a Newton–Krylov alternative that is slower but reaches solutions the mixer cannot hold.

`TF` screens the residual with one Thomas–Fermi length for the whole cell. `local-TF` makes that length a function of $\rho(\mathbf{r})$, which is what a **metal film** needs: the metal wants strong screening and the vacuum wants none, and a single compromise value over-screens one and under-screens the other — charge sloshing between the surfaces. The screened residual is the solution of

$$4\pi e^2 v(\mathbf{G}) + |\mathbf{G}|^2(\alpha v)(\mathbf{G}) = |\mathbf{G}|^2(\alpha\,\delta n)(\mathbf{G}), \qquad \alpha(\mathbf{r}) = 3\left(\tfrac{2\pi}{3}\right)^{5/3} r_s(\mathbf{r}),$$

with (αf) multiplied in *real* space, so the operator is not diagonal in $\mathbf{G}$ and is inverted iteratively at each step. As in `pw.x`, the solve runs on the smooth sphere and the smooth FFT grid, and on an ultrasoft or PAW run the shell between the two cutoffs takes the plain β. On a cobalt film it converges at `mixing_beta = 0.7` in 27 iterations. Plain Anderson takes 30 there, and Kerker at 0.3 is still far out after 250 iterations.

Both are for metal films, and they slow a monolayer or an insulator down, which `pw.x`'s own description of `local-TF` ("for highly inhomogeneous systems") does not say. Kerker damps the residual by $|\mathbf{G}|^2/(|\mathbf{G}|^2 + q_{\mathrm{TF}}^2)$ as though the whole cell were a metal, and in a cell with vacuum the longest wavelength along the normal, $2\pi/c$, is a mode of the vacuum, whose dielectric eigenvalue is 1: damping it only slows the iteration, and by more the longer the cell. `local-TF` screens less in the vacuum but still screens any dense layer as a metal. Iterations at 16 and 64 bohr of vacuum, each cell at its own `conv_thr` and β (`notes/VACUUM-MIXING-NEXT.md`):

cell	`anderson`	`TF`	`local-TF`	`ldos`
$NbSe_2$ monolayer, a 2D metal	12, 9	14, 43	11, 18	10, 10
graphene (c = 20 and 60 bohr)	8, 8	16, 37	12, 22	8, 8
hBN, fixed occupations (c = 20, 60)	11, 11	19, 39	15, 24	11, 11
five Al(100) layers	25, 34	15, 36	13, 20	10, 10

A monolayer, metal or not, converges under plain `anderson` in the same count at every vacuum, and `TF` adds about one iteration for every two bohr of cell length. These counts were taken with a flat fit of the Anderson coefficients. Since 2026-10-13 the fit uses the Hartree metric, as in `pw.x`. With it, plain `anderson` takes the aluminium slab in 14 and 15 iterations instead of 25 and 34, and it converges the cobalt film and the Ni slab of the QE test suite, where the flat fit did not. On bulk cells it takes a few more iterations, about as many as `pw.x`. The keyword `mixing_fit` chooses the fit: `'rho_ddot'` (the default) or `'flat'`. The Hartree metric is the same one `conv_thr` is measured in, so it weights a residual at large G by $1/G^2$. The flat fit also makes this part small. So a quantity that is sensitive to the density at large G can need a smaller `conv_thr` under the default. One case converges only with the flat fit: two canted moments held by `constrained_magnetization = 'atomic'` at `lambda = 10`. `run_scf` warns when `TF` or `local-TF` is asked for with fixed occupations, which is the one case the input can identify.

`ldos` is the preconditioner of Herbst and Levitt (arXiv:2009.01665; DFTK's `LdosMixing`, ABINIT's `iprcel = 200`), and it builds the screening from the **states at the Fermi level** rather than from the charge, which is the property that separates a metal from a dense insulator:

$$\rho_{\text{in}} \;\longrightarrow\; \rho_{\text{in}} + \beta\,\tilde{\varepsilon}^{-1}R, \qquad \tilde{\varepsilon} = 1 - \tilde{\chi}_0 v_H, \qquad \tilde{\chi}_0\,\delta V = -D\,\delta V + D\,\frac{\int D\,\delta V}{\int D},$$

with $D(\mathbf{r})$ the local density of states at ε_F, built at every iteration in the density's own pass over the bands, with a Gaussian of the run's own `degauss`. A uniform D is Kerker with $q_{\text{TF}}^2 = 8\pi D$; a D that vanishes in the vacuum leaves the vacuum alone; a D that vanishes everywhere leaves the plain step. So it is Kerker in the metal, nothing in the vacuum and plain Anderson in an insulator, with nothing to choose. The second term keeps the electron count. On the aluminium slab it takes 10 iterations at both vacua in the table, and on three Co(0001) layers at $\beta = 0.7$ (`tests/data/qe/co-slab-forcetheorem-sr.in`) it takes 20 to 22 at every vacuum from 21 to 59 bohr, where `pw.x` takes 24 with `local-TF`, `local-TF` here 26 to 40, `TF` 37 to 74, and `anderson` does not converge in 150. Each iteration pays an inner conjugate-gradient solve of about 10 to 40 Hartree applications, one FFT pair each.

```
from defuqumat import Calculator
calc = Calculator.from_file("benchmarks/al-slab-v64.in",
                            pseudo_dir="tests/data/pseudo")
scf = calc.get_scf(mixing_mode="ldos")
scf.iterations      # 10, where plain anderson takes 34
```

`ldos` screens the **charge** only, as `TF` and `local-TF` do: the magnetization takes β, so the soft direction of a magnetic film or a 2D magnet is not helped by it. Where there are **no states at the Fermi level** (fixed or `from_input` occupations) the step is the plain one and `run_scf` warns that the run is plain `anderson`, which it is to the bit. Herbst and Levitt name two cases it does not help: a **metal on a semiconductor** (Kerker 26, `ldos` 26 on their Al on GaAs), and **localized orbitals at** ε_F, where it is "as ineffective as homogeneous schemes", which is a d-electron magnet's case.
On a bulk d-metal it is worse than plain `anderson`, measured: bcc iron on a $4 \times 4 \times 4$ grid, nonmagnetic (`benchmarks/fe-noncolin-nonmagnetic.in`), takes 28 iterations against 10, and the one-k-point noncollinear magnet `benchmarks/fe-mag-1k.in` 20 against 11. It is not the sampling (an LDOS smeared at 0.1 and 0.2 Ry takes 20 as well) and not the magnetization: a d-band at ε_F gives $8\pi\langle D\rangle = 7.8$ bohr^{-2} where the free-electron Thomas–Fermi value is 1.9, so the local model screens every wavelength of a small bulk cell four times too hard, and `TF` costs only 13 against 11. A bulk cell has no sloshing to cure; leave it at `anderson`. Near a magnetic transition the mode can also change the state found: on `benchmarks/fe-unstable.in` it lands on a moment of $0.42\,\mu_B$, 6.2×10^{-4} Ry below the nonmagnetic state plain `anderson` stops at.
Tetrahedra are refused: the LDOS needs a smooth delta at ε_F, and with a Gaussian in its place the aluminium slab under `tetrahedra_opt` converged under neither `ldos` nor plain `anderson` in 200 iterations, so the combination has no number behind it. It is refused by name, rather than run as plain `anderson` under its name, wherever a mixer is built with no LDOS to hand it: the Newton–Krylov warm-up (`warmup_mixing`), the ultracell and the response loops. Under `scf_solver = 'newton-krylov'` the solver's own preconditioner is Kerker and `mixing_mode` does not reach it.

`adaptive` is Elk's `mixadapt` and answers the opposite failure. Every other mode here damps an iteration that is *overshooting*; this one lengthens one that is crawling. It carries a separate step length for each component j of the mixed vector and moves it by the sign

of that component's residual,

$$\beta_j \leftarrow \min(\beta_j + \beta_0,\ \beta_{\max}) \quad \text{if } r_j \text{ kept its sign,} \qquad \beta_j \leftarrow \tfrac{1}{2}(\beta_j + \beta_0) \quad \text{if it changed,}$$

and then steps as $\beta_j\,\nu_j + (1-\beta_j)\,\mu_j$. A component pushed the same way for many iterations is one the SCF is crawling along, so its step grows until it moves; one that overshoots and comes back has its step cut. That makes it a stall detector with no threshold in it, which is why it is worth reaching for on a **magnet**: turning every moment in a cell together costs no energy without spin–orbit coupling, so the residual has no component along that rotation and the flat direction has to be crossed rather than descended, and twisting the moments slowly costs only the spin-wave energy Dq^2, which vanishes as the cell grows. The whole packed vector adapts independently, so `becsum` and the Hubbard `ns` get their own step lengths rather than the density's.

`mixing_beta` does not mean the same thing under `adaptive`, and this is the one thing to know before using it. It is Elk's `beta0`: the increment, the starting value and the floor the halving relaxes towards, all three, not a step length. Elk's default is 0.05 and it is small on purpose; 0.7 saturates at $\beta_{\max}$ within two iterations and leaves plain linear mixing at 0.85 under an adaptive name, which is what the warning above that value is for. Each mode supplies its own default when `mixing_beta` is unset, 0.7 for the QE family and 0.05 here, so an input that names neither gets the right one; an input that *does* set `mixing_beta` has that number adopted as β_0, and the same figure therefore means two different things depending on the mode.

Elk's 0.05 is a safe default rather than the best value, and the measurement says so: on bcc iron with its moment in the plane (`tests/data/qe/fe-noncolin-pbe-stress.in`, ultrasoft, PBE, noncollinear) $\beta_0 = 0.2$ converges in **16** iterations where $\beta_0 = 0.05$ takes 24, because the soft directions want the step to grow faster than Elk's default lets it. Anderson on the same cell took 43 at the input's own `mixing_beta` when this was measured (2026-09-14; it takes 15 on the code of 2026-10-04, measured on another machine, the adaptive counts not re-measured) and 24 at the best one for it, and `pw.x` takes 19; all of them reach the same energy within 3×10^{-10} Ry and the same moment *length* within $3 \times 10^{-5}\,\mu_B$, with the transverse components that symmetry makes zero coming out below $2.4 \times 10^{-4}\,\mu_B$. So raising β_0 is the knob to try on a magnet that will not move, and it is the opposite of the reflex a slow SCF usually gets. The saving is smaller than the iteration count suggests, because the diagonalisation threshold is scheduled from the density error and a mixer that drops it faster is handed a tighter eigenproblem: the Davidson work falls by $1.6\times$ where the iteration count falls by $2.7\times$.

On an easy cell it loses, which is what a robustness mixer does: two-atom silicon (`benchmarks/si-1k.in`) at `conv_thr = 1e-10` takes 17 iterations against Anderson's 7, to the same total energy to ten digits.

A mixer can change which solution the run finds, and this is the thing to know before trusting an `adaptive` result on a magnet. On a four-cell hydrogen ultracell under a field turning $90°$ per cell — the case where rotating every moment together costs nothing — that cell has **three** self-consistent solutions, and every one of them reports `converged = True` with an `accuracy` three orders below `conv_thr`. Anderson at $\beta = 0.1$ and 0.7 finds one of them, agreeing to eight digits of the band energy; Anderson at $\beta = 0.3$ finds a second; `adaptive` at both its values finds the third, the one whose moments follow the applied field's ladder. **Which is the ground state is not something the run reports**: the moments differ by tens of degrees per cell, and nothing in a converged result orders them.
The step length does this too, which is the part not to file under "adaptive". Anderson's own β moves between solutions here, so this is not a property of one mixer

and running a second mixer is not a test that the basin was kept. What a growing step buys is more of the same freedom, not a new kind of it.
So on a cell with a soft or degenerate magnetic direction, vary **both** the mixer and β, and compare the **moments** rather than the iteration count: a converged run is not evidence that the basin was kept, and "fewer iterations" is satisfied by a run that found something else. `SCFResult.site_moments` and `UltracellResult.cell_moments()` are what to compare.
`mixing_mode = 'adaptive'` also refuses a **preconditioner** rather than ignoring one. Its step is pointwise in real space with its own factor per point, and Kerker or `local-TF` is a single scalar acting in **G**-space; the two do not compose, so a run asking for both raises. Choose the one that matches the direction the cell is slow in: Kerker for long-wavelength charge, this for a residual that is small and will not turn around.
It is registered under its own name and is **not** aliased to any `pw.x` mode. QE has no adaptive mixer, so a `pw.x` input reaching one of QE's names here still gets what QE would do.

`max_iterations` defaults to 100 — **a hard SCF may need more**, and hitting the cap is reported as not converged rather than as an answer: `run_scf` issues a `RuntimeWarning` naming the accuracy it reached, and `Calculator` raises. Both matter, because a script driving the functional entry point — a spiral scan, a λ ramp, a moment-versus-field sweep — is exactly what would otherwise read `total_energy` off an unconverged density without looking at `converged`.

`mixing_beta_mag` gives the **magnetization** a step length of its own, separate from the charge's `mixing_beta`. It is this code's own knob and a deliberate departure from `pw.x`, which uses one `alphamix` for every component of its density; VASP exposes the same thing as `AMIX_MAG` and Elk gives the magnetic channel its own control. Unset by default, which is QE's rule, and setting it equal to `mixing_beta` reproduces an unset run exactly rather than nearly.

The reason it exists is that the two channels are slow for different reasons and one scalar cannot serve both. The charge's bad direction is the long-wavelength Hartree one, which Kerker and `local-TF` compress; the magnetization's is the Stoner enhancement $\chi_0/(1-I\chi_0)$, which is local, has no $1/q^2$ for a preconditioner to divide out, and amplifies a uniform change at every wavelength equally. Nothing conditions that direction, so a step length is all there is.

Whether to use it is a measurement rather than a preference, and the measurement is `residual_split`. `run_scf(residual_split=True)` puts a five-way split of each iteration's residual into `SCFResult.history` — charge, the length of **m**, the rigid rotation, the rest of the transverse channel, and the magnetic part of `becsum` — and it is what says which channel a given cell is spending its iterations in. The two committed iron benchmarks are opposite cases and the knob follows the split, not the cell:

`mixing_beta_mag`	`fe-noncolin-pbe-stress`	`fe-mag-1k`
unset	43	25
equal to `mixing_beta`	43	25
0.6	**27**	36
1.0	**20**	31

The first cell spends 28 of its 43 iterations in the magnetic channel and the knob takes it to 20, against `pw.x`'s 19 on the same input. The second spends 4 of 25 there, and the same knob makes it a third *worse*. Every run in that table converged to the same total energy as its own unset baseline within 3×10^{-10} Ry, so these are the same states reached

sooner or later and not different ones.

`mixing_beta_mag` is **refused at setup** with `mixing_mode = 'adaptive'` rather than ignored. That scheme already carries one step length per component of the mixed vector, so a second global one for a subset of them is not defined; the refusal is on the same property that makes `adaptive` refuse a preconditioner, and it is raised when the mixer is built rather than at the first mix, so a run that cannot work does not find out hours in. A non-positive value is refused for the same reason any step length would be.
It acts on the **density** alone. `becsum`, `ns` and τ keep the charge's step, because giving them the magnetic one would be a second departure with no measurement behind it. `becsum`'s own residual is now reported every iteration — outside `dr2`, where it cannot be, since `paw_ddot` is not positive definite and `pw.x` comments the term out for that reason — so whether it needs a step of its own can be asked with a number.

`mixing_ndim` sets how many previous iterations the extrapolation spans, default 8 as in `pw.x`. Raising it is the standard first move on a cell that will not converge, and *on the magnetic benchmark here it does not help*: `benchmarks/fe-mag-1k.in` at `conv_thr = 1e-8` takes 27 iterations at 4, **25 at 8**, 33 at 12 and 39 at 20, so the default is already the best value and the deeper histories are worse. The nonmagnetic twin of the same cell (`fe-unstable-nonmagnetic.in`, same β) has the same shape — 26, **21**, 30, 26 — which says the sensitivity is not about magnetism.

`dr2` is reported as its two halves as well as its total, in `scf.history` and on the console. `rho_ddot` weights the charge residual by $1/G^2$ and the magnetization residual by a constant — a factor of 13.6 apart at $G_{\min}$ on `fe-mag-1k` — so a single `accuracy` below `conv_thr` bounds the moment much more weakly than it bounds the charge, and only the split can say whether it is the charge or the magnetization that is still moving.

2.1 An electric field across a slab, and the dipole correction

How do you put a uniform electric field into a periodic cell? A linear potential is not periodic. So the code uses a saw-tooth, as `pw.x` does. The potential rises linearly over most of the cell along the reciprocal vector $\mathbf{b}_{\text{edir}}$. Then it falls back in a thin region of width `eopreg`, which must lie in the vacuum. The field is switched on with `tefield` and has the amplitude `eamp`, in Hartree atomic units.

A slab with a different surface on each side has a dipole. Through the periodic images, this dipole creates an artificial field in the vacuum. The dipole correction (`dipfield`) cancels it (Bengtsson, PRB **59**, 12301 (1999)). It adds a second saw-tooth whose slope is the dipole of the current density. So the total energy of the field is:

$$E_f = \frac{\Omega}{4\pi}\, e^2 \Big[-E_{\text{amp}}\, D + \tfrac{1}{2} D^2 \Big], \qquad D = \frac{4\pi}{\Omega} \Big[\sum_a Z_a\, s(x_a) - \int n(\mathbf{r})\, s(x(\mathbf{r}))\, d\mathbf{r} \Big]\, d.$$

Here s is the saw-tooth, x is the crystal coordinate along the field, and $d = 2\pi/|\mathbf{b}_{\text{edir}}|$ is the distance between lattice planes. The $D^2/2$ term is there only with `dipfield`. The potential is the derivative of E_f with respect to the density, and the force is its derivative with respect to the positions. Both come from `jax.grad` of this one energy.

The input is the `pw.x` input. Here it is the CO molecule of `pw_relax/relax-el.in`, with the dipole correction added:

```
calc = Calculator.from_file("efield-co-dipole.in")
scf = calc.get_scf(conv_thr=1e-13)
```

```
print(scf.total_energy)       # -43.1230004596 Ry
print(scf.efield_energy)      # 8.1867e-05 Ry, part of the total
print(scf.efield_dipole)      # -0.0337687, D Omega/(4 pi) in Ry a.u.
print(calc.get_forces().forces[:, 2])   # [ 0.11697173 -0.11697173]
```

`scf.efield_energy` is always the whole energy of the field, electrons and ions. Without `dipfield`, `pw.x` prints only the part of the ions as "electric field correction". The part of the electrons is then inside its one-electron term. The total energy is the same in both codes.

Refuses. The **stress**, and so a variable-cell relaxation. `pw.x` switches the stress off with this field too. Also every **linear response**, real-time propagation and the ultracell, which have not been checked with the field. `gate` (a charged plate) needs a charged cell, and `tot_charge` is refused. `dipfield` without `tefield` is an error, because in `pw.x` it does nothing. As in `pw.x`, the run uses no symmetry (`nosym`), and the code warns about it.

Validated against `pw.x` on the CO molecule, with and without `dipfield`, both at `conv_thr = 1e-13`. The total energies agree to 4×10^{-10} Ry and the forces to 4×10^{-7} Ry/bohr. The dipole and the field energy agree too. The relaxation of `relax-el.in` ends at the energy of the QE benchmark, -43.1264736 Ry. Two checks need no other code. The derivative of the total energy with respect to `eamp` is the dipole, $-e^2 D\,\Omega/4\pi$, to 10^{-4} relative. And an unpolarized `nspin = 2` run gives the `nspin = 1` energy to 10^{-8} Ry. On the Ni slab with CO of the QE test suite (`pw_dipole`) the total energy agrees to 7×10^{-11} Ry and the forces to 5×10^{-6} Ry/bohr, in 27 iterations against the 26 of `pw.x`.

2.2 A finite electric field in a crystal, by the Berry phase

What does a crystal do in a static uniform field? In a periodic cell we cannot add $-e\mathbf{E}\cdot\mathbf{r}$, because $\mathbf{r}$ is not periodic. So the code minimises the electric enthalpy instead (Souza, Iniguez and Vanderbilt, arXiv:cond-mat/0205633):

$$F[\psi] = E_{\mathrm{KS}}[\psi] - E\left(p_{\mathrm{el}}[\psi] + p_{\mathrm{ion}}\right), \qquad p_{\mathrm{el}} = \frac{2\sqrt{2}\,|\mathbf{a}_g|}{2\pi}\,\frac{1}{N_s}\sum_s \mathrm{Im}\ln\prod_{\mathbf{k}\in s}\det M(\mathbf{k},\mathbf{k}+\Delta\mathbf{k}).$$

Here M is the overlap of the occupied bands at two neighbouring k-points on a string along $\mathbf{b}_g$, and N_s is the number of strings. This is the Berry phase of the polarization. The derivative of F with respect to the wavefunctions is an operator that couples each k-point to its two neighbours. The code builds it from the current states before each diagonalisation, `nberrycyc` times per SCF iteration, as `pw.x` does. The field `efield` is in Ry atomic units, and the dipoles are in Ry atomic units too. This is `lelfield` in `pw.x`, with `gdir`, `nppstr` and an explicit list of k-points: strings of `nppstr` consecutive points.

Switch the field on from a converged state without field. From random wavefunctions the bands at neighbouring k-points are almost orthogonal, and the operator is then very large. Every input of `pw.x` for this does the same:

```
crystal = Calculator.from_file("si8-lelfield-zero.in")   # efield = 0
ground = crystal.get_scf()
calc = Calculator.from_file("si8-lelfield.in")           # efield = 0.001
scf = calc.get_scf(starting_from=ground)
print(scf.total_energy)       # -63.0661033823 Ry
print(scf.berry_dipoles)      # (0.92995686, 115.1735525, 0.0): el, ion, jumps
```

```
print(calc.get_forces().forces[0])   # [-0.01260638 -0.01260638 -0.01251766]
```

`scf.efield_energy` is $-E\,(p_{\mathrm{el}} + p_{\mathrm{ion}})$ and is part of the total energy. The third number of `berry_dipoles` counts the jumps of the phase by 2π during the run, as `pw.x` does.

Refuses. An **ultrasoft or PAW** dataset. The overlap M then needs the augmentation between two k-points, and the force needs its derivative. **`nspin = 2`** and spinors. A **metal**: a Berry phase belongs to a gapped set of bands. The three-direction form `efield_cart`, and an automatic k-grid, which `pw.x` turns into that form. A cell where $\mathbf{a}_g$ is not along the Cartesian axis g. The **stress**: the field terms of the stress are not in `pw.x` either. Also DFT+U, a spin spiral, `tefield` at the same time, and every linear response.

Validated against `pw.x` 7.5 on the 8-atom silicon of its own test, with 9 strings of 7 k-points. The total energy agrees to 1.2×10^{-8} Ry. The electronic dipole agrees to 8.6×10^{-6}, while the value of `pw.x` still moves by 1.6×10^{-5} in its last iteration. The forces agree to 10^{-7} Ry/bohr. One check needs no other code. The energy difference between two fields, without the linear part of the ions, is the integral of p_{el} over the field, to 10^{-7} Ry. The operator is checked against the derivative of the Berry phase itself, to 10^{-12}.

3 Bands, densities of states, projections

A band structure is one diagonalisation per **k** on a fixed density — `run_bands`, or `run_nscf` for a non-self-consistent run on a grid rather than a path. A density of states is

$$g(E) = \sum_{n\mathbf{k}} w_{\mathbf{k}}\,\delta(E - \epsilon_{n\mathbf{k}})$$

with δ replaced by a smearing function or evaluated by tetrahedron interpolation. The projected DOS weights each term by $\left|\langle\phi_\mu|\hat{S}|\psi_{n\mathbf{k}}\rangle\right|^2$, and the spilling parameter is what is *not* captured, $\sum_{n\mathbf{k}} w_{\mathbf{k}} f_{n\mathbf{k}}\big(1 - \sum_\mu |\langle\phi_\mu|\hat{S}|\psi\rangle|^2\big)$.

```
from defuqumat.workflows.bands import run_bands
from defuqumat.workflows.dos import run_dos
from defuqumat.workflows.pdos import run_pdos

bands = run_bands(system, pseudos, scf.density, kpoints=path)   # a k-path

dos, _ = run_dos(system, pseudos, scf.density, grid=(12, 12, 12),
                 scheme="tetrahedra_opt")     # -> (DensityOfStates, NSCFResult)

pdos, _ = run_pdos(system, pseudos, scf)      # -> (ProjectedDOS, NSCFResult)
print(pdos.channels[0])                       # resolved by atom, l and m
print(pdos.charges.charges, pdos.charges.spilling)
```

The DOS schemes live behind a name registry (`DOS_SCHEMES`), and the projected DOS feeds *the same* registry as a per-band weight, so a PDOS and a DOS are the same integration. Projections are $\langle\phi|\hat{S}|\psi\rangle$ on Löwdin-orthogonalised pseudo-atomic orbitals. Validated against a purpose-built `projwfc.x` on seven cases: **6.9e-4** on a projection, **4.7e-5** on a Löwdin charge, and the spilling parameter to all four decimals it prints.

Resolved by j and m_j for a spin-orbit run

With `noncolin` and `lspinorb` the orbitals projected onto are the **spin-angle functions** $|l\,j\,m_j\rangle$ rather than $|l\,m\rangle$ times a spin, so a channel is labelled by j and m_j and the projection separates the two branches spin–orbit coupling splits a shell into. For platinum that is $5d_{3/2}$ against $5d_{5/2}$: one number, resolved, where an l-resolved projection reports their sum and cannot see the splitting at all. The functions are

$$|l\,j\,m_j\rangle \;=\; \sum_{\sigma=\uparrow,\downarrow} C\big(l,\tfrac{1}{2},j;\,m_j-\sigma,\sigma\big)\;Y_{l,\,m_j-\sigma}\;\chi_\sigma,$$

Clebsch–Gordan coefficients against the harmonics, and the number of columns is $\sum_{\text{shells}}(2j+1)$ — which is *not* in general twice the scalar count, because it is read off the dataset's own j channels.

```python
from defuqumat import Calculator

calc = Calculator.from_file("pt-soc-paw-nosym.in",   # noncolin + lspinorb
                            pseudo_dir="../pseudo")
pdos = calc.get_pdos()

print(pdos.channels[0])          # -> #1 Pt1 s_j0.5 m_j=-0.5
print(pdos.charges.format())     # -> s = 0.5032, p = 0.9944, d = 8.4903
print(pdos.charges.charges_lm)   # -> None: a spin-angle function has no m
```

Validated on fully-relativistic **ultrasoft and PAW** platinum datasets against `projwfc.x` runs generated for them, four ways. The *columns* — how many, and each one's (l, j, m_j) — match QE's own label table exactly. The two datasets differ there in a way worth seeing: the PAW file carries both j partners of every l, so its 18 columns are exactly twice the j-averaged 9, while the ultrasoft one has 12 against a scalar 11 — the count is $\sum(2j+1)$ read off the file and not a doubling. The *projections* agree to **6.3e-6**, compared as sums over each degenerate multiplet: a spinor band structure is Kramers-degenerate everywhere, and an individual $|\langle\phi|\hat{S}|\psi_n\rangle|^2$ is not invariant under the mixing inside a multiplet that either code's eigensolver is free to choose, where the sum is. The **Löwdin charges and the spilling parameter agree to every digit `projwfc.x` prints**. And the *curves* — the total and each shell's, on `projwfc.x`'s own energy grid and broadening — agree to **0.16 per cent of the peak** everywhere below the lowest empty band, above which the topmost states are the ones neither eigensolver converges. Timing, one core each from a converged ground state: `get_pdos` costs **1.65 s** against `projwfc.x`'s 0.62 s, and on both sides that is mostly rebuilding from the pseudopotential files rather than projecting — the projection and its integration are 0.137 s of it here.

Norm-conserving is untested rather than refused — no reference case is committed for it.

Two spin channels for a noncollinear run without spin–orbit coupling

With `noncolin` but `lspinorb = .false.` there is no j to resolve by: nothing couples s_z to the orbital motion, so s_z is still a good quantum number and the orbitals are $|l\,m\rangle\chi_\sigma$ — an up and a down copy of every harmonic, 2 natomwfc columns where a collinear run has natomwfc. What comes back is nonetheless *not* twice as many channels: the columns are routed into an **up and a down density of states**, which is `partialdos_nc`'s `nspin0 = 2` and is the shape an LSDA projection has. This is the regime of any magnetic run with scalar-

relativistic datasets, and it is what `h10-chain-noncolin.in` and `alas-magnetoelectric-nosoc.in` are.

```
from defuqumat import Calculator

calc = Calculator.from_file("h-atom-noncolin.in",   # noncolin, no lspinorb
                            pseudo_dir="../pseudo")
pdos = calc.get_pdos(symmetrize=False)

print(pdos.nspin)                # -> 2: an up channel and a down one
print(pdos.channels[0])          # -> #1 H1 s: no s_z, the spin is an axis now
print(pdos.charges.polarization) # -> [0.99857905]: the moment the s shell carries
```

The decomposition is along the **global** z and not along the local moment, exactly as QE's is: an in-plane texture reports two equal channels, which is a statement about the frame the orbitals are built in rather than a defect.

No `projwfc.x` reference for this regime is committed here, so what validates it is an identity instead, and a sharper one: without spin–orbit coupling a state polarized along $+z$ block-diagonalises the noncollinear Hamiltonian into the two collinear ones, so the projection of such a run must reproduce an LSDA run of the same cell channel for channel — and the LSDA route is itself validated against `projwfc.x`. On a hydrogen atom in a 12 bohr box the two runs agree to **3e-12 Ry** in total energy, their majority curves to **2e-5 of the peak** and their minority ones to **0.4 per cent**, and their Löwdin charges to **5e-3** on a polarization of **0.9986** electrons — which is the number a code that binned both spins into one channel, or split the weight evenly, would get wrong while passing every sum rule. The two bounds differ for an arithmetic reason rather than a chosen tolerance: the noncollinear branch reaches $\rho_\downarrow$ as $(n - |m|)/2$, a cancellation of two numbers of order 0.1 where the collinear branch carries $\rho_\downarrow$ itself, and that is worth 1.1×10^{-4} Ry on the *empty* minority eigenvalue where every occupied one agrees to 10^{-6}.

Unlike a spin–orbit run this case *does* have an m to report, so `charges_lm` is filled where a j-resolved projection leaves it `None`. `print_lowdin` allocates it only `IF (nspin /= 4)` and so prints none for either noncollinear branch; the reason it gives — a spin-angle function has no m — is true of the spin–orbit branch alone, and the table here is the richer one on purpose.

A reduced k-set is allowed here, and what makes it so is that the group average runs over the right operator. A point-group operation turns two things at once in this regime — the real harmonics among themselves and the spin frame the spinor is written in — so the matrix averaged over is the tensor product $D^l \otimes \overline{U}$, with U the $SU(2)$ rotation that carries $\mathrm{i}\sigma_y D^*$ for an operation that is a symmetry only together with time reversal. Averaging the m indices alone leaves the spin frame where it was.

```
from defuqumat import Calculator

# fcc nickel, noncollinear, moment along (1,1,1): twelve operations, six of
# them with time reversal, and six of the twelve mix the two spin components.
calc = Calculator.from_file("ni-noncol-111.in", pseudo_dir="../pseudo")
pdos = calc.get_pdos()                   # symmetrize = True is the default
# (nspin, nat, l, m): the majority d weights of the one atom, per m.
print(pdos.charges.charges_lm[0, 0, 2])
# -> [0.94676054 0.9178314  0.9178314  0.94676054 0.91783139]
```

The number: against the same cell run at `nosym` on the closed $4 \times 4 \times 4$ grid, the wedge's Löwdin charges agree to **1.1e-5 electrons** per column with the symmetrisation on and to **5.4e-2** without it. On the four-hydrogen helix (`h4-cycloid-90.in`), where the shell is s and the spin factor is therefore the *whole* symmetrisation, the same pair reads **4.1e-6** against **3.8e-4**.

The conjugation is measured rather than assumed, and the obvious cell cannot measure it. Nickel with its moment along z — which is what `ni-ldau-noncol.in` is — has every operation turning the spin about z alone, so its 2×2 matrix is diagonal, its phase drops out of a modulus, and U, U^T, $U^\dagger$ and the identity all give the same answer to the last digit. Turning the moment onto a three-fold axis separates them: U, U^T and $U^\dagger$ land 3.9e-2, 2.1e-2 and 2.1e-2 from the closed grid, which is worse than not symmetrising the spin at all, and only $\overline{U}$ reaches 1.1e-5.

Refuses. A **symmetrised** projection of a `lspinorb` run — run with `nosym` and the whole **k**-grid, where both codes use every point and neither averages, or pass `symmetrize=False`. Its columns are $|j\, m_j\rangle$ and the operator `sym_proj_so` averages over is D^j for $j = \frac{1}{2} \ldots \frac{7}{2}$, which is a different matrix from the one the branch above uses and is not reachable by relabelling it. *The noncollinear branch without spin–orbit coupling is no longer refused*: its columns are $|lm\rangle \otimes |\sigma\rangle$, the operator is the tensor product $D^l \otimes \overline{U}$ of the harmonic rotation and the conjugated $SU(2)$ one, and it is built. A **fully-relativistic dataset with `lspinorb = .false.`**: QE dispatches the orbitals on `has_so` and their labels on `lspinorb`, so it builds j-resolved columns and calls them up/down ones. An m-resolved charge is *absent* rather than refused for a **spin–orbit** run: `charges_lm` is `None` there, since a spin-angle function has an m_j and no m, and there is no p_x weight to report.

4 Pseudopotentials and functionals

Kinds: norm-conserving, **ultrasoft** and **PAW** — the two-grid split, the augmentation charge, the overlap operator, self-consistent D_{ij}, and PAW's one-centre terms including its radial Poisson solve and spherical quadrature. UPF v2 is read. **The kinds mix within one cell**, which is an ordinary `pw.x` run and is now an ordinary one here: a norm-conserving species in a PAW cell has projectors and therefore rows of the (n_{kb}, n_{kb}) coefficient matrix, and it has no augmentation charge and no one-centre correction, so those rows stay zero and its bare D_{ij} arrives through the same route it always did. Displaced silicon with `Si.pz-n-kjpaw_psl.0.1.UPF` on one site and `Si.pz-vbc.UPF` on the other matches `pw.x` to **4.7e-9** Ry and **1.15e-5** Ry/bohr on a force of 0.0758 — between the same cell run all-PAW (3.9e-7) and all-norm-conserving (2.0e-5), so each species keeps its own accuracy.

Functionals: LDA (PZ), **PBE**, **revPBE**, **PBEsol**, on the plane-wave grid and on the PAW spheres. The functional comes from the pseudopotential headers unless `input_dft` overrides it. Only the *energy* is written down; v_{xc}, and a GGA's v_1 and v_2, come from `jax.grad` of it.

Meta-GGA, potential-only branch: `input_dft = 'tb09'` (Tran–Blaha) and `'bj06'` (Becke–Johnson). These invert the rule above — they *are* potentials, there is no energy — so the consequences are enforced rather than documented: `run_scf` warns that its total is not the value of any functional it minimised, and forces, stress, phonons and response all refuse. Silicon's gap goes from LDA's 0.49 eV to **1.13 eV** against an experimental 1.17.

Van der Waals: Grimme **D2** (`vdw_corr = 'grimme-d2'`). Bilayer graphene matches `pw.x` to **3.1e-9 Ry** and binds at 3.23 Å where PBE alone has no minimum.

Refuses. UPF v1; an unimplemented functional (rather than silently substituting LDA); energy-carrying meta-GGAs (TPSS, SCAN, M06L), whose potential has a $\partial E/\partial\tau$ piece acting on the wavefunction that is not written; plain *ultrasoft* under a meta-GGA, which has no partial waves to reconstruct τ from inside the sphere where PAW does; EXX; and **D3, Tkatchenko–Scheffler, MBD and XDM** — where `pw.x` warns and silently runs with no correction at all.

5 Spin, magnetism and spin-orbit

regime	how to ask	note
unpolarised	default	
collinear	`nspin = 2`	one Fermi level, or two under `tot_magnetization`
noncollinear	`noncolin = .true.`	magnetization is a vector
spin-orbit	`+ lspinorb = .true.`	j-resolved projectors; NC, US and PAW

Three spin numbers are kept apart, and `System` exposes all three: `nspin` says which regime is in force, `npol` is the number of spinor components of a *wavefunction*, and `nspin_mag` the number of components of a *density*. They coincide at 1 and 2 and come apart at 4, where `npol = 2` but `nspin_mag` is **one** for a nonmagnetic spin-orbit run — which is why such a run costs about what a doubled unpolarized one costs.

Fixed occupations under `nspin = 2` fill each channel to its own count rather than searching for a level: `tot_magnetization` gives `nelup` and `neldw`, and `iweights_only` fills NINT($n_\uparrow$) bands in one channel and NINT($n_\downarrow$) in the other. That is the *only* shape the combination has — `pw.x` refuses fixed-occupation LSDA without a `tot_magnetization`, and requires an integer one — and this package makes the same two refusals at input, before anything is allocated. Each channel's highest occupied level is reported as `fermi_energy_up` / `fermi_energy_down` beside the overall `homo` and `lumo`, which is what `iweights` returns. Validated on an oxygen atom at `tot_magnetization = 2`, whose four-up / two-down filling is the Hund's-rule ground state: **4.9e-9 Ry** against `pw.x`. A channel the constraint leaves with no electrons runs, as it does in `pw.x`: it is filled with nothing and reports `iweights`' own -10^{20} as its level, and the fully polarized hydrogen atom at `tot_magnetization = 1` lands on `pw.x`'s -0.94606495 Ry to every printed digit. A $|$`tot_magnetization`$|$ larger than the electron count is refused, where `pw.x` fills the other channel anyway and stops on the wrong charge.

Magnetic fields and constraints: a uniform field over the cell, a field inside one atom's sphere (through a `LOCAL_MAGNETIC_FIELDS` card that `pw.x` has no counterpart for), Elk's `reducebf`, and all four of QE's `constrained_magnetization` schemes. **The field's energy is not part of the reported total** — QE prints `etcon` and never adds it, and Elk excludes its external field's energy by the same convention.

A field lowers the symmetry group, and here that includes a uniform one. A uniform `B_field` is an axial vector on every atom, so in a noncollinear run it enters the magnetic symmetry filter beside the moments, as Elk's `findsym` does with `bfieldc0`; `pw.x` hands its filter the moments alone, so a `pw.x` run with a field off the moment's axis keeps the larger group. On bcc Fe along z with `B_field(1) = 0.01` this drops the C_{4z} that maps B_x onto B_y, which is what had been averaging the canting away. A `LOCAL_MAGNETIC_FIELDS` card labels its atoms in the same way at `nspin = 2`, so a staggered field on two otherwise equivalent atoms keeps only the operations that do not swap them. To change the field of an existing system, use `system.with_b_field(...)`, which rebuilds the k-set with the new group.

A field needs no starting moment, on a cell that is not a magnet already. A noncollinear run with a `B_field` and every `starting_magnetization` zero is magnetic because of the field, and a gradient-corrected functional takes the sign of the magnetization along the field. Where the field-free ground state is nonmagnetic this converges to the induced moment: a hydrogen atom smeared at `degauss = 0.2`, whose seeded field-free run decays to $10^{-4}\,\mu_B$, takes $+0.159\,\mu_B$ along a `B_field(3) = 0.01`. Where the cell polarizes on its own it does not: the same atom at `degauss = 0.02` converges to $-0.103\,\mu_B$, against the field, which is the stationary point near zero moment that an SCF finds as readily as a minimum, 0.052 Ry above the polarized state. A spontaneous magnet still wants a seed of the size of the moment it will carry, and the field does not choose its sign: a seed of 0.6 along the field reaches $+1.0\,\mu_B$, a seed of 0.05 crosses to $-1.0\,\mu_B$ at the same total energy, the field's energy being outside the total. This departs from `pw.x`, whose `domag` reads the starting moments alone: it accepts the unseeded input and converges the *field-free* calculation, because `add_bfield` writes the field into the potential's magnetization channels and `vloc_psi_nc` applies those only `IF (domag)`. A comparison against `pw.x` for such an input is therefore a comparison against no field.

An `r_m` left unset for a species, or set to zero, takes `make_pointlists.f90`'s default radius rather than a sphere of radius zero.

A seed field has no natural size, so the symmetry filter has no scale in it. An infinitesimal field is the standard way to start a texture: small enough not to bias the energy, large enough to pick the state. What the magnetic filter asks of a field is a *pattern* — does the operation map it onto itself, up to time reversal — so each field is compared after being divided by its own largest component, and it is ignored only below 10^{-12} Ry, which is the same floor that decides whether the run is magnetic at all. One rule: a field big enough to make a run magnetic is big enough to cut its group. (It used to be two, seven orders of magnitude apart, and a field in between turned the run magnetic and then contributed nothing to the filter — which is the one thing worse than no field, because the full crystal group then averages away the texture the field was applied to create.)

`reducebf` *decays the field after the convergence test*, so a run can stop with the field still substantially on. The state reported is then the ground state of a functional that includes a Zeeman term, and **the error in the total is second order in the residual field**, not the field energy — the state is stationary. On a hydrogen atom with a 0.1 Ry local field, `reducebf = 0.5` stops with 6×10^{-6} Ry still applied and a total right to 5×10^{-14} Ry, while `reducebf = 0.99` stops after *six* iterations with 9.5×10^{-2} Ry applied and a total 4.7×10^{-5} Ry out. A run that stops above 10^{-4} Ry now says so, naming `field_scale`, the residual and `field_energy`; a field held at full strength (`reducebf = 1`) is a deliberate calculation and is left alone. `reducebf` is also held to Elk's own $[0.5, 1]$ (`readinput.f90:1264`), which this used to accept any value outside: above 1 the symmetry-breaking field *grows* every iteration, and below 0.5 it is gone before the density has responded to it.

Refused. `constrained_magnetization = 'fsm'` together with `lspinorb = .true.`. The fixed-spin-moment search measures $\chi = \delta m_a / \delta B_a$ one cartesian component at a time and inverts it the same way, which models $\partial m_a / \partial B_b$ as *diagonal* — three independent one-dimensional searches. Spin-orbit coupling ties the moment to the lattice and is exactly what makes that untrue: pushing along x moves the moment along z as well. The scheme would still converge sometimes and find the wrong field the rest of the time, with nothing in the output to say which. Use a penalty scheme (`'total'`, `'atomic'`) to hold a moment under spin-orbit coupling.
Also refused: `noncolin = .true.` with `angle1`/`angle2` set and no `starting_magnetization`. The angles are the *direction* of a moment whose length is `starting_magnetization`, so with no

length there is no moment: `domag` goes false, `nspin_mag` collapses to 1, and what runs is an unpolarized calculation with spinor wavefunctions — converged, successful, and not the calculation the angles asked for. `pw.x` runs that input silently. A spinor run with nothing magnetic in it stays accepted, because that is an ordinary calculation; the refusal is about a *stated* direction with nothing to point.

The polar-angle constraint is guarded on the axis. `'total direction'` used to return a NaN potential in all three components whenever the moment lay along z — which is where a run seeded from `starting_magnetization` with no angles starts. $\arccos'$ diverges at the poles while the transverse moment's own derivative vanishes there, so the chain is $0 \times \infty$; and at a *round-off* transverse moment $m_z/|m|$ rounds to bit-exactly 1 and the clamp halves the tangent instead. The angle is written $\mathrm{atan2}(|m_\perp|, m_z)$ with both arguments masked at the value the derivative is taken at, which reproduces QE's `fact1` to 1e-12 off the axis and QE's zeroed transverse factors on it, plus its literal 10^{-14} escape field along x — without which the moment sits at an unstable stationary point of its own penalty and never turns (`add_bfield.f90:184-192`).

Also refused, as `pw.x` refuses them: `'total direction'` at `nspin = 2`, since the total moment of a collinear run has one component and its polar angle is 0 or $180°$, meaning that the penalty is a constant with no gradient (`add_bfield.f90:177-180`); and `tot_magnetization` at any `nspin` other than 2, which is `pw.x`'s own "tot_magnetization requires nspin=2" (`input.f90:778-782`). Both used to run: the first reported a constant constraint energy over an unconstrained state, and the second was stored and read by nothing, so an unpolarized or noncollinear run converged with a moment different from the one asked for. A noncollinear run holds a total moment with `constrained_magnetization = 'total'` instead.

Restarting an SCF from the middle (`checkpoint_dir`, `checkpoint_every`, `max_seconds`). A job killed at its wall clock used to lose every iteration it had run: a state reached disk only after `run_scf` returned. It now writes on a cadence and *resumes from the same directory*, so a resubmitted batch script is the same command and continues.

```
from defuqumat import Calculator
calculator = Calculator.from_file("scf.in", checkpoint_dir="ckpt")
scf = calculator.get_scf()          # run again after a kill: it continues
```

A restart is three things, and the third is the one with no obvious home: the state (density, wavefunctions, `becsum`, `ns`, `tau`); the *mixer's history*, without which Anderson restarts empty and pays back the iterations the checkpoint saved; and the *loop state* — the iteration number, `dr2` and the diagonalisation threshold `ethr`. That last one matters because QE's threshold schedule is indexed on the iteration number, so a resume re-entering at iteration 1 with a fresh threshold converges on a different schedule than the one it left. With all three carried the restart is exact: on silicon at `conv_thr = 1e-12`, stopping and resuming costs the *same* total iteration count as an uninterrupted run at three different mixing parameters (17/17, 13/13, 11/11), with energies agreeing to 2×10^{-15} Ry. Both Quantum ESPRESSO and Elk restart this way, and carry the same three parts.

`max_seconds` lets the loop stop *itself* before the scheduler does, writing a checkpoint on the way out, so a wall-clock kill lands after a checkpoint rather than between two. It is checked between iterations, so set it at least one iteration short of the real limit.

A derived calculator reads the checkpoint rather than the state it was derived from. `with_spin`, `with_positions` and their relatives hand the parent's converged state to the first SCF as a starting point, and `run_scf` looks in its checkpoint directory only when nothing was handed to it — so a promoted run killed at its wall clock and resubmitted used to start from the seed every time and never read the file it had been writing. The checkpoint wins,

for the reason it wins over a seed the caller passed: it is strictly later state. Measured on fcc aluminium, a non-magnetic ground state converged at `1e-8` and promoted into a two-channel run at the input's `conv_thr = 1e-12`: the first line takes two iterations, and the resubmitted one re-enters at iteration 2 carrying the checkpoint's own `ethr = 1e-13` and finishes in one, at the same -4.1854697253 Ry. Before, every resubmission repeated both iterations from the seed, starting at `ethr = 1e-2` and paying the re-diagonalisation the schedule then asks for.

```
calculator = Calculator.from_file("al-metal.in")
calculator.get_scf()                          # the non-magnetic ground state
spinor = calculator.with_spin(2)              # carries it as a seed
scf = spinor.get_scf(checkpoint_dir="ckpt")   # resubmit after a kill: continues
```

The seed and the `magnetization` beside it are withheld together, which is not bookkeeping: a resume refuses a `magnetization` argument by name, there being nothing left for it to decide, and `with_moments` sets one by default — so dropping only the state would turn a resubmitted texture run from one that silently restarted into one that raises.

A field is carried, and only one kind of field is refused. An applied `LOCAL_MAGNETIC_FIELDS` card, and a penalty constraint of any flavour, leave the field exactly as the input built it, so the resume rebuilds it from `scf.in` with the rest of the calculator. Elk's `reducebf` fades a *scale* rather than the field, and that scale is saved and restored — a run whose field had decayed over forty iterations comes back where it left, not at full field.

Refused. Checkpointing is switched off, once and with a message, for a run whose field is *driven* rather than applied: the fixed-spin-moment schemes (`constrained_magnetization = 'fsm'`, `'atomic fsm'`, `'atomic fsm direction'`), where the field is the controller's own state and is replaced after every iteration. No input file says what it reached, so a resume would start again from the input field and converge somewhere else without saying so. `checkpoint_every` must be at least 1.
A DFT+U state used to be refused here as well, on the grounds that `ns` without its manifold is an array of numbers about nothing. That is true of the file and it is not what a resume reads it against: a state is loaded against a system, and the manifold is rebuilt from the `HUBBARD` card before `ns` is looked at. `load_state` restores it from the `calculation` when one is given, and leaves it `None` when only a system is, where what is missing is the label `hubbard_occupations` reads rather than any part of the state.
Know the cost. The payload is the wavefunctions, which is tens of gigabytes on a large cell, so this is a scratch-directory operation and the cadence is a knob rather than every iteration. It has not been timed on a large cell; the default of 10 is a guess until it is.

A starting magnetic texture (`STARTING_MOMENTS` card): one starting moment per *atom*, cartesian, in Bohr magnetons, one line per atom in the order of `ATOMIC_POSITIONS`. *Bohr magnetons* is meant literally: a row of `0.0 0.0 2.0` seeds a $2\,\mu_B$ moment on that atom whatever its valence charge, which is the same number the `atomic` constraint aims at and the same one the per-site report gives back. A row asking for more moment than the atom has valence electrons is clamped to it, with a warning — a superposition of atomic charges cannot express more, and the alternative is a negative spin channel. `starting_magnetization` in the namelist follows Quantum ESPRESSO: below 1 it is a *fraction* of the valence charge, and at or above 1 it is Bohr magnetons and every species is divided by its own valence charge (`input.f90:1448`). QE's `starting_magnetization`, `angle1` and `angle2` are per *species*, so a helix, a cycloid or a skyrmion cannot be stated in a `pw.x` input at all.

It decides three things, and the first is why it matters more than a starting guess usually does. It decides the **symmetry group**; it decides whether the run is **magnetic at all** (a

noncollinear input whose texture is given only through this card is a magnetic run, with a four-component density — this used to be read off the per-species `starting_magnetization` alone, so such an input converged, reported a total energy and never had a magnetization); and it seeds the **starting density**, one moment per atom rather than one per species. The magnetic group is filtered by the per-atom moments $\mathbf{m}_i$ — an operation survives only if it maps that axial vector field onto itself, possibly followed by time reversal — and it is built from the input, once, never from the converged state. Built from per-species values it is a *ferromagnet's* group, which keeps operations a texture does not have, and `sym_rho` then averages the texture away a few iterations in while the charge converges and `dr2` reports nothing.

A per-atom *applied* field is filtered with the moments, together, under one choice of time reversal, exactly as Elk's `findsym.f90` tests `bfcmt0`. So a `LOCAL_MAGNETIC_FIELDS` card that asks for a texture now cuts the group by itself and needs no card of its own.

```
STARTING_MOMENTS
   0.00000000   1.50000000   0.00000000
  −0.75000000   1.29903811   0.00000000
  −1.29903811   0.75000000   0.00000000
```

A texture from Python, without writing a file (`Calculator.with_moments`, `System.with_moments`). The card above is the only way to *state* a texture in an input, and a sweep over magnetic configurations — a 120-degree Néel state against a collinear one, a cone at five angles — wants to state several from one script. `with_moments` takes an `(nat, 3)` array of Bohr magnetons in `ATOMIC_POSITIONS` order, or `None` to drop the card, and returns a new calculator whose k-points have been rebuilt for the new moments' magnetic group.

```
import numpy as np
from defuqumat import Calculator

ferro = Calculator.from_file("h4−chain−ferro.in")  # tests/data/qe
angles = np.deg2rad([0.0, 90.0, 180.0, 270.0])
cycloid = 0.6 * np.stack([np.sin(angles), np.zeros(4), np.cos(angles)], axis=1)

print(ferro.system.symmetry_group().nsym, ferro.system.kpoints.nk)   # 16 9
textured = ferro.with_moments(cycloid)
print(textured.system.symmetry_group().nsym, textured.system.kpoints.nk)  # 4 12
```

The obvious workaround is silently wrong, which is why this is a method rather than an attribute. `dataclasses.replace(system, starting_moments=...)` swaps the field and leaves `kpoints` reduced with the group the *old* moments had, while `symmetry_group()` is a property recomputed from the new ones — 9 k-points reduced with 16 operations, symmetrised with 4, measured on the cell above. On a system built nonmagnetic it also flips `domag`, so the k-set carries a `time_reversal` a magnetic run must not have. Nothing compares the two.

Holding a texture (`constrained_magnetization = 'atomic texture'`) constrains the *full* unit vector of every atom's moment,

$$E_{\mathrm{con}} = \lambda \sum_i \left(1 - \frac{\mathbf{m}_i \cdot \hat{\mathbf{n}}_i}{|\mathbf{m}_i|} \right),$$

with $\hat{\mathbf{n}}_i$ the normalised `STARTING_MOMENTS` rows. It is not one of QE's schemes and has no `i_cons` number. QE's nearest, `'atomic direction'` (`i_cons = 2`), constrains $m_z/|m|$ — the polar angle *alone* — so it is exactly satisfied by every texture lying in a plane that contains z, a helix and a collinear state alike, at zero penalty: it cannot hold one, and reports success while failing to.

Or hold it with a field instead of a penalty. A penalty is a term added to the energy, so at convergence it is still pushing — the residual angle *is* that push, and it cannot be made zero by choosing λ better. Elk's fixed-spin-moment scheme does the other thing: it drives an external field until the moment sits where it was asked to, and the converged state is then a stationary point of the *unconstrained* functional under that field. `'atomic fsm'` and `'atomic fsm direction'` are that scheme resolved by atom (Elk's `fsmtype = 2` and -2),

$$\mathbf{B}_i \leftarrow \mathbf{B}_i - \tau\,(\mathbf{m}_i - \mathbf{m}_{\mathrm{fix},i}),$$

with the targets read from `STARTING_MOMENTS` in Bohr magnetons. The direction-only variant finishes by projecting $\mathbf{B}_i$ perpendicular to $\mathbf{m}_{\mathrm{fix},i}$ (Elk's `r3vo`), so the field can *turn* a moment and can never lengthen or shorten one — and note what that buys: it has no $1/|\mathbf{m}|$ anywhere in it, where the direction-only *penalty* above has one in its gradient and runs away.

On a robust magnet it holds a texture exactly, and it needs Elk's reading, Elk's mixer and Elk's gain to do it. Two iron moments at 90 degrees without spin-orbit coupling (`tests/data/qe/fe2-canted-nosoc.in`, bcc iron in its conventional cell), where exchange wants the pair parallel and holding it costs a real torque:

```
from defuqumat import Calculator
text = open("tests/data/qe/fe2-canted-nosoc.in").read().replace(
    "    nosym = .true.\n",
    "    nosym = .true.\n    constrained_magnetization = 'atomic fsm'\n"
    "    lambda = 0.02\n")
text += "STARTING_MOMENTS\n  1.868 0 0\n  0 1.868 0\n"
calc = Calculator.from_text(text, "tests/data/pseudo")
scf = calc.get_scf(mixing_mode="adaptive", mixing_beta=0.05)
scf.site_moments          # (1.868, 0, 0) and (0, 1.868, 0)
scf.magnetic_field.atomic # the field that holds them, 0.026 Ry per atom
```

It converges in 46 iterations with the pair at 90.002° and both lengths on the target to $10^{-4}\,\mu_B$. The vector penalty on the same cell holds 89.2° (0.39° per site) in 76 at $\lambda = 1$, the largest it tolerates, and Elk itself holds the direction-only variant in 55 loops under 0.0097 Ha per atom, which is 0.019 Ry in this code's units. What a penalty cannot give and this does is the field: $\mathbf{B}_i$ is the torque the texture costs, perpendicular to each moment to within a quarter, and reading it is reading how far the state is from being stationary on its own.

Three choices make it work and each was measured to be necessary. The moment the update reads is the iteration's *output* density, as Elk reads its `mommt`; reading the mixed one instead lets the field climb to 0.25 Ry while the moment it is turning does not move. The mixer carries no history, `mixing_mode = 'adaptive'` at `mixing_beta = 0.05`, which is Elk's `mixadapt` at Elk's `beta0` and is what Elk ran: the field changes the calculation's fixed point every iteration, and Anderson extrapolates over maps that no longer exist, which on this cell runs away at every gain from 0.0025 to 0.04. And the gain is Elk's, below.

The earlier verdict on this scheme, measured on the two-hydrogen cell below, was that it does not converge: a ring with a growing envelope, and a secant that converged to the wrong state. That cell carries $0.000235\,\mu_B$ unconstrained, so its $m(B)$ is nearly a step and it is the wrong cell for any controller; what was wrong with the scheme was the first two choices above. The `secant` update steps only when the inner calculation has converged, and on the iron pair that gate is its failure: under a held field the inner run plateaus at 5×10^{-5}, the field freezes, and 800 iterations end where 200 did. The default update is therefore `'elk'`.

`lambda` means two different things three orders of magnitude apart, and this is the trap of the family. For a *penalty* it is a stiffness, and 1 to 10 is the useful range. For a *feedback* scheme it is Elk's `taufsm`, a gain that multiplies a moment error in Bohr magnetons to give a field in Rydberg. Elk's own default is 0.01 in *Hartree* per Bohr magneton, added to the Kohn-Sham field directly, which is **0.02** here, and 0.02 is the default: at 0.01 the iron pair is still moving at $0.8°$ per site after 300 iterations, and at 0.02 and 0.04 it converges in 63 and 64. Write a penalty-sized value and the run warns by name: at $\tau = 0.2$ the vector scheme on the hydrogen cell drives the moments to $0.82\,\mu_B$ against a target of 0.26.

Which penalty holds a texture, measured. *Not this one.* `'atomic texture'` constrains a direction and not a length, so its potential carries a $1/|\mathbf{m}|$ and *grows* as a site's moment shrinks — positive feedback, and on the two-sublattice cell below no λ converged. `'atomic'` constrains the moment as a vector, so its potential is $2\lambda(\mathbf{m} - \mathbf{m}_{\text{target}})$ and bounded, and it is what to reach for. Two hydrogen atoms of one species asked to sit 120 degrees apart (`tests/data/qe/h2-texture-120.in`), `conv_thr = 1e-8`, targets at the converged sphere moment of $0.26\,\mu_B$:

scheme	λ	converged	iterations	angle	error/site
(none)	—	**yes**	10	$180.00°$	$30.00°$
`atomic`	0.05	yes	10	$170.95°$	$25.47°$
`atomic`	0.2	yes	7	$153.52°$	$16.76°$
`atomic`	1.0	yes	9	$130.30°$	$5.15°$
`atomic`	3.0	yes	17	$123.65°$	$1.83°$
`atomic`	10.0	yes	38	$\mathbf{121.13}°$	$\mathbf{0.55}°$
`atomic`	30.0	**no**	200	$11.91°$	$64.41°$
`atomic texture`	0.1	**no**	200	$118.39°$	$4.30°$
`atomic texture`	0.02	**no**	400	$140.03°$	$10.02°$
`atomic texture`	≥ 2	**no**	200	collapsed	—

Read the first row and the sixth together: left alone the 120-degree state relaxes to the collinear antiferromagnet in ten iterations and *reports success*, and at the largest λ the SCF tolerates the vector penalty holds it to **0.55 degrees per site** in 38. So a magnetic structure that is not the ground state can be converged here, which is what a user mapping out $E(\theta)$ needs and what the cone and spiral collapses recorded elsewhere in this document have no constraint applied to them. The recipe is `'atomic'`, a `STARTING_MOMENTS` card scaled to the moment you *expect* rather than the moment you seed with, and λ raised until the angle stops moving or the SCF stops converging — which on this cell are within a factor of three of each other. `'atomic texture'` warns and names this table.

How far each atom is from its target is on `SCFResult.site_residuals`, one number per atom — degrees for the two direction schemes, Bohr magnetons for `'atomic'`. `constraint_energy` is a single scalar over every site, so under `'atomic texture'` one flipped site out of fifteen reads as a small number indistinguishable from partial convergence everywhere; the per-site array is the only form of the comparison that can tell those apart.

Refused. `'atomic texture'` needs `noncolin = .true.` (a collinear moment has no direction beyond its sign) and a `STARTING_MOMENTS` card with no zero row (a zero vector carries no direction). `STARTING_MOMENTS` itself needs `noncolin` for any x or y component, the same rule `B_field` and `LOCAL_MAGNETIC_FIELDS` are held to, and one row per atom.
Checked, not remembered: the card above parses through `build_system`, reaches `system.starting_moments` unchanged, becomes `m_loc` verbatim, and `constraint_targets` re-

turns it normalised — three unit rows. The starting density built from it carries opposite moment lobes on two antiparallel sites and integrates to zero over the cell.
It reaches all three consumers, which it did not until recently: the charge density, the Hubbard occupation matrix, and a PAW or ultrasoft dataset's atomic `becsum`. The last two used to be split by the per-*species* `starting_magnetization` and so started every site of a species pointing the same way, while the charge carried the texture — so iteration 1 contradicted itself on exactly the systems the card exists for. For PAW that is not merely a poorer guess: the one-centre terms are a *function* of `becsum`, so the two sites got different Hamiltonians than the density asked for. And nothing in the SCF turns a moment, so a spinor DFT+U run — which is forced to `nosym`, leaving the starting matrix as the only steering there is — could converge with the shells polarised along the wrong axes and report success.
Warned about, with a number: a per-atom feedback scheme (`'atomic fsm'`, `'atomic fsm direction'`) under the `'elk'` update with any mixer but `'adaptive'`. The run is legal and will not converge: on the canted iron pair Anderson runs away at every gain tried, where the adaptive mixer holds the pair to $0.001°$. The warning is raised before the first iteration and names the mixer and `beta0` to use.
Warned about, with a number: a textured starting *density* handed to `run_scf` by the caller. The symmetry group is decided from the input and never from the density, so a texture that reaches the run only that way is invisible to the filter and *will* be symmetrised away. `run_scf` measures $\|\rho - \mathrm{sym}(\rho)\|/\|\rho\|$ on the seed before iteration 1 and warns above 10^{-3}, reporting the charge and the magnetization apart. It is not a small effect and it is not a rounding: a four-atom 90-degree cycloid handed to the same cell's *ferromagnetic* group has its magnetization residual come out at **1.0** — averaging four directions 90 degrees apart over a group that permutes the four sites gives exactly zero, and every site moment goes from $0.27\,\mu_B$ to 10^{-19} — while its charge residual is 5.7×10^{-16}, so the charge notices nothing. From a `STARTING_MOMENTS` card the same figure is 6×10^{-16}. Give the texture as a card, or set `nosym = .true.`

The moment on each atom (`SCFResult.site_moments`, `.site_charges`). The two magnetic numbers a run has always printed are integrals over the *cell*, and both are zero for every compensated state — for an antiferromagnet, for a spiral, for a cycloid, and equally for the nonmagnetic state any of those can collapse into. What separates them is the charge and the moment integrated in a sphere around each atom, which is what Quantum ESPRESSO's `report_mag` prints and what Elk computes as `mommt`:

$$q_a = \int w_a(\mathbf{r})\, n(\mathbf{r})\, d^3r, \qquad \mathbf{m}_a = \int w_a(\mathbf{r})\, \mathbf{m}(\mathbf{r})\, d^3r,$$

with w_a one inside a radius r_m around atom a, falling linearly to zero at $1.2\,r_m$, and every grid point belonging to at most one atom.

```
from defuqumat import Calculator

calculator = Calculator.from_file("h2-mirror-afm.in")
scf = calculator.get_scf()

print(scf.magnetization)           # -2.3e-06  -- the cell total, zero
print(scf.absolute_magnetization) #  0.806933
for a, (q, m) in enumerate(zip(scf.site_charges, scf.site_moments)):
    print(f"atom {a}: charge {q:.4f}  moment {m[0]:+.4f} mu_B")
# atom 0: charge 0.6240  moment +0.2909 mu_B
# atom 1: charge 0.6240  moment -0.2909 mu_B
```

They are recorded at every iteration, in `scf.history`, and not only at convergence. That is the point rather than a convenience: a texture that is going to be lost goes early and the run then converges cleanly, so the converged value is exactly the one that cannot show it. The per-iteration console line carries `|m|_site = min..max`, the shortest pair that tells a live compensated state from a dead one, and the full block is printed at the end with θ and ϕ per atom — angles Quantum ESPRESSO does not print and that nobody should be reconstructing by hand from three components.

A collinear run has a magnetic symmetry group too. Until recently it did not: the filter was built only for `noncolin`, so a compensated magnet stated on *one* species kept the operation carrying one sublattice onto the other, and the density symmetriser then averaged $n_\uparrow$ over both sites — which is the statement that the staggered moment is zero. This is not an exotic case: an *altermagnet* is by definition compensated order whose sublattices are related by a rotation, so the swap is a point-group operation of the chemical structure. On two hydrogen atoms related by a mirror with ± 0.6 from the card, the moments were gone at iteration 1 and the run converged to a perfectly good total energy **6.6 meV** above the antiferromagnet, reporting an absolute magnetization of 7×10^{-6}. With the filter the group is cut from 16 operations to the 8 that preserve the moment pattern, and the run agrees with the `nosym = .true.` spelling — the only one that used to work — to 10^{-8} Ry and $10^{-6}\,\mu_B$ in the same iteration count.

The collinear filter compares a *number* per site, not a vector. Without spin–orbit coupling a spatial operation does not turn the spin, so the moment is permuted and never rotated; filtering it as an axial vector instead would cut a C_{2x} or a mirror on a z-ferromagnet and leave a group that is too *small*.

Refused. An operation that reverses *every* moment is a symmetry of the crystal followed by time reversal. Those are **dropped** rather than used, which is Quantum ESPRESSO's own `colin_mag = 1` behaviour: using one means swapping the two spin channels while symmetrising, and the density symmetriser here does not. The result is a subgroup of the true magnetic group — correct, and paid for in k-points rather than in physics.
Where the site moments are not reported: the response stack. All three relaxation drivers carry them now — `RelaxStep`, `VCRelaxStep` and `SpiralRelaxStep` each hold `site_charges` and `site_moments` at their own step, which is the record a *relaxation* needs for the same reason the SCF needs one per iteration: everything else a step reports is zero-sum over the sites, so a compensated magnet that lost its moments between two geometries reads exactly like one that kept them.
The spheres are a choice, not a definition. A plane-wave basis has no muffin tins, so "the moment on this atom" is whatever region you integrate over; r_m defaults to a little under half the nearest-neighbour distance so the tapered spheres are disjoint. The site charges therefore sum to *less* than the cell's charge — the interstitial belongs to nobody — and the site moments sum to less than the cell moment. Compare them against another code only when both used the same radius.

DFT+U: both rotationally-invariant functionals, on `atomic`, `ortho-atomic` and `norm-atomic` projectors, read from the `HUBBARD` card. Forces come from differentiating through projectors that move with the atoms.

The card selects which functional runs, exactly as `pw.x` does it — the namelist's `lda_plus_u_kind` is obsolete there and warned about. A card carrying only U, J_0, `ALPHA` and `BETA` is Dudarev's simplified functional; any J, B, E_2 or E_3 selects Liechtenstein's full one, whose interaction

is the whole Coulomb matrix

$$\mathrm{vee}(m_1m_2m_3m_4) = \sum_k F^k \frac{4\pi}{2k+1} \sum_{q=-k}^{k} a_{kq}(m_1m_3)\, a_{kq}(m_2m_4)$$

rather than one number, with the Slater integrals F^k fixed by U and J. Against `pw.x` on antiferromagnetic FeO: **4.2e-9 Ry** at $J = 1$ eV. *Forces work for both*, and for the full functional `pw.x` does not have them at all (`force_hub.f90` stops on "forces in the DFT+U+J scheme are not implemented"); a central difference of the SCF energy agrees to 1.3e-4 Ry/bohr. **Seed that difference from the undisplaced run** — with a full interaction matrix the DFT+U landscape has more than one solution, and cold-started displacements land on a different one, which does not look like an error, it looks like a force.

```
from defuqumat import Calculator
scf = Calculator.from_file("feo.in").get_scf()
print(scf.energy_terms["hubbard"])
```

```
HUBBARD {atomic}
U Fe1-3d 4.3
J Fe1-3d 1.0      # selects the full functional
```

Around-mean-field double counting (`hubbard_double_counting = 'amf'`) is the alternative to the fully-localised limit that both QE's functionals assume, and `pw.x` has neither the option nor the code. FLL treats the occupations as integers, which is right for a localised magnetic insulator and wrong for a metal; AMF subtracts the shell's mean occupation before the interaction, so a uniformly filled shell is corrected by *exactly* nothing — 0 to 10^{-14} at any filling, where FLL is not. On FeO its Hubbard energy is -0.005 Ry against FLL's $+0.232$. It needs the full functional, there being no interaction matrix in the simplified one to apply the shift to.

Slater integrals from the orbital (`hubbard_slater = 'yukawa'`) computes the interaction instead of parameterising it: F^k is the double radial integral of the manifold's own all-electron partial wave against a screened Coulomb kernel,

$$F^k = \iint \mathrm{d}r\, \mathrm{d}r'\, [r\phi(r)]^2 [r'\phi(r')]^2\, (2k+1)\, \lambda\, i_k(\lambda r_<)\, \tilde{k}_k(\lambda r_>),$$

which becomes the bare $r_<^k / r_>^{k+1}$ as $\lambda \to 0$. Give λ on the card (`LAMBDA Ni-3d 2.1`) or give a U and let the screening length be solved for — either way F^0, F^2, F^4 and J all come out of the orbital. Nickel's unscreened 3d gives $F^4/F^2 = 0.626$ against the 0.625 that QE and Elk both hardcode, and $J = 1.28$ eV.

```
&system
   hubbard_slater = 'yukawa'
/
HUBBARD {ortho-atomic}
U Ni-3d 5.0       # lambda is solved for; F2, F4 and J follow
```

On a spinor (`noncolin`, with or without `lspinorb`) the occupation matrix is one 2×2 matrix in spin space rather than two real matrices, measured on projector columns that are themselves spinors, and its off-diagonal blocks are what let the shell's moment point anywhere. Both functionals work there. Against `pw.x`: **7.7e-9 Ry** on relativistic BN and **1.4e-9 Ry** on fully-relativistic iron carrying $U = 2.20$, $J = 1.75$ eV with its moment turned into the xy plane, with $\mathrm{Tr}[n^{\sigma\sigma}]$ matching every digit `write_ns` prints.

Tensor moments put that matrix in a basis where each element is a multipole rather than a number: an orthonormal, complete set Γ_t^{kpr} with k the rank in the orbital index, p the rank in spin and r their coupling, so that

$$n = \sum_{kprt} w_t^{kpr}\, \Gamma_t^{kpr},$$

with real w. The charge is w^{000}, the spin moment is w^{011}, and $\mathbf{L}\cdot\mathbf{S}$ is w_0^{110} alone — to 10^{-11} over all one hundred moments of a d shell.

Fixing one (a `TENSOR_MOMENTS` card) converges the field to a state carrying it, which selects an orbital or multipolar ordering the field would not find on its own. The penalty is written down and its potential is `jax.grad` of it, and, like a constrained moment's, its energy is not part of the reported total — so the number that comes back is the unconstrained functional at the constrained state.

```
&system
    tensor_moment_penalty = 20.0
/
TENSOR_MOMENTS
 N 2p 1 1 0 0 -0.40        # hold L.S at -0.40
```

Being quadratic, the penalty settles where its gradient balances the material's own restoring force, so the moment *approaches* its target as the penalty stiffens rather than reaching it: on nitrogen's $2p$, whose free $\mathbf{L}\cdot\mathbf{S}$ is -0.0006, holding it at -0.40 gives -0.109, -0.263, -0.355 and -0.388 at $\lambda = 1, 5, 20, 80$, with the energy rising monotonically by 2.6, 15.3, 27.4 and 32.6 mRy.

Refuses. A *collinear* DFT+U run needs `nosym` when its magnetic group carries time-reversed operations: there the occupation matrix is two separate channels and such an operation exchanges them, which is not written. **A spinor run does not** — it keeps both spin indices, so the operation acts on the matrix itself, and both halves of that action are in: the $SU(2)$ representation of each operation beside the rotation of the m indices, and, for a time-reversed one, the transpose that goes with an antiunitary operation. A fixed tensor moment needs a spinor: for a collinear run the spin-rank moments keep only their z component and the constraint degenerates into the fixed spin moment `constrained_magnetization` already is. `hubbard_slater = 'yukawa'` needs a **PAW** dataset, and this is a measurement rather than a preference: only PAW carries the all-electron partial wave. A norm-conserving χ is normalised and gives a pseudo-orbital F^0 far too small (silicon's 3p: 8.8 eV), and an *ultrasoft* χ is not even normalised — iron's 3d has $\int(r\chi)^2 = 0.41$, the rest of the norm living in the augmentation charge, and its F^0 comes out at 2.1 eV where the answer is above twenty. The `norm` that comes back with each manifold is what says whether it is bound inside the cutoff, and the cutoff defaults to the dataset's own augmentation radius because a partial wave *diverges* outside it. Elk's `inpdftu = 2` and `3` (Slater and Racah *input*) have no card syntax: they are a change of coordinates on the three numbers (U, J, B) already spans, and QE's card has spent the names E_2 and E_3 already. The FLL/AMF interpolation (Elk's `dftu = 3`) is not here because it is not in Elk either any more — documented in the 11.0.2 manual, removed from the 11.0.2 source.

Refuses. The intersite V, background channels, the orbital-resolved variant, the `wf`/`pseudo` projector sets, and noncollinear `ns_nc`; and PAW + GGA + a noncollinear magnetization together.
The **residual solver** cannot take a fixed occupation whose filled/empty boundary cuts

a **degenerate multiplet** — which is exactly what an atom's Hund's-rule filling does. Which member of the multiplet the eigensolver returns is arbitrary, so the density map is not a function of the density and a Newton step has nothing to converge on; it is diagnosed by name rather than left as a `NaN`. The mixer damps its way past it, and a *gapped* fixed-occupation LSDA cell agrees between the two solvers to 5e-12 Ry.

5.1 The source-free exchange-correlation field, and the torque

Let us ask what turns a moment. The exchange-correlation field of a local functional is built pointwise: at each point the density is resolved onto the direction of $\mathbf{m}(\mathbf{r})$, the collinear expression is evaluated there, and the splitting is attached back to $\hat{\mathbf{m}}$. So $\mathbf{B}_{xc}(\mathbf{r}) \parallel \mathbf{m}(\mathbf{r})$ everywhere, and the torque the field exerts,

$$\boldsymbol{\tau} = \int \mathbf{m}(\mathbf{r}) \times \mathbf{B}_{xc}(\mathbf{r})\, \mathrm{d}^3 r,$$

is **identically zero at every density**, converged or not, constrained or not. A functional of $|\mathbf{m}|$ alone cannot turn a moment, which is a statement about the approximation rather than about this implementation, and it means a test asserting that this quantity vanishes at self-consistency passes on a broken code exactly as it passes on a working one.

A magnetic field in nature has no sources. `nosource` makes the exchange-correlation one obey that, by projecting out its longitudinal part — Elk's `src/projsbf.f90`, which takes the divergence, solves a Poisson equation for it and adds the gradient of the solution back, and which on a plane-wave grid is one line in reciprocal space, $\mathbf{B}(\mathbf{G}) \to \mathbf{B}(\mathbf{G}) - \hat{\mathbf{G}}\,(\hat{\mathbf{G}}\cdot\mathbf{B}(\mathbf{G}))$, with $\mathbf{G} = 0$ left alone since a Poisson equation cannot move a constant. The projected field is no longer parallel to $\mathbf{m}$, so the torque stops being zero.

```python
from defuqumat import Calculator

# fcc nickel, noncollinear, LDA, with nosource = .true. in &system
calc = Calculator.from_file("ni-noncol-111-nosource.in", pseudo_dir="../pseudo")
torque = calc.get_exchange_torque()

print(torque.parallel_fraction)   # -> 0.982: B_xc is no longer along m
print(torque.sites.shape)         # -> (1, 3): one atom, one torque vector
```

`parallel_fraction` is $\int |\mathbf{m}\cdot\mathbf{B}_{xc}| / \int |\mathbf{m}||\mathbf{B}_{xc}|$, and it is reported beside the torque so that a zero can be told from a silence: one means the field is everywhere along the magnetization and the zero is the approximation speaking. Without `nosource` it is 1 to round-off.

The numbers. On the cell above, $\int |\nabla \cdot \mathbf{B}_{xc}|^2$ falls from 1.5×10^{-2} to 1.1×10^{-32}, and the moment goes from **0.6576 to 0.7045** μ_B — the size of change the source-free literature reports for a transition metal. On the four-hydrogen helix the same flag takes the site moments from 0.4675 to **0.0003** μ_B, and that collapse is the cell rather than the method: the projection removes roughly the radial part of each atom's field, and a Stoner-marginal magnet crosses its own criterion when it does.

And the torque fires. Turning one site's moment inside its own sphere, away from where it sat, on the same nickel cell:

turned by	local functional	source-free field
0°	7e-22 Ry	2e-18 Ry
15°	4e-20 Ry	**1.64e-4 Ry**
45°	5e-20 Ry	**4.47e-4 Ry**
90°	1e-21 Ry	**6.32e-4 Ry**

The left column is the point of the table: the local functional reports zero at *every* angle, not only at the one it converged to.

Note that the *total* torque over the cell is zero for a second reason, and that one is a symmetry rather than an approximation: without spin–orbit coupling the energy does not change under a global rotation of every spin, so the net torque on the cell cannot either. What to read is `torque.sites`; the total is a check on the assembly.

Refuses. Everything that differentiates the total energy — **forces, stress, phonons, the elastic constants and the whole linear response** — because the projection changes the potential and leaves the energy expression alone, exactly as a potential-only meta-GGA does, so the converged state is stationary for a functional nothing writes down. `run_scf` warns for the same reason: the total energy a `nosource` run reports is not the value of anything it minimised and is not comparable with a run without it. **Magnons**, and this is the sharpest one: the transverse kernel $f_{xc}^{+-} = B_{xc}/m$ is a scalar only because the local functional makes $\mathbf{B}_{xc}$ parallel to $\mathbf{m}$, which is precisely what the projection breaks — the quotient is still finite and the magnon it gives is meaningless. A **collinear** run, since a transverse projection gives a field along z components along x and y that an (n, m_z) density has nowhere to carry (Elk's `init0.f90` requires `ncmag` for the same reason). **PAW**, whose one-centre $\mathbf{B}_{xc}$ on the spheres is a second copy of the field that this projection does not reach, where Elk projects its muffin-tin part together with the interstitial one. A **spin spiral**, since $\nabla \cdot \mathbf{B}$ is a laboratory-frame statement and the magnetization is stored in the rotating frame. A **potential-only meta-GGA**, because stacking two potentials with no energy behind them leaves nothing at all that says what the run is a stationary point of. And the **ultracell**, whose per-cell potential stops being a function of that cell's density once a division by $|\mathbf{G}|^2$ couples the whole modulation.
Elk's other half is **not built**: `sxcscf` scales the spin part of the field by a constant, which is the same paper's second knob and a different approximation. Elk labels `nosource` an experimental feature and that label carries here.

5.2 A gradient correction built from ∇m

What does a gradient-corrected functional do with a magnetization that points in different directions at different points? `pw.x`'s answer, which is Kübler's and is `noncollinear_gga = 'kubler'` here, is to rotate onto the local axis: the collinear GGA is evaluated on the pair $(n \pm |\mathbf{m}|)/2$. Where every starting moment is parallel it takes the sign of $\mathbf{m}$ along that axis, so up stays up across a node; where they are not, which is every spin spiral, it takes $|\mathbf{m}|$ itself, which has a kink at every node of $\mathbf{m}$. A kink in the density is a divergence in its gradient, and the self-consistent cycle sees it: the PBE cobalt helix needs 119 iterations to reach 10^{-11} Ry, and `run_scf`'s own defaults do not get there in 150.

`noncollinear_gga = 'grad_m'`, the default since 2026-10-11 for every noncollinear magnetic run with a gradient or meta functional, builds exchange from the gradients of the components instead. With $g = |\nabla n|^2$, $\Gamma = \sum_k |\nabla m_k|^2$, $a_k = \nabla n \cdot \nabla m_k$, $A = |\mathbf{a}|$, $t = \mathbf{a} \cdot \mathbf{m}$ and

$s = |\mathbf{m}|$,

$$E_x = \frac{1}{4} \sum_{\sigma,\tau=\pm1} \left(1 + \sigma\tau \, \frac{t}{sA}\right) e_x(n + \sigma s,\ g + \Gamma + 2\tau A)\,,$$

with e_x the unpolarized gradient exchange, meaning that where $\nabla\mathbf{m}$ is parallel to $\mathbf{m}$ the weight is ±1 and this is the collinear exchange exactly, nodes included, and elsewhere the two branches are mixed by the cosine between $\mathbf{a}$ and $\mathbf{m}$. These are Scalmani and Frisch's invariants with their sign of t replaced by that cosine, which is what makes the energy smooth where $\mathbf{m}$ vanishes; nothing in it refers to an axis. Correlation is unchanged, since it sees the polarization only through an even function. On a spiral the transverse gradient is the laboratory one, $i(\mathbf{G} - \mathbf{q})$ on the transverse pair's own sphere, so the functional is the one the unfolded supercell evaluates, and since that gradient carries $\mathbf{q}$ the exchange enters $\mathrm{d}E/\mathrm{d}\mathbf{q}$ (`get_spiral_relaxation`) as well.

The same energy is evaluated on the **PAW** spheres, from the gradients of the four components' multipoles, with no rotation and no spiral term: the spheres' magnetization carries the spiral's phase only at the atom, and with the covariant term the quarter turn missed its supercell by 7.7×10^{-5} Ry, against 6.7×10^{-11} without it. A **potential-only meta-GGA** (TB09, BJ06) has no energy to differentiate, so its potential is assembled from the collinear channel potential by parity: the sixteen components of $V(x_m, x_a, x_L, x_T)$ odd in a subset of its four signed arguments, put together with the cosines between $\mathbf{m}$, $\nabla n \cdot \nabla\mathbf{m}$, $\nabla^2\mathbf{m}$ and $\boldsymbol{\tau}$. It is the collinear potential exactly wherever the four vectors are parallel, nodes included, and refers to no axis.

```python
from defuqumat import Calculator

# the cobalt helix, PBE, ultrasoft, q = -b3/4, with noncollinear_gga = 'grad_m'
calc = Calculator.from_file("co-helix4-spiral-gradm.in", pseudo_dir="../pseudo",
                            conv_thr=1e-11)
scf = calc.get_scf()

print(scf.converged, scf.iterations)   # -> True 15
```

The numbers. On that cell at 10^{-11}, 15 iterations at 160 Ry and 19 at 80, where the default takes 119 at 160 Ry; the ferromagnet at $\mathbf{q} = 0$ in 14 against the signed default's 35, with the two energies 3×10^{-10} Ry apart, since on a collinear state they are one functional. On the spiral they are not: -74.2833326 against -74.2824789 Ry, which is the choice of functional and not an error of either. Against the four-cell supercell at one density the spiral's potential agrees to 1.4×10^{-13} Ry. On spin-orbit nickel under TB09 both functionals converge in 25 iterations, 6.8×10^{-5} Ry apart, and on a canted spin-orbit cell the total moves by 2.7×10^{-6} Ry (tetragonal cobalt) to 2.7×10^{-4} Ry (iron under DFT+U with the moment in the plane).

Refuses. `nosource`, whose projection was written for a field parallel to $\mathbf{m}$, so that run keeps `'kubler'`, as does an LDA, which has nothing for it to change; an explicit `'grad_m'` on either is refused. A spiral under a meta-GGA stays refused under either, since its kinetic energy density is. It is **not** `pw.x`'s or Elk's functional (Elk regularises the same $|\mathbf{m}|$), so a comparison with either on a canted or spiral state sets `noncollinear_gga = 'kubler'`; on a collinear state the two are one functional.

6 Forces, stress and relaxation

$$\mathbf{F}_I = -\left.\frac{\partial E}{\partial \boldsymbol{\tau}_I}\right|_{\psi \text{ fixed}}, \qquad \sigma_{ij} = -\left.\frac{1}{\Omega}\frac{\partial E}{\partial \varepsilon_{ij}}\right|_{\psi \text{ fixed}}$$

Both are one gradient. With ultrasoft the orthonormality constraint $\langle\psi_n|\hat{S}|\psi_m\rangle = \delta_{nm}$ is carried explicitly, so its multiplier term — QE's Pulay contribution — falls out of the same derivative rather than being added afterwards. A variable-cell relaxation minimises the **enthalpy**, which is why a relaxed crystal carries the applied pressure rather than having zero stress:

$$H = E + P\,\Omega, \qquad \frac{\partial H}{\partial h} = \Omega\,(P\,\mathbf{1} - \sigma)\,h^{-T}$$

```
from defuqumat.forces import compute_forces
from defuqumat.stress import compute_stress
from defuqumat.workflows.relax import run_relax
from defuqumat.workflows.vc_relax import run_vc_relax

forces = compute_forces(calculation, result)      # method=... for QE's own terms
print(forces.forces)                              # (nat, 3) in Ry/bohr

stress = compute_stress(calculation, result, terms=True)
print(stress.pressure)                            # kbar

relaxed = run_relax(system, pseudos, forc_conv_thr=1.0e-4)
cell = run_vc_relax(system, pseudos, press=500.0) # kbar
print(cell.pulay_error)   # the frozen-basis error, reported not hidden
```

QE's `force_lc`, `force_cc`, `force_ew`, `force_us`, `addusforce` and `force_corr` are transcribed too, behind the same registry — **as a cross-check, not as the implementation**. The two share no machinery, which is what makes disagreement informative; it is how the augmentation force's sign and a missing gradient correction were found.

For a noncollinear or spin-orbit run the same expression is evaluated on two-component spinors: the nonlocal term takes the pseudopotential's bare `dvan_so` (a complex 2×2 matrix in spin space) and the orthonormality constraint takes `qq_so`, and the state is one coefficient vector of length $2\,n_{\mathrm{pw}}$ rather than two. Nothing else in the functional changes.

```
from defuqumat import Calculator

calculator = Calculator.from_file("pt2-soc-force.in")   # noncolin, lspinorb
forces = calculator.get_forces()          # (nat, 3) Ry/bohr
stress = calculator.get_stress()          # (3, 3)   Ry/bohr^3
print(forces.max_force, stress.pressure_kbar)
```

Validated against `pw.x` with `tprnfor`/`tstress` on a four-atom noncollinear hydrogen chain (norm-conserving) to 8.9×10^{-7} Ry/bohr, doubled fcc platinum with spin-orbit coupling to 7.5×10^{-6} (ultrasoft) and 7.3×10^{-7} (PAW), and the stress on those three plus QE's own three `pw_spinorbit` cases to $\le 1.2\times10^{-6}$ Ry/bohr^3; and, without any Fortran at all, against a central finite difference of the frozen energy to 6.2×10^{-9} Ry/bohr.

With a gradient-corrected functional the derivative is checked separately, because the gradient correction of a *vector* magnetization is a branch of its own: bcc iron with its moment in the plane, ultrasoft, PBE (`fe-noncolin-pbe-stress.in`) agrees with `pw.x` to $6.7\times$

10^{-9} Ry in energy and 1.6×10^{-7} Ry/bohr^3 in **stress** — 0.019 kbar out of 152.71, and the same level the collinear ultrasoft cases reach. With one atom in a bcc cell the force is zero by symmetry, so the stress is the only derivative that geometry has, which is why it is the quantity compared. Quantum ESPRESSO resolves the density onto a *fixed* quantization axis whenever the starting moments are all parallel to one direction (`compute_ux`), using $(n \pm \mathrm{sign}(\mathbf{m} \cdot \hat{u})\,|\mathbf{m}|)/2$ so that "up" stays up across a node where **m** passes through zero; that is the branch iron takes, and both codes pick the same axis. A genuinely **canted** cell takes plain $|\mathbf{m}|$ instead, with a real kink at every node, and that branch has no external number yet — not for want of trying here: `pw.x` itself limit-cycles at 5×10^{-6} Ry on a canted PBE hydrogen chain that converges to 10^{-11} under LDA in 62 iterations.

A noncollinear or spin-orbit run has forces, a stress and a relaxation, but only through the *autodiff* method: `method='analytic'` is refused by name, because `force_us`/`stres_knl` are transcriptions of QE's hand-derived expressions and have no spinor form. The force on an atom of a **spin spiral** is refused separately — its two spinor components live on different plane-wave spheres, so the nonlocal term needs the projectors of both; `dE/dq` is what a spiral has instead. **Elastic constants and electrostriction** stay refused for `noncolin`: they differentiate the same functional a third time, and while the Sternheimer solve their first-order wavefunctions come from now *does* have a spinor form, no third derivative has been run with one. The refusal is about the assembly rather than the solve, which is the distinction that cost this project a whole quantity once before.

`method='analytic'` is refused for one more reason, and this one is about *storage* rather than about physics: `addusforce` needs the whole $Q_{ij}(\mathbf{G})$ table, and a cell whose stored table would exceed `AUG_MAX_BYTES` (2 GiB; `DEFUQUMAT_AUG_MAX_BYTES` moves it) keeps a radial table and rebuilds the array a chunk at a time instead, so there is no whole table to read. Materialising one to keep the term would undo the thing that scheme exists for, which is why this is a refusal. It costs the cross-check and nothing else: the default autodiff force runs on that route, and on a displaced `si2-us.in` it returns $3.92220321 \times 10^{-2}$ Ry/bohr on *both* storage schemes, identical to every printed digit, agreeing with the resident cell's transcribed force to 1.4×10^{-6} Ry/bohr. Before this the term read `qgm[0]` off an empty tuple and raised a bare `IndexError` from inside a `jax.jit` trace — in a relaxation with `force_method='analytic'`, at the first ionic step with the SCF already paid for.

7 Response: dielectric, phonons, Raman

The **velocity operator** comes first and everything else uses it: it is one `jvp` of $\hat{H}(\mathbf{k})$ at a frozen sphere, because $\partial\hat{H}/\partial k_\alpha = i[\hat{H}, r_\alpha]$ in the periodic gauge. `band_velocities` exposes it — `Calculator.get_band_velocities` is the short way — and because the overlap carries a velocity too, a band velocity is $\langle\psi|\partial_k\hat{H} - \epsilon\,\partial_k\hat{S}|\psi\rangle$.

```
calc = Calculator.from_file("scf.in", pseudo_dir="pseudo")

velocities = calc.get_band_velocities()          # .velocities is (nk, nbnd, 3)
elsewhere = calc.get_band_velocities(kpoints=path)
```

Every first-order quantity then comes from the Sternheimer equation — the response of an occupied orbital to a perturbation, projected outside the occupied manifold so that no

empty states and no energy denominators appear:

$$\left(\hat{H} - \epsilon_n \hat{S} + \alpha \hat{Q}\right) |\Delta\psi_n\rangle = -\,\hat{P}_c^{+}\,\Delta V\,|\psi_n\rangle$$

It is solved self-consistently, because ΔV contains the response of the potential to the response of the density,

$$\Delta V_{\mathrm{scf}} = \Delta V_{\mathrm{ext}} + \int K(\mathbf{r},\mathbf{r}')\,\Delta n(\mathbf{r}')\,d\mathbf{r}'$$

and the kernel K is one `jvp` of `v_of_rho` rather than a second implementation. From its solutions:

$$\epsilon^{\infty}_{\alpha\beta} = \delta_{\alpha\beta} + \frac{8\pi}{\Omega}\sum_{n\mathbf{k}} w_{\mathbf{k}} \langle \Delta_\alpha \psi_n | \partial_\beta \psi_n \rangle, \qquad Z^{*}_{I,\alpha\beta} = \frac{\partial F_{I\alpha}}{\partial \mathcal{E}_\beta}$$

$$C_{I\alpha,J\beta} = \frac{\partial^2 E}{\partial \tau_{I\alpha}\partial \tau_{J\beta}}, \qquad \omega^2 \mathbf{e} = \frac{1}{\sqrt{M_I M_J}}\, C\,\mathbf{e}, \qquad \frac{\partial \epsilon_{ij}}{\partial \tau_{I\gamma}} \ \text{(Raman)}$$

The Raman tensor is the second-order energy differentiated once more along a displacement, at *frozen first-order* wavefunctions — the $2n{+}1$ theorem, which is why a third derivative costs one more `jvp` and not a new solve.

```
from defuqumat.response import (dielectric_tensor, dynamical_matrix,
                                raman_tensors, vibrational_spectrum)

eps = dielectric_tensor(calculation, scf.wavefunctions, scf.eigenvalues,
                        scf.density)
print(eps.epsilon)                      # 3x3, cartesian

ph = dynamical_matrix(calculation, scf.wavefunctions, scf.eigenvalues,
                      scf.density, scf.becsum)
print(ph.frequencies)                   # cm^-1

raman = raman_tensors(calculation, scf, born_charges=True, keep_internals=True)
spectrum = vibrational_spectrum(calculation, scf)   # per-mode Raman and IR
print(spectrum.raman_activity, spectrum.depolarisation)
```

`born_effective_charges` gives Z^* on its own, and `mode_activities` is the bare assembly — a transcription of `dynmat.x`'s `RamanIR`, kept a pure function of arrays so it is testable without a self-consistent run.

The loop stops where `ph.x`'s does, and its linear solves are no tighter than the pass needs. `tr2` is `ph.x`'s `tr2_ph`, in its units: the loop stops when $|\Delta\Delta V_{\mathrm{scf}}|^2$, the summed square of a pass's change in the potential divided by the square of the number of reals it holds and by the number of perturbations, falls below `tr2`/`npol`. The field's three directions are tested together, as `solve_e` tests them; a phonon's modes and a strain's six components are tested one at a time and the worst one decides, which is `ph.x`'s test for a one-dimensional representation. The default, 10^{-14}, is what the committed `ph.x` references use. Two things come with `ph.x`'s formula: a fixed `tr2` admits a larger residual per grid point on a finer grid, and on an fcc cell the number in `history` is twice `ph.x`'s, because `ph.x` perturbs along the lattice vectors and this code along Cartesian axes. What the criterion admits at the threshold is a residual of $\sqrt{N_{\mathrm{tot}}\, n_{\mathrm{pert}}\, \mathtt{tr2}}$ per real in the RMS, 3.8×10^{-5} Ry on silicon's 20^3 grid and 1.8×10^{-4} Ry on an O_2 molecule's 45^3 vacuum box at `nspin = 2`, so a quantity that is a small residue of large terms, such as the raw Born charge of a homonuclear molecule, can be a per cent short at the default, as it can be in `ph.x`. **Born charges on**

a cell with vacuum therefore default to `tr2` $= 10^{-16}$ (`VACUUM_TR2`). A cell has vacuum when more than 10 per cent of its dense grid holds less than 10^{-4} electrons/bohr^3 of the converged charge; none of the four bulk cells measured (silicon norm-conserving and ultrasoft, AlAs, aluminium) has a single such point (silicon's minimum is 1.4×10^{-3}, aluminium's 2.1×10^{-4}), while 56 per cent of the O_2 box is below it and 21 per cent of the graphene bilayer's. On the O_2 molecule 10^{-16} takes `ph.x`'s eight passes and puts Z^* 1.5×10^{-4} from `ph.x`'s, where 10^{-14} stops a pass earlier at 2.6×10^{-3}. One `tr2` governs the whole solve, so the dielectric tensor that comes with those charges is converged to 10^{-16} as well; asked for alone (`born_charges=False`) it keeps 10^{-14}, an explicit `tr2` always wins, and the result's `tr2` says which was used. The CG threshold follows `ph.x`'s schedule, 10^{-2} on the first pass and min $\left(0.1\sqrt{|\Delta\Delta V_{\mathrm{scf}}|^2},\ 10^{-2}\right)$ after, and each solve starts from the previous pass's solution. A number for `threshold` holds the CG at that value on every pass instead, which together with a smaller `tr2` is what a comparison of two routes below `ph.x`'s own convergence needs.

```
calc = Calculator.from_file("si-epsilon.in", pseudo_dir="pseudo")
eps = calc.get_dielectric_tensor()          # ph.x's tr2 and CG schedule
print(len(eps.history), eps.average_iterations)   # 5 passes, as ph.x; 12.6
tight = calc.get_dielectric_tensor(tr2=1e-24, threshold=1e-12)
print(eps.isotropic - tight.isotropic)      # 7.9e-6, the scheduled CG's residue
```

At Γ this code carries a term `ph.x` does not, and `origin_tangent` is the knob that puts `ph.x`'s convention back. The plane wave with $\mathbf{k}+\mathbf{G} = 0$ exists only at Γ exactly, and there the $l = 1$ projector's $\mathbf{k}$-derivative is finite: the product $f_1(q)\, Y_{1m}(\hat{\mathbf{q}}) \to c\sqrt{3/4\pi}\, q_m$ is linear in the vector, so its gradient is $c\sqrt{3/4\pi}\, \delta_{m\alpha}$ rather than zero. Quantum ESPRESSO zeroes that row, which makes its $\mathrm{d}H/\mathrm{d}\mathbf{k}$ discontinuous at Γ — a $\mathbf{k}$ of 10^{-4} off it computes the term. The default here is the derivative the operator has; `Calculation(..., origin_tangent=False)` reproduces `ph.x`. What it is worth is the weight of Γ in the k-sum, so a *shifted* mesh cannot see it at all, an unshifted one sees a fraction, and a Γ-only cell sees all of it: on the O_2 molecule ϵ_{xx} reads 1.116446 against 1.110915, and Z^*_{xx} 0.10110 against 0.13372, where `ph.x` gives 1.110915996 and 0.13367.

A Born charge is validated on a polar crystal, and the reason is worth one sentence. In silicon Z^* is zero by symmetry, so the -0.0757 both codes print is a basis-set residue and agreeing about it says nothing about the antisymmetric part, which is what a Born charge physically is. On zincblende AlAs the two sites carry different species and Z^* is a real charge: $+2.101065$ on the Al and -2.165827 on the As, against `ph.x`'s $+2.10106$ and -2.16581. The three silicon datasets agree with `ph.x` to every printed digit, 1.0×10^{-5} and 9×10^{-6} for norm-conserving, ultrasoft and PAW.

Ultrasoft and PAW work for the dielectric constant ($\leq$1.2e-4 against `ph.x` on four cases) *and for the dynamical matrix* — silicon's optical mode comes out at **513.295** and **513.421** cm^{-1} against `ph.x`'s 513.275 and 513.404, which is tighter than the norm-conserving case's 0.05. Four things make the difference and all four switch themselves off when S is the identity: the source term is $(\mathrm{d}H/\mathrm{d}u - \varepsilon\, \mathrm{d}S/\mathrm{d}u)|\psi\rangle$; the first-order state has an occupied block the Sternheimer solve does not produce; the mixed state changes at *frozen* states, because the augmentation charge and the projectors travel with their atom; and the orthonormality multipliers are variables of the functional that move — as a **matrix**, since a diagonal one is not invariant under the occupied-manifold rotation the state tangent is free in. Metals work for the response and for a norm-conserving dynamical matrix; and a symmetry-reduced wedge works as of the rank-n symmetriser.

The Raman tensor is ultrasoft and PAW as well, at **1.2e-4** on both against a central

difference of ϵ over re-converged displaced cells, where the norm-conserving control on the same script is 6.8e-4. It took **two tangents that are only right together**, and either alone is worse than neither: the state tangent is $P_c\,\mathrm{d}\psi + \psi^{\mathrm{ort}}$, the occupied block again; and $|b\rangle = P_c\,\mathbf{r}|\psi\rangle$ is *not* the solution of its own linear equation once `adddvepsi_us` has applied S to it and added the augmentation dipole, so $\mathrm{d}b$ is the tangent of a *composition* rather than of a solve. What located them is that $\partial\epsilon/\partial\tau$ is a sum of five partial derivatives and each one can be finite-differenced on its own against a re-converged run: three agreed to 7e-4 and two did not.

A spinor run has a dielectric constant and Born charges, on all three datasets — what an augmented one adds is in the amber box below, with its own numbers. A non-magnetic one is the same physics on a doubled space, four bands of two electrons against eight of one, and it comes out the same to 5×10^{-14} in ϵ and 3×10^{-15} in Z^*; the symmetrised 8-point wedge of an unshifted grid gives the closed 64-point grid's tensor to 7×10^{-13}, and `ph.x`'s own number to 4.3×10^{-5}, which is the scalar route's figure to the digit.

```
silicon = Calculator.from_file("si-epsilon.in")
spinor  = silicon.with_spin(4)          # 8 bands of one electron, not 4 of two
spinor.get_dielectric_tensor().isotropic    # 13.806641, against 13.806646
```

`with_spin` promotes the converged state rather than starting again, so the second ground state is not paid for twice.

A magnetic spinor is refused, and the reason is a measurement rather than a missing term. An iodine atom with spin–orbit coupling and its moment along z (`i-atom-soc.in`, `nosym`, a 0.164 eV gap) gives $\epsilon = \mathrm{diag}(1.3565721,\ 1.3565721,\ 1.5744824)$: uniaxial along the moment, off-diagonals at 10^{-15}, and *nothing imposes it* — the run carries no symmetry, so the two equal entries are a measurement. Turn the moment to x and the distinct axis moves with it, returning the same two numbers to nine digits, which is what makes the anisotropy a statement about spin–orbit coupling rather than about the box. And yet `ph.x`, on a ground state the two codes agree on to the printed digit, gives 1.494594 along the moment against this 1.574482 — **5.3%**, where the two components across it agree to 5×10^{-4}. Every threshold QE's `dmxc_nc` has that a derivative of the potential does not (a clamped ζ, a zeroing above $|\zeta|=1$, an $|\mathbf{m}|\le10^{-10}$ rule) fires at **zero** of that cell's 157464 grid points, so it is not a convention at an edge.

What is *not* in question is the bare response underneath. On the same cell, with no kernel and no field in the calculation at all, χ_0 under a potential probe reproduces a central difference of the density to 6.8×10^{-7} for a charge probe and 1.1×10^{-6} for a field along the moment, both unchanged over a factor of eight in the step, and this is the first fully relativistic dataset the solve has run on, so `dvan_so` is live in it. The disagreement therefore sits above χ_0: in the exchange–correlation kernel, in the field's own source term, or in `ph.x`. It is unlocated between those three, and a number that is 5% out while looking like a working calculation is what a refusal is for.

A subset of the atoms can be displaced on its own. The cost of a dynamical matrix is that $3\,n_{\mathrm{at}}$ bare perturbations and $3\,n_{\mathrm{at}}$ first-order wavefunctions are held at once, which is what stops it running on a large cell; `atoms=` solves only the perturbations asked for and returns that subset's own block, diagonalised with its own masses. That is the frozen-substrate approximation — a molecule on a fixed slab — and the modes it leaves are the frustrated translations and rotations, which are the physics of such a calculation rather than an artefact. A `on_row` callback receives each row as it is assembled, so a run that outlives its job keeps what it paid for.

```
phonons = calc.get_phonons(atoms=range(57))   # the molecule, slab held fixed
phonons.frequencies                            # (3 * 57,) in cm^-1
```

It works on norm-conserving, ultrasoft and PAW datasets. The validation needs no reference of any kind, which is unusual here: a partial dynamical matrix is not an approximation — it solves fewer perturbations of the *same* equations — so the whole-cell run is exact for what the two share, and the subset's rows reproduce it to **9.3e-15** (norm-conserving), **2.8e-14** (ultrasoft) and **4.9e-15** (PAW) Ry/bohr^2. The physical check is that freezing one of silicon's two atoms turns the optical mode's reduced mass $m/2$ into m, so the frequency falls by $\sqrt{2}$: $519.205 \to 367.146$, a ratio of 1.414173.

A soft mode converges in `ecutrho`, and the finite-difference route hides it. The analytic Hessian and a finite difference of the forces are the same quantity in exact arithmetic and are not on a grid: the FFT box breaks continuous translation invariance, so the energy carries a small ripple as an atom moves across it (the "egg-box" effect). The analytic second derivative takes the *point* curvature of that ripple; a finite difference averages it over the displacement. The two therefore differ by the amount the dense grid is not translationally invariant — and because that error is roughly a fixed curvature, it matters in proportion to how *small* the mode's own curvature is. A stiff mode barely notices; a soft one can be several per cent out.

It is `ecutrho` and not `ecutwfc` that fixes it, the ripple living on the dense grid where the density and the local potential do. Measured on N_2 in a 12 bohr cube (SG15 ONCV PBE, `nosym`, `K_POINTS gamma`, one atom frozen, the other at 1.15 Å), with the finite difference taken as a central difference of the forces at ± 0.03 Å:

	transverse (cm^{-1})	gap	stretch (cm^{-1})
40/160 Ry, analytic	314.12		1483.49
40/160 Ry, finite difference	296.59	17.5	1490.77
40/320 Ry, analytic	306.35		1488.1
40/320 Ry, finite difference	301.9	4.4	1495.2

Raising *only* `ecutrho` moves the frustrated-rotation pair by 8 cm^{-1} analytically and 5 cm^{-1} by finite difference, closing their gap from **17.5 to 4.4** cm^{-1}, while the stretch is 0.5% off either way at both cutoffs — which is the signature. The analytic asymmetry is 3e-8 and 1.4e-8 Ry/bohr^2, so neither number is a convergence failure of the solve.

What makes this worth stating rather than discovering: both routes converge, each agrees with itself, and they drift *together*, so nothing in either calculation announces the error. The practical rule is that a soft mode wants its own `ecutrho` convergence pair, and the shift between them is the error bar to quote — not the agreement between the analytic and finite-difference routes at one cutoff, which is a weaker statement than it looks. (Measurements contributed from a production study of a molecule on a surface, where the modes of interest are 10–15 meV.)

A trap worth knowing: a response on a reduced k-set is a *polar vector field* and must be symmetrised as one. Running the whole grid instead is sound only if the grid is closed under the point group — a **shifted** Monkhorst–Pack grid is not.

7.1 Phonons away from Γ

A displacement pattern $\mathbf{u}_s(\mathbf{R}) = \mathbf{u}_s e^{i\mathbf{q}\cdot\mathbf{R}}$ is not a displacement of the unit cell — it is periodic only on a supercell commensurate with $\mathbf{q}$, which is exactly what a frozen-phonon calculation pays for. Linear response does not pay it, because the *first-order* quantities are still lattice periodic once their $e^{i\mathbf{q}\cdot\mathbf{r}}$ is taken out. The perturbation is $e^{i\mathbf{q}\cdot\mathbf{r}}$ times a periodic function, so $\Delta V_{\mathbf{q}}|\psi_{\mathbf{k}}\rangle$ is a Bloch state at $\mathbf{k}+\mathbf{q}$: the calculation stays in the unit cell, with one extra plane-wave sphere.

That asymmetry is the whole of it. The operator, the projector and the preconditioner belong to $\mathbf{k}+\mathbf{q}$; the eigenvalue subtracted is the one of the band being perturbed, at $\mathbf{k}$:

$$\left(\hat{H}_{\mathbf{k}+\mathbf{q}} - \epsilon_{n\mathbf{k}}\hat{S}_{\mathbf{k}+\mathbf{q}} + \alpha\hat{Q}\right)|\Delta\psi_{n\mathbf{k}}\rangle = -\,\hat{P}_c^{\mathbf{k}+\mathbf{q}}\,\Delta V_{\mathbf{q}}\,|\psi_{n\mathbf{k}}\rangle$$

and the response density is $\Delta n_{\mathbf{q}}(\mathbf{r}) = (2/\Omega)\sum_{n\mathbf{k}} w_{\mathbf{k}}\,\psi^*_{n\mathbf{k}}\Delta\psi_{n\mathbf{k}}$, which is complex — at Γ it collapses to twice a real part and nowhere else.

```
# nosym = .true., noinv = .true., on an unshifted grid -- see the box below
calc = Calculator.from_file("si-nosym.in", pseudo_dir="pseudo")

ph = calc.get_phonons_at_q(q=(0.5, 0.5, 0.5))          # crystal coordinates, L
print(ph.frequencies)                                   # cm^-1
print(ph.matrix.shape)                                  # (nat, 3, nat, 3), complex

xpoint = calc.get_phonons_at_q(q=(0.0, -1.0, 0.0), q_cartesian=True)
```

`q` is in crystal coordinates of the reciprocal lattice unless `q_cartesian` is set, which reads it in the $2\pi/a$ units `ph.x` prints. The matrix is **complex**: $D(\mathbf{q})$ is hermitian rather than symmetric, and only at $\mathbf{q}=0$ — or where the crystal has inversion — does it collapse to a real one.

Silicon on a 64-point unshifted grid, against the vendored `ph.x`: at L the six modes are 101.789, 101.789, 382.235, 405.099, 488.028 and 488.028 cm^{-1} against 101.843, 101.843, 382.241, 405.121, 488.019 and 488.019, the worst of them **0.054** cm^{-1}; at X the worst is **0.038**. That is the same floor the zone centre has and for the same reason (QE interpolates its radial form factors from a d$q = 0.01$ table where this integrates them directly). `ph.x` runs the eight-point wedge with its response symmetrised where this runs the whole grid with nothing symmetrised, so the comparison is also the only check the two routes get against each other.

Only the Ewald sum knows about q, among the terms that do not involve the response. The second derivative of the external potential at frozen density is diagonal in the atom index — both derivatives are taken at the same site, so the two modulation phases cancel — and it is therefore the same matrix at every wavevector. The ion–ion sum is not diagonal, and it is the one piece rebuilt.

Refuses. A symmetry-reduced k-set: a response has to be averaged over the operations that leave the *perturbation* invariant, and for a phonon at $\mathbf{q}$ those are the ones with $S\mathbf{q} = \mathbf{q} + \mathbf{G}$ — the small group of $\mathbf{q}$, which is not the crystal's group and is not written. Run with `nosym` and `noinv` on an **unshifted** grid, which is the grid that is closed under the point group. There is no **dispersion** either: that needs the star of $\mathbf{q}$ and `q2r`/`matdyn`'s Fourier interpolation on top of this. Ultrasoft and PAW are refused (S moves with the atoms, and the orthonormality multipliers carry a term whose bra is at $\mathbf{k}$ and whose ket is at $\mathbf{k}+\mathbf{q}$, where q_{ij} pairs projectors on one sphere); so are spin polarization, spinors, spin spirals, meta-GGA and DFT+U. A **metal** runs at every $\mathbf{q}$ but zero, and at $\mathbf{q}=0$ (or a reciprocal lattice vector) it is refused and pointed at `get_phonons()`: only there can a displacement change the number of electrons, so only there does the Fermi level move (`ph.x`'s `ef_shift`), and this route carries no shift. A **nonlinear core correction** is refused for a reason that belongs to $\mathbf{q}$ rather than to the dataset: two atoms' core charges overlap inside a nonlinear E_{xc}, so unlike every other frozen term that one *is* a function of $\mathbf{q}$.

Electron–phonon coupling

What is the lifetime of a phonon in a metal? It decays by exciting an electron–hole pair across the Fermi surface, and the rate is set by the matrix element of the change of the self-consistent potential between a state at $\mathbf{k}$ and one at $\mathbf{k}+\mathbf{q}$,

$$g^{\mu}_{mn}(\mathbf{k},\mathbf{q}) = \langle \psi_{m\mathbf{k}+\mathbf{q}} |\, \partial_\mu V_{\text{bare}} + \partial_\mu V_{\text{scf}} \,| \psi_{n\mathbf{k}} \rangle,$$

with μ an atom and a cartesian direction, so g is in Ry/bohr. Both halves are already there after the phonon at $\mathbf{q}$: the bare one is the right-hand side of the Sternheimer equation above, and the screened one is the converged $\Delta V_{\mathbf{q}}$ of its loop. Summed over the Fermi surface with the mode's displacement $z_\nu = e_\nu/\sqrt{M}$,

$$\gamma_{\mathbf{q}\nu} = \frac{\pi}{2} \sum_{\mathbf{k}} w_{\mathbf{k}} \sum_{mn} \left| z_\nu^\dagger g_{mn}(\mathbf{k},\mathbf{q}) \right|^2 \delta(\epsilon_{n\mathbf{k}} - E_F)\, \delta(\epsilon_{m\mathbf{k}+\mathbf{q}} - E_F), \qquad \lambda_{\mathbf{q}\nu} = \frac{\gamma_{\mathbf{q}\nu}}{\pi N(E_F)\, \omega^2_{\mathbf{q}\nu}},$$

meaning that $\gamma_{\mathbf{q}\nu}$ is the linewidth, a half-width in energy that inelastic neutron or x-ray scattering resolves, and $\lambda_{\mathbf{q}\nu}$ the dimensionless coupling of that mode, whose average over the zone is the λ of McMillan's T_c. $w_{\mathbf{k}}$ sums to 2 and $N(E_F)$ is per spin. This is `ph.x`'s `electron_phonon = 'simple'`, transcribed: for each broadening σ the Fermi level and $N(E_F)$ are recomputed at that width with Methfessel–Paxton smearing, and the two deltas are Gaussians of width σ, so you can watch how far the result is from converged in the k-mesh by how much it moves with σ.

```
# a metal: occupations = 'smearing', nosym and noinv on an unshifted grid
calc = Calculator.from_file("al-elph-nosym.in", pseudo_dir="pseudo")

el = calc.get_electron_phonon(q=(0.25, 0.0, 0.0), q_cartesian=True)
print(el.frequencies)        # cm^-1, those of el.phonons
print(el.sigmas)             # Ry: 0.02, 0.04, ..., 0.20 as in ph.x
print(el.gamma_ghz[1])       # (3 nat,) linewidths at sigma = 0.04, GHz
print(el.lambdas[1])         # (3 nat,) lambda_(q nu)
print(el.dos[1], el.fermi_energies[1])   # N(E_F) per spin per Ry, E_F in Ry
```

`sigmas`, `el_ph_sigma`, `el_ph_nsigma` and `el_ph_ngauss` choose the broadenings as `ph.x`'s variables of the same names do; `degeneracy_cmm1` (0.05 cm^{-1}) is how close two frequencies have to be for their linewidths to be averaged as one degenerate set. The result also carries `el_ph_sum`, the $3n_{\text{at}} \times 3n_{\text{at}}$ matrix $\sum |g|^2 \delta\delta$ before it is projected on the modes, and `phase_space`, the double delta alone.

Aluminium on the $8 \times 8 \times 8$ grid without symmetry, against serial `ph.x` 7.5 on the same input: at $\mathbf{q} = (1/4, 0, 0)\, 2\pi/a$ the frequencies are 73.9960, 73.9961 and 132.4755 cm^{-1} against 73.9956, 73.9957 and 132.4752, and at $\sigma = 0.02$ Ry the linewidths are 2.67, 2.67 and 1.53 GHz with $\lambda = 0.2120$, 0.2120, 0.0381, which is what `ph.x` prints; E_F, $N(E_F)$ and the double delta agree to every printed digit at all ten broadenings. Test-suite `ph_interpol_metal` has the same cell and the same $\mathbf{q}$, and is *not* the reference: its benchmark was written by QE 6.5, whose frequencies there are 76.57 and 133.66 cm^{-1} from a ground state identical to 7.5's to every printed digit, so the difference is between the two versions of `ph.x`.

What is invariant and what is not. The sum over band pairs with weights that depend only on eigenvalues does not care which basis a degenerate eigensolver returned, at $\mathbf{k}$ or at $\mathbf{k}+\mathbf{q}$, so γ is well defined; a single $|g_{mn}|^2$ is not, and is not exposed. Inside a degenerate pair of *modes* the split of γ between the two depends on which basis of the pair the diagonalisation chose, so the pair's average is reported, which is the number `ph.x` prints for each member.

Refuses. An insulator (with fixed occupations the double delta is empty and λ is not defined), and everything the phonon at **q** refuses: so ultrasoft and PAW, whose screened term carries the augmentation charge's $\int \Delta V\, Q_{ij}$ on the projectors (`adddvscf`), spin polarization and spinors, a symmetry-reduced k-set, and $\mathbf{q} = 0$, where the Fermi shift has to be taken out of ΔV first. There is no $\alpha^2F(\omega)$, no λ averaged over the zone and no T_c: `ph.x`'s `'interpolated'` route gets them from a dense-grid band structure and `q2r`/`matdyn`'s interpolation, and the dispersion they need is not written here.

Refuses. $\chi^{(2)}$ and the electro-optic tensor (the second-order source term has nothing to build the $\langle u_i|r_k|u_j\rangle$ piece from — and it is **42%** of the answer, measured); a partial dynamical matrix (`atoms=`) of a *symmetrised* run, since `symdvscf` acts on the $3\,n_{\text{at}}$ stack as one object — an operation rotates the cartesian direction *and* permutes the atom, so it mixes a mode inside the subset with one outside it, and such a run wants `nosym`; the dynamical matrix of an ultrasoft or PAW *metal* (the `wg`/`wk` weight split was derived for a response whose `becsum` dependence is entirely inside `dpsi`, and an insulator cannot say which weight the three further tangents belong with); any response under a potential-only meta-GGA; and a shifted grid with a reduced k-set.

A **spinor** run (`noncolin`, with or without `lspinorb`) has a dielectric constant and Born charges on all three datasets, provided it carries *no net moment*. A **textured** one (`nspin_mag = 4`) is refused by name and the refusal carries its own measurement: it runs, its tensor is uniaxial along the moment and follows the moment when that is turned, and it is 5.3% from `ph.x` in the component along the moment with the two across it agreeing to 5×10^{-4}. An *ultrasoft or PAW* one runs, and the object it was refused for is not written here either: dD_{ij} is a 2×2 matrix in spin space (`set_int3_nc`), and one `jvp` of `newd` already passes through the noncollinear recombination, whose `fcoef` sandwich is linear, so the tangent comes out dressed. What had to be written was the *position* operator's spin blocks, the augmentation charge having a dipole that couples the two components (`compute_qdipol_so`). Fully-relativistic ultrasoft AlAs reaches `ph.x` to 3.5×10^{-5} on a spin-orbit shift of 8.6×10^{-3}, and its Born charges to 3.2×10^{-6} and 3.9×10^{-5} on the two atoms (2.101143 against 2.10114, -2.165831 against -2.16587); a relativistic *PAW* insulator has no reference here, so that combination is validated only through the identity against the scalar run. A spinor *metal* is refused, and for the reason every metal is: an `epsilon_infinity` is not defined there.

The **phonons, the Raman tensor, the strain response** and everything built on them stay refused for a spinor, and for their own reason rather than this one: `symmetrize_displacement` carries the axial landmine the field path had — three of the four `nspin_mag` channels are a magnetization and rotate with $\det(R)$ and the time-reversal sign — with the atom permutation on top, and a symmetriser that has never been checked against the whole zone is worse than a refusal. That is not rhetoric: on the field path the wrong rotation was **66%**, worse than no symmetrisation at all.

Also refused: a ground state converged under a **magnetic field or a constrained moment**. The whole stack rebuilds its potential from the field the *input* asked for, and that is not the field the density belongs to — `reducebf` scales a field away as the run proceeds (7% of its input value after 25 iterations at 0.9) and the fixed-spin-moment scheme replaces it outright, which is why `SCFResult` carries `magnetic_field` and `field_scale` separately. Threading that pair through is the missing piece for a field put in by hand; a `constrained_magnetization = 'fsm'` moment needs the *induced* field as well, and its field comes from a feedback update rather than from a penalty, so it is not a derivative of anything. Re-run the ground state without the field.

LO–TO splitting and the static dielectric constant

At Γ a polar crystal's dynamical matrix is not analytic. An optical mode that moves the ions against each other builds a macroscopic electric field, and what the field costs depends on the *direction* the zone centre is approached from, so the matrix acquires a rank-one term:

$$D_{I\alpha,J\beta} \;\to\; D_{I\alpha,J\beta} + \frac{4\pi e^2}{\Omega}\,\frac{(\mathbf{q}\cdot Z^*_I)_\alpha\,(\mathbf{q}\cdot Z^*_J)_\beta}{\mathbf{q}\cdot\epsilon^\infty\cdot\mathbf{q}}$$

It raises the longitudinal mode and leaves the transverse ones alone — the LO–TO splitting — and it vanishes identically where Z^* does, so a non-polar crystal is untouched. Contracting the same two ingredients the other way gives the ionic screening of a *static* field, one oscillator per polar mode:

$$\epsilon^0_{ij} = \epsilon^\infty_{ij} + \frac{4\pi e^2}{\Omega}\sum_\nu \frac{p^\nu_i\,p^\nu_j}{\omega^2_\nu}, \qquad p^\nu_i = \sum_{I\gamma} Z^*_{I,i\gamma}\,z^\nu_{I\gamma}$$

where p is the same mode dipole the infrared activity is the square of. So ϵ^0 and ϵ^∞ are two ends of one contraction, and together with the frequencies they satisfy the Lyddane–Sachs–Teller relation $\epsilon^0/\epsilon^\infty = (\omega_{\mathrm{LO}}/\omega_{\mathrm{TO}})^2$.

```
spectrum = calc.get_vibrational_spectrum(neutralize=True)
print(spectrum.static_permittivity)          # eps_0, 3x3
print(spectrum.ionic_permittivity)           # the polar modes' share of it

lo = calc.get_vibrational_spectrum(loto_direction=(1, 0, 0), neutralize=True)
print(lo.frequencies)                        # the LO modes along x
```

`nonanal` adds the term to a dynamical matrix on its own and `loto_modes` diagonalises the result; `polar_mode_permittivity` is the oscillator sum, and `neutral_born_charges` the sum-rule correction below. All four are pure functions of arrays, so they can be checked without a self-consistent run behind them.

Both halves agree with `dynmat.x` — which applies the same term and the same oscillator sum to the tensors this code writes to its `fildyn` — to every digit it prints. **The Lyddane–Sachs–Teller relation is the check neither half can pass alone**, since one side is a ratio of two diagonalisations and the other a sum of dipoles over ω^2; on AlAs it holds to **5.0e-11**.

It needs a charge-neutral Z^* and that is worth knowing before reading an LO frequency. The sum rule $\sum_I Z^*_I = 0$ says a rigid translation builds no field; a computed Z^* misses it by the basis-set error, which *charges the crystal* — AlAs at `ecutwfc = 10` misses it by -1.257, and the non-analytic term then lifts a longitudinal *acoustic* mode from 1.8 to 33.8 cm^{-1} and puts the LO frequency 7.7 cm^{-1} low. `neutralize=True` shares the violation out over the atoms, which is what `dynmat.x` does under any `asr` but `'no'`. Comparing against another code does not see this: it is handed the same charges and reproduces the same wrong number. LST does.

Refuses. Everything the Raman tensors and the Born charges refuse, since it is built from both: PAW Z^*, metals, and the rest of the list above. The splitting is a Γ quantity by construction — it is what $q \to 0$ leaves behind — so a dispersion through it needs phonons at $q \neq 0$, which are not implemented.

Elastic constants, electrostriction and the elasto-optic tensor

The same machinery in the *strain* coordinate rather than the atomic one. **The strain *response* itself works on all three pseudopotential kinds** (`strain_response`): the augmentation charge $Q_{ij}(\mathbf{r})$ is a function of the *cell*, so a homogeneous strain deforms the table it is tabulated on where a displacement only translates it, and on top of that come the four terms the dynamical matrix needed. Against a central difference of *re-converged* strained runs — which needs no `ph.x`, since `ph.x` has no strain perturbation — it agrees to **4.6e-4** (ultrasoft) and **4.7e-4** (PAW) against a norm-conserving 1.9e-4 on the same cell. The *derivatives* below are norm-conserving still — but the variational second-order energy they all stand on is not: it reproduces `dielec.f90`'s dielectric constant to **3.4e-10** (ultrasoft) and **6.9e-11** (PAW) against a norm-conserving 8.4e-10, which took four terms that each vanish when S is the identity. One of them is worth stating on its own, because it is a distinction rather than a term: a *state* is projected with $1 - \sum |\psi\rangle\langle\psi| S$ and a *right-hand side* with $1 - \sum S|\psi\rangle\langle\psi|$, and collapsing the two — which is exact at $S = 1$ — costs a factor of fourteen. **The third derivative in this *strain* coordinate runs on all three pseudopotential kinds too**, and what had it refused was a reference rather than a term. Three tangents are what it takes: the two that closed the Raman tensor, and the commutator that sources db written with the multiplier *matrix* rather than the frozen scalar eigenvalue, which is the right form for a reason that needs no measurement: the equation defining b carries that eigenvalue on both of its sides, so differentiating it produces $\mathrm{d}\varepsilon_n$ twice, in the operator and in the source. Against a central difference of ϵ over re-converged strained cells, extrapolated in the step: **7.2e-6** (ultrasoft) and **2.2e-5** (PAW), on a floor of **2.5e-5** measured by running the norm-conserving control through the identical script, where the analytic answer is exact. The frozen scalar reads 1.3e-2 there and the source written with the traced diagonal alone 1.4e-3.

That term had been excluded by measurement, and the measurement was a step size: it read the *displacement* coordinate's Raman tensor going from 1.2e-4 to 1.14e-3 against a central difference at $h = 0.02$, and that difference is itself 1.1e-3 from its own limit, reading -69.1942, -69.1350 and -69.1205 at 0.02, 0.01 and 0.005 on ultrasoft silicon, so its truncation error had been cancelling the missing term. Extrapolated, the two forms swap places. **The elastic constants are the one thing here that stays refused on an augmented dataset**, and for terms of their own: C_{ijkl} is a second derivative with no electric-field solution in it, so none of the above reaches it, and measured against a five-point second difference of the total energy it is **22 per cent** out on C_{1111} and 2.9 per cent on C_{1212}, the same on ultrasoft and PAW where the norm-conserving control is exact to 6.2e-08. Ask `electrostriction(elastic=False)` for everything else; M and Q need the compliance and go with them. `strain_response` solves the first-order problem for a strain — Abinit's metric-tensor formulation, obtained for nothing here because the cell was already written in reduced coordinates — and two things follow from it:

$$C_{ijkl} = \frac{\partial \sigma_{ij}}{\partial \varepsilon_{kl}}, \qquad \frac{\partial \chi_{ij}}{\partial \varepsilon_{kl}} \ \text{(elasto-optic)}$$

The second is a **third** derivative of the energy, and it comes from one `jvp` of the second-order energy at frozen first-order wavefunctions — the $2n{+}1$ theorem again. Through the thermodynamic identity of Tanner, Bousquet and Janolin it gives the four electrostriction tensors m, q, M and Q.

```
from defuqumat.response import elastic_constants, electrostriction

es = electrostriction(calculation, scf)     # solves the strain response itself
```

```
print(es.m, es.q, es.M, es.Q)              # the four electrostriction tensors
print(es.photoelastic)                      # the elasto-optic tensor
print(es.elastic.voigt)                     # C_ij in Voigt notation, GPa

# or the elastic constants alone, from a strain response you already have
C = elastic_constants(calculation, scf.wavefunctions, scf.eigenvalues,
                      scf.density, es.strain)
print(C.tensor.shape, C.voigt)

# on an ultrasoft or PAW dataset, everything but the elastic constants and
# the two families built on them: es.M and es.Q come back as None
es = electrostriction(calculation, scf, elastic=False)
print(es.depsilon_dstrain, es.photoelastic, es.m, es.q)
```

Converged silicon gives C_{11} = 198.5 GPa and C_{12} = 68.9, against a measured 165.7 and 63.9; the three independent components of $\partial\epsilon/\partial\varepsilon$ match a central difference over re-converged strained cells to **2e-4**, which is that difference's own floor, and the rank-4 tensor is cubic to 3e-14 with nothing imposing it.

The six columns of C_{ijkl} are six `jvp`s of the stress at one point, and they are evaluated together, as one `vmap` over the six tangents, whenever the compiled program's own scratch (`memory_analysis().temp_size_in_bytes`, read before it runs) is at most `SHARED_PRIMAL_BYTES`, 1 GiB, and one at a time above it. The shared form computes the stress once instead of six times: 3.4x faster than the one-at-a-time form on silicon with 64 k-points and 204 MB of scratch, which it takes, and 3.7x on a rhombohedral boron nitride cell at 1.6 GB, 3.5 times one column's, which is above the gate and so runs one column at a time unless the gate is raised. It moves the tensor by round-off, 1.6e-11 on 26.2; `shared_primal_bytes=0` (or `DEFUQUMAT_ELASTIC_SHARED_BYTES=0`) keeps the one-at-a-time form and its last bits. The gate costs the shared program's compile even when it refuses, once per shape with the kernel cache on. It applies where the states are held in memory; a run whose states are walked through host memory a chunk at a time takes its columns one at a time as before and never shares.

Refuses. Both are `nspin = 1`, insulators and **clamped-ion**. `d(chi)/d(strain)` and the elasto-optic tensor run on norm-conserving, ultrasoft and PAW datasets; **the elastic constants are norm-conserving only** (`require_a_measured_elastic_regime`), with 22 per cent on C_{1111} measured on the augmented pair rather than assumed, and M and Q go with them because they need the compliance, so ask for `elastic=False` to get the rest. A diverged first-order solution is refused rather than consumed in silence (`require_converged_responses`): at QE's `alpha_mix = 0.7` the strain response of a *slab* diverges, and consuming it gave a C_{ijkl} that was not even symmetric under $C_{ijkl} = C_{klij}$. The elastic constants additionally need the whole k-grid rather than a wedge, because their functional builds its own density and symmetrises it as a scalar inside the chain rule.

7.2 The piezoelectric tensor

The polarization a strain induces is the stress an electric field induces — one mixed second derivative read either way round:

$$e_{k,ij} = \frac{\partial P_k}{\partial \varepsilon_{ij}} = \frac{\partial \sigma_{ij}}{\partial \mathcal{E}_k} = -\frac{1}{\Omega}\frac{\partial^2 E}{\partial \varepsilon_{ij}\,\partial \mathcal{E}_k}$$

It is the Born charge's construction with one coordinate changed. Z^* is a `jvp` of the *force* along the field's response; the force is `jax.grad` of the frozen-state energy in the atomic positions and the stress is `jax.grad` of the same functional in a strain, so this is a `jvp` of the *stress* along the same response — three of them, one per field direction, on top of a dielectric constant that was going to be solved anyway. The orthonormality constraint contributes nothing here, because $\langle\psi|\hat{S}|\psi\rangle$ is a sum over a sphere of integers and carries no cell.

```
calc = Calculator.from_file("alas.in", pseudo_dir="pseudo")

piezo = calc.get_piezoelectric_tensor()
print(piezo.voigt)              # (3, 6) in C/m^2, columns xx yy zz yz xz xy
print(piezo.e14)                # -0.7638: zincblende's one component
print(piezo.e.shape)            # (3, 3, 3) in e/bohr^2, e[k, i, j]
print(piezo.dielectric.epsilon) # the field response is shared, not re-solved

# The same number contracted rather than taped: no forward-over-reverse tape,
# so it is the route for a dense mesh or a large cell.
cheap = calc.get_piezoelectric_tensor(method="zstar_eu")
print(abs(cheap.e14 - piezo.e14))   # 3.4e-15
```

`pw.x` computes no piezoelectric tensor — the word occurs once in the whole distribution, in a citation in a comment — and Elk's task 380 reaches it by a finite difference of the Berry-phase polarization over one full ground state per strain tensor. So the validation is internal, and there are four independent statements. Silicon is centrosymmetric and its whole tensor vanishes (2.4e-5 C/m^2 against AlAs's 0.764 from the same code, with nothing imposing it on a `nosym` run); AlAs is $\bar{4}3m$ and only $e_{14} = e_{25} = e_{36}$ survive, to 1.7e-14; an eight-point wedge reproduces the sixty-four-point closed grid to **4.5e-9**, and on an ultrasoft cell, where one term of this derivative is quadratic in a per-k tangent and no average of the finished tensor can repair it, a three-point wedge reproduces its own closed grid to **4.8e-6** — which is the shared field response's floor on that cell and not the tensor's, since ϵ_∞ itself splits between the same two runs by five times as much; and the same mixed derivative contracted the other two ways — `zstar_eu.f90`'s expression with a strain label, and the strain response against the field's bare perturbation — agree to **6.2e-15** and **1.3e-7**. What anchors the sign and the normalisation is that the *same* assembly run in the position coordinate is the Born charge, and that is `ph.x`'s number.

All four of those statements are blind to the k-mesh, and the k-mesh is worth thirteen per cent. The three routes contract the same field response, and the Z^* anchor is the same assembly in the position coordinate, so none of them can see the Brillouin-zone sum this quantity is. Measured against Elk's route on the same AlAs cell, one converged ground state per strain and a finite difference of the Berry-phase polarization, which shares no machinery with any of the above:

e_{14}, C/m^2	4 4 4	6 6 6	8 8 8	10^3
this code, a `jvp` of the stress	−0.763786	−0.687475	−0.672897	−0.669907
Berry-phase finite difference	−0.661386	−0.661964		

`method` is what that second call is for, and on a big run it is not a detail: both routes consume the *same* field response, so neither checks the other's solve, but `'autodiff'` holds a forward-over-reverse tape of every radial and reciprocal-space intermediate a cell derivative rebuilds, where `'zstar_eu'` contracts the response against the bare strain perturbation and holds nothing. Measured on ultrasoft AlAs, the first peaks at 139.6 GiB

at 64 k-points and grows by about 1.6 GiB a point, which puts a 6^3 mesh on a whole cluster node. The default is the taped one because it is the route that extends to the next derivative; reach for the other when the mesh is what you are converging, which on an ultrasoft cell is the difference between a workstation and a cluster node.

`zstar_eu.f90` line 90 hands an augmented dataset to `zstar_eu_us.f90`, and this route was refused on one until what those three hundred further lines are worth in the strain coordinate had been found: it is two contractions, because only one leg of this derivative moves S. They are the multipliers' own first-order change together with the `add_for_charges` sandwich beside it, and the screening of the frozen-state density response — the second of which is not about the constraint at all, but about what "bare" means when the coordinate is a strain, since freezing the density as an *array* misses that the density is a function of the cell as well as of the states. With all of them in, the two routes read 1.473304405 and 1.473304408 on a small ultrasoft AlAs cell where they had been 1.6 per cent apart, and $+0.8213303507$ against $+0.821330452$ on the cell above at 64 k-points, which took 3.6 GiB and nineteen minutes on the contracted route where the taped one took 139.6 GiB. **PAW is still refused by name**, because its one-centre energy is a function of `becsum` directly and the cross term that reaches is on no grid.

The -0.7638 printed above is the `4 4 4` of the committed input, and it is thirteen per cent from where the calculation is going; the Berry-phase value stays inside 0.7 per cent over four string meshes and two ground states, so it is the response route that is unconverged rather than the reference. The successive differences shrink by about a factor of five a rung, so on that pattern roughly 0.0008 would be left after 10^3 and the two routes would sit about one per cent apart, which is comparable with what the reference itself is known to. That last step is an extrapolation from four points rather than a measurement, so what is established is the thirteen per cent, not where the last per cent goes. The reason is that e_{14} is a mixed derivative of an energy that is already stationary in the density, so what is left is a zone sum with no variational protection, and it needs a denser mesh than the ground state does: the `6 6 6` and `4 4 4` rows above share one SCF, to -16.89293132223534 Ry in every printed digit, and that same change of mesh moves the Berry value by 0.0006 and this code's value by 0.076. **Converge the mesh on this quantity rather than on the total energy.**

The code says so now, and it can measure it. Nothing inside this quantity can see its own zone sum — the three routes share one field response and the Z^* anchor is the same assembly in the position coordinate — so `get_piezoelectric_tensor` warns with the curve above, and the result carries `nk` and `grid`, the sample it actually integrated. What replaces the warning with a number for *your* crystal is a ladder: one ground state and one response per mesh, and the relative change across the last step.

```
ladder = calc.get_piezoelectric_kmesh_ladder(meshes=(4, 6))

print(ladder.nk)       # (64, 216): what each rung integrated
print(ladder.e14)      # (-0.763786, -0.687475) C/m^2
print(ladder.drift)    # 0.111: the last step moved the tensor 11 per cent
print(ladder.settled)  # False, and the warning above says so with that number
print(ladder.tensor)   # the densest rung, carrying kmesh_drift
```

The rungs are the irreducible wedge of each unshifted grid by default, which is the same sample at a fraction of the cost — on an ultrasoft cell the wedge and the closed grid gave the same tensor to 1.6×10^{-6} — and `wedge = False` runs the whole grid. A rung is a whole calculation, so a ladder costs what its densest rung costs: those two rungs took 73 s and 191 s and peaked at 14.4 GB, all of it the taped route's tape at 216 k-points,

which is the case for `method = 'zstar_eu'` on anything larger. **The last step is smaller than the error**: across this crystal's rungs the steps are 10, 2.1, 0.44 and 0.12 per cent to $12\times12\times12$, a geometric approach that leaves about a third of the last step still to travel, so `settled` means the mesh has stopped being the largest thing wrong rather than that it is converged. The $10 \times 10 \times 10$ rung sits 1.2 per cent from the Berry value and that is the cell's 10 Ry cutoff, not the mesh: a finite difference at constant cutoff feels the basis change with the strain and the response does not, and at 30 Ry the $12 \times 12 \times 12$ rung and the Berry value agree to 0.25 per cent. A ladder converges the k-sum only; the cutoff is converged on its own.

Refuses. Clamped-ion — the atoms are carried by the strain and do not relax, which is also what Elk's task computes; the internal-strain term that makes the constant comparable with experiment is not here, and for zincblende it very nearly cancels this one. And **polar crystals by name**: what the derivative above gives is the *improper* tensor, and the proper piezoelectric response differs from it by $\delta_{ki}P_j - \delta_{ij}P_k$, which needs the spontaneous polarization itself. Those terms vanish identically whenever the two Cartesian labels they pair are different — so e_{14} of a zincblende crystal never carries the ambiguity — and they vanish for every component of a class that admits no invariant vector, which is what is checked. **PAW** — the ultrasoft half of what used to be one refusal is lifted, on every mesh since 2026-10-11. At a converged cutoff on zincblende AlAs the deficit of e_{14} from each cell's own Berry-phase finite difference is 15.30, 3.73 and 0.86 per cent at `4 4 4`, `6 6 6` and `8 8 8` for the ultrasoft dataset, against 14.54, 3.65 and 1.01 for the norm-conserving calibration, so the dataset moves it by less than a point and the mesh is the error at every one of them; a coarse ultrasoft run therefore gets the same k-mesh warning a norm-conserving one does, with those offsets in it, where until then it was refused below eight divisions (`allow_a_coarse_mesh`, which exempted the ladder's coarse rungs, is still accepted and does nothing). What follows is the history of the dataset refusal, and it is a gap rather than a missing term: nothing in the assembly is norm-conserving and the displacement leg of it is the Born charge, which is now validated on a *polar* ultrasoft crystal as well (`alas-epsilon-us.in`, 2.101065 and -2.165827 against `ph.x`'s 2.10106 and -2.16581), so what is left refused is the *strain* leg alone. The one run taken on an ultrasoft cell so far does not lift it: `alas-piezo.in` gave $+0.815929$ against a Berry-phase finite difference's $+0.687757$, but that is the same `4 4 4` mesh at which the norm-conserving calibration cell is itself thirteen per cent out, so it measures the k-convergence above rather than the dataset. What has since been settled is the *internal* half of that question: the two assemblies agree on an augmented dataset to 2.6×10^{-9} C/m^2 where they had been 1.6 per cent apart, so what the refusal is waiting on is a comparison against something outside this code and not a missing term. What the refusal is no longer standing in front of is *untested* code: the PAW half of the wedge completion, which reaches the one-centre terms that no grid integral follows, is now exercised on a committed PAW zincblende cell, where it takes a three-point wedge from 6.96×10^{-4} away from its own closed grid to 1.58×10^{-6}, a factor of 440. Beyond that: insulators, `nspin = 1`, and everything the Sternheimer regime and the differentiable cell refuse (a metal, a spin spiral, a magnetic field, a shifted grid run with `nosym`).

7.3 Spin-polarized response

The Sternheimer equation is solved per spin channel,

$$\left(H_\sigma - \varepsilon_{n\sigma} S + \alpha_{pv} Q_\sigma\right) |\Delta\psi_{n\sigma}\rangle = -P^{+}_{c,\sigma}\, \Delta V_\sigma\, |\psi_{n\sigma}\rangle,$$

with the occupied manifold $P_{c,\sigma}$ built from *that channel's* filling: $N_{\uparrow}^{\mathrm{occ}} = \mathrm{NINT}(n_{\uparrow})$ and $N_{\downarrow}^{\mathrm{occ}} = \mathrm{NINT}(n_{\downarrow})$, which for a magnetic insulator are different numbers. Quantum ESPRESSO never meets this because LSDA doubles `nks` there and each spin channel is a separate k-point.

```
# chi_0 for a spin-polarized run -- the part that is validated unconditionally.
from defuqumat.response.sternheimer import make_sternheimer

solver = make_sternheimer(calculation, result, spin_polarized=True)
drho = solver.chi0(dv)          # dv is (2, n1, n2, n3) on the dense grid
```

Validated against a central difference of the density under a spin-dependent probe potential — 1.8×10^{-6} on an antiferromagnetic hydrogen chain (a smeared metal) and 1.1×10^{-6} on triplet O_2 (the sliced branch, ultrasoft) — and against the identity that a cell with no magnetization gives the same ε_∞ at `nspin = 1` and `nspin = 2`: 6.2×10^{-14} on 13.806646105. The probe must differ between the channels, because χ_0 is block diagonal in spin and one equal in both would not tell the blocks apart.

The *screened* response and the Born effective charges are validated for `nspin = 2` as well, on triplet O_2 against the vendored `ph.x`: $\varepsilon_\infty = \mathrm{diag}(1.11091441, 1.11091441, 1.19799977)$ against $(1.110915996, 1.110915996, 1.198004867)$, and Z^* against the *raw* (no acoustic sum rule) block, 0.13372 and 0.20042 against 0.13367 and 0.20023. Two things had to be right for those and neither was a new derivation. The exchange–correlation kernel is *defined* to be zero where the magnetization reaches the charge — `dmxc_lsda` does not regularise $|\zeta| = 1$, it skips it — so what was missing was that convention, applied to the *argument* the derivative is taken at rather than to its value. And the orthonormality multipliers carry a weight on the column of Λ_{mn} and need a *mask* on its row: the solver keeps $\max(N_\sigma^{\mathrm{occ}})$ bands in every channel because the shape is static, so a magnetic insulator's minority channel carries empty bands whose rows are not removed by the zero weight. That leak is worth a factor of three on Z^*_{xx}, and nothing already validated can see it — the term it feeds is dS/du, which vanishes for a norm-conserving dataset, and the mask is all ones whenever the two channels are filled to the same depth.

Refuses. `tot_magnetization` together with a smearing: the two channels have separate Fermi levels and `Smearing` carries one scalar `ef`. A filling whose boundary lands inside a *degenerate multiplet* — a Hund's-rule atom is exactly this case — where the solve converges and returns an answer 100 % away from a finite difference, because which member falls below the cut is arbitrary; that is the physics, not a missing term. The dynamical matrix, the strain response, the elastic constants, the Raman tensor and the electrostriction tensors for `nspin = 2`: the solve is spin-polarized, their second-derivative assembly is not — *except* the Born effective charges, which are validated above. The *screened* response of a magnetic system with vacuum: for `nspin = 2` the kernel `dv_of_drho` is the second derivative of the LSDA energy in the two channel densities and diverges wherever a channel density reaches zero — measured on triplet O_2, 1504 of 91125 grid points and exactly 1504 NaN — *lifted for the LDA*, which is the paragraph above, and standing for a **GGA**, whose `dgcxc_spin` carries its own density and ζ gates that nothing here has transcribed. And any potential-only meta-GGA (`tb09`, `bj06`), which has no total energy to differentiate — refused by name now rather than surfacing as `v_of_rho` asking for `tau`.

7.4 Magnons: the transverse spin susceptibility

A magnon is the collective precession of a magnet's own magnetization: a spin wave. It lives at the pole of the transverse susceptibility, which obeys a Dyson equation with *no* Coulomb term — a transverse spin fluctuation moves no charge, and the Hartree interaction belongs to the charge channel:

$$\chi^{+-} = \left[1 - \chi_0^{+-} f_{xc}^{+-}\right]^{-1} \chi_0^{+-}, \qquad f_{xc}^{+-}(\mathbf{r}) = \frac{B_{xc}(\mathbf{r})}{m(\mathbf{r})}.$$

The Kohn–Sham part χ_0^{+-} is a sum over pairs with one member in each spin channel, the majority state at $\mathbf{k}$ and the minority at $\mathbf{k}+\mathbf{q}$; its own spectral weight is the **Stoner continuum** of independent spin flips, which costs at least the exchange splitting. The kernel pulls a mode out from underneath that continuum, and that mode is the magnon.

The kernel is not a second derivative and this is why it works for a GGA. Rotate the whole ground state rigidly by an angle θ: then $\delta\mathbf{m}_\perp = \theta\,\mathbf{m}$ and $\delta\mathbf{B}_{xc,\perp} = \theta\,\mathbf{B}_{xc}$, so the transverse response of the exchange–correlation field to the magnetization is B_{xc}/m — an exact consequence of rotational invariance of $E_{xc}[n,|m|]$, whatever the functional. It also means the Goldstone theorem holds *by construction*, so the residual below tests the assembly and the truncations rather than the physics of f_{xc}.

```
import numpy as np
from defuqumat import Calculator

calc = Calculator.from_file("ni-fcc-magnon.in")

# one wavevector: the spectral function, and the Stoner continuum under it
chi = calc.get_spin_susceptibility((0.0, 0.0, 0.0), np.linspace(0.0, 0.06, 9),
                                   nbnd=30, ecut_response=60.0)
chi.goldstone              # how far X_0 B_xc = m is from holding: the error bar
chi.enhancement            # lambda_max(X_0 F) at omega = 0; one at q = 0
chi.magnon                 # the pole, in Ry, or None where there is none
chi.magnon_mev
chi.spectral_function      # -Im chi_00 / pi
chi.kohn_sham_spectral_function
chi.method                 # "crossing", "peak" or "unstable"

# a path: one fixed-density run serves every q
disp = calc.get_magnon_dispersion([(0, x, x) for x in (0.0, 0.25, 0.5)],
                                  np.linspace(0.0, 0.06, 9),
                                  nbnd=30, ecut_response=60.0,
                                  goldstone_correction=True)
disp.energies_mev          # (nq,), nan where the grid holds no pole
disp.enhancements          # > 1 is an instability, not a magnon
disp.kernel_scale
```

The functional entry points are `defuqumat.workflows.run_spin_susceptibility` and `run_magnon_dispersion`.

q must be a difference of two k-points of the run's own grid. That is where the states at $\mathbf{k}+\mathbf{q}$ come from — there is no second diagonalisation — and it is Elk's requirement that `vecql` be commensurate with the mesh, reached by construction. What survives of the $\mathbf{k}+\mathbf{q}$ problem is an umklapp shift of one gather index, and $\chi_0(\mathbf{q}+\mathbf{G}) = \chi_0(\mathbf{q})$ under a relabelling of the matrix's own $\mathbf{G}$ index holds to 1.3×10^{-17}, which is what says that shift is right.

The pole is found as a root, not as a peak. Below the Stoner onset $\lambda_{\max}(\chi_0(\omega)f_{xc})$

is real and rises monotonically, so the magnon is the root of $\lambda_{\max} = 1$ — no broadening enters it, a handful of frequencies suffice, and it reports *below zero* where the state is unstable instead of returning the frequency grid's own edge.

The Goldstone residual is the number to read first. A global spin rotation costs no energy, so $\chi_0(\mathbf{q}{=}0, \omega{=}0)\, B_{xc} = m$ exactly — but only with a complete band set *and* a complete **G**-set, so what `goldstone` reports is a double truncation. **Which axis binds depends on the element.** On hydrogen the band count does it: $8.0 \to 0.50$ per cent over $n_{\text{bnd}} = 12 \to 140$. On fcc nickel the band count does nothing at all (10.02, 10.13, 10.22 per cent at 30, 60, 100 bands) and only `ecut_response` moves it: $10.0 \to 6.1 \to 2.0$ per cent at 12, 30, 60 Ry, because B_{xc} of a 3d shell has structure a small sphere cannot represent.

Against Elk's committed reference, nickel's magnon agrees to 0.8%. Elk ships an `Ni-magnetic-response` example — fcc nickel at $\mathbf{q} = (0.1, 0.1, 0)$ on a 10^3 shifted grid — whose pole sits at 117.92 meV. Its input applies a small magnetic field, which is worth 1.99 meV of Zeeman gap, so its field-free magnon is 115.93 meV against **115.04** here on the same cell, grid and wavevector — and 123.08 with the Goldstone correction applied, which moves it *away*, so neither is chosen for agreeing. That is across an all-electron LAPW basis versus a norm-conserving plane-wave one, and across a 25 Ry response sphere versus 60. The moment agrees on the same run (0.6178 against 0.6152 μ_B), and so does the sign of the static χ. **That 0.7% of kernel is worth 7% of magnon** is the number to take away: it is how steeply $\lambda(\omega)$ crosses one, and so how far the Goldstone residual has to fall before a magnon energy is worth a second digit.

An enhancement above one is an instability, and that is a result. It says the collinear state is not a minimum at that wavevector — the transverse susceptibility has changed sign — so the ferromagnet is not the ground state there. Nickel's enhancement *falls* away from $\mathbf{q} = 0$, which is what a stable ferromagnet looks like. Most simple models do not: a half-filled band on any lattice prefers antiferromagnetic order, so an fcc hydrogen crystal comes out soft at finite **q** at every spacing tried, and the calculation says so rather than producing a plausible dispersion anyway.

Do not compare that against a spiral $E(q)$ without checking what the spiral converged to. It is the obvious cross-check and it does not work: on fcc hydrogen the 90-degree spiral at $q_3 = \frac{1}{2}$ converges *off* the magnetic branch, so its energy gain is demagnetization and not a spin wave; a 15-degree cone collapses the same way, because nothing holds it. (The collapsed $|m|$ is not a number to quote — two runs of the same input give 0.0001 and 0.0435, which is where a wandering SCF stopped rather than a property of the state.)

A constraint does not rescue it on this cell, and the reason is the cell. A spiral SCF accepts `constrained_magnetization` and a magnetic field — neither is refused — and the quantity a constraint acts on is the **rotated-frame** magnetization, which is the right object for a helix. But `'fsm'` does not converge here, and it does not converge at $q = 0$ either, where there is no rotated frame at all: fcc hydrogen at this spacing is a *marginal* magnet, so $m(B)$ is nearly a step — 0.057 Ry of field takes the moment from 0.027 to 0.719 — with very little in between for a constraint to hold. The moment lands saturated, the inner SCF then stops converging under the field that put it there, and since the field steps only on self-consistent pairs it freezes. Two numbers say that cell was the wrong choice before the run: its bare moment is 0.0435 and the target was 0.0273, so the constraint asked for a moment *below* the cell's own while the only other stable point was saturated in the opposite direction. Use a robust magnet, and give `electron_maxstep` room: it is **shared** between the inner SCF and the outer field loop.

What to read when one does not converge. `'fsm'` drives a *feedback field* rather than a penalty, so its `constraint_energy` is 0 by construction and tells you nothing. `SCFResult.constraint_residual`

is $\mathbf{m} - \mathbf{m}_{\text{target}}$, signed and per component, with the same per iteration in `history`; a residual that has changed *sign* is an overshoot rather than a slow approach. And `converged = False` on such a run is often not a density failure — `accuracy` can sit below `conv_thr` while the constraint is unmet, which is a separate warning with its own advice. Elk's own magnon-spiral example runs `fsmtype = -1` with a large field along `momfix` instead: the *direction* fixed hard rather than a small magnitude pinned. That scheme is not implemented here.

`goldstone_correction=True` scales the kernel by $1/\lambda_{\max}(\mathbf{q}{=}0)$, the same factor at every $\mathbf{q}$, so the uniform mode sits at exactly zero. It is what the magnon literature does, it is off by default, and it is honest only if the residual error is $\mathbf{q}$-independent — check that by running the dispersion at two (n_{bnd}, `ecut_response`) settings.

An ultrasoft magnet needs one more term in every matrix element, and without it there is no magnon at all rather than a slightly wrong one. $\langle\psi_{n\mathbf{k}\uparrow}|e^{-i(\mathbf{q}+\mathbf{G})\cdot\mathbf{r}}|\psi_{m\,\mathbf{k}+\mathbf{q}\downarrow}\rangle$ is the plane-wave overlap of the pseudo states only when all of the charge is in $|\psi|^2$; with an ultrasoft dataset what is missing is

$$\sum_a \sum_{ij} Q^a_{ij}(\mathbf{q}+\mathbf{G})\,\langle\psi_{n\mathbf{k}}|\beta^{\mathbf{k}}_i\rangle\langle\beta^{\mathbf{k}+\mathbf{q}}_j|\psi_{m\,\mathbf{k}+\mathbf{q}}\rangle,$$

the augmentation charge at the same wavevector the exponent carries. Nothing is derived for it: $q^a_{ij}(\mathbf{b})$ at an arbitrary wavevector is the primitive the ultrasoft Berry phase already uses, called once per $\mathbf{G}$ of the response sphere (13 ms each, so a sphere of 59 vectors is under a second).

The number is the Goldstone identity on fcc nickel with an ultrasoft dataset: **0.984** without the term against **0.071** with it, at the same sphere and band count. The bare figure is not an independent error bar and the guide will not pretend it is — the identity's $\mathbf{m}$ is read off the dense-grid density, which already carries `addusdens`'s augmentation, so 0.984 is the augmentation's *share of the moment*. What measures the assembly is the augmented number, and it is truncation rather than a missing term: over `ecut_response` of 6, 8, 12, 16, 24 and 48 Ry it reads 0.1063, 0.0706, 0.0992, 0.0987, 0.0854 and **0.0128**. *Not* monotone — the 8 Ry point is the default and sits below the next two, reproducibly — so read the identity at the setting a dispersion is to be run at rather than trusting a trend.

Refuses. `nspin != 2`: without a collinear magnetization the transverse block does not decouple and B_{xc}/m does not exist, and a **noncollinear** ground state needs the full 4×4 spin-density response, which is a different object. **PAW**, and no longer ultrasoft. $\langle u_i|e^{-i(\mathbf{q}+\mathbf{G})\cdot\mathbf{r}}|u_j\rangle$ is not the plane-wave overlap once the charge is not all in $|\psi|^2$, and the missing augmentation charge $Q_{ij}(\mathbf{q}+\mathbf{G})$ is now carried — it is the primitive the ultrasoft Berry phase already uses, called once per $\mathbf{G}$ of the response sphere. What PAW is refused for is a *second* term that is not this one: its exchange-correlation field has a one-centre part on the spheres, where the kernel B_{xc}/m is built from the grid field alone, so a PAW magnon would carry the right matrix element and the wrong enhancement. A **spin spiral**, whose rotating frame is not the one the kernel was derived in. A **Hubbard** U, which responds to the magnetization too. An **applied magnetic field or a constrained moment**, because the rotation argument assumes the energy is invariant under a global spin rotation and a field is exactly what breaks it — a magnon in a field is gapped by the Zeeman energy, which this construction does not carry. A **potential-only meta-GGA**, there being no E_{xc} for that argument to be about. And a **symmetry-reduced k-set**: $\chi_0(\mathbf{G},\mathbf{G}')$ is a matrix in two $\mathbf{G}$ indices that nothing here symmetrises, and the $\mathbf{k}+\mathbf{q}$ fold needs the grid closed under translation by $\mathbf{q}$ in any case.

8 Optical spectra and excitons: TDDFT

The response above is a *static* one, solved orbital by orbital. An absorption spectrum needs the frequency axis and needs the susceptibility as a **matrix** in reciprocal lattice vectors, so this is the one place where a sum over states earns its keep. `run_absorption` builds

$$\chi^0_{\mathbf{G}\mathbf{G}'}(\omega) = \frac{1}{\Omega}\sum_{\mathbf{k}}\sum_{ij}\frac{(f_i - f_j)\,\rho^{ij}_{\mathbf{G}}\,\rho^{ij\,*}_{\mathbf{G}'}}{\epsilon_i - \epsilon_j + \omega + i\eta}, \qquad \rho^{ij}_{\mathbf{G}} = \langle u_i | e^{-i\mathbf{G}\cdot\mathbf{r}} | u_j \rangle$$

and solves the Dyson equation with an exchange-correlation kernel:

$$\epsilon^{-1} = 1 + X\big(1 - X - \tilde{f}_{\mathrm{xc}}X\big)^{-1}, \qquad X = v^{1/2}\chi^0 v^{1/2}, \qquad \tilde{f}_{\mathrm{xc}} = v^{-1/2} f_{\mathrm{xc}} v^{-1/2}$$

The **bootstrap** kernel of Sharma, Dewhurst, Sanna and Gross (*PRL* **107**, 186401 (2011); Elk's `fxctype = 210`) is that equation's other half, so the two are solved together to self-consistency:

$$f^{\mathrm{BS}}_{\mathrm{xc}} = -\frac{\epsilon^{-1}(\mathbf{q},0)\,v(\mathbf{q})}{\epsilon_0(\mathbf{q},0) - 1}, \qquad \epsilon_0 = 1 - v\chi^0$$

It is parameter-free and diverges as $1/q^2$, which is what binds an electron–hole pair; ALDA's kernel is finite at $\mathbf{q} = 0$ where v diverges, so its head and wings vanish *identically* and no amount of it can bind one.

Quantum ESPRESSO has no counterpart. `TDDFPT/` is a Liouville–Lanczos solver with RPA and ALDA; it has no bootstrap kernel and no Dyson equation in **G** space. The reference is Elk.

```python
from defuqumat.workflows import run_absorption
import numpy as np

omega = np.arange(0.0, 0.6, 0.005)          # Ry
spectrum = run_absorption(system, pseudos, scf.density, omega,
                          kernel="bootstrap",   # or rpa / alda / lrc / bootstrap-1
                          nbnd=60, ecut_response=8.0,
                          broadening=0.012, scissor=0.05)

print(spectrum.absorption)                  # Im eps_M, isotropic average
print(spectrum.frequencies_ev)              # the same grid in eV
print(spectrum.alpha, spectrum.iterations)  # the LRC-equivalent head, and the
                                            # bootstrap's fixed-point count
print(spectrum.static_residual)             # how far the band truncation reaches
```

`chi_0` is the whole cost and the kernel is nearly free, so comparing kernels means building it once: `independent_response` returns it and `solve_dyson` screens it, which is what `run_absorption` does internally and what the notebook shows.

Three knobs here are this code's own. `ecut_response` (Elk's `gmaxrf`) selects the **G**-set the matrix is built over and has its own convergence: too small and the local-field effect is missing, and the cost is quadratic in it. `scissor` is *part of the method* rather than a convenience — the paper's Eq. (3) replaces χ^0 by a gap-corrected model response, and the velocity matrix elements are renormalised by $e_{ij}/(e_{ij} \mp \Delta)$ along with it. And `static_residual` is the diagnostic for the one error this phase cannot refuse: how many empty states the sum ran over. It is $\epsilon_M(0)$ from this run, in RPA, minus the same quantity from the Sternheimer solve, which is band-*complete*; it is kernel-matched on purpose, since differencing a bootstrap number against a Sternheimer one that contains f_{xc} would measure the kernel instead.

Two memory dials bound the working set without touching the answer, since the pairs and the frequencies are each independent. `pair_batch` is how many occupied–empty pair densities are formed at once (each is a whole box on the smooth grid): the band dial's value on a CPU, and on a GPU a chunk sized to about 256 MB of boxes rather than every pair. `w_batch` (`DEFUQUMAT_W_BATCH`) is how many frequencies are assembled and screened at once: the whole axis on a CPU, and a chunk sized to about 256 MB on a GPU.

```
spectrum = run_absorption(system, pseudos, scf.density, omega,
                          nbnd=60, ecut_response=8.0, w_batch=16)
```

A trap that is invisible in the answer: the macroscopic dielectric function is the inverse of the 3×3 *head* of ϵ^{-1}, not the head of the inverse of the whole matrix. The second is the microscopic ϵ, whose head in RPA is exactly the no-local-field result; it is smooth, positive, has the right peaks and is 9% too large on silicon. Both are returned (`epsilon` and `epsilon_no_local_fields`), because the gap between them *is* the local-field effect.

Validated by an identity rather than against another code's spectrum: the same $\epsilon_M(0)$ is reached by this sum over states plus a Dyson inversion and by the projected conjugate-gradient solve of `dielectric_tensor`, which shares no machinery with it and never sees an empty state. On silicon the two agree to **1.3e-2** on a constant of 22 — and that residue is the band truncation, which is why it is reported rather than tuned away. The `screening = "hartree"` switch on `dielectric_tensor` exists for this comparison: the identity holds only when both sides use the same kernel.

Refuses. Finite **q** (the perturbed states would live at $\mathbf{k}+\mathbf{q}$); ultrasoft and PAW (every matrix element gains the augmentation charge $Q_{ij}(\mathbf{G})$); metals (the $f_i - f_j$ weight kills the $i = j$ term, so the intraband Drude response is absent entirely — Elk's `tddftlr` does not add it either); `nspin` $\neq 1$, noncollinear magnetism, spin–orbit coupling, spin spirals and a Hubbard U; a symmetry-reduced k-set, because symmetrising $\chi^0_{\mathbf{GG}'}$ rotates *two* **G** indices at once and the rank-n symmetriser is Cartesian; the `alda` kernel with a gradient-corrected functional, since ALDA's kernel is pointwise in the density and a GGA's is not; and **non-convergence of the bootstrap fixed point is an error**, as it is in `tddftlr.f90`, because a spectrum from a kernel still in motion is the spectrum of nothing.

8.1 The conductivity tensor, the Kerr angle and the anomalous Hall effect

The whole complex tensor, interband plus a Drude term:

$$\sigma_{ij}(\omega) = \frac{i}{\Omega} \sum_{\mathbf{k}} \sum_{n,m} \frac{W_n \, (1 - f_m)}{\epsilon_{mn}} \left[\frac{v^i_{nm} v^j_{mn}}{\omega - \epsilon_{mn} + i\eta} + \frac{\overline{v^i_{nm} v^j_{mn}}}{\omega + \epsilon_{mn} + i\eta} \right] + \frac{\omega^2_{p,ij}}{4\pi(\gamma - i\omega)}$$

with $v = \partial H / \partial \mathbf{k}$ from one `jvp` of $H(\mathbf{k})$ at a frozen sphere. *That* is the one thing here not transcribed from `dielectric.f90`, and it is not a refinement: `epsilon.x` accumulates $\langle \psi_1 | \mathbf{G} | \psi_2 \rangle$, and $[H, \mathbf{r}] \neq \mathbf{p}$ when the pseudopotential is nonlocal.

What is new beside `run_absorption` is the **antisymmetric** part, σ_{xy} — the response that rotates light reflected off a magnet, and whose static limit is the intrinsic anomalous Hall conductivity. Time reversal forces it to zero, so a nonmagnetic crystal has none however strong its spin–orbit coupling; a *collinear or coplanar* magnet without spin–orbit coupling has none either, because a spin rotation composed with conjugation survives

as an antiunitary symmetry. (A *noncoplanar* magnet does have one with no spin–orbit coupling at all — the topological Hall effect — so the usual pairing is a route and not a theorem.)

```
calc = Calculator.from_file("ni-soc.in", pseudo_dir="pseudo")

sigma = calc.get_optical_conductivity(nbnd=36, window=0.6, nw=200)
print(sigma.sigma_s_per_cm.shape)   # (nw, 3, 3) in S/cm
print(sigma.kerr)                   # complex Kerr angle, degrees, per frequency
print(sigma.hall_conductivity)      # the static antisymmetric part, S/cm
print(sigma.plasma_ev)              # hbar omega_p, eV, a (3, 3) tensor
print(sigma.dielectric)             # 1 + 4 pi i sigma / omega
print(sigma.fsum, sigma.band_cut_gap)   # two of the three diagnostics
print(sigma.degenerate_pairs)       # the third: pairs the guard dropped, usually 0
```

There is no reference output for any of it, so the validation is internal and analytic. The *symmetric* part reproduces the independent-particle dielectric function of the TDDFT stack to **3.6e-14**, which is a different assembly (a Dyson solve over a response sphere) reaching `ph.x` through `dielectric_tensor`. The **f-sum rule** is the absolute anchor: $\int_0^\infty \mathrm{Re}\,\sigma_{aa}\,d\omega$ must equal $(\pi/2\Omega)\sum W_n\langle n|\partial^2 H/\partial k_a^2|n\rangle$, and the familiar $\pi n_e/2$ is only the *local* Hamiltonian's special case — silicon's diamagnetic weight is measurably **0.943** and not one. Measured against a central difference of the velocity operator, the ratio is 1.258 on a 4^3 grid, 1.045 on 6^3 and **0.990** on 8^3: it converges in the *k-grid*, because what separates the two is $\sum_k w_k\,\partial^2\epsilon_n/\partial k^2$, which vanishes over the zone by periodicity and not on a coarse mesh. Aluminium's plasma frequency comes out at **11.89 eV** against a free-electron 16.27, which is the zone-boundary gaps removing Fermi surface, and its tensor is isotropic to **2.5e-4** eV with nothing imposing that.

Refuses. Ultrasoft and PAW run, and did not until P99: the current operator of a generalised eigenproblem is $\langle n|\partial_a H - \varepsilon_m\partial_a S|m\rangle + (\varepsilon_m - \varepsilon_n)K^a_{nm}$ and the last piece — the augmentation dipole — is neither tangent of the `jvp` that gives the first two. It is the Kubo curvature's own term and its numbers are there; what this module adds is that the two assemblies are the *same* object, agreeing to **5.1e-15** on norm-conserving and on ultrasoft AlAs, which is what says an index order transposed in either would have shown. A **spin spiral**, whose two spinor components live on different spheres. A **symmetry-reduced k-set**, because the antisymmetric part is an axial vector — the escape is the whole unshifted grid, which is closed under the point group. And **`nspin` = 2**, whose two channels are two band structures whose conductivities add; a spinor run is the regime a magneto-optical spectrum wants anyway, since σ_{xy} needs spin–orbit coupling and collinear spin has none.

Three things are reported rather than refused, and all three must be read. `band_cut_gap` is the gap between the last band summed and the first dropped: stopping *inside* a degenerate multiplet keeps some members and drops others, and silicon's antisymmetric σ — zero by time reversal — comes out at 4.0e-13 when the cut falls at a real gap and at 1.0e-5 when it does not. `degenerate_pairs` counts the occupied/empty pairs that came back closer together than `degeneracy_tol` (1e-5 Ry, what an SCF converges an *empty* band to) and were therefore dropped. Zero is the ordinary case. When it is not zero it usually still does not matter — on the smeared frequency route the two orderings of such a pair cancel analytically, because what survives the $1/\epsilon_{mn}$ is $w_k[f(\epsilon_n)-f(\epsilon_m)]$ and that is itself linear in the gap — and the code warns exactly where that cancellation fails: `method = "curvature"`, whose $1/\epsilon_{mn}^2$ leaves a $1/g$ divergence, and a fixed-occupation run, where f is not a function of energy at all. And the **anomalous**

Hall conductivity of a metal is mesh-hungry: the static Berry-curvature integral is concentrated on near-degeneracies at the Fermi surface, so it is quoted with its k-convergence and not on its own.

8.2 The shift current: the bulk photovoltaic effect

Illuminate a crystal with no inversion centre and it carries a direct current, with no junction and no built-in field. The intrinsic part of that is the *shift current*: the photoexcited electron is born displaced, and what the current counts is the real-space shift between the valence and conduction Wannier centres. It is a second-order optical response, $J^a = 2\sigma^{abc}(0;\omega,-\omega)E_b(\omega)E_c(-\omega)$, with

$$\sigma^{abc}(\omega) = -\frac{i\pi e^3}{4\hbar^2}\int [d\mathbf{k}] \sum_{n,m} f_{nm}\left(I^{abc}_{mn} + I^{acb}_{mn}\right)\left[\delta(\omega_{mn}-\omega)+\delta(\omega_{nm}-\omega)\right], \qquad I^{abc}_{mn} = r^b_{mn}\, r^{c;a}_{nm}.$$

The whole difficulty is $r^{c;a}_{nm}$, the *generalised derivative* $\partial_a r^c_{nm} - i(A^a_{nn} - A^a_{mm})r^c_{nm}$ — neither piece of which is separately computable, since the diagonal Berry connection is gauge dependent and is not a function of H at one $\mathbf{k}$. The escape is the Aversa–Sipe sum rule, which writes the covariant object entirely in gauge-free quantities living at a single $\mathbf{k}$:

$$r^{c;a}_{nm} = \frac{i}{\omega_{nm}}\left[\frac{v^c_{nm}\Delta^a_{nm} + v^a_{nm}\Delta^c_{nm}}{\omega_{nm}} - w^{ca}_{nm} + \sum_{p\neq n,m}\left(\frac{v^c_{np}v^a_{pm}}{\omega_{pm}} - \frac{v^a_{np}v^c_{pm}}{\omega_{np}}\right)\right],$$

with $v^a_{nm} = \langle n|\partial_a H|m\rangle$ and $w^{ab}_{nm} = \langle n|\partial_a\partial_b H|m\rangle$. Both are derivatives of code that already exists: v is one `jvp` of $H(\mathbf{k})$ at a frozen sphere and w is that `jvp` differentiated by a second one. Nothing is transcribed. The interband dipole $r^a_{nm} = -i\, v^a_{nm}/\omega_{nm}$ is *exact* for an eigenstate of the full $H(\mathbf{k})$, which is why a plane-wave code needs no counterpart to Wannier90's position-operator matrices.

```
calc = Calculator.from_file("alas-raman.in", pseudo_dir="pseudo")

sc = calc.get_shift_current(nbnd=14, window=0.9, nw=180)
print(sc.sigma.shape)            # (nw, 3, 3, 3) in A/V^2, real
print(sc.component(0, 1, 2))     # sigma^xyz(omega), -43m's only free component
print(sc.frequencies_ev)         # the photon energy axis in eV
print(sc.truncation)             # the band-truncation diagnostic: read it
print(sc.band_cut_gap)           # where the band set was cut, in Ry
```

There is no reference binary — `pw.x` computes no photocurrent of any kind, and Elk's `nonlinopt.f90` is second-harmonic generation, a different response — so the validation is internal and it is three statements. The **sum rule against a parallel-transport finite difference** of the dipole, which shares nothing with it: no second derivative, no intermediate sum, only the phases of neighbouring $\mathbf{k}$-points fixed to one another. That comparison is decisive in exactly one form, because the sum rule is an *identity over a complete basis*: held on a frozen sphere of 158 plane waves, AlAs agrees to **2.6e-3** at 158 bands against 0.52 at 120 and 0.69 at 20. A residue that falls off a cliff at completeness is a correct assembly with a truncated sum; one that plateaus is a bug. Then the **crystal class**: AlAs comes out exactly $\bar{4}3m$ on a `nosym` grid, only the components with all three labels different surviving, while centrosymmetric silicon gives zero from the same machinery. And the **absorption edge**: σ^{xyz} is 4.3e-24 A/V^2 at 2 eV against a peak of 1.1e-4 near 4.4 eV,

which is the one check no index bookkeeping can pass by accident — no absorption, no photocurrent.

All three are blind to an overall constant, and this document says so rather than leaving it to be discovered: a σ wrong by a factor of two is still exactly $\bar{4}3m$, still zero on silicon and still zero below the gap. Two further checks close that gap. The **spin sum** — the same cell run `nspin` = 1 and as a two-component spinor gives the same tensor to **4.5e-9**, peak ratio 1.000000. And the **overall scale against the published literature**, where the *ordering* is what makes the comparison sharp rather than merely consistent. AlAs's first peak converges to **35.1** μA/V^2 at 4.17 eV (33.9 at `ecutwfc` = 16, then 33.7, 34.9 and 35.1 at 22 with 14, 22 and 30 bands), where calculations across the fourteen III–V and II–VI zincblende semiconductors span 14 μA/V^2 (CdSe, smallest) to 83 (AlSb, largest) and find the aluminium compounds the strongest of the family. A factor of two either way breaks that ordering rather than moving the number: 17 would put AlAs at the bottom of the family, below the cadmium chalcogenides, and 70 would put it beside AlSb, the heaviest and narrowest-gap member. The number to compare is the peak below about 6 eV, which is the range published spectra cover. That sweep also showed `truncation` to be *predictive* rather than merely indicative: it reads 3.5% at 14 bands, where the value is 4.0% from its 30-band one, and 1.0% and 1.3% at 22 and 30.

The convention is declared, because the literature has two. What is implemented is $j^a = 2\sigma^{abc}(0;\omega,-\omega)\,\mathrm{Re}[\mathcal{E}_b(\omega)\mathcal{E}_c(-\omega)]$, the definition of the paper the formulae above come from. Some authors write $J = \sigma\mathcal{E}\mathcal{E}$ and report numbers twice as large for the same physics, so a comparison against a paper that does not state its normalisation is worth nothing to better than a factor of two.

Refuses. Ultrasoft and PAW, one order further out than the Kubo curvature, the optical conductivity and second-harmonic generation, none of which refuse them any more. The dipole carries $\partial S/\partial k$, its derivative carries $\partial^2 S/\partial k_a \partial k_b$, and the augmentation dipole $\langle\psi_n|T^\dagger\partial_k T|\psi_m\rangle$ that belongs with them is here too. What is not here is *that term's own k-derivative*, which is built from $\langle\beta|\psi\rangle$ and $\partial\beta/\partial k_a$ and so wants $\partial^2\beta/\partial k_a\partial k_b$ — a second derivative of a bare projector, which nothing in this package forms. A **spin spiral**. A **symmetry-reduced k-set**, because σ^{abc} is a polar rank-3 tensor and a wedge sum is not the cell's until it is averaged over the point group; the escape is the whole unshifted grid. A **metal**, whose partially filled bands contribute pairs with vanishing ω_{nm} where every denominator above diverges — the Fermi-surface terms that replace them are a different assembly. **DFT+U**, whose Hubbard term is built from $\phi_U(\mathbf{k}+\mathbf{G})$ and so carries a velocity and a second velocity of its own. And **`nspin` = 2**. A *spinor* run is supported and tested.

The band count is reported rather than refused, and it is the largest approximation here. The intermediate sum over p converges only as it approaches completeness, so a production band count carries a few per cent that no symmetry check sees — `truncation` is the in-run estimate and is taken over the top *quarter* of the bands, because dropping one band reports 5.3e-5 where the error is 4 per cent. The route that would remove it is written down in `notes/NONLINEAR.md`: the two sums are a Sternheimer solve rather than a spectral sum, and what stops it being taken is that the operator to invert is indefinite. `band_cut_gap` carries the same warning it does for the conductivity.

8.3 Second-harmonic generation

Shine light of frequency ω on a crystal with no inversion centre and some of it comes back out at 2ω. The tensor that says how much is the second-order optical susceptibility $P^a(2\omega) = \chi^{abc}(-2\omega;\omega,\omega)\,E_b(\omega)E_c(\omega)$, a polar rank-3 tensor, symmetric in its last two labels and identically zero in every centrosymmetric crystal. It is a sum of three physically distinct pieces — the pure interband term χ_{II}, the intraband modulation of it η_{II}, and the modulation of the transition energy $\frac{i}{2\omega}\sigma_{II}$ — each of which is a contraction of the interband dipoles $r^a_{nm} = -i\,v^a_{nm}/\omega_{nm}$ over two resonant denominators, one at ω and one at 2ω:

$$\chi^{abc}(-2\omega;\omega,\omega) = \frac{1}{\Omega}\sum_{\mathbf{k},m,n}\left[\frac{c^{(1)}_{mn}}{\omega_{mn}-\omega+i\eta} + \frac{c^{(2)}_{mn}}{\omega_{mn}-2(\omega-i\eta)}\right],$$

where the coefficients $c^{(1,2)}_{mn}$ carry a further sum over an intermediate state and are built once, before the frequency axis is touched. This is the same sum over states the shift current uses, contracted differently; it needs no second derivative of H at all, because that triple sum *is* the sum-rule expansion of the generalised derivative written out rather than collapsed.

```
calc = Calculator.from_file("alas-shg.in", pseudo_dir="pseudo")

shg = calc.get_shg(nbnd=23, window=0.6, nw=240, broadening=0.010)
print(shg.chi.shape)                  # (240, 3, 3, 3), complex, in pm/V
print(shg.component(0, 1, 2)[0])       # chi^xyz at the lowest frequency
print(shg.d_coefficient(0, 1, 2)[0])   # d = chi/2, what the literature quotes
print(shg.frequencies_ev[:3])          # the FUNDAMENTAL photon energy, in eV
print(shg.truncation)                  # the band-truncation diagnostic
print(shg.chi_ii, shg.eta_ii, shg.sigma_ii)   # the three parts, kept apart
```

There is a real reference here, which is unusual in this corner of the package: Elk's `nonlinopt.f90` (task 125) computes the same tensor, and `tests/data/elk/` carries its output for the same AlAs crystal, the three parts separately as well as their sum. `pw.x` has none — its `el_opt.f90` is the *static* electro-optic response, and the branch that reaches even that is the one this document's Raman section records as broken in the vendored 7.5.

What the comparison is *not* is digit-for-digit, and saying so is part of quoting it: an all-electron LAPW basis is not a norm-conserving pseudopotential one, and a second-order susceptibility carries two energy denominators, so a gap difference is not a scale factor. On the same crystal and the same 6^3 mesh, at 23 bands where Elk ran 37 states with Al 2p and As 3d among them: the **resonance position** agrees to **0.5%** (2.152 eV against 2.163), the **peak height** to **0.8%** (25.90 a.u. against 25.70), and the **static value** to **8%** (-3.21 a.u. against -3.50). With `origin_tangent=False`, `ph.x`'s zero at $\mathbf{k}+\mathbf{G}=0$ described in the response section, the height reads 7% and the static value 11%, so Γ's row of $\mathrm{d}H/\mathrm{d}\mathbf{k}$ is worth 6% of this tensor on a 6^3 mesh. The static value is the one that moves with the basis, and it was shown converged on that convention: `ecutwfc` 10, 30 and 45 give -2.71, -3.10 and -3.13, so the residue is the pseudopotential rather than the cutoff.

Four internal checks close what the reference cannot. $\bar{4}3m$: AlAs comes out exactly zincblende on a `nosym` grid with nothing imposing it. **Silicon**: an inversion centre forbids every component, and the centrosymmetric control runs through the same machinery. **The two-photon edge**: $\mathrm{Im}\,\chi$ turns on at *half* the smallest direct gap, not at the gap, because the second channel's denominator is $\omega_{mn}-2\omega$ — the only check here blind to both the scale and the symmetry, and a lost factor of two there moves the absorption edge

by an octave. And **the spinor**, which is the scale's own anchor: the same cell run `nspin = 1` and as a two-component spinor must give the same tensor, which is what catches a factor of two in the spin sum.

The convention is declared, because the literature has two. What is computed is $\chi^{(2)}$, defined by $P^a = \chi^{abc} E_b E_c$; nonlinear-optics papers more often quote $d^{abc} = \chi^{abc}/2$, so a number compared against a published d without halving it is wrong by exactly two and nothing about its shape, symmetry or zeros would say so. `d_coefficient` returns the halved form.

The scissors shift is the one knob worth explaining. An LDA or GGA gap error does not scale a second-order susceptibility, it moves its resonances, and `scissor` applies the usual rigid shift to the empty states. The dipoles are built from the *unshifted* gaps — a scissors correction is a statement about energies, not about wavefunctions — which is what Elk reaches by scaling its momentum matrix elements instead. Validated against Elk's own scissored run: at $\Delta = 0.05$ Ha the 2ω peak moves **0.0502** Ry against a half-scissor of 0.0500, and its height falls to **0.62** of the unscissored value against Elk's 0.58.

Refuses. Four, and they are what a *sum over velocity matrix elements* refuses rather than what the shift current does: a **spin spiral**, a **symmetry-reduced k-set** (a polar rank-3 tensor is not the cell's until averaged over the point group; the escape is the whole unshifted grid), a **metal**, **DFT+U**, and **`nspin = 2`**. A *spinor* run is supported and tested.

Ultrasoft and PAW are not among them, and were until P99 because this module inherited the shift current's guard whole. That refusal belongs to the generalised derivative $r^{c;a}$, which the shift current differentiates and this assembly does not: the triple sum over l above *is* that derivative's sum-rule expansion written out, so what an augmented dataset needs here is the velocity matrix element and nothing beyond it. That element is $\langle n|\partial_a H - \varepsilon_m \partial_a S|m\rangle + (\varepsilon_m - \varepsilon_n) K^a_{nm}$, whose last piece is the augmentation dipole of the Kubo section, and deleting it moves $|\chi^{(2)}|$ on ultrasoft AlAs by **34.1** pm/V on a peak of **1578.3** at 23 bands — 2.2 %, where the same deletion is worth 0.5 % in the linear conductivity, because $\chi^{(2)}$ carries the velocity matrix element three times rather than twice.

This does not lift the `chi^(2)` refusal in the Raman section, and the distinction matters. That one is about the $2n+1$ route through the Sternheimer solver, where a field enters only through a source term and the $\langle u_i|r_k|u_j\rangle$ piece has nothing to build it from; a sum over states never needed that object. The electro-optic tensor and a truncation-free static $\chi^{(2)}$ are still refused for the same missing term.

The band count is reported rather than refused, exactly as for the shift current and for the same reason: the sum over the intermediate state is an identity only over a complete basis. `truncation` is the in-run estimate, taken over the top quarter of the bands.

Two warnings, here and in the shift current, mark a part of the tensor the crystal does not have: a `nosym` run whose dense FFT grid is not a multiple of the factors the fractional translations need breaks the dropped operations at the grid's sampling error (0.72 pm/V on two-atom silicon, where inversion forbids every component, against 0.00074 with symmetry kept for the SCF and the whole unshifted mesh passed as `kpoints`), and a `band_cut_gap` below 10^{-8} Ry means the band set was cut inside a degenerate multiplet, so which of its members were kept is the eigensolver's arbitrary choice. On AlAs 22 bands do that, on the 4^3 grid the snippet above runs on as on the 6^3 mesh (at 13 of its 216 points), and 23 do not, which is why the snippet asks for 23.

9 Real-time propagation: the current under a pulse, high harmonics and the third harmonic

What does a crystal do when an intense laser pulse goes through it? The simplest way to find out is to let the occupied Kohn-Sham states evolve in time under the field and read off the current they carry, which is what this section does, and two quantities come out of the same engine: the high-harmonic spectrum, which is a non-perturbative response and can only come from a propagation, and the third-harmonic susceptibility, which is the weak-field limit of the lowest odd harmonic above the fundamental and comes out as a derivative of the propagation with respect to the field strength.

We take a uniform field $\mathbf{E}(t)$ in the velocity gauge, a vector potential with $\mathbf{E} = -(1/c)\,\partial_t\mathbf{A}$, and carry it as the shift of crystal momentum it is,

$$\boldsymbol{\kappa}(t) = \mathbf{A}(t)/c, \qquad \partial_t\boldsymbol{\kappa} = -\mathbf{E}(t),$$

in 1/bohr and Hartree atomic units of time. The periodic part of each occupied state then evolves as

$$i\,\partial_t|u_{n\mathbf{k}}(t)\rangle = H\big(\mathbf{k}+\boldsymbol{\kappa}(t)\big)\,|u_{n\mathbf{k}}(t)\rangle,$$

meaning that the Hamiltonian at time t is the ordinary Bloch Hamiltonian at a displaced k-point, on the plane-wave sphere built for $\mathbf{k}$ and never rebuilt. For a norm-conserving pseudopotential that is the whole coupling to the field: the kinetic energy becomes $|\mathbf{k}+\mathbf{G}+\boldsymbol{\kappa}|^2$ and the projectors are evaluated at $\mathbf{k}+\mathbf{G}+\boldsymbol{\kappa}$, since the gauge factor $e^{-i\boldsymbol{\kappa}\cdot\mathbf{R}}$ cancels between the two projectors of one atom, so nothing is expanded in the field and no empty state is needed. The current is the derivative of the band energy with respect to $\boldsymbol{\kappa}$ at frozen states,

$$\mathbf{J}(t) = -\frac{1}{\Omega}\sum_{n\mathbf{k}} w_{n\mathbf{k}}\left\langle u_{n\mathbf{k}}(t)\right|\frac{\partial H}{\partial\mathbf{k}}\Big|_{\mathbf{k}+\boldsymbol{\kappa}(t)}\left|u_{n\mathbf{k}}(t)\right\rangle,$$

which already contains the diamagnetic current (the derivative of $|\mathbf{k}+\mathbf{G}+\boldsymbol{\kappa}|^2$) and the commutator of the position with the nonlocal potential (the derivative of the projectors), so nothing is added to it by hand: a code that adds $-(1/c)\mathbf{A}N$ to a current computed this way counts the diamagnetic term twice.

The potential is the ground state's and is held fixed, which is the independent-particle response. It is what the published silicon calculation found sufficient for its conditions (arXiv:1609.09298, for one material, the LDA and one weak mid-infrared pulse), and it is what makes the k-points independent, so a chunk of them is propagated through the whole pulse at a time and the peak memory is one chunk whatever the mesh. The potential can be updated in time instead, with `potential="hxc"`; its own subsection, at the end of this section, says what that changes and what it costs.

The step is the fourth-order Taylor expansion of $\exp[-iH(t+\Delta t/2)\,\Delta t]$, four Hamiltonian applications, which is stable for $|\Delta t\,(\epsilon - c)| \le 2\sqrt{2}$ for every eigenvalue ϵ, with c the energy the step subtracts. c is the carried states' mean energy over every k-point and is never moved: a component at ϵ loses $(\Delta t(\epsilon-c))^6/144$ of its norm a step, so a centre pushed off the bands to keep a long step stable damps the very states the current is made of, which on AlAs at 12 Ry took the first-order current from 9.5×10^{-9} to 2×10^{-20} in 800 steps of 0.305 that a half-width criterion accepted. So the run bounds the spectrum before it starts (the top by Weyl's inequality on the kinetic, local and nonlocal terms; the bottom from the carried states, since the local potential's minimum puts Weyl's bottom ten times too low), refuses a step for which the farther edge is out of reach of c, warns when the carried bands will lose more than 10^{-4} of their norm over the run, and stops if a norm grows, since a Taylor instability grows exponentially and reads as physics for the first thousand

steps. `dt` is in Hartree atomic units of time, 24.189 as, as Elk's `dtimes` is; the published silicon runs use 0.08 to 0.1. The projectors at $\mathbf{k}+\boldsymbol{\kappa}$ come from a Chebyshev table of the radial functions in q^2 built once per dataset, within 10^{-13} of the radial transform, because rebuilding them by the transform at every step cost five times a Hamiltonian application.

The pulses are Elk's three shapes, so that an Elk input reads the same here (`Gaussian`: a Gaussian-enveloped sine with a phase and a chirp, `genafieldt.f90`'s; `Ramp`, a polynomial ramp; `Kick`, a step in $\mathbf{A}$), the $\sin^2$ envelope of the high-harmonic literature (`Sin2`), and the adiabatic switch-on $e^{\eta t}\cos\omega t$ (`Adiabatic`) the perturbative orders use, all importable as `from defuqumat.realtime import Sin2`. A peak intensity in W/cm^2 becomes $E_0=\sqrt{I/3.51\times10^{16}}$ atomic units, which is what Elk prints as the peak power density. **The sign of the coupling is the electron's and is not Elk's**: here an electron of charge -1 sees $H(\mathbf{k}+\mathbf{A}/c)$, where Elk adds $-\mathbf{A}\cdot\mathbf{p}/c$ and writes $\sum\langle\mathbf{p}\rangle-\mathbf{A}N/c$ with no charge; for the same $\mathbf{A}$ the conductivity, the dielectric function and the odd orders agree in sign with Elk's `JTOT_TD`/Ω, and the even orders are opposite.

```python
from defuqumat import Calculator
from defuqumat.realtime.pulse import Sin2

calc = Calculator.from_file("si2-symmetric.in", pseudo_dir="pseudo")
calc.get_scf(conv_thr=1e-10)

pulse = Sin2.from_intensity(5e11, 1.55, 2, polarization=(1, 0, 0))
run = calc.get_realtime(pulse, grid=(4, 4, 4), dt=0.2)
print(run.times.shape, run.current.shape)   # Hartree a.u.; J is (nt, 3)
print(run.symmetry_operations)              # 8: the little group of a [100] field
print(run.work, run.energy_gained)          # Omega int J.E dt against E(T) - E(0)
print(run.norm_drift, run.excited)          # electrons per cell left across the gap
```

Where each piece is. The five `Calculator` methods of this section delegate to `run_realtime`, `run_realtime_dielectric`, `run_hhg`, `run_harmonic_orders` and `run_third_harmonic` (`defuqumat.workflows.realtime`, which also has `field_little_group`, `chi3_from_orders`, `chi2_from_orders` and `CHI3_AU_TO_SI`), and those to `propagate` and `propagate_orders` (`defuqumat.realtime`), which take a `Calculation`, the states, their weights and the potential to freeze, and return a `RealTimeResult` or an `OrdersResult` (`fourier_component` projects an order on a harmonic). `conductivity_from_kick` and `harmonic_spectrum` turn a current into a `KickResponse` or a `HarmonicSpectrum`. The pulses are behind `pulse.get_pulse` and `register_pulse`, the steps behind `propagators.get_propagator` and `register_propagator` (`taylor4`, with `largest_stable_step` saying how long a step the cell admits), the projectors' table is `radial_table`, and the dense frequency-domain hierarchy the orders are checked against is `dense_orders`.

The k-set is a whole unshifted grid, because the current is a polar vector and time reversal does not hold under a field (the state at $-\mathbf{k}$ evolves under $-\boldsymbol{\kappa}$). What reduces it is the **little group of the field**, the crystal's operations that leave every polarisation of the pulse invariant, with time reversal off, as Elk's `tdinit.f90` reduces it: each point of the wedge stands for its star and the current is averaged over the group as a polar vector. A field along $[100]$ in silicon keeps 8 of the 48 operations and 18 of the 64 points of a 4^3 grid, and the wedge gives the whole grid's current to 6×10^{-11} on a scale of 8.8×10^{-4}, the fixed-density solve's own threshold, in 19 s against 63.

Two checks hold for any run and are what the result carries: **with no field the states are stationary** (the band energy constant to 2×10^{-10} Ha over 200 steps from exact eigenstates, the norm to $(\Delta t\,\Delta\epsilon)^6/144$ a step), and **the work the field does is the energy it puts in**, $\Omega\int\mathbf{J}\cdot\mathbf{E}\,dt=\Delta E$, to 2.4×10^{-6} on a two-cycle pulse. The second is blind to the

time unit and to any constant on **J** or **E**; what it sees is a current that is not the derivative of the Hamiltonian the step applies and a step that is not unitary. The scale is pinned by the comparisons below.

9.1 The dielectric function from a kick

A step in $\boldsymbol{\kappa}$ at $t = 0$ is a field $\mathbf{E}(t) = -\kappa_0 \delta(t)$, the same at every frequency, so the current after it is the linear response at all frequencies at once,

$$\sigma_{ab}(z) = -\frac{1}{\kappa_0} \int_0^\infty J_a(t)\, e^{izt}\, dt, \qquad z = \omega + i\eta, \qquad \epsilon_{ab}(z) = \delta_{ab} + \frac{4\pi i\, \sigma_{ab}(z)}{z},$$

which is Elk's task 481. The window $e^{-\eta t}$ is the imaginary part of z, so the transform is exactly the response at the complex frequency, prefactor included. `get_realtime_dielectric` takes the *exact* first order in the kick by default, one `jvp` through the propagation, so no kick strength has to be chosen small; `strength=` runs a finite kick as Elk does.

```
eps = calc.get_realtime_dielectric(direction=(1, 0, 0), broadening=0.5,
                                   window=10.0, nw=200, grid=(4, 4, 4), dt=0.2,
                                   subtract_static=True)
print(eps.frequencies_ev[:3], eps.epsilon[:3, 0])   # eps_xx(w + i eta)
```

It is an identity with the Kubo sum, up to one term that a coarse mesh makes large. The propagation carries the exact diamagnetic term, $\sum w \langle \partial^2 H / \partial k^2 \rangle$, where the interband `get_optical_conductivity` replaces it by its f-sum value. On a finite set of k-points the two differ by the mesh sum of the band curvature,

$$\sigma^{\mathrm{RT}}(z) = \sigma^{\mathrm{Kubo}}(z) + \frac{iD}{\Omega z}, \qquad D = \sum_{n\mathbf{k}} w_{n\mathbf{k}} \frac{\partial^2 \epsilon_{n\mathbf{k}}}{\partial k^2},$$

which is exponentially small in the mesh density for an insulator and not small on a coarse one: a spurious Drude term. Measured on two-atom silicon at 6 Ry on two k-points, against the Kubo resolvent sum fed every eigenstate of the dense $H(\mathbf{k})$ and D from a difference of the dense eigenvalues: the two agree to 1.6×10^{-5} of the scale from 0.5 to 16 eV with the term, and differ by the whole scale, 0.81 against 0.81, without it. On the 4^3 grid of the snippet the term makes ϵ_{xx} at $\omega \to 0$ read -278 with $\eta = 0.5$ eV, since it grows as $1/\eta^2$ there. `subtract_static=True` removes the constant part of **J** first, as Elk's `jtconst0` does, and on that grid it brings the kick to the interband `get_optical_conductivity` on the same mesh to 0.1 to 0.3 per cent from 2 to 6 eV ($32.87 + 10.59i$ against $32.83 + 10.61i$ at 2 eV) and to 3 per cent at $\omega \to 0$ (22.59 against 23.27), where the time average of the oscillating current over a finite run is not exactly zero. Both read 23 because a 4^3 grid is far from converged for silicon. On a converged mesh the term, and the subtraction, are not needed.

Against Elk, on its own `Si-dielectric` example at scissor 0 (the same lattice, grid and field, the 100 points of the field's wedge, 4000 steps of 0.2), both currents through one transcription of Elk's task 481: the static ϵ_{xx} is 15.11 here against Elk's independent-particle task 121 at 15.19, while Elk's own real-time route reads -4.13 at $\omega \to 0$, the low-frequency failure of a propagation in a truncated band basis that the full sphere does not have. The first two absorption peaks are at 3.57 and 3.67 eV here and 3.50 and 3.64 in Elk's real-time spectrum, whose potential is updated in time (local fields and the ALDA) where this one is frozen. On one core each, Elk's propagation took 1300 s and this one 486 s.

9.2 The high-harmonic spectrum

Driven hard enough, a solid emits at multiples of the drive's frequency, and the spectrum $\omega^2|\mathbf{J}(\omega)|^2$ of the current is what is measured: a plateau of nearly equal odd harmonics up to a cutoff that grows with the field, then a fall. With an inversion centre only the odd orders appear; without one, in zincblende AlAs, the even ones do too.

```
from defuqumat.realtime.pulse import Sin2

pulse = Sin2.from_intensity(1e12, 1.55, 10, polarization=(1, 0, 0))
hhg = calc.get_hhg(pulse, grid=(6, 6, 6), dt=0.2, highest=25)
print(hhg.harmonics(15))          # the peak intensity at orders 1 to 15
print(hhg.cutoff(floor=1e-6))     # the end of the unbroken odd run above 1e-6
print(hhg.realtime.excited)       # electrons per cell promoted by the pulse
```

Notebook 50 runs silicon and AlAs under the same eight-cycle 800 nm pulse at 3×10^{12} W/cm^2 along $[111]$, which keeps the same six operations in both crystals: silicon's comb is odd to the 19th harmonic (29 eV) at a floor of 10^{-6} of the fundamental, its 4th harmonic 0.0095 of its 5th, where AlAs's is 0.26 and its 8th 0.35 of its 7th. The direction is not a free choice: what forbids an even harmonic is any operation that reverses the field, and along $[100]$ zincblende's two-fold axis about $[010]$ does, so AlAs along $[100]$ is odd-only (its 4th, 6th and 8th harmonics 1.5×10^{-2}, 7×10^{-4} and 9×10^{-5} of the fundamental, against 0.45, 0.12 and 0.033 along $[111]$). The plateau's end moves with the field (9, 15, 19, 27 and 31 from 1 to 5×10^{12} W/cm^2 on silicon's 4^3 grid) and with the basis (the 19th at 12 Ry, the 23rd at 20).

Two properties of a solid's spectrum belong here rather than in a surprise. The spectrum of one cell without dephasing is noisy above the gap and converges slowly in k, and published work either adds a dephasing time of a few femtoseconds or propagates the pulse through the sample (arXiv:1705.10707); what is offered is the window (`window='hann'` by default, since a current cut off while it oscillates puts a sinc on every harmonic and hides the weak ones) and the mesh. And a velocity-gauge propagation on a fixed mesh has no structure-gauge problem, since each k-point evolves alone and no k-derivative of a Bloch state appears.

The frozen sphere under a large shift. The sphere holds the plane waves with $|\mathbf{k}+\mathbf{G}|^2 \le E_{\mathrm{cut}}$ while the state at time t lives at $\mathbf{k}+\mathbf{G}+\boldsymbol{\kappa}$, so the cutoff along the field is $(\sqrt{E_{\mathrm{cut}}}-\kappa_{\max})^2$: 11.2 Ry instead of 12 for the published silicon pulse. The result carries it as `effective_cutoff`. Measured on two-atom silicon at 30 Ry, the occupied eigenvalues of $H(\mathbf{k}+\boldsymbol{\kappa})$ on the sphere of $\mathbf{k}$ differ from those on a sphere rebuilt at $\mathbf{k}+\boldsymbol{\kappa}$ by at most 1.8×10^{-5}, 3.4×10^{-4} and 2.9×10^{-4} eV at κ = 0.1, 0.3 and 0.5 bohr^{-1}; at 12 Ry by 10^{-2} eV, which is that cutoff's own error.

9.3 The perturbative orders and the third harmonic

Scale the field, $\boldsymbol{\kappa}(t) = \lambda\,\mathbf{a}(t)$, and expand the current, $\mathbf{J} = \sum_n \lambda^n \mathbf{J}^{(n)}$. Each order is a derivative of the propagation with respect to λ at $\lambda = 0$, and it is taken as one, by nested forward differentiation through every step rather than by fitting runs at several amplitudes, so the orders come out separated exactly. With the adiabatic switch-on $\mathbf{a}(t) = e^{\eta t}\cos(\omega t)\,\hat{\mathbf{e}}$ on $[-T, 0]$ the response of order n is $e^{n\eta t}$ times a periodic function,

$$\mathbf{J}^{(n)}(t) = e^{n\eta t} \sum_m \mathbf{J}_{(n,m)}\, e^{-im\omega t},$$

whose coefficients are the response functions at the complex frequencies $m\omega+in\eta$, so that η is a broadening per photon, the same parameter a sum over states calls its broadening, and a unitary evolution needs no dephasing invented for it. The third harmonic is $\mathbf{J}_{(3,3)}$, the intensity-dependent index $\mathbf{J}_{(3,1)}$, the second harmonic $\mathbf{J}_{(2,2)}$ and the optical rectification $\mathbf{J}_{(2,0)}$. With $E(z)=i\lambda z/2$ the field's amplitude at z and $P=iJ/\Omega_s$ at the output frequency,

$$\chi^{(3)}(-3\omega;\omega,\omega,\omega)=-\frac{8\,J_{(3,3)}}{3z^4},\qquad \chi^{(3)}(-\omega;\omega,\omega,-\omega)=\frac{8\,J_{(3,1)}}{3(2z-\bar{z})\,z^2\bar{z}},$$

in atomic units, $4\pi/E_{\text{au}}^2=4.75\times10^{-23}$ m^2/V^2 each.

```
orders = calc.get_harmonic_orders(frequency=1.55, broadening=0.3, order=3,
                                  grid=(2, 2, 2), eta_t=10)
print(orders.component(3, 3, axis=0))      # J_(3,3) along x, Hartree a.u.
thg = calc.get_third_harmonic(frequency=1.55, broadening=0.3, grid=(2, 2, 2),
                              eta_t=10)
print(thg.chi_xxxx_3w, thg.chi_xxyy_3w)    # m^2/V^2
print(thg.chi_xxxx_w)                      # chi(-w; w, w, -w)
```

The check is a second route to the same orders that shares nothing but $H(\mathbf{k})$. At order n and harmonic m the steady-state component of each occupied state solves a linear equation at $\epsilon_n+m\omega+in\eta$ with the derivatives $\partial^p H/\partial k^p$ along the field on its right-hand side; built densely with every band of the sphere at a small cutoff, it is a frequency-domain hierarchy with no time step, no propagator and no Fourier projection. On zincblende AlAs at 4 Ry, at Γ (where the projector row at $\mathbf{k}+\mathbf{G}=0$ lives) and at a general point, with $\omega=0.05$ and $\eta=0.01$ Ha over $\eta T=20$: the five components $(1,1)$, $(2,2)$, $(2,0)$, $(3,3)$, $(3,1)$ agree to 1.4×10^{-5} to 3.0×10^{-5} at 400 steps a period and to 4.9×10^{-6} to 7.6×10^{-6} at 800, the midpoint rule's Δt^2. The even orders there are the size of the odd ones, so they are compared and not a residue.

The second order against `get_shg` pins the factors the first order cannot: the $\frac{1}{2}$ and $\frac{1}{4}$ of the cosine, $J=\dot{P}$ and the normalisation of $\chi^{(2)}$. With $\chi^{(2)}(-2\omega;\omega,\omega)=-2iJ_{(2,2)}/z^3$ (`chi2_from_orders`) on AlAs at 12 Ry, 4^3, along $[111]$, the propagation reads $124.92+20.12i$ pm/V at 0.7 eV against the dense hierarchy's $124.93+20.16i$, and the relation to `get_shg` is $\chi^{\text{RT}}=-\overline{\chi^{\text{SoS}}}$ at every frequency, broadening, mesh and cutoff tried. The conjugate is the sum over states' time convention ($\omega-i\eta$ in both denominators, as Elk's `nonlinopt.f90` writes it); the minus is the charge, to which $\chi^{(2)}$ is odd and which the sum over states does not carry, so below the gap the real-time χ_{xyz} of AlAs with Al at the origin is positive where `get_shg`'s and Elk's are negative. And with every band of the sphere the sum over states is still 5 to 9 per cent short, on every mesh: with the projectors switched off the two close to 2×10^{-5}, so what the sum misses is the projectors' curvature $\langle n|\partial^2 V_{NL}/\partial k_a\partial k_b|m\rangle$, which its sum rule replaces by the free electron's δ_{ab}. A truncated sum agrees better (1.35 per cent at 23 bands) because the truncation cancels part of the missing term; the regression test pins the relation as it stands.

The third order against a finite difference of four full runs at $\lambda=\pm h,\pm2h$, which shares the propagator and checks only the differentiation: 6.5×10^{-6} at $h=3\times10^{-4}$ over the whole $J^{(3)}(t)$, falling as h^2 above that.

Silicon at 1.55 eV against arXiv:1810.06500, whose LDA values are $|\chi^{(3)}_{xxxx}(\omega)|=2.2\times10^{-18}$ and $|\chi^{(3)}_{xxxx}(3\omega)|=1.3\times10^{-18}$ m^2/V^2. Converged in the mesh at 12 Ry (20^3 to 28^3, the field's wedge), the frozen potential gives 0.94×10^{-18} and 0.66×10^{-18} at $\eta=0.2$ eV and 1.36×10^{-18} and about 1.0×10^{-18} at 0.1 eV (the third harmonic oscillating by 10 per cent with the mesh there), with the phase delays 0.52 and 0.64 fs against the paper's 0.6 and

0.7, and $\chi^{(1)} = 14.3$ against its 15.2. It is the same scale and smaller by about two, and three differences could not be separated from that factor: their potential is updated in time (no local fields and no ALDA kernel here), their lattice is 10.26 bohr against 10.2, and their extraction is a fit to a filtered 20 fs pulse where this is a Lorentzian η per photon. Those mesh points come from the dense hierarchy, which agrees with the propagation on every cell both were run on (4.8×10^{-5} on 2^3 at $\eta T = 12$) and costs 0.35 s a k-point at 169 plane waves, two orders of magnitude below the propagation at this cutoff. **The velocity gauge's mesh artefact is the size of the answer on a coarse mesh**: the band-curvature term of the linear response is 135, 30, 7.5 and 1.8 in $\chi^{(1)}$ on 2^3 to 8^3, and its quartic sibling as large as the whole $\chi^{(3)}(3\omega)$ on 2^3, so a third harmonic on a mesh below 12^3 is not silicon's.

What it costs. One run serves one frequency, and its length is $T = \ln(C/\varepsilon)/\eta$ for a start transient of relative size ε, which does not decay because the evolution is unitary: at $\eta T = 6$ the same comparison reads 0.4 to 4.6 per cent. Order n of nested forward mode carries 2^n copies of each state. A converged spectrum of $\chi^{(3)}$ is therefore the job of the frequency-domain hierarchy below, and this route is the one it is checked against.

9.4 The spectrum of the orders: the frequency-domain hierarchy

One propagation gives the orders at one frequency, and a spectrum of $\chi^{(3)}$ is the same steady state solved directly at every frequency. We will now see that route, which shares the physics of the previous subsection and none of its time stepping. At order N and harmonic M the steady-state component of each occupied state solves

$$\left(\epsilon_n + M\omega + iN\eta - H_0\right) c_M^{(N)} = \sum_{p=1}^{N} \frac{1}{p!\,2^p} \sum_{s=0}^{p} \binom{p}{s} h_p\, c_{M-(2s-p)}^{(N-p)}, \qquad h_p = \frac{\partial^p H(\mathbf{k} + x\hat{\mathbf{e}})}{\partial x^p}\Big|_{x=0},$$

meaning that each order is driven by the lower ones through the derivatives of the Bloch Hamiltonian along the field, and the current of order N at harmonic M is the occupation-weighted sum of $\langle c_{m_1}^{(n_1)} | \partial_{\boldsymbol{\kappa}} h_p | c_{m_2}^{(n_2)} \rangle$ over the pairs with $n_1 + n_2 + p = N$. A frequency then costs nine linear solves per band and k-point, $c_{\pm1}^{(1)}$, $c_{\pm2,0}^{(2)}$, $c_{\pm3,\pm1}^{(3)}$, where the propagation pays a run of $\eta T \approx 12$ to 20 for each.

The bands the ground-state solve resolved are held exactly, and the rest of the sphere is a Krylov solve. Each component is split as $c = Pc + (1-P)c$, with P the projector on the computed bands (the occupied ones and the conduction bands below the top four of `nbnd`, which an unconverged root would occupy): $Pc = \sum_j |u_j\rangle\langle u_j|\mathrm{rhs}\rangle/(z - \epsilon_j)$ is arithmetic, and $(1-P)c$ solves $(H_0 - z + \alpha P)\, x = -(1-P)\,\mathrm{rhs}$, Quantum ESPRESSO's `cch_psi_all` operator with α past every $M\omega$. The two ill-conditioned pieces of the hierarchy are both in P, where they are exact: the secular $1/(2i\eta)$ of $c_0^{(2)}$ along the state itself, and the denominators between two occupied states, which cancel only in the sum over bands. $H_0 - z$ is complex, normal and indefinite above the gap, so the conjugate gradients of the static response do not apply; the solve is BiCGStab, right-preconditioned with the kinetic preconditioner of the Sternheimer stack and batched over the bands and harmonics of one order. It was chosen on two-atom silicon against the alternatives: with twelve computed bands it takes 40 to 52 matrix-vector products to a relative residual of 10^{-10} at 40 Ry and $\eta = 0.1$ eV, 1.3 times full GMRES, while the restarted GMRES(4) of `solve_e_fpol.f90`, written for an imaginary frequency, takes about a thousand above the gap. The derivatives h_p are kinetic and nonlocal only, since the local potential carries no $\mathbf{k}$, and come from the table of the projectors with no transform.

```
spec = calc.get_nonlinear_spectrum([1.0, 1.55, 2.0], broadening=0.3, grid=(4, 4, 4))
third, first = spec.chi3(axis=0)          # m^2/V^2, chi_xxxx(3w) and chi_xxxx(w)
print(abs(third), abs(first))
print(spec.iterations, spec.computed_bands)
```

`spec.component(n, m)` is $\mathbf{J}_{(n,m)}$ at every frequency, with the same normalisation as `get_harmonic_orders`' `component`, and `chi2(axis)` and `chi3(axis)` apply the formulas above to it (pm/V and $\mathrm{m}^2/\mathrm{V}^2$). `iterations` is the largest BiCGStab count of each frequency. On the 4^3 mesh of the snippet the numbers are the mesh's and not silicon's (the previous subsection says why); the route is what makes the mesh that converges them affordable.

The check is the dense hierarchy, as an identity. On zincblende AlAs at 4 Ry at Γ and a general point, all five components $(1,1)$, $(2,2)$, $(2,0)$, $(3,3)$, $(3,1)$ at 0.02, 0.05 and 0.11 Ha with $\eta = 0.01$ Ha agree with `dense_orders`, which solves with every band of the sphere, to 10^{-13} to 8×10^{-12}, in 18 to 21 BiCGStab iterations; with half the computed bands in P to 2×10^{-10}, so the projector moves the iteration count and not the answer. **Against the propagation** at one frequency, on AlAs at 6 Ry on the 2^3 mesh along $[111]$ at 1.5 eV and $\eta = 0.3$ eV, the five components agree to 3.5×10^{-5} to 9.6×10^{-4} at 400 steps a period and to 4.2×10^{-6} to 4.1×10^{-5} at 800, the propagation's step and its start transient at $\eta T = 12$; the hierarchy took 13.6 s for that frequency and the propagation 302 s. On AlAs at 8 Ry on the 4^3 mesh along $[111]$ the second harmonic at 1.5 eV and $\eta = 0.3$ eV is $\chi_{xyz} = 145.2015 + 119.0707i$ pm/V, the dense hierarchy's to every printed digit and the propagation's $145.1967 + 119.0447i$ to 1.4×10^{-4}, in 15 s for two frequencies against the propagation's minutes for one. **Silicon's third harmonic on converged meshes**, 12 Ry and $\eta = 0.2$ eV: at 1.55 eV on 20^3 the route gives $|\chi^{(3)}_{xxxx}(3\omega)| = 0.658$ and $|\chi^{(3)}_{xxxx}(-\omega;\omega,\omega,-\omega)| = 0.936$ in units of 10^{-18} $\mathrm{m}^2/\mathrm{V}^2$, the dense hierarchy's 0.66 and 0.94, and the spectrum on 16^3 from 0.4 to 3 eV, 27 frequencies, took 19 minutes on ten cores of a workstation where the propagation took a day of one core for one frequency on 12^3: $|\chi(3\omega)|$ peaks at 1.2×10^{-18} near 1.0 eV, the three-photon resonance at a third of the gap. Below about 0.6 eV the numbers on that mesh are the band-curvature term's, which grows as the frequency falls.

What it costs. A frequency is the nine solves, each about 20 to 50 BiCGStab iterations of two Hamiltonian applications, so a few hundred applications per band and k-point where the propagation pays tens of thousands; the current's assembly needs no transform and is a few per cent of a call. The k-points are independent and are solved in chunks of `k_batch` (one on a CPU, one `vmap` over a chunk on a card), each frequency starting from the previous one's solutions, and what is resident per k-point is the nine components, BiCGStab's six vectors for the largest order and the computed bands, `(10 + 24) nocc + nb` vectors of `npwx`.

At first order the potential can respond too. With `potential="hartree"` or `"hxc"` and `order=1`, the right-hand side gains the induced potential, $\delta v_+ c_0$ at $+\omega$ and $\delta v_+^* c_0$ at $-\omega$, with $\delta v_+ = K\,\delta\rho_+$, $\delta\rho_+ = \sum w\,(c_0^* c_+ + c_-^* c_0)$ and K the Hartree kernel or the derivative of the whole potential at the states' own density, the propagation's update of the previous subsection linearised. The fixed point in δv_+ is a GMRES solve, each of whose products is one pass over every k-point; the mixing the static response uses (`ph.x`'s, history four) took 200 passes and diverged on an interband transition with the Hartree kernel where GMRES took 28. The result's `epsilon(axis)` is $1 + 8\pi J_{(1,1)}/z^2$ and `epsilon(axis, frozen=True)` the same run's at the ground state's potential, and on a coarse mesh the two differ from the converged ϵ_M by the same band-curvature term, which their difference does not carry.

```
lf = calc.get_nonlinear_spectrum([0.1, 2.0], broadening=0.1, order=1, potential="hxc")
print(lf.epsilon(0) - lf.epsilon(0, frozen=True))   # the local fields' shift of eps_xx
```

Against the dense self-consistent solve on AlAs at 4 Ry, at the three frequencies of the frozen check, it agrees to 9×10^{-12} to 5×10^{-11} in 9 to 12 products, where the induced potential moves the current by 2 to 12 per cent with the Hartree kernel and 0.08 to 2 per cent with the exchange-correlation one. Against the static response of the Sternheimer stack, which shares only the functional's derivative, on two-atom silicon's 4^3 mesh at $\omega = 0.1$ eV and $\eta = 0.1$ eV: the shifts are -2.119995 and -0.883446 against -2.120012 and -0.883444, 8×10^{-6} and 2×10^{-6}, which is also the check that the induced density of the field's wedge is completed with the right group. And against the propagation with the potential updated, from kicks without the static subtraction (so that the band-curvature term is the same in both and cancels in the shift), on the same mesh at 1, 2 and 3 eV and $\eta = 0.5$ eV: the frozen ϵ agree to 6×10^{-7} to 1×10^{-6} and the shifts to 5×10^{-4} of the shift, 10^{-5} of ϵ, which is the propagation's second-order step at $\Delta t = 0.2$.

9.5 The potential updated in time: local fields and the adiabatic kernel

So far the potential was the ground state's. We will now see what changes when the Hartree and exchange-correlation potentials are rebuilt from the density at every step, which is what makes the response carry local fields (the density the field induces inside a cell screens it) and, with the exchange-correlation part, the adiabatic kernel of the ground state's functional (the ALDA for an LDA run). Every method of this section takes

- `potential="frozen"`, the default, the independent-particle response;
- `potential="hartree"`, the Hartree potential updated and the exchange-correlation potential kept at the ground state's, which is the Dyson route's `rpa` kernel with local fields;
- `potential="hxc"`, both updated, the Dyson route's `alda` for an LDA run and the adiabatic GGA for a PBE one;

and `corrector`, below. The potential is written as the ground state's plus the change,

$$v(t) = v_{\mathrm{scf}} + U[\rho(t)] - U[\rho_0], \qquad \rho(t) = \sum_{n\mathbf{k}} w_{n\mathbf{k}} \, |u_{n\mathbf{k}}(\mathbf{r}, t)|^2,$$

with U the Hartree potential or the Hartree and exchange-correlation one and ρ_0 the propagated states' own density at the start, meaning that the start is a stationary state on any k-mesh. Written as $v(t) = U[\rho(t)]$ it would not be: the states come from a fixed-density solve on the propagated mesh, whose density is the ground state's only on the ground state's own mesh, and on a 6^3 mesh of two-atom silicon converged on 4^3 the two potentials differ by 10^{-2} Ry, which a run with no field would start to follow. The energy the dynamics conserves is $E = E_{\mathrm{kin+NL}}(\boldsymbol{\kappa}) + \int (v_{\mathrm{loc}} + v_{\mathrm{scf}} - U[\rho_0])\rho + E_U[\rho]$, whose derivative in ρ is the potential the states feel, and it is what `energy` carries.

The applied field is the whole macroscopic field. The Hartree potential has no $\mathbf{G} = 0$ term, so the induced macroscopic field is not fed back, the current over the applied field is the macroscopic conductivity, and the dielectric function from a kick is ϵ_M with local fields, $1/[\epsilon^{-1}]_{00}$, which is what `get_absorption` and `get_dielectric_tensor` compute and what Elk's propagation does with `tafindt` off.

The k-points are no longer independent, since every step needs the density of all of them, so the whole state set is resident and the run walks time outside and the k-chunks inside each step. **The step** is the same midpoint rule with the potential at the midpoint extrapolated from the last two, $\frac{3}{2}v(t) - \frac{1}{2}v(t-\Delta t)$, the states stepped, the density and potential rebuilt at $t+\Delta t$, and with `corrector=1` (the default) the step taken again from t at $\frac{1}{2}[v(t)+v(t+\Delta t)]$. Both are second order in Δt; the corrector costs a second step and buys a smaller constant. On a wedge of the field's little group the density is completed with that group and its fractional translations, and never with the crystal's whole group, under which the first-order density of a field along x in silicon, odd under inversion, would average to zero.

```
eps = calc.get_realtime_dielectric(direction=(1, 0, 0), broadening=0.5,
                                   window=10.0, nw=200, grid=(4, 4, 4), dt=0.2,
                                   subtract_static=True, potential="hxc")
print(eps.epsilon[:3, 0])        # eps_M(w + i eta) with local fields and the ALDA
```

The check is a dense self-consistent solve that shares no time step. At first order each occupied state's components at $\pm\omega$ solve $(\epsilon_n \pm \omega + i\eta - H_0)c_\pm = \frac{1}{2}h_1 c_0 + \delta v_\pm c_0$ with $\delta v_+ = K\,\delta\rho_+$, $\delta\rho_+ = \sum w\,(c_0^* c_+ + c_-^* c_0)$ and $\delta v_- = \delta v_+^*$, K the Hartree kernel or the derivative of the whole potential at ρ_0, solved densely with every band of each sphere and to self-consistency by GMRES (`realtime.dense.dense_first_order`). On AlAs at 4 Ry, Γ and a general point, $\omega = 0.05$ and $\eta = 0.01$ Ha: the propagation's first order agrees with it to 1.36×10^{-5} (Hartree) and 1.64×10^{-5} (Hartree and exchange-correlation) at 400 steps a period, as the frozen potential does (1.37×10^{-5}), and to 8.3×10^{-7} and 3.7×10^{-6} at 800, while the local fields move the current by 3.5 per cent and 0.35 per cent from the frozen value: on that cell the exchange-correlation kernel undoes most of what the Hartree term does. The higher orders go through the second and third derivatives of the functional, which a finite difference of full self-consistent runs in the field amplitude checks: the third order agrees with the stencil to 7.7×10^{-6} over the whole series (9.0×10^{-6} and 5.4×10^{-6} on $J_{(3,3)}$ and $J_{(3,1)}$), falling by 3.9 when the stencil halves, against 6.5×10^{-6} at a frozen potential. After a kick of 0.05 bohr^{-1} with the field off the energy is conserved to the frozen route's own floor with the corrector (1.6×10^{-7} Ha over 1000 steps of 0.1, against 1.4×10^{-7} frozen, both the norm the Taylor step loses) and to a bounded 1.6×10^{-6} without it, which falls by 3.9 when the step is halved; and the work a pulse does is the energy it deposits to 1.4×10^{-6} at $\Delta t = 0.1$, as at a frozen potential. **Against the static response of the Sternheimer stack**, which is band-complete and shares only the functional's derivative: on two-atom silicon at 12 Ry on its 4^3 mesh the local fields lower the static ϵ_{xx} from 23.788 (`get_dielectric_tensor(screening="none")`) to 21.668 with the Hartree kernel alone and 22.904 with the exchange-correlation one (`"hartree"`, `"full"`), and the first order of the propagation at $\omega = 0.5$, $\eta = 0.2$ eV moves ϵ by $-2.176 - 0.057i$ and $-0.902 - 0.019i$ from the frozen value, 2.6 and 2.0 per cent from the static shifts, which is what $|z|^2$ over the square of the gap leaves at that frequency. The shift and not ϵ itself is compared, because at first order the update leaves the band-curvature term of the frozen route untouched and the shift is free of it.

Against Elk, now like for like in the physics. Elk's propagation updates its potential, so the `Si-dielectric` run of the previous subsection is repeated with `potential="hxc"` (the same 100 points and 4000 steps of 0.2, through the same transcription of Elk's task 481). Below the absorption edge the update lowers ϵ_{xx} by 4.6 to 5.7 per cent (14.38 against 15.10 at 0.5 eV, 21.39 against 22.42 at 2 eV), and Elk's real-time spectrum lies 1.5 to 5.9 per cent below its own independent-particle one there; the largest $\mathrm{Im}\,\epsilon$ falls from 64.0 to 61.7, against 62.6 in Elk's real-time spectrum. What is left between the two codes is

what was there at a frozen potential, the basis (LAPW in 25 bands against the plane-wave sphere), about 3 per cent below the edge and 0.07 to 0.2 eV in the peak positions. The lowest frequencies, below 0.5 eV, are not compared: there the transform of a finite run divides the run's last oscillation by ω^2 in both codes, and Elk's own real-time value at $\omega \to 0$ is -4.13.

What it costs. The whole mesh's states are resident, twice inside a step, and 2^n times at order n of the orders; each step is $1+$`corrector` Taylor steps and as many densities and potentials, about twice the frozen step: on Elk's `Si-dielectric` run, one quiet core each, 1085 s against 505 s frozen and against Elk's 1300 s for the same physics, Elk's step being first order in the potential where this one is second. With `corrector=0` a step is one Taylor step and one density, 1.08 times the frozen one on silicon's whole 4^3 mesh against 2.04 with the corrector, at the price of the extrapolation's ten times larger energy drift above.

9.6 Magnetic, spin-orbit, ultrasoft and PAW ground states

Which ground states can the field be applied to? Every route of this section, the propagation, its orders, the frequency-domain hierarchy and, for the second harmonic, `get_shg`'s sum over states, takes a collinear spin-polarized run, a spinor run with spin-orbit coupling or noncollinear magnetism, an ultrasoft or PAW dataset, and their combinations. We will now see what each adds to the equation of motion and to the current; the entry points and their arguments are the ones above.

A collinear run (`nspin = 2`) is two band structures, one per channel, each with its own potential. At a frozen potential they are independent and every order of the current is the sum of the channels', and with `tot_magnetization` the two carry different numbers of bands, which the run reads off the occupations. With the potential updated in time they couple through the density: the Hartree potential of the two channels' sum and the exchange-correlation potential of both, the adiabatic local spin density kernel for an LSDA run.

A spinor run is one Hamiltonian on twice the plane waves, and its coupling to the field is the scalar one: both components sit on the sphere of $\mathbf{k}$ and meet the same projectors at $\mathbf{k}+\boldsymbol{\kappa}$ through the 2×2 coefficients $D_{ij}^{\sigma\sigma'}$, which carry the spin-orbit coupling (`dvan_so`). So the spin-orbit term is minimally coupled to the field, as the nonlocal potential is (Ismail-Beigi, Chang and Louie, PRL 87, 087402 (2001)), where Elk's real-time spin-orbit term is written with the canonical momentum (arXiv:1406.6607), a difference between the two codes in the physics and not in the numerics. The magnetization enters through the local 2×2 potential $v+\mathbf{m}\cdot\boldsymbol{\sigma}$ alone, and with the potential updated the four components $(n,\mathbf{m})$ are rebuilt from the spinors at every step. The field's little group of a magnetic run keeps no operation that is a symmetry only together with time reversal, since the field breaks it.

An ultrasoft or PAW dataset has an overlap $S(\mathbf{k}+\boldsymbol{\kappa})$ that moves with the field, and the gauge factor $e^{i\boldsymbol{\kappa}\cdot\mathbf{r}}$ that takes every operator from $\mathbf{k}$ to $\mathbf{k}+\boldsymbol{\kappa}$ meets the time derivative through it:

$$i\,S\,\partial_t|\phi\rangle = \big[H-\dot{\boldsymbol{\kappa}}\cdot\mathbf{X}\big]|\phi\rangle, \qquad \mathbf{X}=\tilde{\mathbf{r}}-S\,\mathbf{r}=\sum_{ij}|\beta_i\rangle\big[\mathbf{d}_{ij}\langle\beta_j|+i\,q_{ij}\langle\partial_{\mathbf{k}}\beta_j|\big],$$

every operator at $\mathbf{k}+\boldsymbol{\kappa}(t)$, with $\mathbf{d}_{ij}$ the dipole of the augmentation charge about its atom and $\partial_{\mathbf{k}}\beta$ the projectors' derivative about their own atom, meaning that $\mathbf{X}$ is what the augmented density adds to the position operator beyond $S\mathbf{r}$. It is the moving-ion term of ultrasoft real-time propagation (Qian, Li, Lin and Yip, PRB 73, 035408 (2006)) with $\boldsymbol{\kappa}$ in the place of the ions' coordinates, and its anti-Hermitian part, $-\frac{i}{2}\,\partial_t S$, is exactly what

keeps $\langle\phi|S|\phi\rangle$ constant. The current is the velocity of the augmented states,

$$\mathbf{J} = -\frac{1}{\Omega}\sum_{n\mathbf{k}} w_{n\mathbf{k}}\Big[\langle\phi|\partial_{\mathbf{k}}H|\phi\rangle - 2\,\mathrm{Im}\,\langle\phi|HS^{-1}\mathbf{X}|\phi\rangle\Big],$$

whose matrix element between two eigenstates is the generalised velocity $\langle n|\partial H - \epsilon_m \partial S|m\rangle + (\epsilon_m - \epsilon_n)K_{nm}$ the optical conductivity is built from, and whose work on the field is the energy the states gain. S^{-1} is applied by Woodbury's identity, one small solve per k-point and time, as `sm1_psi.f90` does; PAW's one-centre coefficients are the ground state's, frozen with the potential; a kick is a step in $\boldsymbol{\kappa}$, which the augmented states cross by the same equation without H (the all-electron state does not move under it). The frequency-domain hierarchy solves the same equation expanded in the field, with S, its derivatives and those of $\mathbf{X}$ beside the Hamiltonian's.

```
from defuqumat import Calculator

calc = Calculator.from_file("se-trigonal-soc.in")   # spin-orbit, three atoms
calc.get_scf()
spec = calc.get_nonlinear_spectrum([0.8, 1.2], broadening=0.2,
                                   direction=(1, 0, 0), grid=(2, 2, 2))
print(spec.chi2(0))       # chi_xxx(-2w; w, w), pm/V
print(spec.chi3(0)[0])    # chi_xxxx(-3w; w, w, w), m^2/V^2
```

What checks it. Each of these is an identity on a small cell rather than an agreement with a reference. The first order of a kick against the optical conductivity's sum over every eigenstate of the dense $H(\mathbf{k})$, $S(\mathbf{k})$ with the generalised velocity, plus the band curvature the sum replaces by its f-sum value, which is how the norm-conserving propagation is checked: 2.4×10^{-6} of the scale on ultrasoft AlAs, 2.5×10^{-6} on ultrasoft AlAs with spin-orbit coupling and 1.5×10^{-7} on trigonal selenium with it. Every operator of the nonlinear problem enters there, as a function of $\mathbf{k}$, and the review of this route measured that dropping $\mathbf{X}$, flipping its sign or dropping the current's correction each moves the first order by 10^{-3} of itself. The work the field does on ultrasoft AlAs under a strong pulse equals the energy the states gain to 2.7×10^{-7}, with $\langle\phi|S|\phi\rangle$ constant to 7×10^{-9}; with $\mathbf{X}$ left out the norm grows and the run stops. A collinear magnet written as spinors, AlAs with two unpaired electrons, gives the collinear route's currents in the hierarchy to 10^{-12} at every order and with the potential updated, and in the propagation to the step's own error, 2×10^{-7} at 800 steps a period and seventeen to twenty-two times that at 400, where the two share neither the Hamiltonian, the density nor the kernel's structure. The hierarchy agrees with the propagation at the full switch-on ($\eta t = 12$) to 2.0×10^{-5} to 1.2×10^{-4} of each order's scale on the magnet, ultrasoft and PAW AlAs and selenium with spin-orbit coupling, the level norm-conserving AlAs reaches between the same two routes. As a reference for what the dataset itself is worth, the ultrasoft and PAW pslibrary datasets give $\chi^{(2)}$ and $\chi^{(3)}$ of AlAs at 1.5 eV 0.8 per cent and 6×10^{-4} apart on one cell, with each converged in the cutoff to 0.1 per cent.

Refuses. With an **ultrasoft or PAW** dataset, **the potential updated in time**: the augmentation charge of $\rho(t)$ would come from the projections at $\mathbf{k}+\boldsymbol{\kappa}(t)$, $D(t)$ would be rebuilt by `newd` at every step and PAW's one-centre D from the one-centre densities, none of which is written; and **a kick after the start of the run**, whose step the augmented states cross by a transformation the time grid cannot hold. **DFT+U**, until the Hubbard projectors' own row at $\mathbf{k}+\mathbf{G}=0$ is repaired (their first derivative at Γ is 19 per cent off for a p shell) and the linear check is run. **A spin spiral** and **gamma-only storage**. **A ground state converged under a magnetic field or a constrained**

moment, for the response stack's reason. **A symmetry-reduced k-set passed as `kpoints`** (pass `grid=`, which builds the field's own wedge) and **a shifted grid**, which is not closed under the group. **`occupations = 'fixed'` cutting a degenerate multiplet**: the current would depend on the rotation the eigensolver returned. A metal runs with its ground-state occupations held, and its intraband response converges slowly in k. With the potential updated: **a meta-GGA** with `potential="hxc"` (a potential-only one has no energy to conserve, and either kind would need $\tau(t)$ propagated beside $\rho(t)$) and **a checkpoint**, since every state and two potentials would have to be saved. The frequency-domain hierarchy refuses **a component whose solve does not converge** (`HierarchyError`, with its residual and count, since an unconverged first order is amplified by $1/(2\eta)$ into the second), **the potential updated in time above first order** (the induced potentials at 2ω and 3ω would each be a fixed point at every frequency, and they would carry the second-order density artefact, 7 per cent of ρ on a 2^3 mesh; the propagation has them at one frequency a run), **two FFT grids** with the potential updated, **an outer solve that does not converge**, **an order above three**, and an `nbnd` that leaves no conduction band below the top four.

10 Topological invariants

Everything is built from one primitive — the overlap of occupied manifolds at neighbouring k-points,

$$M^{(\mathbf{b})}_{mn} = \langle u_{m\mathbf{k}} | \hat{S} | u_{n,\mathbf{k}+\mathbf{b}} \rangle$$

because a determinant of overlaps is blind to the unitary mixing a degenerate eigensolver leaves. The Berry curvature is the phase of a plaquette of them and the Chern number their lattice sum, which is an **exact integer on any mesh**:

$$F_{12}(\mathbf{k}) = \ln\left[U_1(\mathbf{k})U_2(\mathbf{k}+\hat{1})U_1(\mathbf{k}+\hat{2})^{-1}U_2(\mathbf{k})^{-1}\right], \qquad C = \frac{1}{2\pi i}\sum_{\mathbf{k}} F_{12}(\mathbf{k})$$

Z_2 has two independent routes — the flow of Wannier charge centres, and, when there is an inversion centre, the parity product over the eight TRIM points:

$$(-1)^{\nu} = \prod_{i=1}^{8} \prod_{n\ \mathrm{occ}}^{\mathrm{Kramers}} \xi_{2n}(\Gamma_i)$$

```python
from defuqumat.workflows.topology import (run_z2, run_z2_3d,
                                          run_berry_curvature)

z2 = run_z2(system, pseudos, scf.density, axis=2)   # Wilson loop by default
print(z2.z2, z2.gap_step)   # read gap_step before believing the integer

chern = run_berry_curvature(system, pseudos, scf.density, shape=(12, 12))
print(chern)                # an exact integer on any mesh

nu = run_z2_3d(system, pseudos, scf.density)   # the four 3D indices
print(nu)                   # (nu0; nu1 nu2 nu3)
```

Refuses. A collinear spin-polarized run (two Hamiltonians, so two independent sets of invariants and no statement of which is meant); a Z_2 without spin–orbit coupling,

where every invariant is zero for a reason that has nothing to do with the band structure; the parity route without an inversion centre; and a manifold that is not separated from the band above it (`gap_tol`). A PAW run needs the converged `becsum` passed beside the density — `run_z2(..., becsum = scf.becsum)`, which `Calculator` does for you — because `ddd_paw` is a property of the wavefunctions and cannot be rebuilt from a density; leaving it out is worth 1.03 eV in the eigenvalues, and an invariant computed from those is still an integer.

Running both Z_2 routes is the check. Where they disagree the parity one is the answer, because it has no mesh and the Wilson route does. **Two things bite in a plane-wave code and both are silent:** neighbouring k-points do not share a G-sphere, so coefficients are aligned by Miller index; and the wrap at the zone edge is a *shift* of that index, without which the Chern number comes out smooth and non-integer.

10.1 The curvature as a map: the Kubo route

Two constructions of the same quantity, chosen by name:

$$\Omega_n^{12}(\mathbf{k}) = -2\,\mathrm{Im}\sum_{m\neq n}\frac{A^1_{nm}A^2_{mn}}{(\varepsilon_n-\varepsilon_m)^2}, \qquad A^a_{nm} = \langle\psi_n|\,\partial_{k_a}H - \varepsilon_m\,\partial_{k_a}S\,|\psi_m\rangle + (\varepsilon_m-\varepsilon_n)K^a_{nm}$$

with `method="kubo"`, against the lattice field strength of Fukui, Hatsugai and Suzuki with `method="fhs"` (the default). The velocity operator is one `jvp` of $H(\mathbf{k})$ at a frozen plane-wave sphere — no dense $H(\mathbf{k})$ is ever formed — and the tangent for a crystal direction d is the reciprocal lattice vector $\mathbf{b}_d$.

The last term is the one a dataset with an augmentation charge needs, and it is zero for every other. With $\hat{S} = T^\dagger T$ the states a Berry phase is about are $T|\psi\rangle$ rather than $|\psi\rangle$, so the connection splits as $\langle\Psi_n|\partial_a\Psi_m\rangle = \langle\psi_n|\hat{S}|\partial_a\psi_m\rangle + K^a_{nm}$ with

$$K^a_{nm} = \sum_{ij}\langle\psi_n|\beta_i\rangle\,q_{ij}\,\langle\partial_{k_a}\beta_j|\psi_m\rangle - i\sum_{ij}\langle\psi_n|\beta_i\rangle\,d^a_{ij}\,\langle\beta_j|\psi_m\rangle, \qquad d^a_{ij} = \int r_a\,Q_{ij}(\mathbf{r})\,\mathrm{d}\mathbf{r},$$

the augmentation dipole `adddvepsi_us` adds to an electric field's position operator.

Written that way A^a is Hermitian and is the generalised velocity matrix, which is what `defuqumat.response.conductivity` contracts; the code here writes the equivalent form with the *outer* band's energy ε_n in both factors and the two blocks $K^\dagger$ and K split between them, because that is what falls out of resolving the identity inside $-2\,\mathrm{Im}\langle\partial_1 n|\hat{S}|\partial_2 n\rangle$. The two agree identically, and the reason they do is the identity below. Only $\partial_{k_a}H$ and $\partial_{k_a}S$ can be got from the two tangents of one `jvp`, and K is neither: $K^\dagger + K$ is *exactly* $\langle\psi_n|\partial_{k_a}S|\psi_m\rangle$, the dipole cancelling between the two and the rest adding, so the two factors of the sum take different blocks and no rearrangement of $\partial S/\partial k$ supplies it. That identity holds to 1.2×10^{-16} on ultrasoft silicon, 2.1×10^{-16} on PAW silicon and 2.9×10^{-16} on fully-relativistic ultrasoft AlAs — where the blocks are q^{SO}_{ij} and d^{SO} and carry a spin index each — which pins the index order and says nothing about the dipole; the dipole is pinned against a central difference of the FHS overlap itself, where at a step small enough that all three k-points hold the same plane-wave sphere the connection comes out at 1.67×10^{-4} (ultrasoft) and 1.67×10^{-4} (PAW) of a norm-conserving control's own 1.70×10^{-4}, while dropping K leaves 1.57×10^{-2} — 94 times larger, and *not moving* as the difference shrinks.

```
from defuqumat.workflows.topology import run_berry_curvature
```

```
curvature = run_berry_curvature(
    system, pseudos, density, shape=(4, 4), nocc=4, nbnd=12, method="kubo"
)
curvature.curvature          # (4, 4), Omega(k)
curvature.curvature_by_band  # (4, 4, nocc) -- see the refusal box
curvature.truncation         # what the highest empty band was worth
```

Validated against the FHS flux, which shares no machinery with it: a centred plaquette shrunk around one k-point of zincblende AlAs converges onto the pointwise Kubo value to 3.2×10^{-3}, and the whole 24×24 mesh agrees plaquette by plaquette to 1.45×10^{-4}, improving 65-fold over a sixfold refinement. The velocity matrix elements themselves reproduce a central finite difference of $H(\mathbf{k})$ to 1.8×10^{-9}, and on centrosymmetric silicon the curvature vanishes *pointwise* to 3.5×10^{-5}.

Refuses. `method="kubo"` is for the map, never for the integer: its denominator $(\varepsilon_n - \varepsilon_m)^{-2}$ is singular at a degeneracy and its Brillouin-zone sum is an ordinary Riemann sum, which converges to an integer without ever being one — `method="fhs"` is the default and is exact on any mesh. **Ultrasoft and PAW run**, and what the `method="kubo"` comparison against `method="fhs"` cannot tell you is whether the augmentation dipole is there: on ultrasoft AlAs it is worth **0.34 %** of Ω where the two methods sit **50 %** apart pointwise at 18×18, so that comparison would read the same with the term deleted, and the numbers that carry it are the overlap's own finite difference above. The sum over empty states is truncated at `nbnd` and the truncation is *reported* rather than hidden — read `BerryCurvature.truncation`, or `truncation_abs` wherever symmetry forces the curvature to vanish, since the ratio is then noise over noise. `curvature_by_band` is gauge invariant only for a non-degenerate band; inside a degenerate multiplet only the sum over the members is defined, and `curvature` (the manifold total) always is.
A mesh point where an occupied and an empty band actually *touch* — a Dirac cone, a Weyl node, the closing a topological transition runs through — is a genuine singularity and is **dropped, with a warning naming how many**. `BerryCurvature.singular_points` carries the count, and a nonzero one means the mesh sum is not a Chern number of that state set: the curvature diverges there and a zero was put in its place, which is smooth and plausible and wrong. Move the mesh off the point, or use `method="fhs"`, which needs no gap in a denominator. A band pair counts as touching below 10^{-8} of the band width *over the whole mesh* — a fraction rather than an energy, since a model Hamiltonian carries no unit, and over the whole mesh because for a two-band model the spectrum at one $\mathbf{k}$ *is* the gap.

10.2 The Berry-phase polarization

The polarization of a crystal is not the dipole of its unit cell — that integral depends on where the cell is cut. What is a property of the crystal is a *phase*: the Berry phase the occupied manifold accumulates as $\mathbf{k}$ crosses the zone once along a reciprocal lattice vector $\mathbf{b}_i$ (King-Smith and Vanderbilt, PRB **47**, 1651 (1993)),

$$\Omega\,\mathbf{P} = \sum_i \frac{\varphi_i}{2\pi}\,\mathbf{a}_i, \qquad \varphi_i = \underbrace{\operatorname{Im}\ln \prod_j \det M^{(\mathbf{b}_i/N)}(\mathbf{k}_j)}_{\text{electronic}} + \underbrace{2\pi \sum_a Z_a^{\mathrm{v}}\, s_{ai}}_{\text{ionic}}$$

with M the same overlap the invariants above are built from and s_{ai} the atom's crystal coordinate. It is the *determinant* of the overlap that is kept here where a Wilson loop keeps the eigenvalues, which is the whole difference between the two calculations.

The value is defined modulo a quantum and the result object carries it. A single number is not a physical statement: what is meaningful is a *difference* between two geometries taken on the same branch, which is what a Born effective charge and a piezoelectric constant are made of.

```
from defuqumat.workflows.polarization import run_polarization

p = run_polarization(system, pseudos, scf.density,
                     gdir=2,            # 0-based; pw.x's gdir = 3
                     nppstr=6,          # as pw.x counts it
                     transverse=(2, 2))
print(p.total_phase, p.quantum)         # -0.2412383  (mod 1)
print(p.polarization_si)                # -0.34909 C/m^2, mod 1.44708
print(p.strings.phases)                 # one phase per string, four here
```

`Calculator.get_polarization()` reads `lberry`, `gdir` and `nppstr` straight off a `pw.x` input, so a polarization run transfers unchanged. `nppstr` counts the repeated endpoint the way `kp_strings` does; the mesh built here carries `nppstr - 1` distinct points and closes the string with the reciprocal lattice vector, which is the same links and one diagonalisation fewer per string.

Refuses. A **metal** — a Berry phase is a property of a gapped manifold, and a smeared occupation does not say which bands the string carries. **`nspin = 2`**: two channels are two independent string sets and one phase each, which is a layout this does not have rather than a missing term. And a **spin spiral**, whose two spinor components live on spheres centred at $\mathbf{k} \pm \mathbf{q}/2$, so a string overlap is not a single gather. A *spinor* run (`noncolin`) is supported and has its own `pw.x` reference; its quantum is half the scalar one, because a spinor band holds one electron. Ultrasoft and PAW are *not* refused, and both are validated: zincblende SiC matches `pw.x` string by string with an electronic phase of -0.009696 against -0.00970. It had to be SiC rather than silicon, because a centrosymmetric crystal's electronic phase is zero whatever $q_{ij}(\mathbf{b})$ does and so agrees with zero however wrong the augmentation is.

Validated three ways, two of which need no other code. Against `pw.x`'s own `lberry` run on AlAs it agrees *string by string* and on every digit printed (electronic phase 0.00876, total -0.24124). The Zak phase of the SSH model is pinned to 0 or π by symmetry and comes out exactly that on a mesh of *any* size (10^{-10}), and silicon is pinned by inversion to 0 or half a quantum, landing on the half (10^{-8}). Silicon is also the second `pw.x` reference, and the one that exercises the quantum of 2: both its species carry an even valence, where AlAs's arsenic does not. The strongest is $\mathrm{d}P/\mathrm{d}u$ against the Born charges, which come from a Sternheimer solve and share no machinery with a string of overlaps: AlAs on an $8 \times 8 \times 8$ ground state gives $Z^*(\mathrm{Al}) = +2.148$ against `ph.x`'s $+2.142$, charge-neutral to 5×10^{-5}.

10.3 The magnetoelectric tensor

How much electric polarization a magnetic field induces,

$$\alpha_{ij} = \frac{\partial P_i}{\partial B_j},$$

computed Elk's way (task 390): a full ground state at $B \mp \delta/2$ for each Cartesian direction, the Berry-phase polarization of each, and a central difference. The cheap route — one derivative of the magnetization along an electric field's response — needs a noncollinear

Sternheimer solve, and an electric field as a ground-state perturbation, which this package does not have.

Two symmetries must both be broken or the answer is zero for a reason that has nothing to do with the calculation: time reversal and inversion each map α to $-\alpha$. Elk's own example is built around the first, and so is the case here — a non-magnetic semiconductor has no linear tensor at all, so a *large* external field is applied to break time reversal by hand and the derivative taken against a small further change in it. The magnetism is induced rather than spontaneous, which is what makes a plain gapped semiconductor usable.

```
from defuqumat import Calculator

alas = Calculator.from_file("alas-magnetoelectric.in", pseudo_dir)
me = alas.get_magnetoelectric_tensor(delta=0.02, directions=(2,))
print(me.magnitude)        # 4.4376e-06
print(me.largest_step)     # how close a field step came to a branch
```

Refuses. An unconverged ground state, which is what makes the next limitation visible rather than silent. **Only the column parallel to the seeded magnetization is reachable** on the committed cell: a field along the direction `starting_magnetization` points converges in 12 iterations, and one with a transverse component does not converge in 150, because rotating the moment is the slow mode. It is not the tilt — a pure transverse field fails on its own. Seed the magnetization along the column you want. A run carrying no magnetization vector ($n_{\mathrm{spin,mag}} \neq 4$): the Zeeman term then multiplies nothing and every polarization comes out identical, which looks like a symmetry result. And a field step that moves the Berry phase by more than a quarter of its quantum, since a difference taken across a branch is the quantum divided by δ — large, smooth and meaningless. Everything the polarization refuses is refused here too.

The validation is the spin-orbit null. Without the coupling a global spin rotation maps B to $-B$ and leaves every charge observable fixed, so α vanishes identically. On the same cell, same field, same ultrasoft family at the same valence, with only `lspinorb` and the scalar/fully relativistic datasets changed: 3.43×10^{-9} against the coupled 4.44×10^{-6}, a ratio of **1293**. Two things had to be right together to get there, and neither helps alone: `chain` (seeding one field run from the other, which breaks the very $B \to -B$ symmetry the null rests on — off by default here, unlike Elk) and the *polarization's* threshold rather than the self-consistent field's. **The absolute value is not calibrated against another code**: Elk's `bfieldc` and this package's `B_field` differ in normalisation by about two orders of magnitude, and Elk's own tensor on this cell is not converged.

11 Effective masses, site angular momenta, nesting, structure factors and STM images

Quantities Elk computes and `pw.x` mostly does not. The first two cost one NSCF at a single k-point; the nesting function costs one NSCF on a dense grid, and nothing after it; the structure factors cost one transform of a density that already exists, and an STM image one band sum with different weights.

11.1 The effective mass tensor

$$\left(\frac{1}{m^*}\right)_{ab} = \frac{1}{2}\frac{\partial^2 \varepsilon_n(\mathbf{k})}{\partial k_a \partial k_b},$$

in units of $1/m_e$. The factor of a half is Rydberg atomic units and not a normalisation: $\hbar^2/2m_e$ is exactly 1 Ry bohr^2, so a free electron has $\varepsilon = |\mathbf{k}|^2$ and the tensor is the identity.

The first derivative is analytic and only the second is a difference. $\partial\varepsilon_n/\partial k$ is the generalised Hellmann–Feynman expression built from one `jvp` of $H(\mathbf{k})$; its second derivative is not reachable the same way, because an individual band's $|\partial_k \psi_n\rangle$ is not what the Sternheimer projector produces (it removes the whole occupied manifold, and the band whose mass is wanted is usually empty). So the construction is one central difference of an *exact* first derivative — six stencil points against Elk's twenty-seven, and no polynomial fit. Elk's route, differencing eigenvalues, is kept beside it as `method="eigenvalue"` because it shares nothing with the velocity operator.

The whole stencil is built on one plane-wave sphere, the centre's, and its points are then moved onto it. Which plane waves satisfy $|\mathbf{k}+\mathbf{G}|^2 \le$ `ecutwfc` is a step function of $\mathbf{k}$, so two points a stencil apart can hold different numbers of them: at L on two-atom silicon at `ecutwfc = 30` the centre holds 754, the $\pm h$ pair 752 and the $\pm 2h$ pair 744. A difference taken across that step carries it divided by h^2, which is not $O(h^2)$ and which the Richardson step therefore returns rather than removes; on one sphere what is differenced is one band of one basis. It is worth **0.5 %** on the eigenvalue route at L at the default stencil.

```
from defuqumat import Calculator

calc = Calculator.from_file("si.scf.in")
mass = calc.get_effective_mass((0.0, 0.0, 0.0), nbnd=10)  # crystal coordinates

mass.inverse_mass          # (nbnd, 3, 3) in 1/m_e, cartesian
mass.principal(band=7)     # the three principal masses and their axes
mass.density_of_states_mass(band=7)   # (m_x m_y m_z)^(1/3)
mass.truncation            # the O(h^2) the Richardson step removed
mass.multiplets            # per multiplet, including the degenerate ones
```

On two-atom silicon at Γ against the vendored **Elk** binary (all-electron LAPW, PBE, $a = 10.26$ bohr) with a PAW dataset here: the non-degenerate Γ_{1v} agrees to **0.02%** (0.8601272 against 0.8603044, i.e. $m^* = 1.16262\,m_e$ against 1.16238) and $\Gamma_{2'c}$ to 0.36% ($m^* = 0.170440\,m_e$). The two routes here agree with each other to 1×10^{-7} on Γ_{1v}, and the tensors come out isotropic to 2.9×10^{-8} with nothing imposing cubic symmetry.

Refuses. A band inside a **degenerate multiplet** has no tensor of its own — the eigensolver's arbitrary rotation within the manifold rotates it — so those bands come back as `nan` and what is reported instead is the multiplet's *summed* inverse mass, which is the trace and is invariant. Silicon's threefold $\Gamma_{25'}$ is the case, and Elk's own output is the evidence: it prints -5.8938, -3.7690, -3.7690 for three states whose energies agree to 5×10^{-8} Ha. A **spin spiral** is refused, since a k-stencil moves both spheres. `delta` is the stencil's outermost displacement in both routes; the $O(h^2)$ truncation is removed by a Richardson step and then *reported* in `truncation` rather than tuned away.

11.2 $\langle L\rangle$, $\langle S\rangle$ and $\langle J\rangle$ on each atom

Where the orbital moment of a spin-orbit magnet actually sits. From one site density matrix per atom and shell, built out of the same projection a projected density of states uses,

$$\rho^{a}_{(ms),(m's')} = \sum_{n\mathbf{k}} w_{n\mathbf{k}}\, c^{a}_{ms,n\mathbf{k}}\, \overline{c^{a}_{m's',n\mathbf{k}}}, \qquad c = \langle\phi|S|\psi\rangle,$$

and then $\langle L_i\rangle = \sum_s \mathrm{Tr}_m(L_i\rho_{ss})$ and $\langle S_i\rangle = \frac{1}{2}\sum_m \mathrm{Tr}_s(\sigma_i\rho_{mm})$, with $J = L+S$. The L matrices are the complex-basis ones conjugated into the *real* spherical harmonics the code works in, using the same `rot_ylm` unitary the spin-orbit projectors are built from.

```
from defuqumat import Calculator

calc = Calculator.from_file("ni.scf.in")   # noncolin, lspinorb, nosym
momenta = calc.get_angular_momenta()

print(momenta.table())      # what Elk writes to LSJ.OUT
momenta.atoms[0].l          # <L> on atom 1, cartesian, units of hbar
momenta.atoms[0].j          # <J> = <L> + <S>
momenta.total_orbital       # summed over the cell
```

Validated by identities, not by another code's number — Elk integrates over a muffin tin of a stated radius and this projects onto an orbital set, so the two differ by definition the way Löwdin and muffin-tin charges do. $\langle L\rangle$ is *quenched to zero* without spin-orbit coupling, measured at 1.7×10^{-16} on silicon; with the coupling on, fully-relativistic nickel gives $\langle L_z\rangle = 0.0364767\,\hbar$ against a measured orbital moment of about $0.05\,\mu_B$, with $|L|/|S| = 0.11665$ against an experimental $m_L/m_S \approx 0.1$ and $\langle L\rangle \parallel \langle S\rangle$ (Hund's third rule). Driving the moment along z, x and y gives $|\langle L\rangle| = 0.0364767$ in all three, with a spread of 7.3e-11 over the three cubic axes, with nothing imposing that a magnitude is a scalar.

A symmetry-reduced k-set works, and what makes it work is that, in a magnetic noncollinear run, $\langle L\rangle$ and $\langle S\rangle$ are averaged over the group as **axial** vectors — each operation carrying $\det(R)$ and the time-reversal sign beside the rotation, the site charge carrying only the atom permutation. The distinction is not a refinement: the *polar* average of an axial vector is exactly zero on any centrosymmetric crystal, so writing the obvious one would return a clean and plausible nothing rather than a wrong number. **A collinear run is different**, because there the spin is not coupled to the lattice: at `nspin = 2` the site $\langle S_z\rangle$ is a scalar carried from atom to atom by the operations, and averaging it as an axial vector under the spatial group gave exactly zero on bcc Fe's 48 operations. $\langle L\rangle$ of a collinear run is completed to zero by time reversal, since each channel has a real Hamiltonian and $\langle L\rangle_{-\mathbf{k}} = -\langle L\rangle_{\mathbf{k}}$. A **nonmagnetic spinor run** is completed to zero in the same way, and for both vectors: with no magnetization the ground state is time-reversal symmetric, so every state at $\mathbf{k}$ has a Kramers partner at $-\mathbf{k}$ with $\langle L\rangle$ and $\langle S\rangle$ reversed, and on a wedge cut with $\mathbf{k}\to-\mathbf{k}$ both are set to zero at every site. The spatial average alone does not do it on a noncentrosymmetric cell, or at $n_{\mathrm{sym}} = 1$, where it returned the uncancelled half of the zone.

Refuses. A **fully-relativistic ultrasoft or PAW** dataset, because the spinor overlap's off-diagonal spin blocks are `qq_so` and the projection here applies the scalar S to each component; a fully-relativistic *norm-conserving* dataset has $S = 1$ and is exact. And a **spin spiral**, whose two components do not share a set of atomic orbitals. `pw.x`'s `lorbm` computes the *cell's* orbital magnetization and has no per-atom counterpart

to check this against — that quantity is the next subsection, and the two are worth reading together.

11.3 The orbital magnetization of the cell

The other half of a magnet's moment, and the half no integral over the unit cell can give. A Bloch state's current circulates through the whole crystal, so $\int \mathbf{r}\times\mathbf{j}$ depends on where the cell is cut — the same difficulty the electric polarization has. What is well defined is a k-space quantity (Thonhauser *et al.*, PRL **95**, 137205 (2005); Ceresoli *et al.*, PRB **74**, 024408 (2006)):

$$\mathbf{M} = \frac{e}{2\hbar c}\,\mathrm{Im}\sum_n \int [d\mathbf{k}]\,\langle\partial_{\mathbf{k}}u_n|\times(H_{\mathbf{k}}+E_{n\mathbf{k}}-2\mu)\,|\partial_{\mathbf{k}}u_n\rangle,$$

a *local* circulation inside each cell plus an *itinerant* one along the boundary. Neither is separately measurable and their sum is. The derivative is the covariant finite difference over a uniform mesh — the dual states $|w_m\rangle = \sum_n (M^{-1})_{nm}|u_{\mathbf{k}+\mathbf{b},n}\rangle$ of the neighbouring manifold — so nothing is divided by $E_n - E_m$ and the answer is invariant under any unitary mixing of the occupied bands.

```
from defuqumat import Calculator

calc = Calculator.from_file("i-atom-soc.in")    # noncolin, lspinorb, gapped
orbm = calc.get_orbital_magnetization(divisions=(3, 3, 3))

orbm.lc, orbm.ic     # pw.x's two Kubo terms, Bohr magnetons per cell
orbm.total           # their sum, at mu = 0
orbm.chern           # the Chern vector; zero for an ordinary insulator
```

Against `pw.x`'s `lorbm` on the same $3\times3\times3$ grid, an iodine atom in a twelve-bohr box: $M_{LC} = -0.5986388$ against -0.5986370 and $M_{IC} = -0.5757995$ against -0.5757987, so the total is -1.1744384 against -1.1744357. What is left at 2×10^{-6} is the difference between two separately converged densities: re-running at `conv_thr` of 1e-6, 1e-8 and 1e-12 moves M_{LC} by 5e-6, 2e-7 and 2e-6 against the same reference. **The physics check is independent of both**: iodine is $5s^2 5p^5$, one hole in the p shell, and Hund's rules give $L = 1$ locked parallel to $S = 1/2$. The site projection of the previous subsection measures $\langle L_z\rangle = 0.99977\,\hbar$ on the same run, and the modern theory gives $1.1744\,\mu_B$ — 17% more, which is what a projection onto atomic orbitals does not see: the circulation outside those orbitals, and the nonlocal pseudopotential's own contribution to the velocity, since $\langle L\rangle$ is $\mathbf{r}\times\mathbf{p}$ where this contracts H.

Two conventions are QE's and are worth knowing. What both codes print is $M(\mu = 0)$: `orbm_kubo` imports the Fermi level and never uses it, and the term it drops is μ times the Chern vector, which vanishes for any topologically trivial manifold. And the two printed terms are *not* the papers' LC/IC split — the Fortran prints $\mathrm{Im}\langle\partial u|H|\partial u\rangle$ and $\mathrm{Im}\langle\partial u|E|\partial u\rangle$, which differ from $(H-E)$ and $2(E-\mu)$ term by term and agree in the sum. This follows the Fortran, so the two halves can be compared and not only the total.

Refuses. Ultrasoft and PAW, which `pw.x` refuses in the same place (`setup.f90`: 'Orbital Magnetization not implemented with USPP/PAW'): the overlap between neighbouring manifolds would need $q_{ij}(b)$ and the dual states would have to be dual in the S metric. A run **without spin-orbit coupling**, where the orbital moment is quenched

identically — measured here as 10^{-9} with `soc_scale = 0` in the same relativistic dataset. **`nspin = 2`**, which is the same statement: each channel is separately time-reversal symmetric. A **metal**, or any manifold not separated from the band above it, which the gap check refuses by name. A **spin spiral**. And a mesh with **two divisions** along a direction that carries a derivative, where a point's two neighbours are the same k-point and the central difference is an alias; one division is allowed and sets that direction's derivative to zero, which is what a slab means.

11.4 The Fermi-surface nesting function

How much of the Fermi surface maps onto itself when translated by **q**:

$$N(\mathbf{q}) = \frac{1}{N_k} \sum_{\mathbf{k}} g(\mathbf{k})\, g(\mathbf{k}+\mathbf{q}), \qquad g(\mathbf{k}) = g_s \sum_{s,n} \delta(\varepsilon_{sn\mathbf{k}} - E_F).$$

$g(\mathbf{k})$ is the density of states *at one k-point* — a picture of the Fermi surface on the grid — so $N(\mathbf{q})$ is the $\omega \to 0$ imaginary part of the Lindhard function stripped of its matrix elements: the *geometric* half of an instability. Where it is large, a perturbation of wavevector **q** connects many occupied states to many empty ones at no energy cost, which is what softens a phonon, opens a charge-density-wave gap, or picks a spin spiral's pitch.

It is a convolution. Elk writes an $O(N_q N_k)$ double loop with $\mathbf{k}+\mathbf{q}$ folded back onto the grid; that fold makes the sum a cyclic cross-correlation, so $N = \mathrm{ifftn}(|\mathrm{fftn}(g)|^2)/N_k$ gives every **q** in one transform. Measured on a 12^3 aluminium grid that is **0.001 s** here against Elk's 0.37 s. A symmetry-reduced wedge is *unfolded* onto the complete grid rather than refused, since $\varepsilon_n(R\mathbf{k}) = \varepsilon_n(\mathbf{k})$: 72 diagonalisations instead of 1728.

```
from defuqumat import Calculator

calc = Calculator.from_file("al.scf.in")
nest = calc.get_nesting(grid=(12, 12, 12))   # the grid is the convergence knob

nest.nesting        # (nq,) N(q), in (states/Ry/cell)^2
nest.qpoints        # (nq, 3) crystal coordinates, on the unshifted grid
nest.ratio          # N(q) / D(E_F)^2, dimensionless and 1 on average
nest.peak()         # the largest N(q) away from q = 0, and where it is
nest.along(2)       # (q_3, N) along one crystal axis, for a line plot
nest.as_grid()      # (n1, n2, n3), for a slice
nest.fermi_dos      # D(E_F) in states/Ry/cell
nest.elk_units      # the same array in NEST3D.OUT's normalisation
```

The functional entry point is `defuqumat.workflows.run_nesting`, which takes the converged density directly.

Two identities come with the normalisation and both are worth reading. The mean of N over the whole q-grid is exactly $D(E_F)^2$ (`sum_rule` reports the residue; it is 3×10^{-16} relative). And $N(0) \geq N(\mathbf{q})$ for every **q** by Cauchy–Schwarz, so **the nesting peak is always a peak away from the origin** — `peak()` excludes $\mathbf{q} = 0$ for that reason, not as a plotting convenience.

Validated against a closed form and against a quantity computed by an unrelated route. For free electrons $N(q) = \Omega/4\pi^2 q$ below $2k_F$ and zero above: the code reproduces the $1/q$ law to 10^{-4} and the hard cut to 10^{-16}. A half-filled hydrogen chain has $2k_F = \pi/c$, so it must nest at $\mathbf{q} = (0, 0, \frac{1}{2})$ — it does, at **99.8%** of the Cauchy–Schwarz bound, and `relax_spiral_q`

relaxes the corresponding spin spiral to 0.500014 by going downhill in a magnetic total energy, which shares nothing with this. Against the vendored Elk binary on aluminium, $N(0)/D(E_F)^2$ agrees to 1.5%; the raw $N(0)$ differs by 4.7%, which is the all-electron Fermi surface underneath and is quoted rather than tuned away.

Refuses. A constrained `tot_magnetization`, which has one Fermi level per channel, so $g(\mathbf{k})$ is two different surfaces; a **fixed-occupation** run, which never searches for a level, so $\delta(\varepsilon - E_F)$ has no argument; and a **spin spiral**, because the quantity is a statement about the state a spiral grows out of — compute it on the paramagnet and compare its peak with `relax_spiral_q`'s answer. The delta defaults to a **Gaussian** even when the run used Methfessel–Paxton or cold smearing: those go negative on the wings, so $g(\mathbf{k})$ can be negative and $N(\mathbf{q})$ would have no sign at all. Pass `smearing=` to have the run's own anyway.

11.5 Magnetocrystalline anisotropy

The energy it costs to point a magnet's moment one way rather than another — what makes a hard magnet hard and what pins a spiral into a plane. It is tenths of a meV per atom against total energies of hundreds of Rydberg, so taking it as a difference of two self-consistent total energies asks two SCF runs to agree in their ninth digit. The *force theorem* does not ask that. Converge the magnet **without** spin-orbit coupling, rotate the converged density so its magnetization points along $\mathbf{n}$, and diagonalise **once** with the coupling switched on. At frozen density every term of the total energy except the band energy is a functional of ρ alone, so

$$E(\mathbf{n}_1) - E(\mathbf{n}_2) = E_{\mathrm{band}}(\mathbf{n}_1) - E_{\mathrm{band}}(\mathbf{n}_2), \qquad E_{\mathrm{band}} = \sum_{n\mathbf{k}} w_{n\mathbf{k}}\, \varepsilon_{n\mathbf{k}},$$

exactly, and the whole calculation is one diagonalisation per direction.

The cheaper thing gives almost none of it. Freezing the wavefunctions as well and taking $\langle\psi|H_{\mathrm{SOC}}|\psi\rangle$ once, at the coupling-free states, gives the first-order term, and the coupling enters there as $\xi\,\langle\mathbf{L}\rangle\cdot\mathbf{n}$, which vanishes because the orbital moment of a scalar-relativistic collinear state is quenched. What survives is the exchange field's augmentation charge seen through the j-resolved projectors, the spin times a time-even orbital tensor, which a quenched state does not remove: on tetragonal cobalt it is 1.79×10^{-3} meV of anisotropy with the easy axis along c, against 0.552 meV of free-energy anisotropy from the diagonalisation on the same density, and on a cubic cell it is the same 1.26×10^{-2} meV along every axis. So the anisotropy is second order in the coupling to about 0.3 per cent, and what supplies it is the repulsion between levels that a diagonalisation performs and an expectation value does not. `get_first_order_soc(spinor, direction=)` computes the first-order term, $\sum w\,\langle\psi|\delta D - \varepsilon\,\delta S|\psi\rangle$ with δD and δS the coupled nonlocal coefficients and overlap minus the reduced ones at the same potential, which is the derivative of the free energy with respect to `soc_scale` at 0.

Two pseudopotential files, one density. The first leg runs with a scalar-relativistic dataset and the one-shot leg with the fully-relativistic dataset of the *same generation*; only the density crosses, and a density does not know which file made it. That is what makes an ultrasoft anisotropy reachable, the alternative — averaging one relativistic file's j channels back — being refused for ultrasoft and PAW by `pw.x` itself.

```
from defuqumat import Calculator
```

```
scalar = Calculator.from_file("scf.in")          # nspin = 2, Co.pbe-nd-rrkjus
spinor = Calculator.from_file("nscf.in")         # noncolin, lspinorb, rel- file
mae = scalar.get_anisotropy(spinor, directions="xyz")

print(mae.anisotropy_mev, mae.easy_axis)
print(mae.difference(0, 2))                      # in-plane minus out-of-plane
```

`directions` takes a list of vectors, an integer asking for that many points spread evenly over the sphere, or one of `"cardinal"`, `"xz"` and `"xyz"`. `projected=True` adds the decomposition of the band energy over atomic orbitals, resolved by atom, ℓ, m and spin, which is what says *which* orbitals supply the anisotropy; comparing its total against E_{band} measures the spilling.

`soc_scale` scales the spin-orbit part of the nonlocal potential, the overlap and the augmentation charge while keeping the same dataset and the same k-points. At 0 every one of them is replaced by the spin trace of its spin-orbit form, $\frac{1}{2}\sum_{ss'} F^{ss'} M F^{s's}$ for each spin-independent matrix M the coefficients F dress, the density's `becsum` included, so the reduced Hamiltonian is the derivative of the energy it reports. At 0 the anisotropy is exactly zero, which is the control for the whole calculation: without the coupling the Hamiltonian is invariant under a global spin rotation, so a band energy *cannot* depend on where the moment points. On tetragonal cobalt the two directions of a self-consistent total agree to 2×10^{-10} meV, and switching the coupling off costs 0.44 mRy of the total. On a fully-relativistic PAW dataset it also scales the magnetization the Dirac small component carries on each sphere, $-2(\mathbf{m}_{\mathrm{s}}\cdot\hat{\mathbf{r}})\,\hat{\mathbf{r}}$, which is the spin reflected about the radial direction and so ties the spin to the lattice as the coupling does; its charge is left alone, being invariant under a spin rotation. Left on at 0 it gave tetragonal nickel (`Ni.rel-pbe-spn-kjpaw_psl`, 40 Ry, a $2\times2\times2$ mesh) 8.3×10^{-3} meV between x and z; switched off with the rest, the two agree to 7×10^{-6} meV at `conv_thr = 1e-12`, a residue that does not fall further with `conv_thr` and belongs to the truncated cutoff: at the dataset's own $75/480$ Ry the two agree to 3.9×10^{-9} meV.

A PAW dataset needs one file rather than two, and `becsum` beside the density. A PAW Hamiltonian's one-centre coefficients D^1_{ij} are a functional of `becsum` exactly as the grid potential is a functional of ρ, so what the theorem freezes on this dataset is the *pair*: the potential has two representations and both are held fixed. `becsum` cannot cross between the two files above — a scalar-relativistic dataset carries one projector per (n,ℓ) channel and its fully-relativistic partner one per (n,ℓ,j), so the two arrays are indexed by different things — and that is why `pw.x` refuses the path outright rather than for a reason that could be plumbed around. The way round it is `soc_scale`: run the *self-consistent* leg on the fully-relativistic file with `soc_scale = 0` and the one-shot leg on the same file with the coupling on, so that both legs share a projector set by construction. `becsum` is then laid along the requested direction with the density, off the axis the *density* defines rather than its own, which is what an antiferromagnet needs: its two sublattices point opposite ways while the cell has one frame to turn. It is a projection onto that axis rather than a rigid rotation, exactly as it is for the density, so a canted first leg hands over the collinear part of its moment and not the canting.

```
from defuqumat.scf import run_scf
from defuqumat.workflows.anisotropy import run_anisotropy

# one fully-relativistic PAW file, run twice: the coupling off for the
# self-consistent leg and on for the one-shot.
scf = run_scf(system.with_soc_scale(0.0), pseudos, conv_thr=1.0e-8)
```

```
mae = run_anisotropy(system, pseudos, scf.density,
                     directions=[(0, 0, 1), (1, 0, 0), (1, 1, 0)],
                     becsum=scf.becsum)
```

`Calculator.get_anisotropy` passes `becsum` for you when the first leg's projector set is the one the second leg has, and hands over nothing when it is not, so the two-file route above is unaffected. With the coupling off the identity holds on a PAW dataset as it does on an ultrasoft one: 3.1×10^{-10} meV over five directions on antiferromagnetic oxygen. Leaving the rotation out — carrying `becsum` but laying it along z for every direction — gives **468 meV** on that same cell, whose answer is exactly zero, which is why the identity is worth running.

The torque: the same number from one angle. Taking the anisotropy as $E(\mathbf{n}_1) - E(\mathbf{n}_2)$ is 10^{-5} Ry out of 10^2 — seven digits of cancellation. The torque takes it as a *derivative* evaluated once, where nothing cancels: for $E(\theta) = K_1 \sin^2\theta$,

$$-\frac{\mathrm{d}F}{\mathrm{d}\theta} = -K_1 \sin 2\theta, \qquad \text{so at } \theta = 45^\circ \text{ the torque is } -K_1.$$

```
torque = scalar.get_torque(spinor)              # 45 degrees by default
print(torque.anisotropy_constant_mev)           # K1 in meV
```

It differentiates the free energy, and for a smeared metal that is not the band energy. A Hellmann–Feynman derivative at frozen occupations gives $\mathrm{d}F/\mathrm{d}\theta$ with $F = \sum w\varepsilon - TS$; $\sum w\varepsilon$ carries an extra $\sum(\mathrm{d}w/\mathrm{d}\theta)\varepsilon$ that the entropy cancels. The difference is not small. On tetragonal cobalt, as the smearing narrows:

`degauss` (Ry)	band difference	free difference	torque
0.020	1.2353	0.5523	0.5523
0.005	0.5434	0.5291	0.5291
0.002	0.5308	0.5308	0.5308

all in meV. The torque tracks the free-energy difference at every width and both converge on $K_1 = 0.531$; at a production `degauss` of 0.02 the *band*-energy difference reads 2.3 times that. Use `MagneticAnisotropy.free_energies` to compare like with like.

The torque as a vector: every spin turned by one rotation. The angle above turns a collinear moment in one plane. `get_orientation_torque` turns the whole texture by a rotation R, $\mathbf{m}(\mathbf{r}) \to R\,\mathbf{m}(\mathbf{r})$ with the charge kept, diagonalises once with the coupling at that orientation, and returns

$$\boldsymbol{\tau} = -\left.\frac{\partial F}{\partial \boldsymbol{\omega}}\right|_{\boldsymbol{\omega}=0}, \qquad R(\boldsymbol{\omega}) = e^{[\boldsymbol{\omega}]_\times} R_0,$$

the torque for the three generators of a rigid rotation about R_0, in Ry per radian, meaning that component a is how fast the free energy falls when every spin turns together about $\hat{\mathbf{e}}_a$. For a collinear magnet turning in the plane $(\hat{\mathbf{e}}_1, \hat{\mathbf{e}}_2)$ the plane torque above is $\boldsymbol{\tau} \cdot (\hat{\mathbf{e}}_1 \times \hat{\mathbf{e}}_2)$, and the component about the moment is zero, since that turn moves nothing. On the tetragonal cobalt pair of the table, 45 degrees from c towards a, the y component is the plane torque to 10^{-10} relative on the same states, -4.0595×10^{-5} Ry, and the component that tilts the moment out of the (z, x) plane, which the mirror $y \to -y$ with time reversal forces to zero, is 5.8×10^{-11} Ry, the diagonalisation's level.

```
import numpy as np
from defuqumat.workflows.anisotropy import rotation_from_euler
R = rotation_from_euler(0.0, np.pi / 4, 0.0)     # ZYZ, radians: c tilted 45 deg to a
vector = scalar.get_orientation_torque(spinor, rotation=R)
print(vector.torque)                              # -> [~0, -4.06e-05, ~0] Ry/rad
print(vector.direction, vector.along_moment)      # where the moment points; ~0
```

The orientation is given as a matrix and read back as ZYZ Euler angles (`vector.euler_angles`), but nothing is differentiated in the angles: at $\beta = 0$ the angles α and γ are the same rotation, and that is an easy axis along z. The source may be a four-component density whose moments are not collinear, a commensurate spiral in a supercell for instance, and it is then turned as it is, with the system's `angle1`/`angle2` describing it. Two checks that share no automatic differentiation with the torque: the closed form

$$\boldsymbol{\tau} = \int \mathbf{m}_{\mathrm{out}}(\mathbf{r}) \times \mathbf{B}(\mathbf{r})\, \mathrm{d}\mathbf{r},$$

the frozen exchange field acting on the magnetization of the states solved with the coupling, which agrees with it to 1.7×10^{-12} on ultrasoft cobalt; and turning the lattice by R^{-1} under fixed spins in place of turning the spins by R, which agrees to 4×10^{-11} once the eigensolver's floor is lowered, and to 10^{-6} at the default floor, which is the torque's resolution there (about 10^{-10} Ry per radian).

Relaxing the orientation. `get_relaxed_orientation` is the structural relaxation with the rotation as the coordinate: at each step one diagonalisation with the coupling at the frozen source density turned rigidly, the free energy and the torque above, and a step of the same BFGS the atoms use, in the rotation vector about the start, so a length is an angle in radians. It returns the easy axis of a magnet, or the plane (and phase) of a spiral in a supercell.

```
start = rotation_from_euler(0.4, 0.9, -0.3)      # 51.6 degrees off c, oblique
relaxed = scalar.get_relaxed_orientation(spinor, rotation=start, curvature=True)
print(relaxed.converged, relaxed.direction)      # -> True, (~0, ~0, 1): the c axis
print(relaxed.curvature_eigenvalues)             # -> [~0, 2K1, 2K1] Ry/rad^2
```

Start it off every symmetry element: an orientation a symmetry fixes has no torque, whether it is an easy axis or a hard one, and a relaxation started there reports convergence without having searched (it warns). The curvature, a central difference of the torque, is what tells a minimum from a saddle; one of its eigenvalues is zero for a collinear texture, the turn about the moment itself. The gradient threshold sits above the torque's noise, and the energy minimised is the free energy $\sum w\varepsilon - TS$, for the reason above.

The same inside a self-consistent run with the coupling. The orientation is an electronic degree of freedom, so an SCF with spin-orbit coupling turns the moments itself, but it is a soft mode: the lean of the output magnetization off the field it was solved in is the anisotropy over the exchange, and a mixer takes a fraction of it per iteration. Seeded 45 degrees off c, tetragonal cobalt does not converge in 100 iterations, its angle wandering between 36 and 54 degrees while $\mathrm{d}r^2$ falls to 10^{-8} and rises again. Every iteration has done the force theorem's calculation with the input density frozen, so the torque on a rigid turn of that density is the closed form above, one integral per iteration, recorded as `SCFResult.orientation_torque` and in `history`; it sits at -3.3×10^{-5} Ry per radian through that whole run, which is $-K_1$ of the self-consistent anisotropy read from one run. On PAW the torque has a one-centre share, three forward derivatives an iteration through

every output state, so a PAW run computes it only when something reads it, `rotate_moments` or the printed iteration line (`verbose` with the coupling on), and records `None` otherwise. `rotate_moments=True` turns the input density by $\boldsymbol{\omega} = -H^{-1}\mathbf{G}$ after every mix, the density, `becsum`, the mixer's history and the carried states together, with H built by BFGS from the torques and every step bounded by `rotation_trust`, and the run is converged when the torque is below `torque_conv_thr` as well.

```
oblique = Calculator.from_file("oblique.in")     # lspinorb, nosym, angle1 = 45
scf = oblique.get_scf(rotate_moments=True, conv_thr=1e-12, torque_conv_thr=1e-9)
print(scf.converged, scf.magnetization_vector)   # -> True, along c
print(scf.orientation_torque)                    # -> ~1e-9 Ry/rad
```

It converges in 44 iterations to 0.002 degrees from c, and its total energy is that of the SCF started exactly along c to 1.3×10^{-8} Ry. Without the coupling every orientation has the same energy and the torque is the eigensolver's noise, which a step divided by a vanishing curvature would turn into motion, so the option is refused there rather than left to do nothing.

Refuses, for `rotate_moments`: no coupling (or `soc_scale` $= 0$); a spin spiral, whose transverse magnetization is kept in the frame that turns with **q**; a Hubbard U, whose `ns` would have to turn with the density; a magnetic field or a constrained moment, which holds the orientation the step moves; and a run without `nosym`. `get_relaxed_orientation` and `get_orientation_torque` refuse everything the force theorem refuses. **PAW is taken by both**: the one-centre coefficients are rebuilt from the turned `becsum` inside the differentiated energy, from the one-file route's `soc_scale` $= 0$ leg for the force-theorem routes, and on nickel the one-centre share of the torque is larger than the whole and of the opposite sign. With the coupling, `becsum` is not a vector under a spin rotation (it keeps only the blocks diagonal in j), so `rotate_moments` turns it through the states. On PAW nickel `rotate_moments` still stalls near 10^{-4}, its steps running along a nearly flat in-plane direction; prefer `get_relaxed_orientation` there.

Refuses. **PAW without `becsum`**, and **PAW on the two-file route at all**: the one-centre coefficients are built from a `becsum` that a run on the other file cannot supply, its projectors being indexed by (n, ℓ, j) where the other's are indexed by (n, ℓ). The refusal names the one-file route in its message, and a `becsum` whose shape does not match this leg's projector count is refused rather than reshaped. A **Hubbard** U, whose `ns` is not in the handoff either; a potential-only **meta-GGA**, whose τ is not; a converged **magnetic field** or constrained moment, whose energy sits outside the reported total; and a **spin spiral**, which cannot carry the coupling switched on and so has no one-shot leg; its first-order term is `spiral_spin_orbit_energy`. A direction other than the system's own `angle1`/`angle2` needs `nosym`: a magnetic noncollinear run reduces its k-grid with the *magnetic* symmetry group, which depends on where the moment points, so two directions would otherwise be sampled on two different wedges. A four-component source density that is genuinely noncollinear is refused by name, as it already was on the path that builds the states: the torque lays the moment down along one axis with its sign, and an antiferromagnet keeps its antiparallel regions only because of that sign. On an **ultrasoft or PAW** dataset `soc_scale` takes 0 or 1 and nothing in between: the coupling-free end of the overlap is spin-independent, so the anisotropy vanishes identically there whatever else it is, but a blend of it with the true overlap partway is not the overlap of any set of projectors and S stops being a usable metric. On a calculation whose every dataset is norm-conserving there is no overlap to blend, the

scale reaches `dvan_so` alone and linearly, and any value in $[0, 1]$ is admitted as $H_0 + s\,\delta D$ exactly; outside $[0, 1]$ it is refused on every dataset.

11.6 The relaxed anisotropy: total energies instead of band sums

The section above buys its precision by freezing the density: every term of the total energy except the band sum is a functional of ρ alone, so it cancels between two directions exactly. The price is that the density is not allowed to respond to the spin-orbit field. `run_relaxed_anisotropy` pays the other way round — one whole self-consistent noncollinear run per direction, and a difference of two **total** energies:

$$E_{\mathrm{MAE}} = E_{\mathrm{tot}}[\hat{\mathbf{n}}_1] - E_{\mathrm{tot}}[\hat{\mathbf{n}}_2], \qquad \text{each } E_{\mathrm{tot}} \text{ converged at its own } \hat{\mathbf{n}}.$$

The extra energy the relaxation finds is variational, so it lowers *both* directions, and what survives in the difference is second order on a quantity that is already second order in the coupling. Nothing bounds it in general, which is why it is measured rather than argued about.

```
from defuqumat import Calculator

soc = Calculator.from_file("ni-tetragonal-relaxed-mae-paw.in",
                           pseudo_dir="tests/data/pseudo")
mae = soc.get_relaxed_anisotropy(directions="xz")

print(mae.energies_mev)        # relative to the first direction
print(mae.easy_axis, mae.anisotropy_mev)
print(mae.converged, mae.drifts)   # read these BEFORE the number
```

There is **one** calculator here and no second leg. That is the whole practical difference from the force theorem, and it is what lets this route reach **PAW** and a Hubbard U: the theorem hands a density from a scalar-relativistic run to a fully-relativistic one, and `ddd_paw` and `ns` are properties of *wavefunctions* that cannot cross that gap. A relaxed run hands nothing over — each direction converges its own `becsum` from its own states with one dataset throughout.

Read `.converged` and `.drifts` first, and the second one is the trap. Nothing holds the moment in a relaxed run, which is the point of it; so a direction that is not a stationary point of the anisotropy energy will turn towards one that is, and its total energy then belongs to a state other than the one asked for. A cardinal axis of a cubic or uniaxial crystal is stationary by symmetry and does not move; an oblique direction on a strongly anisotropic magnet need not be. A drift past one degree raises a warning naming the direction and where the moment went.

A **magnetic field** or a constrained moment, because the field's energy is deliberately outside the reported total, so two directions' totals would differ by a Zeeman term neither accounts for — and because a constraint defeats the drift check, the constraint being what holds the moment rather than the anisotropy. A potential-only **meta-GGA**, which has no energy functional at all, so its printed total is not the value of anything the run minimised. A **spin spiral**, which cannot carry the coupling switched on; its first-order term is `spiral_spin_orbit_energy` instead. A **collinear** run, which has no direction to point. And `nosym` is required rather than merely advised: a magnetic noncollinear run reduces its k-grid with the *magnetic* symmetry group, which depends on where the moment points, and here that is worse than it is for the force theorem

— a total energy carries the Ewald and Hartree terms too, so the contamination is not confined to the bands.

11.7 X-ray and magnetic structure factors

What a diffractometer measures is not a density but its Fourier coefficients, one per reflection:

$$F(\mathbf{H}) = \int_\Omega \rho(\mathbf{r})\, e^{i\mathbf{H}\cdot\mathbf{r}}\, d^3r, \qquad F_j(\mathbf{H}) = \int_\Omega m_j(\mathbf{r})\, e^{i\mathbf{H}\cdot\mathbf{r}}\, d^3r$$

for every reciprocal lattice vector with $|\mathbf{H}| \le$ `hmax` — X-rays scattering off the charge, neutrons off the magnetization. Elk's tasks 195 and 196; `pw.x` has neither. In a plane-wave code the integral *is* an array that already exists, up to the crystallographers' two conventions (the positive phase, and no $1/\Omega$), so the whole step is one transform and a gather: **6 ms** against the seconds its SCF cost. $F(0)$ is the electron count of the cell and $F_j(0)$ its magnetic moment, which is what anchors both.

```python
from defuqumat import Calculator

calc = Calculator.from_file("si.scf.in")
factors = calc.get_structure_factors(hmax=5.0)

factors.charge               # (nh,) complex F(H), in electrons
factors.magnetization        # (nh, ncomp) in Bohr magnetons, or None
factors.vectors.indices      # (nh, 3) the (h k l) after the output transform
factors.vectors.length       # (nh,) |H| in 1/bohr
factors.vectors.multiplicity # the size of each star that was collapsed
factors.of((2, -2, -2))      # one reflection, by the index the table prints
print(factors.table(limit=10))

# Elk's wsfac: rebuild the density from the states in an energy window (Ry),
# which is what makes this a probe of bonding rather than of the whole charge.
bonding = calc.get_structure_factors(hmax=3.0, window=(-1.0, 0.2))
```

The functional entry point is `defuqumat.workflows.sfac.run_structure_factors`. `transform="conventional"` labels the reflections on the conventional cubic axes (Elk's `vhmat`), `reduce=False` keeps every member of every star, and `core=True` adds a dataset's frozen core-correction charge.

A pseudopotential density is valence-only, and the comparison with an all-electron code is structured by reflection class rather than by one tolerance. On silicon, against the vendored Elk binary on the same cell and k-grid: the origin is 8 electrons here and 28 there, and every *allowed* reflection is core-dominated in the same proportion — (111) is 1.7495 against 15.14, and no tolerance makes that agree. What *is* comparable is a **forbidden** reflection, where the spherical part of every atom cancels and the core with it: silicon's (222) is **0.347406** here against **0.33416**, 4%, on a quantity whose entire value is the aspherical bonding charge. That last statement is checked without any other code, since this package's own SCF starting guess is a superposition of free atoms: it gives 0 at (222) to 10^{-10}. The space-group forbidden reflections — diamond's (200), (420) — vanish to 10^{-9} here and 10^{-13} there.

Norm-conserving, ultrasoft and PAW all work, and the difference between them is measurable. $F(0) = 8.0000000000$ on each, which is the check that the augmentation charge reached the transform at all — for a soft dataset most of the density near the nucleus *is* that charge. The like-for-like reference is not Elk's all-electron total but its

valence-only run, which `wsfac` above the 2p core produces: against it the augmentation charge is worth a factor of two, (111) coming out -0.7% for ultrasoft and PAW against -1.5% norm-conserving, and (004) $+31\%$ against $+61\%$. It is worth *nothing* on the forbidden (222), which is 0.3474 on all three to 3×10^{-4}: a silicon atom in diamond sits at $\bar{4}3m$, whose lowest non-spherical invariant is $l = 3$, and an s, p dataset's augmentation charge carries multipoles only to $L = 2$. That is also what the 4% against all-electron is, by elimination — not the k-grid, not either basis, not the functional (0.07%), and **not the core**, since Elk's own valence-only run gives the identical 0.334165.

Magnetic reflections are a low-$|\mathbf{H}|$ quantity and the boundary is measured. Ferromagnetic bcc iron against Elk's task 196, comparing the normalized $F_{\mathrm{mag}}(\mathbf{H})/F_{\mathrm{mag}}(0)$ because the moments differ (2.2145 μ_B here, 2.0613 there): 5.8% at the first reflection, 17% at the second, worse outwards. Raising `ecutwfc` from 30 to 45 moves those by less than 0.1%, so it is the pseudisation and not the basis — iron's moment is in the 3d shell, inside the radius the dataset smooths. The cheapest cell carries the sharpest magnetic statement instead: an antiferromagnetic hydrogen chain has charge factors $2.000000, 0, 0.8587, 0$ along the chain and magnetic ones $0, 1.2254, 0, 0.3862$, so it scatters neutrons exactly where it scatters no X-rays, and the zeros are exact.

Refuses. An `hmax` past $\sqrt{\texttt{ecutrho}}$: the FFT box carries a coefficient at every frequency out to its corner, and past the density's own cutoff those are the aliased tail of $|\psi|^2$ rather than Fourier components — small, smooth and entirely plausible, which is why this is a guard and not a check on the answer. An empty or inverted energy window, and `core=True` on a dataset with no nonlinear core correction. **Said rather than refused**, because the numbers are right and only a claim about them would be wrong: these are *valence* structure factors, and a star's multiplicity applies to the charge, the magnetic members of a star being related by $\det(R)\,R$ rather than being equal.

11.8 Scanning-tunnelling microscopy images

Tersoff and Hamann's result is that the current an s-wave tip draws at $\mathbf{r}$ is the sample's local density of states there, at the energy the bias selects. So an STM image is not a new calculation, it is the *density built from different occupations*:

$$\rho_{\mathrm{STM}}(\mathbf{r}) = \sum_{n\mathbf{k}} w_{\mathbf{k}}\, \tilde{\delta}(E - \varepsilon_{n\mathbf{k}})\, |\psi_{n\mathbf{k}}(\mathbf{r})|^2,$$

with $\tilde{\delta}$ the same smeared delta a density of states is built from. Elk's task 162 and `pw.x`'s `plot_num = 5` both say exactly that. Two energy selections: with no `bias` the weight is a delta at E (Elk's, a density of states per unit volume, in $1/(\mathrm{bohr}^3\,\mathrm{Ry})$), and with one it is the window $[E, E + V]$ (QE's, a density in electrons/bohr^3) — a *negative* V imaging the filled states, which is a real experiment's sign convention and the mode a semiconductor surface needs.

```
from defuqumat import Calculator

calc = Calculator.from_file("graphene.scf.in")

# constant height: the tip plane at crystal coordinate 0.85 along c
image = calc.get_stm(height=0.85, shape=(64, 64))

image.values        # (64, 64) the tunnelling density on the plane
```

```
image.coordinates   # (64, 64, 2) in-plane cartesian coordinates, bohr
image.extent()      # (x0, x1, y0, y1) for imshow
image.integral      # int rho_STM d3r = D(E_F): the sum rule
image.density       # the same field on the FFT grid

# filled states, 0.15 Ry below the Fermi level
filled = calc.get_stm(height=0.85, bias=-0.15)

# constant current: the corrugation, in bohr along the surface normal
scan = calc.get_stm(height=0.70, mode="constant-current",
                    current=1.0e-5, heights=(0.0, 6.0), nheights=80)
scan.heights_bohr   # (n1, n2), nan where the set-point is not crossed

# a magnetic tip: the image depends on which way it points
up = calc.get_stm(height=0.85, spin="up")            # collinear
along_x = calc.get_stm(height=0.85, spin=(1, 0, 0))  # noncollinear

# Elk's three corners, in crystal coordinates, for any other plane
side = calc.get_stm(plane=((0.15, 0, 0), (0.15, 0.3, 0), (0.15, 0, 1)))
```

The functional entry point is `defuqumat.workflows.stm.run_stm`. A delta at the Fermi level wants a denser k-grid than the SCF's, which `grid=` re-solves the bands on and **recomputes the Fermi level for**; `smearing` picks the delta (a Gaussian by default whatever the run used, because Methfessel–Paxton and cold smearing go negative on their wings and a negative tunnelling current is meaningless); and `band_cutoff` is `stm.f90`'s fixed truncation at three widths, off by default because it is an approximation.

The plane is read out of the grid exactly. A density is a finite sum of plane waves, so its value at a point between grid points is that sum evaluated there rather than a spline through neighbours. It matters for the one thing an image is about: the corrugation in the vacuum is a small modulation on a quantity falling by orders of magnitude across one grid spacing, and an interpolant of it carries the grid's own periodicity as a false corrugation. The three corners are in *crystal* coordinates, Elk's `vclp2d`, and the sampling excludes the far edge (i/n, not $i/(n-1)$) so a whole-cell image tiles.

Against `pp.x`: 6.7×10^{-10} on every point of the 15^3 grid of QE's own fcc aluminium, reading its `filplot` text dump so that nothing is interpolated on either side. Three of QE's conventions are needed to make it the same sum: `stm.f90` divides by the volume and not by `degauss`, so its image is the zero-bias one times the width; it uses the run's own smearing; and it truncates the band sum at three widths. That truncation is worth 0.4% *and the complete sum is the smaller one* — Marzari–Vanderbilt's delta is negative for $x > \sqrt{2}$, so the states QE drops carry negative weight. The check that involves no other code is the sum rule $\int \rho_{\mathrm{STM}}\, d^3r = D(E)$ against `compute_dos`, which holds to 10^{-10} on a scalar run and on a spinor one, and in its window form on an ultrasoft one — the augmentation charge follows the tunnelling weights, which is why an ultrasoft or PAW image works here and `stm.f90`'s does not.

A magnetic tip is the part neither code has. It counts the states whose spin is along its own moment, so with P the tip's polarization

$$\rho_{\mathrm{STM}}(\mathbf{r};\hat{\mathbf{n}}) = \tfrac{1}{2}\left[\rho(\mathbf{r}) + P\,\hat{\mathbf{n}}\cdot\mathbf{m}(\mathbf{r})\right],$$

and $P = 1$ makes the image a genuine spin channel. On an antiferromagnetic hydrogen chain the charge image is flat to six figures while the two spin projections are each other's mirror ($\pm2.58\%$, antisymmetric to 10^{-3}), and they add back to the charge to 10^{-14}. On a chain whose four moments each turn 90 degrees from the last, a tip along x sees atoms 1

and 3 with opposite sign and atoms 2 and 4 **not at all** (10^{-16} against a contrast of 0.063), while a tip along z sees none of the four — with nothing imposing any of it.

The physics case is graphite. In AB-stacked bilayer graphene one sublattice of the surface layer sits above an atom of the layer below and the other above a hollow, so the two are inequivalent and only one is bright at the Fermi level: 1.64× here, on two atoms of the same element carrying the same charge. That is the textbook result that half the atoms of graphite are invisible to an STM, and it is a statement about the density of states *at the Fermi level* rather than about the charge. The tunnelling density falls off log-linearly into the vacuum ($R^2 = 0.9992$ over five orders) at a rate of at least $2\sqrt{V_{\mathrm{vac}} - E_F}$, which is Tersoff–Hamann's own statement and involves none of the assembly; the bound is one-sided because graphene's states at E_F sit at K, whose in-plane momentum steepens the decay (measured 1.86, against 0.98 for $k_{\parallel} = 0$).

Refuses. A spin spiral (its two spinor components live on different plane-wave spheres, so $|\psi(\mathbf{r})|^2$ is not the lattice-periodic object this sum builds), a constrained `tot_magnetization` (one Fermi level per channel, and a tip sees one energy), an applied magnetic field (its energy is outside the reported total, so the level the tip would be put at is not the field-free one the image would be read as), a spin direction on a run with no magnetization, and a *transverse* direction on a collinear run — there m_x is absent rather than zero, and projecting onto the axis would be a claim the calculation cannot make. A constant-current scan longer than the lattice period along the plane's normal, where the tip meets the periodic image of the surface and the density rises again. **Said rather than refused**: a constant-current image returns `nan` at every point where the set-point is not crossed inside the scan, with a warning saying how many; and a zero-bias image of a gapped run is identically zero, the delta sitting in the gap, which is what `bias` is for.

11.9 Tunnelling spectra: $\mathrm{d}I/\mathrm{d}V$ over an energy axis

An image is the local density of states at one tip energy, which is one picture at one bias. The other section of the same function is the curve at one place over many biases, which is what a scanning tunnelling microscope measures when it is used as a spectrometer, and what resolves a gap, a band edge or a state sitting inside a gap — none of which an image shows, since an image at a single energy reports only how much weight there is and not where in energy it sits,

$$\frac{\mathrm{d}I}{\mathrm{d}V}(\mathbf{r}, V) \;\propto\; \rho(\mathbf{r}, E_F + eV) \;=\; \sum_{\mathbf{k},n} w_{\mathbf{k}}\, \delta(E_F + eV - \varepsilon_{\mathbf{k}n})\, |\psi_{\mathbf{k}n}(\mathbf{r})|^2 \,,$$

the same sum `run_stm` builds, evaluated at every energy of an axis.

This is the measurement a modulation is read with, and it is why the quantity is here rather than only in the unit cell: a charge density wave is a *gap* that opens in antiphase with the charge maxima, a spin density wave moves the two spin channels in opposite directions from cell to cell, and a domain wall carries a state that lives nowhere else. Each of the three is an energy axis at every position, and an ultracell, the subject of *A modulation over many unit cells*, is the calculation that has them.

The energy axis is nearly free, and that is the whole of the implementation. $\psi_{\mathbf{k}n}(\mathbf{r})$ does not depend on the energy, so the states are sampled at the tip points *once* and each energy after that is one matrix product against the per-state weight. On an eight-cell spin density wave with a 96×12 map, one, twenty-one and forty-one energies cost 2.50, 2.56 and 2.61 s, against 2.47 s for a single image: forty-one energies are 4.4 per cent

more than one, and 39 times less than the forty-one images that are the only other way to get them. Neither `pw.x` nor Elk has an axis — `stm.f90` takes one scalar `sample_bias` and `wfplot.f90` sets one delta — so both compute a spectrum by being run again at every bias, paying the whole sum each time.

```
import numpy as np
from defuqumat import Calculator

calc = Calculator.from_file("si-ultracell-mag.in")
field = lambda x: 0.02 * np.cos(2 * np.pi * x[..., 0] / 8)
wave = calc.get_ultracell(supercell=(8, 1, 1), kgrid=(1, 2, 2), nbnd=32,
                          magnetic_field=field)

# a spectrum above the middle of each unit cell: one tip point per cell,
# which is the line cut across the modulation
tips = np.stack([np.arange(8) + 0.5, np.full(8, 0.5), np.full(8, 0.35)], axis=-1)
bias = wave.fermi_energy + np.linspace(-0.06, 0.06, 41)
sts = calc.get_ultracell_sts(energies=bias, tip=tips, width=0.02, spin="up")

sts.values        # (41, 8) dI/dV, one column per cell
sts.bias_axis     # (41,) the bias in Ry, measured from the Fermi level
sts.current       # (41, 8) I(V), the same axis integrated outwards from V = 0
sts.integral      # (41,) D(E) per unit cell -- the *charge's* sum rule, which
                  # is what the weights give exactly whatever the tip is

# a whole map at every energy instead, and the ordinary unit-cell spectrum
maps = calc.get_ultracell_sts(energies=bias, height=0.35, shape=(48, 8))
plain = calc.get_sts(energies=bias, tip=tips[:1], width=0.02, grid=(4, 4, 4))
```

The entry points are `run_sts` and `run_ultracell_sts`, or `Calculator.get_sts` and `Calculator.get_ultracell_sts`.

Reading it. `energies` is the axis in Rydberg and has no default: the axis *is* the measurement and a range has no natural width. What comes back is `values`, of shape `(nE,)` + whatever the sampling was — a row of tip points, or a plane. `current` is $I(V)$ within this approximation, a tip whose own density of states is flat and a barrier that does not depend on the bias; the decay of each state into the vacuum *is* in it, the prefactor is not, and the experiment's sign convention is kept, so the filled states below the Fermi level come back negative. A magnetic tip works exactly as it does for an image, through `spin` and `polarization`, and on a spin density wave it is what is wanted: an unpolarized tip cannot see the wave at first order and sees its square instead.

Refuses. Everything an image refuses, and two more of its own, both because $\psi(\mathbf{r})$ is sampled and squared here where an image sums into a density. A **symmetry-reduced k-set**: `run_stm` goes through the density, which is symmetrised, so a wedge gives it the whole zone's answer, while nothing symmetrises this one and a wedge would return the sum over that wedge alone — a smooth, plausible, wrong density of states everywhere off a symmetry axis. On silicon's $4\times4\times4$ grid reduced to eight k-points the wedge sum differs from the whole grid's by 98 per cent of the peak, while its *integral* is right to 1 per cent and the image on that same reduced set is right to 0.3 per cent — so a sum rule would not catch it and the image is genuinely unaffected. Pass `grid` (which builds the *complete* grid here, where an image's reduces) or run `nosym`. A tip inside an **augmentation sphere** of an ultrasoft or PAW dataset, where the pseudo-wavefunction is not the true one and the norm is short by 3 to 9 per cent. On a dense bulk crystal that leaves nowhere to put a tip at all — SiC's radius is 2.4 bohr and

its layers are closer together than that — which is the right answer rather than an obstacle: a tip sits above a *surface*, in vacuum, where a pseudo-wavefunction is the true one. An ultracell is exempt from the *first* of these, requiring `nosym` already, and takes the second: its image, its spectrum and its transmission all ask the guard on the unit cell's own coordinates, every copy of the ultracell holding the same atoms at the same heights. Also refused: `bias` together with an energy axis, since the window is the integral the axis already carries and `current` is it; and a constant-current spectrum, where the tip moves as the bias is swept, which is a different experiment and n_{heights} times the cost. **Said rather than refused**: `current` raises on an axis whose step is coarser than the smeared delta, because a trapezoid that steps over a level returns a number of the wrong order rather than a slightly wrong one — one level of width w on an axis of step h integrates to 1.00004 at $h = w$, 1.14 at $2w$ and exactly zero by $250w$, which is what any sane axis does against the 10^{-5} Ry width a fixed-occupation run gets by default. A run with no Fermi level takes the middle of the gap as the zero of bias, with a warning, rather than 0 Ry.

11.10 Vertical tunnelling transport through a two-dimensional material

An STM image says what the tip *sees*. This says what gets *through*: an electron enters at a point $\mathbf{r}$ above the material and leaves into an infinite plane below it — the substrate the sheet sits on — so what decides the current is the *nonlocal* Green's function between the two rather than the local density of states at the tip. A **point** tip makes Γ_t rank one, which collapses the Landauer–Büttiker trace exactly:

$$T(\mathbf{r};E) \;=\; \mathrm{Tr}[\Gamma_t G^r \Gamma_s G^a] \;\propto\; \int_{\text{plane}} \mathrm{d}^2 r' \,\big|G(\mathbf{r},\mathbf{r}';E)\big|^2 .$$

The substrate is taken to be *infinite and featureless*, so it is invariant under every lateral lattice translation and conserves the lateral momentum. The exit integral is then diagonal in $\mathbf{k}$, and everything reduces to one quadratic form per k-point,

$$T(\mathbf{r};E) = \sum_{\mathbf{k}} w_{\mathbf{k}} \sum_{nm} a_{n\mathbf{k}}(\mathbf{r})\, a^*_{m\mathbf{k}}(\mathbf{r})\, S_{\mathbf{k}}[n,m], \qquad S_{\mathbf{k}}[n,m] = \int_{\text{plane}} \psi^*_{n\mathbf{k}} \psi_{m\mathbf{k}}\, \mathrm{d}^2 r',$$

with $a_{n\mathbf{k}} = \psi_{n\mathbf{k}}(\mathbf{r})\sqrt{\tilde{\delta}(E-\varepsilon_{n\mathbf{k}})/\eta}$. So **the sum over k is incoherent and the interference is between bands at the same k**, which is what "the substrate is featureless" means physically. $S_{\mathbf{k}}$ is a Gram matrix, so the transmission is non-negative by construction and is blind to a rotation inside a degenerate multiplet.

Either electrode can be magnetic. A spin-polarized substrate is a 2×2 acceptance $P_s = (1 + P_s\,\hat{\mathbf{n}}_s\cdot\boldsymbol{\sigma})/2$ sitting *inside* $S_{\mathbf{k}}$, between two bands. A spin-polarized *tip* is the same matrix on the other side, $\Gamma_t = \gamma_t P_t$, and it is not a weight but a contraction: with the spinor Green's function $G_{ss'}(\mathbf{r},\mathbf{r}') = \sum_n a_{n,s}(\mathbf{r})\,\psi^*_{n,s'}(\mathbf{r}')$ the Landauer trace is no longer taken over the tip's spin, and the two spinor components stop adding and start interfering,

$$T(\mathbf{r}) = \int_{\text{plane}} \mathrm{d}^2 r'\, \mathrm{Tr}_{\text{spin}}\Big[\Gamma_t\, G\, \Gamma_s\, G^\dagger\Big] \;=\; \sum_{\mathbf{k}} w_{\mathbf{k}}\, \mathrm{Tr}_{\text{spin}}[P_t\, M_{\mathbf{k}}(\mathbf{r})]\,, \qquad M_{\mathbf{k}}[s,s'] = \mathbf{a}_s^{\mathsf{T}} S_{\mathbf{k}}\, \mathbf{a}^*_{s'}.$$

M is itself a Gram matrix, $A^{\mathsf{T}} S A^*$ for the $(n_{\text{bnd}}, 2)$ amplitude block, so it is positive semi-definite and $\mathrm{Tr}[P_t M] \geq 0$: the non-negativity survives. With *both* leads polarized the map depends on the angle between the two moments, which is a **tunnelling magnetoresistance** image. **Note the asymmetry of the names**: here `spin` is the *substrate* and `tip_spin` the *tip*, which is the opposite way round from `get_stm`, where `spin` is the tip. The substrate was the polarizer first and the name was kept.

```
from defuqumat import Calculator

calc = Calculator.from_file("graphene.scf.in")

# tip plane above the sheet, substrate plane below it, atoms in between
t = calc.get_vertical_transport(exit_height=0.38, height=0.62,
                                shape=(48, 48), broadening=0.02,
                                grid=(6, 6, 1))

t.image            # (48, 48) the transmission map, arbitrary units
t.interference     # coherent minus incoherent: what no local picture gives
t.notes["channels"]  # open transmission channels the substrate offers
t.extent()         # (x0, x1, y0, y1) for imshow

# a transport dI/dV at one place: the tip sampling is paid once
import numpy as np
spectrum = calc.get_vertical_transport(
    exit_height=0.38, tip=np.array([[0.0, 0.0, 0.62]]),
    energies=calc.get_scf().fermi_energy + np.linspace(-0.2, 0.2, 41),
    broadening=0.02, grid=(6, 6, 1))

# a spin-polarized substrate: only on a magnetic run, and the two
# directions partition the unpolarized answer exactly
magnet = Calculator.from_file("h-sheet-lsda.scf.in")
up = magnet.get_vertical_transport(exit_height=0.15, height=0.85,
                                   spin="up", broadening=0.05)
down = magnet.get_vertical_transport(exit_height=0.15, height=0.85,
                                     spin="down", broadening=0.05)

# a magnetic tip on a noncollinear run, and both electrodes at once:
# the map then measures the angle between the two moments
spinor = Calculator.from_file("h-sheet-noncolin.scf.in")
valve = dict(exit_height=0.15, height=0.85, broadening=0.05,
             shape=(24, 24), spin=(1.0, 0.0, 0.0), polarization=0.9)
parallel = spinor.get_vertical_transport(tip_spin=(1.0, 0.0, 0.0),
                                         tip_polarization=0.9, **valve)
antiparallel = spinor.get_vertical_transport(tip_spin=(-1.0, 0.0, 0.0),
                                             tip_polarization=0.9, **valve)
tmr = antiparallel.image.mean() / parallel.image.mean()   # 5.29 on this cell

parallel.tip_spin          # (1.0, 0.0, 0.0), beside .spin for the substrate
parallel.tip_polarization  # 0.9
```

The amplitude is the on-shell one and that is a measurement, not a preference. The literal Landauer denominator $1/(E-\varepsilon+i\eta)$ cannot be evaluated by a sum over states: the states far from E build the barrier's evanescent decay entirely by cancellation between themselves. On a cell small enough to diagonalise completely (367 plane waves, 367 exact bands) the running sum for one $G(\mathbf{r},\mathbf{r}')$ wanders over more than an order of magnitude and lands only at the complete basis, with a cancellation ratio of **349**. Replacing the denominator by its modulus — which is Bardeen's golden rule, the sample visited on shell — keeps the interference between bands degenerate at the tip energy, which is the interference a real experiment lets happen, and converges: to **3e-7** in the mean between 8 and 45 bands. `method="resolvent"` is reachable and warns.

What a map holds is one k-point, the bands near the tip energy and a block of pixels, so a large cell or a large image costs time rather than memory. A band whose on-shell

amplitude is below 10^{-18} of the largest at every energy asked for, among the spin channels the leads accept, is not sampled at all, which for a Gaussian delta is about nine broadenings away and for a Fermi–Dirac one about 84, meaning that what it would have added is below round-off of the map; a degenerate block is kept whole, and `t.notes["window"]` reports how many bands were sampled out of how many there are. The exit-plane overlaps and the diagnostics are still built on every band, and `method="resolvent"` samples every band, since no term of its denominator is small. The tip pixels are taken in blocks of a fixed size, and a flat tip plane is folded onto its in-plane plane waves before it is sampled, as the exit plane is. None of this runs on an accelerator: on a card the band solve is the only part of the map that does, and the wavefunctions are read from it one k-point at a time.

Validated by identities, since neither `pw.x` nor Elk computes this. (QE's `pwcond.x` is a Landauer transmission of a different geometry: two semi-infinite crystalline leads with the current along one axis, one conductance per energy, and no point contact and so no map. Elk has none.) Widening the substrate from a plane to the whole cell makes $S_{\mathbf{k}}$ the identity by orthonormality, so the quantity becomes the Tersoff–Hamann tunnelling density of states *exactly* — agreeing with `run_stm` to **4.6e-13** with no factor between them, which is the check a normalisation error could not hide in. $S_{\mathbf{k}}$ swept along its own normal integrates to the identity to 1.4e-15, and agrees with a real-space quadrature to 5e-16. A substrate polarized along $\hat{\mathbf{n}}$ plus one along $-\hat{\mathbf{n}}$ is a substrate that takes both, to round-off; one across the moment takes exactly half. The same holds for the *tip*, to **3.1e-16**, with the substrate polarized along a third direction at the same time, and a tip at $P = 0$ is exactly half the unpolarized map (1.5e-16), the convention a nonmagnetic tip has in the STM images. A *magnetic* tip in the whole-cell limit reproduces the spin-polarized tunnelling density of states to **5.8e-13**, on a sheet whose moment is put in a generic direction: the transposed index order of M would return the answer for a tip at $(n_x, -n_y, n_z)$, which is real, non-negative and — on a moment lying along an axis — identical, so the check is only a check with $m_y \neq 0$. On that cell the mirrored tip's image differs by a factor of two. Both electrodes polarized at $P = 0.9$ give a parallel-to-antiparallel contrast of a factor **5.3**, with a perpendicular tip at the mean of the two to 5e-6. The same cell run as $n_{\text{spin}} = 1$, $n_{\text{spin}} = 2$ and as a spinor agrees to 1e-10 and 1e-5. And the physics: graphene's two Dirac states at K are degenerate partners, so the symmetry of that point makes the substrate's overlap on the pair a multiple of the identity and leaves nothing off-diagonal for the current to interfere through; its map correlates with the STM image at **1.000000** with zero interference, while an AB bilayer — whose current must cross both layer-polarized sheets — falls to **0.16**, its coherent map a factor of 26 below the incoherent one.

Refuses. A k-set with more than one division along the stacking axis: the lateral momentum is conserved exactly and the perpendicular one is not, so two $k_\perp$ at the same $k_\parallel$ interfere with a phase that depends on where the exit plane sits. A *symmetry-reduced* k-set, because a wedge sums to the map symmetrised over the whole point group while only the subgroup that leaves the exit plane in place belongs to this geometry — a mirror through the slab exchanges the tip side with the substrate side; `grid=` builds the whole grid for that reason. A tilted exit plane, which has no closed-form in-plane orthogonality to stand on. A tip or exit plane *inside* an augmentation sphere on an ultrasoft or PAW dataset — in the vacuum a pseudo-wavefunction is the true one, which is why those datasets otherwise need nothing extra. A spin-selective substrate *or a spin-polarized tip* on a run with no magnetization, there being nothing for either moment to couple to and every direction taking half of everything, which is the charge map again; and a transverse direction on a collinear run for either lead,

where m_x and m_y are absent rather than zero. And, inherited from the STM images unchanged: a spin spiral, a constrained `tot_magnetization`, an applied magnetic field. A `bias=` window whose axis does not resolve `broadening`: the current is a trapezoid over a smeared delta, so an axis coarser than one point per width steps over the levels rather than integrating them and the answer is wrong by orders. Measured on the hydrogen sheet at $\eta = 0.02$ Ry over a 0.24 Ry window, against an axis eight times finer: 0.997 of it at $h = \eta$, **1.010 at** 2η, 0.298 at 4η and 0.121 at 12η — not monotone, so a coarse axis reads high before it collapses. The refusal names the `nenergies` that would do. **Said rather than refused**: a geometry whose atoms do not lie between the two planes warns — a cell is periodic, so with both planes on the same side the electron tunnels through the vacuum and around the periodic image, which is a real number and not this one. **Not claimed**: a finite contact patch (which breaks the k-diagonality), a tip with structure beyond an s-wave, and an absolute conductance — the tip and substrate couplings are unfixed prefactors, so what the result carries is the map and its contrast.

11.11 Which k-points the tunnelling current comes out of

The section above asks *where on the surface* the current goes. This asks the conjugate question: *which states* carry it. Replace the point tip by a plane parallel to the surface and the real-space map collapses; what is left is one number per k-point, and on a two-dimensional metal that number is the answer to "which pocket of the Fermi surface does an electron actually tunnel out of".

Integrating the point-tip map over one cell's worth of the tip plane turns $\int a_{n\mathbf{k}}(\mathbf{r})\, a^*_{m\mathbf{k}}(\mathbf{r})\, \mathrm{d}^2 r$ into $g_{n\mathbf{k}} g_{m\mathbf{k}} S^{\mathrm{tip}}_{\mathbf{k}}[m,n]$, where S^{tip} is the *same* Gram matrix as the exit plane's, evaluated at the tip's height. So

$$W(\mathbf{k}) \;=\; w_{\mathbf{k}} \,\mathrm{Tr}\Big[D_{\mathbf{k}}\, S^{\mathrm{exit}}_{\mathbf{k}}\, D_{\mathbf{k}}\, S^{\mathrm{tip}}_{\mathbf{k}}\Big], \qquad D_{\mathbf{k}} = \mathrm{diag}\big(g_{n\mathbf{k}}\big), \qquad \sum_{\mathbf{k}} W(\mathbf{k}) = \int_{\mathrm{tip}} T(\mathbf{r})\, \mathrm{d}^2 r.$$

That last equality is a theorem, not an approximation: this is the same object as the map, and nothing is sampled in real space, so it costs a small fraction of one — two Gram matrices and an n^2_{bnd} trace, against an $n_k \times n_{\mathrm{bnd}} \times n_{\mathrm{points}}$ sampling.

Both sides of it are Brillouin-zone integrals, so it holds mesh for mesh. Passing `grid=` re-solves the bands on the grid you name, which is what the refusals below require; comparing the result against a quantity built on the k-points the SCF converged on then compares two different integrals of the same integrand. `run_stm` is that case and has no `grid` of its own, so a comparison against it takes no `grid` here either. On the hydrogen sheet of the test suite, whose SCF runs a $4 \times 4 \times 1$ grid, $\sum_{\mathbf{k}}$ `tersoff_hamann` is the image's plane integral to 1.2×10^{-13} on the SCF's own k-points and a factor of 7.7 out on a $3 \times 3 \times 1$ one.

Three columns come back together, because they are the same trace with one or both matrices replaced by the identity, and *the physics is in their ratio*. `weight` is $W(\mathbf{k})$; `tersoff_hamann` is its $S^{\mathrm{exit}} \to 1$ limit, which is the tip plane's local density of states and so what an STM would see; `bare` is the further limit in which the tip plane stops discriminating too, the plain Fermi surface $\sum_n \tilde{\delta}(E - \varepsilon_{n\mathbf{k}})$. A fourth, `incoherent`, is every channel tunnelling on its own, taken in the basis that diagonalises S^{exit} inside each degenerate multiplet — that column is a *diagonal* and so is not invariant under the rotation a degenerate eigensolver is free in, where W is a trace and needs no such care.

Why the ratio is large. A state at in-plane momentum $\mathbf{k}_\parallel$ decays into the vacuum as

$\psi \sim e^{-\sqrt{\kappa_0^2+k_\parallel^2}\,z}$, so a zone-corner pocket is suppressed against a zone-centre one by a factor that grows exponentially with the gap. That is also the **check that shares no machinery with either code**: fit a decay constant to each pocket over a height sweep and they must satisfy $\kappa_1^2 - \kappa_2^2 = |\mathbf{k}_1|^2 - |\mathbf{k}_2|^2$. κ is the *amplitude* decay and W is quadratic in ψ on each plane, so a sweep of the tip alone has $\mathrm{d}\ln W/\mathrm{d}z = -2\kappa$ and a sweep of both planes -4κ; halving on one side only makes the identity come out four times too large, which is what it is for. A height sweep costs *one* band solve, not one per height — the bands do not know where the tip is.

```python
import numpy as np
from defuqumat import Calculator
from defuqumat.transport.momentum import decay_constants, decay_identity, pocket_mask

calc = Calculator.from_file("nbse2.in")          # a 1H-NbSe2 monolayer

# the two planes straddle the slab; a sweep of tip heights costs one band solve
run = calc.get_momentum_transport(exit_height=0.212, height=[0.677, 0.732, 0.788],
                                  grid=(6, 6, 1), broadening=0.006,
                                  smearing="gaussian")

run.weight            # (nheights, nk) -- the transmission per k-point
run.tersoff_hamann    # the same states, substrate made structureless
run.bare              # the plain Fermi surface
run.interference      # coherent minus incoherent
run.as_grid("weight") # reshaped to the Monkhorst-Pack grid, for a BZ image
run.per_kpoint()      # with the k-weight divided out, for another code

# the zone partition: closer to a corner than to Gamma, which on a hexagon is
# exactly 2/3 of the zone by area
corners = pocket_mask(run.kcartesian, calc.system.cell)
run.shares(corners, index=2)        # each column's share, and the suppression
run.reweighting(corners, index=2)   # (W_G/W_K)/(D_G/D_K): how the junction re-ranks

# the vacuum decay, and the identity that checks it
sweep = run.sweep("tersoff_hamann")
heights = np.array([1.5, 2.5, 3.5]) / 0.529177210903      # bohr
kappa = decay_constants(heights, sweep[:, [0, 14]])       # Gamma and K
decay_identity(kappa[1], kappa[0], run.kcartesian[14], run.kcartesian[0])
```

The index order is the whole difficulty, and it needs two distinct planes to see at all. The elementwise-looking alternative $\sum_{nm} g_n g_m S^{\mathrm{exit}}[n,m]\, S^{\mathrm{tip}}[n,m]$ is $\mathrm{Tr}[D S^{\mathrm{exit}} D (S^{\mathrm{tip}})^{\mathsf{T}}]$: real, non-negative, and *exactly* equal to the right answer whenever either matrix is diagonal — so it agrees in the Tersoff–Hamann limit, agrees on a single-band metal, and passes every check that does not involve two planes at once. On random orthonormal bands it is **3%** out, which is far above round-off and far below anything a plot would show. Only the literal plane integral separates them, and it does: the two routes agree to **1e-12**, one evaluating a Green's function on a real-space quadrature of the tip plane and the other never leaving reciprocal space.

Validated against a second implementation, which is new here. Neither `pw.x` nor stock Elk computes this: Elk's `fermisurf.f90` (tasks 100/101) writes bands for a plotting program without weighting them by anything, and its only tunnelling task is 162, the Tersoff–Hamann image. The one other implementation known is elkpy's Elk patch (task 9007), and it was written independently — same on-shell Bardeen amplitude, same full Gram matrices on both planes, same trace of a product, and the same index-order trap

recorded on its own side. On 1H-NbSe2 the two agree on the physics; the numbers are in `PLAN.md` P77.

Refuses. A *magnetic plane tip*: a polarized tip contracts the two spinor components through its own 2×2 projector, which the plane integral does not collapse the way it collapses the trace over position, so it is left to `get_vertical_transport`'s map rather than approximated here. The substrate's own acceptance does go in, inside the Gram matrix, exactly as it does there. A tilted tip plane, where the point-tip map allows one: the closed-form Miller-index orthogonality both sides rest on needs a plane spanned by two lattice vectors, and it is what makes this cheap. **Inherited from the vertical transmission unchanged**: a k-set with more than one division along the stacking axis, a symmetry-reduced k-set (`grid=` builds the whole grid), a plane inside an augmentation sphere on an ultrasoft or PAW dataset, gamma-only storage, a spin-selective substrate on a run with no magnetization, and — from the STM images — a spin spiral, a constrained `tot_magnetization` and an applied magnetic field. Both planes on the same side of the slab **warns** rather than refuses. **Not claimed**: an absolute conductance. The lead couplings are unfixed prefactors, so what this carries is the weights and their ratios — which is also why `shares` reports shares of a total rather than absolute weights.

12 Things with no `pw.x` counterpart

Spin spirals by the generalized Bloch theorem: a spiral of wavevector $\mathbf{q}$ is a spinor whose two components live on *different* plane-wave spheres,

$$\psi_{\mathbf{k}}(\mathbf{r}) = \begin{pmatrix} e^{i(\mathbf{k}+\mathbf{q}/2)\cdot\mathbf{r}}\, u^{\uparrow}(\mathbf{r}) \\ e^{i(\mathbf{k}-\mathbf{q}/2)\cdot\mathbf{r}}\, u^{\downarrow}(\mathbf{r}) \end{pmatrix}$$

so in the rotated frame the density is lattice periodic and the SCF, the functional and the mixer are untouched. On a norm-conserving dataset only two terms carry $\mathbf{q}$ — $|\mathbf{k} \pm \mathbf{q}/2 + \mathbf{G}|^2$ and v_{nl} — and $dE/d\mathbf{q}$ at frozen coefficients is `jax.grad` of them.

An ultrasoft or PAW dataset adds a third, and it is the one that used to refuse the combination. The transverse block of the density is $\langle\psi^{\uparrow}|\ldots|\psi^{\downarrow}\rangle$, so it pairs projectors at $\mathbf{k}+\mathbf{q}/2$ with projectors at $\mathbf{k}-\mathbf{q}/2$, and the augmentation charge reaching it is the lattice sum $\sum_{\mathbf{R}} e^{-i\mathbf{q}\cdot\mathbf{R}}\, Q_{ij}(\mathbf{r} - \boldsymbol{\tau}_a - \mathbf{R})$ rather than the periodic one. That sum is $e^{-i\mathbf{q}\cdot\mathbf{r}}$ times a lattice-periodic function — the same factor the smooth transverse density carries, which is what makes the rotated frame work at all — and the periodic factor is the transform of

$$Q_{ij}(\mathbf{G} - \mathbf{q})\, e^{-i(\mathbf{G}-\mathbf{q})\cdot\boldsymbol{\tau}_a},$$

the resident table displaced by $-\mathbf{q}$. The charge and m_z blocks pair projectors at the *same* $\mathbf{k}$ and keep the resident table. Nothing else changes: the displaced table is built by the same assembly, and at $\mathbf{q} \to 0$ it is the resident one term for term.

The transverse pair is kept on the laboratory sphere, and that is what makes the spiral its own supercell. The laboratory magnetization is $m_+ = m'_+ e^{-i\mathbf{q}\cdot\mathbf{r}}$, so the sphere an unfolded supercell keeps, $|\mathbf{G}'| < G_\rho$, is $|\mathbf{G} - \mathbf{q}| < G_\rho$ in the spiral's labels; the transverse augmentation charge, its `newd` integrals, and the smooth-to-dense and dense-to-smooth interpolation of the transverse density and potential all run on that sphere (and on $|\mathbf{G}-\mathbf{q}| < G_s$ for the smooth grid), while the charge and m_z stay on $|\mathbf{G}| < G_\rho$. You can think of it as cutting m'_+ where the supercell cuts m_+. On the resident sphere instead the two discretise the transverse charge on spheres displaced by $\mathbf{q}$, and the shell between them is

not small: on the cobalt helix of `tests/data/qe/co-helix4-spiral.in` at 20/160 Ry the supercell, diagonalised once at the unfolded density, differed from the spiral by 7.3×10^{-5} Ry in the eigenvalues and 4.2×10^{-3} in the output magnetization, and by 4.0×10^{-3} Ry at 80 Ry; on the laboratory sphere it is 3×10^{-14} Ry and 1.2×10^{-9}, and 1×10^{-13} at 80. The oxygen chain's quarter turn against its four-cell supercell went from 1.6×10^{-9} to 3.5×10^{-12} Ry (ultrasoft) and from 3.3×10^{-7} to 2.1×10^{-11} (PAW). The sphere's membership is a step function of $\mathbf{q}$, like the wavefunction spheres, so $dE/d\mathbf{q}$ freezes it with them, and an $E(\mathbf{q})$ scan rebuilds it at every point; its size changes with $\mathbf{q}$ by a few G-vectors (4325 against 4344 on cobalt at 160 Ry), so an augmented scan compiles its density kernels again where the count is new. Members past the box's $(n-1)/2$ are held at the wrapped index, which is exact as long as no two share one; the box is checked for that.

What the laboratory sphere does not fix, measured rather than assumed. The PBE cobalt helix does not converge on it: at 160 Ry it stops at $\sim 10^{-10}$ Ry with the total energy wandering by 5×10^{-5}, and at 80 Ry near 2×10^{-7}, where the resident sphere converged in 22 and 36 iterations. In LDA the same cell converges in 14 iterations either way, and on the resident sphere the PBE cell already failed at 320 Ry, so this is the gradient correction's $|\mathbf{m}|$ on a turning texture (a spiral has no fixed axis to take its sign along) rather than the sphere; the supercell limit-cycles the same way. Note what `converged` then means: the grid residual is read on the resident sphere, which still sees 99.9 per cent of the transverse residual (the shell holds 2×10^{-5} of it at 160 Ry), but it does not see `becsum`, whose magnetic residual stays at 1.7×10^{-3}, so on this cell a run can stop at `conv_thr = 1e-10` with its energy still moving by 5×10^{-5} Ry. The identities above are taken at one diagonalisation of a fixed density and do not need the state converged.

$dE/d\mathbf{q}$ on such a dataset differentiates three terms the norm-conserving one does not have. The displaced table is a function of $\mathbf{q}$ exactly as $|\mathbf{k} \mp \mathbf{q}/2 + \mathbf{G}|^2$ and v_{nl} are, so it is rebuilt inside the differentiated path rather than held fixed; the orthonormality constraint the frozen-coefficient gradient carries is $\langle\psi|S|\psi\rangle - 1$ rather than $\langle\psi|\psi\rangle - 1$, and S pairs each spinor component with the projectors of its *own* shifted sphere, so it moves with $\mathbf{q}$ and its derivative is the spiral's Pulay term; and PAW's one-centre energy is part of the total being differentiated. The last of those is the one a test would have caught immediately — without it the functional misses the whole of E_{PAW} — and the first two are the ones that would have been silent, which is why each is measured separately below.

What it agrees with, and what PAW's own floor is. Against a central difference of the *re-converged* energy at a frozen plane-wave sphere, on the oxygen chain at $q_3 = 0.3$, an ultrasoft dataset converges onto the finite difference as δ^2 and reaches 2.4×10^{-7} Ry per unit $\mathbf{q}$ at $\delta = 0.0025$. PAW stops falling at 1.4×10^{-6}, and that floor is *not* the spiral's.

The sharpest way to see it needs no finite difference at all: at $q_3 = 0$ and $q_3 = \frac{1}{2}$ symmetry forces $dE/d\mathbf{q}$ to vanish exactly, so what comes back there *is* the gradient's own error — 4.6×10^{-10} at the zone centre for ultrasoft, and at the zone boundary 1.3×10^{-7} (ultrasoft) and 1.2×10^{-8} (PAW). PAW's used to read 2.5×10^{-6} there, falling by seven when `ecutrho` was doubled, and was put down to PAW's spherical terms against the grid; it was the shell between the resident sphere and the laboratory one, which the same run with the resident spheres reproduces (2.48×10^{-6}). The ladder's floor is a different thing and did not move: 1.40×10^{-6} against 1.38×10^{-6} on the resident spheres, so that one is not the shell, and what it is is open.

Two consequences are worth knowing rather than discovering. The transverse pair is augmented as one *complex* field, because $Q_{ij}(\mathbf{r})$ is real and so the resident table satisfies $Q_{ij}(-\mathbf{G}) = Q_{ij}(\mathbf{G})^*$ while the displaced one does not; and the identity against a supercell closes to 3.5×10^{-12} Ry on an ultrasoft dataset and 2.1×10^{-11} on a PAW one. The 1.7×10^{-9} and 3.3×10^{-7} this page used to give were the shell between the resident sphere and the

supercell's, and the second was put down to PAW's one-centre path, which a laboratory-sphere run with the same path refutes.

```
from defuqumat.workflows.spiral import (run_spiral_scan, relax_spiral_q,
                                        heisenberg_exchange)

scan = run_spiral_scan(system, pseudos, wavevectors)   # E(q)
best = relax_spiral_q(system, pseudos)   # BFGS on the reciprocal metric
print(best.wavevector, best.scf.total_energy)

J = heisenberg_exchange(scan, system.cell, shells)   # fit E(q) for the J(R)

scan = run_spiral_scan(system, pseudos, wavevectors, gradients=True)
print(scan.relative)     # E(q) - E(q_0) in mRy, from the energies
print(scan.integrated)   # the same curve, from dE/dq along the path
```

`heisenberg_exchange` fits $E(\mathbf{q}) - E(0) = m^2 \sum_s J_s \sum_{\mathbf{R}\in s}[1 - \cos(\mathbf{q}\cdot\mathbf{R})]$, one J_s per neighbour shell s, which is how a spiral scan becomes a spin model. Each shell is named by one representative lattice vector, and the inner sum runs over its whole star under the lattice point group (or the `point_group` passed in), each $(\mathbf{R}, -\mathbf{R})$ bond counted once, so the fitted J_s does not depend on which member of the star was typed. `compute_spiral_gradient` gives $dE/d\mathbf{q}$ on its own.

$E(\mathbf{q})$ two ways. With `gradients=True` the scan also takes $dE/d\mathbf{q}$ at every point, and `scan.integrated` accumulates it along the path,

$$E(\mathbf{q}_n) - E(\mathbf{q}_0) = \sum_m \int \mathrm{d}\mathbf{q} \cdot \frac{\partial E}{\partial \mathbf{q}},$$

by the trapezoid rule on the segments between successive wavevectors (both factors in lattice coordinates, so the contraction needs no metric and the path may bend). The two curves are the same physics and the gap between them is the useful part: the energies are computed on a plane-wave sphere *rebuilt* at every point and step by the Pulay error of a finite basis wherever a plane wave crosses the cutoff, while the gradient is taken at a *frozen* sphere and does not see those steps. On the hydrogen chain of `tests/data/qe/h-chain-spiral.in` at `ecutwfc = 25` the direct energies rise, fall and rise again over a 13-point scan where the integrated curve descends monotonically; the two agree to 0.005 mRy at `ecutwfc = 60` once the quadrature error is removed, against 0.130 at 25.

Two error sources live in that gap and refining tells them apart: the trapezoid rule's own h^2 shrinks when the $\mathbf{q}$ path is refined ($0.051 \to 0.016 \to 0.003$ mRy for $7 \to 13 \to 25$ points) and the basis noise does not (0.139, 0.138, 0.137). What the integrated route does *not* buy is worth stating, since the intuition that it might is a common one: it is not cheaper (every point still needs its own SCF), it does not converge on a coarser k-mesh (the gradient is the exact derivative of the *same* fixed-mesh energy, so it inherits the same Brillouin-zone error), and it needs a *tighter* `conv_thr` rather than a looser one, an energy's error being second order in the density's where a derivative's is first.

Also here: **per-atom magnetic fields**, the **topological invariants** of the previous section, and **rank-n tensor symmetrisation** — `symme.f90`'s `symmatrix3`/`symtensor3` written at any rank.

Refuses. A spiral with spin-orbit coupling switched on, permanently — it breaks the theorem, and Elk refuses it too; at `soc_scale = 0` a fully-relativistic dataset is admitted, as the zeroth order of the next subsection; a spiral with symmetry, until the spin space

group is written, so a spiral needs `nosym`; `mixing_space = 'g'` on a magnetic spiral, since that layout keeps every channel on the resident smooth sphere and the transverse pair lives on the displaced one, so each step would cut the shell between them (the default, `'r'`, mixes the whole dense grid); a spiral whose FFT box would fold two members of the displaced sphere onto one index, which no cell measured here does, since `pw.x`'s FFT sizes leave room; $dE/d\mathbf{q}$ on a dataset whose augmentation charge is held as a *radial* table rather than on the dense **G** set — the branch a large cell falls onto when $n_h^2 n_{\mathrm{gm}}$ is over `DEFUQUMAT_AUG_MAX_BYTES` — because that route reads $|\mathbf{q}|$ on the host to size its interpolation and running past the end of the table is a NaN rather than a clamp, so a tracer cannot go through it at all; and spiral relaxation under a magnetic field, whose energy is outside the reported total, so the state is stationary for a different functional than the one being differentiated. `heisenberg_exchange` refuses a shell vector that is not a nonzero integer lattice vector in crystal coordinates, and two representatives of the same star, whose two J would be one parameter and leave the fit singular.

What bounds its memory. $dE/d\mathbf{q}$ is walked `k_batch` k-points at a time wherever the calculation carries a chunk size — a CPU, or `memory_mode='memory'` on a card whose mesh does not fit whole — and in one pass otherwise. On an augmented dataset the chunks are exact by the chain rule: the density carries **q** there and the Hartree energy is quadratic in it, so the terms that see **q** through the whole `becsum` are differentiated once at the whole `becsum` and their gradient is pulled back through each chunk's. Measured on ultrasoft spiral silicon on a GTX 1060, the single pass peaks at 343, 1013, 2257 and 4351 MB at 8, 27, 64 and 125 k-points, and the chunked route at 104, 70, 74 and 121 MB, the two gradients agreeing to 1e-15. (Until 2026-09-28 an augmented dataset was forced down the single pass with a warning.)

12.1 Spin-orbit coupling on a spiral, to first order

We will now see what spin-orbit coupling does to a spiral when it is small. A spiral with the coupling is not a spiral any more, since the coupling ties the spin to the lattice and translating by **R** while turning every spin by $\mathbf{q}\cdot\mathbf{R}$ stops being a symmetry, but the spiral without it is exact in the unit cell and the coupling can be added on top as a perturbation. Its first order is the Dzyaloshinskii–Moriya energy of the spiral, the energy that tells one sense of rotation from the other and that sets whether a polar magnet prefers a cycloid, and it is what STM on a chiral surface texture and the pitch of a helimagnet measure.

The zeroth order is the spiral on a fully-relativistic dataset with the coupling switched off, `soc_scale = 0`, which is the one form of the pair the reader admits: `soc_scale` blends the coupling's part of `dvan_so` linearly, so the first-order operator is the coupled nonlocal coefficients minus the reduced ones, $\delta D = D^{\mathrm{so}}(1) - D^{\mathrm{so}}(0) = \sum_a \delta D^a \sigma_a$, and at zero the Hamiltonian commutes with every spin rotation. What matters is that δD is the same matrix in every cell while the spins turn, so in the frame of the spiral's axis $\hat{\mathbf{n}}$ only its σ_n part keeps each spinor component on its own sphere; the transverse part moves the state at **k** to the ones at $\mathbf{k}\mp\mathbf{q}$ and has no expectation value. The first-order energy is therefore

$$E_1(\mathbf{q},\hat{\mathbf{n}}) = \hat{\mathbf{n}}\cdot\mathbf{V}(\mathbf{q}), \qquad V_a = \sum_{\mathbf{k}} w_{\mathbf{k}} \sum_n f_{n\mathbf{k}} \Big[\langle u^{\uparrow}_{n\mathbf{k}} | \delta D^a | u^{\uparrow}_{n\mathbf{k}} \rangle_{\mathbf{k}+\mathbf{q}/2} - \langle u^{\downarrow}_{n\mathbf{k}} | \delta D^a | u^{\downarrow}_{n\mathbf{k}} \rangle_{\mathbf{k}-\mathbf{q}/2} \Big],$$

each component projected on the projectors of its own sphere, meaning that the energy is linear in the axis and one vector gives it for every orientation of the spiral plane. Here $\hat{\mathbf{n}}$ is the spin direction of the up component, about which the moment turns by $-\mathbf{q}\cdot\mathbf{r}$. Two properties are exact: the spiral at $(\mathbf{q},\hat{\mathbf{n}})$ is the texture at $(-\mathbf{q},-\hat{\mathbf{n}})$, so $\mathbf{V}(-\mathbf{q}) = -\mathbf{V}(\mathbf{q})$ and

the whole first-order energy is the chirality; and inversion through a site sends $(\mathbf{q}, \hat{\mathbf{n}})$ to $(-\mathbf{q}, \hat{\mathbf{n}})$, so $\mathbf{V} = 0$ on a centrosymmetric crystal. At small $\mathbf{q}$, $\mathbf{V} = \mathrm{D}\,\mathbf{q}$ with D the micromagnetic tensor of $E = D_{ij}\, n_i q_j$. By Hellmann and Feynman E_1 is $\mathrm{d}F/\mathrm{d}(\texttt{soc_scale})$ at zero, the free energy at frozen occupations.

```
calc = Calculator.from_file("nii-chain-spiral.in", "../pseudo")  # spiral_q, lspinorb,
    ↪ soc_scale = 0
first = calc.get_spiral_spin_orbit_energy()
print(first.vector_mev)            # V(q), meV per cell, cartesian
print(first.energy_mev((0, 1, 0))) # E1 with the spiral's axis along y
print(first.easy_axis)             # the axis the coupling prefers, -V/|V|
```

The result is a `SpiralSpinOrbit`: `vector` is $\mathbf{V}$ in Ry per cell, `energy(axis)` and `energy_mev(axis)` are $\hat{\mathbf{n}}\cdot\mathbf{V}$, `easy_axis` is $-\mathbf{V}/|\mathbf{V}|$, `by_k` resolves $\mathbf{V}$ by k-point (the weights inside), `dropped` carries the transverse contraction the spiral's layout would have added if the coupling turned with the spins, and `trace` the operator's spin trace, zero to round-off.

What it agrees with. There is no reference code for the quantity, so the checks are a supercell and a limit, on a nickel chain with an iodine beside each bond (`tests/data/qe/nii-chain-spiral.in`; 40 Ry and eight k-points for the numbers here). The iodine breaks inversion and leaves one mirror, the plane of the atoms, so $\mathbf{V}$ can only lie along y, the normal to that plane: at $q = \frac{1}{4}$ it is 2.78×10^{-2} meV per cell along y, with the other two components at 2×10^{-5}. The four-cell supercell diagonalised at the unfolded spiral density, where both spinor components share one sphere and the whole of δD is contracted with nothing masked, gives the same first-order energy to 1.8×10^{-11} Ry per cell, the spread between two spiral runs at 34 and 48 bands. With the coupling scaled, $H_0 + \lambda\,\delta D$ on that supercell ($\texttt{soc_scale} = \lambda$, admitted between 0 and 1 because every dataset here is norm-conserving), the odd part of the free energy over λ is 0.0727, 0.1143 and 0.2941 meV at $\lambda = 0.1$, 0.2 and 1, and a quadratic through them reaches 0.0267 at zero, against 0.0278. The remainder is linear in λ, not quadratic: the transverse part enters second order as a quadratic form whose antisymmetric piece changes sign with $\mathbf{q}$, so the chirality energy has a second-order term, and on this chain it is seventeen times the first-order one, which is itself a remainder of k-point contributions two hundred times larger than their sum. So E_1 is the right limit and not, on this cell, the energy at the physical coupling; the even part, $-206\lambda^2$ meV, is second order exactly as it should be. The test cell is the same chain at 34 Ry, where $V_y = -1.2179$ meV, the run at $-\mathbf{q}$ gives $+1.2179$ to 1.3×10^{-5} of it, the supercell agrees to 9×10^{-12} Ry, the scaled coupling reaches V_y to 2.3×10^{-5}, and the second-order odd part is a tenth of the first-order one rather than seventeen times it. **None of these values is the chain's**: the nickel dataset wants about 90 Ry, the first order is -2.35, -1.22 and $+0.028$ meV at 30, 34 and 40 Ry on eight k-points, and a remainder of cancelling k-point contributions converges only with the cutoff and the mesh together.

The supercell's grid has to be the spiral's repeated. The unfolding is exact on any grid that holds the density's components, but the exchange-correlation potential built from it is evaluated point by point, and `pw.x`'s grid rule gives the supercell four times the unit cell's points only for some cutoffs: at 30 Ry the chain's grid is 15 and the supercell's 64, and the first order moved 1.3 per cent. `unfold_spiral_density` warns.

The supercell's starting moments have to follow the texture. With every starting moment parallel, a gradient-corrected noncollinear run takes the sign of $\mathbf{m}$ along that axis to resolve the density (`pw.x`'s `lsign`), and on a texture that turns in the laboratory frame the sign flips where the moment crosses perpendicular to it, so the resolved density jumps there. That is a different functional from the spiral's, which takes $|\mathbf{m}|$: started all along x the supercell's PBE potential was 0.83 Ry from the spiral's at the same density and its

first-order energy 33 times larger. `fixed_density_states` warns when it is handed such a density.

Refuses. The spiral with the coupling switched on, at `soc_scale = 1`, in the reader and again inside the calculation, since `with_soc_scale` is a field replacement that never meets the reader. A **q** in the reciprocal lattice, where the two spheres coincide and the transverse part has an expectation value of its own: that is a uniform magnet, and its first-order term is `frozen_expectation`. A scalar-relativistic dataset, whose difference from a relativistic file is not a perturbation of anything. An **ultrasoft or PAW** dataset, where `soc_scale` also changes `qq_so` and `newd_so`'s sandwich of the augmentation integrals, and the exchange field inside that sandwich turns with the spiral while `fcoef` does not, so which of its blocks keeps a component on its own sphere is a derivation not yet written. A spiral converged under a magnetic field or a constrained moment is refused by the fixed-density step it passes through, which needs the converged field handed over.

13 A modulation over many unit cells: the ultracell

A spin density wave in chromium has a period of 21 unit cells. A charged impurity in silicon is screened over about 20. A skyrmion is hundreds across. Each of them is a slow *envelope* on a crystal that is still atomically periodic, and a supercell pays the same price per degree of freedom for the envelope as for the atoms — which is why that length scale is the one nobody computes from first principles.

The ultracell separates the two. Take $N = n_1 n_2 n_3$ copies of the unit cell. The reciprocal lattice vectors of that ultracell lying inside the unit cell's Brillouin zone are N wavevectors $\mathbf{Q}$, and everything allowed to be long ranged is written

$$\rho(\mathbf{r}) = \sum_{\mathbf{Q}} \rho_{\mathbf{Q}}(\mathbf{r})\, e^{i\mathbf{Q}\cdot\mathbf{r}}, \qquad \rho_{\mathbf{Q}} \text{ lattice periodic.}$$

The ultracell's Kohn-Sham states at an ultracell Brillouin zone point $\mathbf{k}$ are then expanded in the *unit cell's own* Kohn-Sham states at the N points that fold onto it,

$$|\Psi\rangle = \sum_{\mathbf{Q},n} c_{\mathbf{Q}n}\, |\psi_{\mathbf{k}+\mathbf{Q},n}\rangle, \qquad n \le \texttt{nbnd},$$

and what is diagonalised is the $N\,\texttt{nbnd} \times N\,\texttt{nbnd}$ matrix

$$H_{(\mathbf{Q}m),(\mathbf{Q}'n)} = \delta_{\mathbf{Q}\mathbf{Q}'}\delta_{mn}\,\varepsilon_{\mathbf{k}+\mathbf{Q},n} + \langle\psi_{\mathbf{k}+\mathbf{Q},m}|\,\delta V\,|\psi_{\mathbf{k}+\mathbf{Q}',n}\rangle,$$

with δV the ultracell potential *minus* the unit cell's — the eigenvalue on the diagonal already carries the whole unmodulated potential, so putting it in twice would be the one easy mistake here.

Why this is cheap. The unit-cell states at the folded k-points are computed **once, before the self-consistent loop**, and frozen. Every iteration after that builds a small dense matrix and diagonalises it; no plane-wave Hamiltonian is touched again. The variational dimension per k-point is $N\,\texttt{nbnd}$ where a real supercell's is $N\,\texttt{npw}$, and the ratio $\texttt{npw}/\texttt{nbnd}$ is typically 10^2 to 10^3. It is a prefactor, however, not a change of scaling: the dense solve is still $O((N\,\texttt{nbnd})^3)$, so this reaches a few thousand cells along one direction rather than a micron in three.

What the approximation is, exactly. As `nbnd` grows to the full plane-wave count, the basis above spans precisely the ultracell's own plane-wave basis. So the method is a *variational truncation of the exact N-cell supercell problem to `nbnd` bands per folded k-point*,

and nothing else. An eigenvalue's convergence carries no sign — the Hamiltonian moves with its own truncated density, so an ultracell eigenvalue is not an upper bound on the supercell's, and neither is a density — but it does converge, which is the property the implementation is checked against: a two-cell silicon ultracell under an applied modulation reproduces a real four-atom supercell's induced density, and the disagreement falls monotonically with `nbnd` (`tests/regression/test_ultracell.py`).

The total energy is the exception, and it does carry a sign. What the run reports as `total_energy` is the Kohn-Sham free energy of the state the loop converged to, and the bases are *nested* in `nbnd`, so it is a monotone non-increasing upper bound on the N-cell supercell's own energy: $+8.15\times10^{-5}$, $+3.52\times10^{-6}$, $+3.62\times10^{-7}$ and $+1.43\times10^{-7}$ Ry at `nbnd = 12, 24, 48, 64` on that same silicon cell. Comparing against a supercell needs the *same* FFT box on both sides, since two discretisations of one functional differ by about 10^{-6} Ry per cell here, which is larger than the top of that ladder; within the ladder the box is the same one and the monotonicity is exact.

```
from defuqumat import Calculator
import numpy as np

cal = Calculator.from_file("si.in")        # the two-atom unit cell

# a potential of one ultracell period along a_1, in Ry; x runs over [0, n_i)
modulation = lambda x: 0.05 * np.cos(2 * np.pi * x[..., 0] / 8)

ulr = cal.get_ultracell(supercell=(8, 1, 1), kgrid=(1, 2, 2),
                        nbnd=24, external=modulation)

print(ulr.iterations, ulr.accuracy)    # dr2 on the ultracell box, in Ry
print(ulr.density.shape)               # (nspin, 8*n1, n2, n3): the whole ultracell
print(np.abs(ulr.modulation).max())    # rho - tile(rho_0): the induced envelope
print(ulr.fermi_energy, ulr.band_energy)
```

`supercell` is three integers and the ultracell lattice vectors are derived from them; `kgrid` samples the *ultracell's* Brillouin zone, which is N times smaller than the unit cell's, so the folded set the frozen states are computed on is `supercell * kgrid`. `external` is an applied potential in Ry, given either as an array on the ultracell grid or as a function of unit-cell crystal coordinates, which run over $[0, n_i)$ across the ultracell. `ulr.density` is normalised *per unit cell*, so it integrates to N times the electron count over the ultracell and equals the tiled unit-cell density exactly when nothing is modulated.

`nbnd` is the convergence parameter and it is the only one. It is the size of the variational basis per folded k-point, so the default — which for an insulator is the occupied bands and nothing above them — leaves the envelope nothing to be built from. Pass it. `ulr.multiplet_gap` reports the smallest gap the truncation opens anywhere on the folded k-set: where that is small the cut has fallen inside a degenerate multiplet, which member of it survives is whatever the eigensolver returned, and the basis is arbitrary there. **Whether that matters depends on where the cut is**, and the two cases were measured rather than assumed. Low — no more empty bands than occupied ones — the arbitrary rotation is among the very states the response is built from, and `run_ultracell` warns: `nbnd = 8` on silicon cuts at 2.6×10^{-14} Ry and does not converge at all. High in the empty manifold a cut this degenerate is both common and harmless — `nbnd = 32`, `48` and `80` each open one (6×10^{-15}, 5×10^{-12}, 4.7×10^{-11} Ry) and each converged in ten iterations to a monotonically improving answer — so the gap is reported there without a warning. Raise `nbnd` until the cut lands in a gap, or until it lands high enough not to be one.

Convergence. The mixed quantity is the density on the ultracell box and the convergence test is the Hartree energy of its residual — QE's `dr2`, generalised to $|\mathbf{G}+\mathbf{Q}|$ — so `conv_thr` means here what it means everywhere else in this package. Kerker preconditioning is on by default and matters more here than in a unit cell: the smallest non-zero $|\mathbf{G}+\mathbf{Q}|$ an ultracell carries is N times smaller, so the Hartree kernel there is N^2 larger and long-wavelength charge sloshing is the whole difficulty. Elk's own example of the method runs at a mixing parameter of 0.001 for up to two thousand iterations; a two-cell silicon ultracell under an applied modulation converges here in nine.

The total energy, and what it is for. `ulr.total_energy` is the Kohn-Sham free energy per unit cell, and it is what says whether a modulation is the ground state rather than merely what a modulation does: the energy *gain* of a spin density wave over the uniform state is a difference of two of these. Neither reference code has it — Elk's `energyulr.f90` computes the occupied eigenvalue sum alone and `pw.x` has no ultracell — and the eigenvalue sum is not the total: it double-counts the Hartree and exchange-correlation energies exactly as an ordinary `eband` does, because the frozen eigenvalue carries the whole self-consistent potential. The assembly is therefore QE's own,

$$E = (\texttt{eband} + \texttt{deband}) + E_{\mathrm{H}} + E_{\mathrm{xc}} + E_{\mathrm{Ewald}} + \texttt{demet},$$

with `deband` the integral of the density against the potential over the ultracell, and `ulr.energy_terms` carries the pieces. The Ewald term is the *unit cell's*, unchanged, because the atoms do not move.

```
# the same run as above -- the energy comes with it and costs nothing extra
print(ulr.total_energy)                # Ry, per unit cell
print(ulr.energy_terms)                # the pieces, summing to it
print(ulr.energy_history[-3:])         # one per iteration, approaching from above
print(ulr.field_energy)                # None without a magnetic_field
```

This is the one quantity in the method that converges with a sign. An ultracell eigenvalue is not an upper bound on the supercell's and neither is a density, for the reason above — the Hamiltonian moves with its own truncated density. The energy is a bound because the bases are *nested*: raising `nbnd` keeps every function already in the basis, and the union over $\mathbf{Q}$ of the folded k-point spheres is the supercell's own plane-wave space. So `total_energy` falls monotonically towards the real supercell's from above, and on two-cell silicon under an applied modulation it does: $+8.15\times10^{-5}$, $+3.52\times10^{-6}$, $+3.62\times10^{-7}$ and $+1.43\times10^{-7}$ Ry at `nbnd = 12, 24, 48, 64`.

Comparing against a supercell needs the same FFT box on both sides. Two discretisations of one functional differ by about 10^{-6} Ry per cell here, which is larger than the top of that ladder, and a supercell picks its dense grid from its own cutoff: at the default `ecutwfc = 12` it takes $(32,15,15)$ where the tiled unit cell's is $(30,15,15)$, and the third rung then comes out 1.49×10^{-7} Ry *below* the supercell. Choose a cutoff at which the two grids agree — `ecutwfc = 13` puts the unit cell on $(18,18,18)$ and the two-cell supercell on $(36,18,18)$ — and do it through `ecutwfc` rather than `ecutrho`, since an ultracell refuses a double grid. Within one ladder the box is the same on every rung and the monotonicity is exact.

An ultrasoft or PAW dataset. The augmentation charge changes nothing structural about the method, and saying why is shorter than the code. A matrix element between two ultracell basis functions runs over the whole ultracell, so it sums over the N copies of every atom, and each copy's projectors carry the Bloch phase $e^{i(\mathbf{k}_0+\mathbf{Q})\cdot\mathbf{R}}$; what survives

is

$$\sum_{\mathbf{R}} e^{i(\mathbf{Q}'-\mathbf{Q})\cdot\mathbf{R}}\, Q^a_{ij}(\mathbf{r}-\boldsymbol{\tau}_a-\mathbf{R}),$$

the ordinary augmentation charge *displaced* by $\mathbf{Q}'-\mathbf{Q}$, the ket's wavevector minus the bra's. That is the same object a spin spiral already uses, which is its special case at $\mathbf{Q}=+\mathbf{q}/2$ and $\mathbf{Q}'=-\mathbf{q}/2$. The overlap operator does *not* enter: the augmentation part of $\langle\psi_{\mathbf{k}_0+\mathbf{Q}}|S|\psi_{\mathbf{k}_0+\mathbf{Q}'}\rangle$ carries the same lattice sum, and $\mathbf{Q}-\mathbf{Q}'$ is a reciprocal vector of the ultracell, so that sum is $N\delta_{\mathbf{Q}\mathbf{Q}'}$ — the frozen states are exactly S-orthonormal across $\mathbf{Q}$ and the eigenproblem stays an ordinary one. The density gains the augmentation charge of every copy, built from a `becsum` resolved by $\mathbf{Q}$-difference, and PAW gains its one-centre terms per copy, the copies' occupations being the transform of that same `becsum` over the same index.

```
from defuqumat import Calculator
import numpy as np

cal = Calculator.from_file("si-ultracell-paw.in")   # PAW, ecutrho = 4 ecutwfc

ulr = cal.get_ultracell(supercell=(4, 1, 1), kgrid=(1, 2, 2), nbnd=32,
                        external=lambda x: 0.02 * np.cos(2 * np.pi * x[..., 0] / 4))

print(ulr.total_energy)                          # -89.18116879 Ry per unit cell
print(ulr.energy_terms["one_center_paw"])        # -67.17927909, the one-centre term
print(float(np.abs(ulr.modulation).max()))       # 1.7738e-04 e/bohr^3, the response
```

`ecutrho = 4 ecutwfc` is not what such a dataset wants, and `run_ultracell` says so with a warning: the run is then self-consistent and comparable against a supercell at the same cutoffs while its absolute energy is not converged in `ecutrho`. The dual the dataset was generated for, 8 to 12 times `ecutwfc`, runs as well and is what an absolute number should be taken at. One thing to know before comparing a run at a dual against a real supercell: the supercell chooses its own FFT box, which at a dual is not the tiled one — $(54,25,25)$ against $(50,25,25)$ on this cell — and it then sits 2.20×10^{-6} Ry per cell above its own unit cell with nothing applied. Compare the energy of the modulation, each side against its own unmodulated state, and that offset cancels. Both datasets were measured against a real four-atom supercell under the same applied potential: the induced density's error falls 1.18×10^{-1}, 1.56×10^{-2}, 3.72×10^{-3} ultrasoft and 1.17×10^{-1}, 1.71×10^{-2}, 5.09×10^{-3} PAW at `nbnd = 12, 24, 48`, with the total above the supercell's at every rung and falling. With no modulation applied the total reproduces the unit cell's own SCF to 8.5×10^{-12} Ry and 2.5×10^{-11} Ry, the second including a -67.18 Ry one-centre term.

A spinor works the same way, with one line that is not the unit cell's. At `npol = 2` the four integrals are recombined into 2×2 spin blocks and sandwiched between `fcoef` — `newd_so`, with a wavevector on it — and PAW's one-centre coefficients are added to the *scalar* components before that transform rather than after, which is the order the unit cell uses and the one that puts them in the right spin structure. What differs is the lower off-diagonal block. In a unit cell it is the conjugate transpose of the upper one, because both are built from the same two real components; here every component is a function of the $\mathbf{Q}$-difference and $D_a(\mathbf{Q}_d)^*=D_a(-\mathbf{Q}_d)$, so that conjugate transpose belongs to the *opposite* difference. Both blocks are written out from the components, and the matrix is Hermitian across the pair $(\mathbf{Q}_d,-\mathbf{Q}_d)$ rather than within one entry.

With nothing applied the tiled total reproduces the unit cell's own SCF to 2.8×10^{-12} Ry ultrasoft, 8.5×10^{-13} Ry PAW, and 7.5×10^{-13} Ry on fully relativistic platinum, which is the one cell where `fcoef` is not the identity. Against a real supercell under the same modulation

the total sits above it at every rung and falls: $+4.80 \times 10^{-5}$, $+4.48 \times 10^{-6}$, $+6.37 \times 10^{-7}$ Ry at `nbnd = 24, 48, 96` for spinor PAW silicon, and $+3.09 \times 10^{-5}$, $+1.55 \times 10^{-5}$, $+5.21 \times 10^{-6}$ at `nbnd = 16, 24, 40` for platinum. **A spinor band holds one electron where a collinear band holds two**, so the comparison against the collinear route is at half the band count, and there the two agree to every digit printed — $+6.37 \times 10^{-7}$ on both at the last rung.

```
from defuqumat import Calculator
import numpy as np

cal = Calculator.from_file("pt-ultracell-soc.in")   # fcc Pt, lspinorb, ultrasoft

ulr = cal.get_ultracell(supercell=(2, 1, 1), kgrid=(2, 2, 2), nbnd=40,
                        external=lambda x: 0.05 * np.cos(np.pi * x[..., 0]))

print(cal.system.npol, cal.system.nspin_mag)  # 2 1 -- a spinor carrying no moment
print(ulr.total_energy)                       # -69.34673356 Ry per unit cell
print(ulr.augmentation_residual)              # 2.8266e-07
```

One number comes back with an augmented run that a norm-conserving one does not report. `ulr.augmentation_residual` is how complex the augmented density came back, relative to its own largest value, and it is a convergence number in `ecutrho` rather than an error: the ultracell's reciprocal cutoff set is the unit cell's dense sphere at *every* **Q**, and the negative of **G** + **Q** is that sphere *displaced* rather than that sphere, so a real function represented on it is not exactly real. It is **exactly zero** at $N = 1$, and on ultrasoft silicon it reads 1.71×10^{-4}, 7.21×10^{-5} and 2.96×10^{-5} at `ecutrho` of 64, 96 and 144 Ry. The Hartree term of a norm-conserving ultracell carries the same truncation and does not report it.

Two spin channels are in and are measured separately, since everything above runs one channel and would not see the other. The null at `nspin = 2` reproduces the unit cell to 7.2×10^{-13} Ry, and a uniform applied field at $N = 1$ — where each channel gets its own δV and therefore its own $\int \delta V\, Q$ in D_{ij}, with opposite signs — reproduces an ordinary SCF's moment to 5.0×10^{-3}, 1.8×10^{-3} and 4.9×10^{-4} relative at `nbnd = 12, 24, 40`. What has *not* been compared against a supercell is a *modulated* moment on an augmented dataset.

An applied field's energy is not in the total, which is QE's and Elk's shared convention and this package's everywhere else: `ulr.field_energy` carries $-\int B \cdot m$ beside it. An applied *scalar* potential is treated the other way and its energy stays in the total, once, because that is what `vltot` is — which is also what makes a comparison against a supercell carrying the same modulation a comparison of the same quantity.

Under a field, the quantity that carries the sign is `total_energy + field_energy`, not the total on its own. What the calculation minimises is the full energy with the Zeeman term in it, and the reported total has that term taken out, so the total alone is a bound on nothing — measured on a partly-polarized hydrogen cell under a uniform 0.02 Ry field, it sits 7.4×10^{-7} Ry above an ordinary SCF at `nbnd = 12` and 9.7×10^{-7} and 1.6×10^{-6} Ry *below* it at 24 and 40, moving monotonically the wrong way, where the sum is $+4.15 \times 10^{-6}$, $+2.47 \times 10^{-6}$ and $+5.15 \times 10^{-7}$ — above at every rung and falling. Add the field energy back whenever two magnetic states are being compared.

A spin density wave. `nspin = 2` is in, and it is what the method was published for: a modulation of the *magnetization* over tens of cells is the case a supercell cannot afford. Without spin-orbit coupling the ultracell matrix is block diagonal in spin, so two channels cost two solves of the same size rather than one of twice it; what the channels share is a **single Fermi level** over all $2N$ `nbnd` states, and that sharing is the physics — it is what

lets an electron cross from the minority channel in one cell to the majority channel in the next, which is what a spin density wave is.

Nothing in a calculation breaks spin symmetry on its own, so a modulated moment has to come from somewhere — and the tiled state is an *exact* fixed point of this loop, so a magnetic ultracell given nothing converges in one iteration and stays exactly as ferromagnetic as its unit cell. There are two doors, and **they are different quantities rather than two routes to one**. An applied field *drives* a modulation, and what comes back is the **Q**-resolved spin susceptibility: a response. A *seed* hands the loop the texture as its initial condition and the loop keeps it: the ordered state itself. The field comes first here because it is the honest measurement of a susceptibility; the seed is under *Seeding the texture* below. The two doors meet in one place: a *noncollinear* run given a field and no seed starts in the field's own direction by default, which is under *A texture, not just an amplitude* below.

`magnetic_field` is the first knob: a collinear $B(\mathbf{r})$ in Ry, entering as $v_\uparrow \mathrel{-}= B$ and $v_\downarrow \mathrel{+}= B$, in the same two forms `external` takes.

```
from defuqumat import Calculator
import numpy as np

cal = Calculator.from_file("si-mag.in")     # the same cell at nspin = 2

# a field of one ultracell period along a_1, in Ry
field = lambda x: 0.05 * np.cos(2 * np.pi * x[..., 0] / 2)

ulr = cal.get_ultracell(supercell=(2, 1, 1), kgrid=(1, 2, 2),
                        nbnd=12, magnetic_field=field)

print(ulr.cell_moments())          # the moment of each cell: the wave itself
print(np.abs(ulr.magnetization).max())         # rho_up - rho_dw on the box
print(ulr.charge_accuracy, ulr.magnetic_accuracy)
```

`ulr.cell_moments()` integrates $\rho_\uparrow - \rho_\downarrow$ over each unit cell of the ultracell, which needs no sphere and no radius: the cells tile the ultracell exactly, so the partition is the geometry. On the cell above the two come back equal and opposite, which is the two-cell wave the field asked for.

Seeding the texture

`seed_magnetization` is the second door. It is a factor $s(\mathbf{r})$ on the *converged unit cell's own* magnetization,

$$\mathbf{m}(\mathbf{r}) \;\longmapsto\; |s(\mathbf{r})|\, R(\hat{\mathbf{e}}_0 \to \hat{s}(\mathbf{r}))\, \mathbf{m}_0(\mathbf{r}), \qquad |s| \le 1,$$

which reduces to $m \mapsto s\, m_0$ for a collinear run, where s is a number. That is `starting_magnetization`'s role one level up — a fraction of a moment that is already there, with its range — and it is a factor rather than a magnetization because a caller knows the texture, one direction per cell, and does not know the magnetization profile *inside* a unit cell. $s = 1$, and $s = \hat{\mathbf{e}}_0$ for a vector, are the identity. $|s| > 1$ is refused: the reference's $|\mathbf{m}| \le n$ pointwise, so a factor above one asks for more magnetization than the cell has and makes a channel density negative.

Nothing is faded, and that is the design decision. Elk seeds a random *field* (`rndbfcu`) and takes it away geometrically (`reducebf`), and it has to: its self-consistency runs on the potential, where a seed is a term in the functional and has to leave before the converged

state means anything — which leaves the awkward criterion, a fixed point of a field that is going to zero. Here the mixed quantity is the *density*, so a seed is an initial condition and not a term: the functional is the field-free one from the first iteration onwards, and `conv_thr` means exactly what it means in every other run in this package. The price of that choice is the other half of the same sentence — a seed cannot *hold* a texture the way a field can, so a state whose basin the seed misses decays back to the tiled one, and the loop says so by converging to it.

```
from defuqumat import Calculator
import numpy as np

cal = Calculator.from_file("h-mag.in")   # partly polarized hydrogen, nspin = 2

# one period of the wave over two cells, as a factor on the cell's own moment
stagger = lambda x: np.cos(np.pi * x[..., 0])

ulr = cal.get_ultracell(supercell=(2, 1, 1), kgrid=(2, 2, 2), nbnd=24,
                        seed_magnetization=stagger)

print(ulr.iterations, ulr.converged)   # 8 True
print(ulr.total_energy)                # -0.93602520 Ry/cell: the staggered state
print(cal.get_scf().total_energy)      # -0.93300778: the ferromagnet it started in
print(ulr.cell_moments())              # [ 0.07381, -0.073736]
```

What that buys, as a number. The seeded run converges to a state **3.085 mRy per cell below** the tiled ferromagnet — a state the method could not previously reach at all, since the ferromagnet is a fixed point and nothing moves off it. And it is the right state: against the same two atoms run as a real supercell through this package's own SCF, seeded the same way with `starting_magnetization` of opposite sign on two species, the wave's own Fourier component agrees to 1.51×10^{-3}, 5.04×10^{-4}, 1.74×10^{-4} and 3.10×10^{-5} relative at `nbnd = 16, 24, 40, 64`, and the energy sits $+1.53\times10^{-4}$, $+6.80\times10^{-5}$, $+2.32\times10^{-5}$ and $+5.57\times10^{-6}$ Ry above the supercell's, above at every rung and falling. The supercell's amplitude is $0.36546537\,\mu_{\mathrm{B}}$ per cell.

The seed is evaluated pointwise, and a smooth seed is not a per-cell sign. $\cos(\pi x)$ changes sign halfway through each cell, so the *cell* moments it leaves are small (0.0738 against the atoms' 0.365) while the *atoms* are cleanly opposite — which is the physical state, and the supercell agreement is what says so. Read a seeded wave off its Fourier component, or off `cell_moments()` only for a seed that is constant within each cell, which is `floor(x)` in the callable. Turning pointwise rather than in steps is also what makes a helix's pitch exact, below.

Two refusals, and both would otherwise be silent. A reference with **no moment**: the seed scales a moment, so on a cell that converged unpolarized it scales zero, every iteration is a no-op and the unpolarized state comes back as a converged answer. And, for a vector seed, a **compensated** reference — an antiferromagnetic unit cell, whose net moment is a residue of cancellation whose direction is round-off, so there is no direction to turn the texture rigidly from. Turning such a cell sublattice by sublattice is not written.

An ultrasoft or PAW dataset is seeded in both of its mixed variables. Most of a transition metal's moment is inside the projector spheres rather than on the grid, so a seed that reached the density alone would seed almost nothing on a PAW magnet and would leave the first iteration's one-centre potential disagreeing with its own density. `becsum` is therefore seeded from the *same* evaluated field, read at each atom's own grid point of the box, so the spheres and the grid cannot disagree about the period. That half is checked on

constructed occupations rather than in a converged run (`tests/unit/test_ultracell_seed.py`), because the copy index is a lattice translation in C-order over the integer triple and a seed landing on the wrong copy is a converged, plausible and wrong modulation that no tiled null can see — a tiled seed being the same on every copy.

Read `magnetic_accuracy`, not `accuracy`, on a magnetic run. `dr2` is the Hartree energy of the density residual, so its charge half is weighted by $1/|\mathbf{G}+\mathbf{Q}|^2$ and its magnetization half by a constant — and in an ultracell the gap between them is wider than in a unit cell, because the smallest $|\mathbf{G}+\mathbf{Q}|$ is N times smaller and the charge half's weight at the envelope's own wavevector is N^2 larger. A converged total can therefore leave the moment still moving; the two halves are reported apart for exactly that reason, and `ulr.history` carries both per iteration.

What a magnetic run is checked against. At $N = 1$ a *uniform* applied field is the same physics as an ordinary SCF with `B_field(3)`, which shares nothing with this code path — it goes through `add_bfield.f90`'s expression inside a plane-wave SCF. On the silicon cell at $B = 0.02$ Ry the reference moment is 0.71159883 and the ultracell gives it to 9.6×10^{-3} relative at `nbnd = 12`, 3.2×10^{-3} at 24 and 7.7×10^{-4} at 40: the same convergence in `nbnd` the charge shows, now for the moment and against a route with no ultracell in it. Against a real two-cell supercell of polarized hydrogen under a scalar modulation, the charge and the magnetization agree to 1.7×10^{-4} and 1.4×10^{-4} at `nbnd = 24`.

A texture, not just an amplitude. `nspin = 4` is in as well, and it buys the modulation a collinear run cannot write down: one in which the moment's *direction* turns from cell to cell rather than its length growing and shrinking along a fixed axis. A helix, a cycloid, a domain wall and a skyrmion are all of that kind. **Spin-orbit coupling comes with it and costs nothing**: it is a term in the Hamiltonian the frozen unit-cell states were diagonalised with, and the ultracell only ever sees their eigenvalues and their coefficients, so there is no spin-orbit term anywhere in the method.

What changes is the shape of one matrix. A collinear potential is diagonal in spin, so the $2N$ `nbnd` problem is two blocks of half the size; a spinor basis function has two components and the potential acts on it as the 2×2 matrix

$$V(\mathbf{r}) \;=\; v_0(\mathbf{r})\,\mathbb{1} \;+\; \mathbf{B}(\mathbf{r}) \cdot \boldsymbol{\sigma}, \tag{1}$$

whose off-diagonal entries couple the components at every point — and that coupling is exactly what lets the magnetization turn. There is one block, one Fermi level over all N `nbnd` states, and each state holds *one* electron rather than two.

The applied field changes shape with the regime: `magnetic_field` is a scalar $B(\mathbf{r})$ for `nspin = 2` and a **vector** one for `nspin = 4`, either as a `(3, *box)` array or as a function of unit-cell crystal coordinates returning three components per point. A field that *turns* is what drives a helix, and it cannot be written down at all in the collinear form.

```
from defuqumat import Calculator
import numpy as np

cal = Calculator.from_file("h-noncolin-ultracell.in")  # moments free to turn

# a field of 0.01 Ry turning once over the ultracell, in the xy plane
def turning(x):
    phase = 2 * np.pi * x[..., 0] / 4
    return 0.01 * np.stack([np.cos(phase), np.sin(phase),
                            np.zeros_like(phase)], axis=-1)

ulr = cal.get_ultracell(supercell=(4, 1, 1), kgrid=(1, 2, 2), nbnd=16,
```

```
                        magnetic_field=turning, mixing_beta=0.3)

print(ulr.cell_moments())         # (4, 3): one moment VECTOR per cell
print(ulr.magnetization.shape)    # (3, *box): m_x, m_y, m_z on the grid
```

`cell_moments()` returns an $(N, 3)$ array for a noncollinear run where it returns $(N,)$ for a collinear one, and `magnetization` likewise becomes the three cartesian components on the box. Neither needs a sphere or a radius: the cells tile the ultracell exactly, so the partition is the geometry.

A field given with no seed is its own seed (`seed_from_field`, the default since 2026-10-10 for a noncollinear reference that carries a direction, on the Kramers-closed basis and without spin-orbit coupling): the loop starts with every cell's moment turned onto the field, $\mathbf{B}/|\mathbf{B}|$ at the reference's own length, and the reference's direction kept where the field vanishes. On the cell above that run converges in **11** iterations to a flat helix, the four moments at $35.2°$, $125.2°$, $-144.8°$ and $-54.8°$, exactly $90°$ apart, each of length 0.488 with no net moment and nothing out of the plane. Started from the tiled reference instead, `seed_from_field = False`, the same call converges in 142 iterations to a compromise at $28°$, $66°$, $-42°$ and $-51°$ with a net moment of 0.25, which is **2.15 mRy per cell higher** in `E + field` (the tiled start on the reference's states alone gave $27°$, $62°$, $-40°$ and $-48°$). The helix is not something the field imposes on a stiff ferromagnet: the same four hydrogens as a real supercell, seeded as a helix with no field at all, sit 0.739 mRy per cell *below* the ferromagnet, so what the field selects is the phase of a texture the lattice already prefers, and the compromise is a fixed point the tiled start stops at.

What the default changes, and when to turn it off. A run under a field that starts tiled is the response about the reference's own state; one seeded from the field is the state the loop relaxes to from the field's texture, and on a cell with more than one self-consistent solution the two need not be the same one. At 0.01 Ry on four cells of hydrogen with the reference along $(1, 1, 1)/\sqrt{3}$ they are: 9 iterations seeded against 55 tiled, `E + field` equal to 2.8×10^{-10} Ry and the moments to 1.4×10^{-4} after one global rotation. At 0.002 Ry the seeded run converges in 9 to a flat helix of 0.347 where the tiled one does not converge in 1500, and the helix is the state the exact four-atom supercell converges to under the same ladder of fields from a helical start (25 iterations, $90.0°$ a site, `E` per cell 3.1×10^{-6} Ry from the ultracell's); but the tiled start's iterates on both sides pass through states lower in `E + field` than that helix, 0.54 mRy per cell in the ultracell and 0.10 mRy in the supercell, neither converging in 400 iterations, so at a field this weak the helix is a converged answer and not shown to be the lowest one. **Where the field turns faster than the exchange lets the moments follow, the seed can be worse than the tiled start**: on two cells of the same hydrogen with the moment along x and 0.01 Ry along $+y$ in one cell and $-y$ in the other, the seeded loop converges in 8 iterations to moments of 0.262 where the reference has 0.623, 0.35 mRy per cell *above* the canted ferromagnet the tiled start reaches in 3. `seed_from_field = False` is the start for a linear response about the reference's own state and for that comparison, and `result.seed_origin` says which start a run took. A paramagnetic or compensated reference has no direction to turn from and runs tiled with `result.field_seed_skipped` saying so; on the reference's states alone (`kramers_pairs=False`) nothing is seeded. **With spin-orbit coupling the default starts tiled as well** (since 2026-10-11), with `field_seed_skipped` saying so, because an easy axis can hold the moment against a weak field and the seed was measured only without the coupling; `seed_from_field = True` takes the field's seed there all the same, and the two starts are compared by `total_energy + field_energy`.

On the reference's states alone a noncollinear ultracell converges slowly when

the field is weak, and lowering `mixing_beta` is the wrong response. That basis prefers the reference's own direction, so turning *every* moment together is a nearly flat manifold the mixer has to *traverse* to reach the place the field pins, and `mixing_beta` is how fast it travels. On the four-cell hydrogen ultracell above, every run given 300 iterations: at a 0.01 Ry turning field every `beta` converges to the same state and the best is 0.5 (36 iterations, against 48 at 0.7 and 115 at 0.2). At 0.002 Ry `beta = 0.7` converges in **263** iterations and `beta = 0.3` does *not converge in 300*, its residual sitting between 2×10^{-5} and 9×10^{-5} for the whole run without ever leaving: stuck rather than unstable. **The default basis has no such manifold**, since it prefers no direction: a seeded helix converges in 10 iterations in either frame, and what is slow under a weak field there is leaving the tiled ferromagnet, which is what the field's own seed does in one step. The run says which of these applies by name when it fails to converge, and names `seed_magnetization` first wherever a seed can be taken.

What a noncollinear run is checked against. At $N = 1$ a uniform vector field is the same physics as an ordinary SCF with `B_field(1:3)`, which goes through `add_bfield.f90`'s expression inside a plane-wave SCF and shares nothing with this code path — and for a vector field that check reaches the transverse components, which no collinear identity can. On a hydrogen cell whose moment is free, that reference run says something clean on its own: the converged moment's direction equals $\hat{\mathbf{B}}$ to *six decimals*, which is what a magnet with no anisotropy does and what fixes the sign of the coupling. Against it, at $\mathbf{B} = (0.005, 0.0005, 0)$ Ry, the ultracell reproduces the moment to 1.9×10^{-2} relative at `nbnd = 16`, 6.6×10^{-3} at 40, 1.5×10^{-3} at 64, 4.5×10^{-4} at 96 and 1.6×10^{-4} at 128 — monotone, and a factor of 116 across the range. What is converging is the *transverse* component: m_x is right to 1.5×10^{-3} at the first rung, and m_y walks $0.0601, 0.0634, 0.0691, 0.0694, 0.0732, 0.0740$ towards the reference's 0.0743. That ladder is the reference's states alone. On the default basis there is none: without spin-orbit coupling the closed span holds the rotated states exactly, and the moment is right to 7.5×10^{-7} at `nbnd = 16` with `E + field` equal to twelve digits. Reading that needs the reference converged in *direction*, which a flat manifold makes slow: at `conv_thr = 1e-11` it stops 2.5×10^{-2} degrees short of $\hat{\mathbf{B}}$, and at `1e-13` it is within 5.6×10^{-5}.

On the reference's states alone, expect to need more bands than a collinear run, by about a factor of two. A spinor state holds one electron and lives in a space of $2\,$`npwx`, so a given `nbnd` is half as complete a basis as the same number would be for one collinear channel. The ladder above is at 37 per cent of the basis at `nbnd = 128`, which is where stage 1's charge ladder was at `nbnd = 80` on silicon. Convergence is also slower the larger the *rotation* asked for: a field that turns the moment by 57 degrees converges in `nbnd` several times more slowly than one that tilts it by 6, because a change of the local spin frame has to be built out of empty states of the opposite spin channel, which the exchange splitting puts far away.

Seeding a texture, and which axis to write the seed about. `seed_magnetization` takes a vector here, so the seed turns the moment rather than scaling it, and a helix of pitch N cells is the case to have in mind. This paragraph and the three after it describe the reference's states alone, `kramers_pairs=False`; the default basis, closed under time reversal and described after them, removes the preferred axis, the canting and the lean they measure. Such a helix is invariant under a translation by one cell followed by a spin rotation of $360/N$ about its own axis, and the self-consistent map commutes with both — so that sector is closed and a run started in it stays in it. **In the truncated problem it is closed only if the basis is**, and the basis is the unit cell's own spinors: a rotation about the reference's own magnetization is a *phase* on each of them, since without spin-orbit coupling they are eigenstates of $\boldsymbol{\sigma} \cdot \hat{\mathbf{e}}_0$, and a rotation about any other axis is not. There is therefore *one* closed sector and its axis is $\hat{\mathbf{e}}_0$, which makes the rule short: **write the**

seed about the direction the reference's moment already points along, which is to say converge the unit cell with its moment along the axis the helix is to turn about.

What that buys is the iteration count and the frame, not the pitch. On four cells of hydrogen at `nbnd = 16`, seeded about z with the reference along z, the run converges in **14** iterations and the pitch comes back at 90.0001, 90.0000, 89.9999 degrees. The same seed on a reference along $(1,1,1)/\sqrt{3}$ takes **290** iterations and arrives at a helix turning about $(1,1,1)/\sqrt{3}$ instead — and it is the *same state*: after one global rotation the two runs' cell moments agree to 1.23×10^{-3} on moments of 0.272, and their energies to 3.5×10^{-9} Ry. A global spin rotation costs nothing without spin-orbit coupling, so the unprotected run is not wrong; it is traversing a flat manifold to reach the frame its own basis prefers. That is what the 290 iterations are, and it is why the fix is the axis rather than `mixing_beta` — the same lesson the weak-field case above teaches, arriving from the other side. A reference in the helix plane is worse: the same seed then converges in 23 iterations to steps of 94, 74 and 84 degrees with a uniform moment of 0.069 along the reference, which is a wrong answer rather than a slow one. **The run says so**: a seed whose directions do not sit on a cone about the reference's magnetization is warned about by name, and the warning names the fix below.

What the truncation costs instead is the canting, and that is the half a pitch check cannot see. The converged moments stand off the helix plane by a *uniform* angle — which the closed sector allows, a cone being as invariant under the pair as a flat helix is — and that is a ferromagnetic remnant at $\mathbf{Q} = 0$, towards the direction the basis was built around, which the wave's own Fourier component is blind to. It falls with `nbnd` where the pitch has nothing left to converge:

`nbnd`	pitch error	cone	net moment	E above the supercell
16	$1.35 \times 10^{-4}\,^\circ$	10.64°	0.0628	$+4.37 \times 10^{-4}$
24	$8.50 \times 10^{-5}\,^\circ$	6.30°	0.0370	$+2.54 \times 10^{-4}$
40	$3.27 \times 10^{-5}\,^\circ$	3.00°	0.0176	$+1.22 \times 10^{-4}$
64	$1.20 \times 10^{-5}\,^\circ$	0.91°	0.0054	$+3.74 \times 10^{-5}$

so reading the pitch alone at `nbnd = 16` would call the state exact while 18 per cent of its moment stood off the helix. Across that range the cone falls by 11.65 and the energy error by 11.68, which is worth recording and is not explained here.

On a real magnet this is the number to watch, and it is not cosmetic. A seeded 15-cell helix on a fully relativistic PAW NiBr2 cell at `nbnd = 40` came back with a pitch of 24.51° against the 24.0° asked for and nothing measurable out of the plane, and with a 22.8 per cent remnant along the reference direction that the pitch could not see: the moment's *length* then ran 9 per cent with the texture's own period, which reads as an amplitude wave and is a uniform vector added to a helix. Removing it took the pitch to 24.18°. What it cost is the harmonic content, which is usually the reason for the calculation — a coplanar helix of uniform length has no charge response at $\mathbf{q}$ at all, and that run's charge came out at $\mathbf{q}$ over $2\mathbf{q}$ by a factor of two, because a remnant along the reference feeds the charge at $\mathbf{q}$ directly. So read `cell_moments().mean(axis=0)` beside the pitch, and treat it as a convergence parameter in `nbnd` rather than as a property of the material. (Measured in another session and reported here.) The reference is the same four atoms as a real supercell with the helix seeded per species, whose site moments are flat to 10^{-5} and whose energy is 0.739 mRy per cell *below* the ferromagnet the ultracell expands around — which is what makes reaching it worth a seed at all. At `nbnd = 8` the pitch still comes back to 4.2×10^{-3} degrees, the closure not needing convergence to hold, while the run does not reach `conv_thr` in 300 iterations and its energy sits $+9.06 \times 10^{-4}$ Ry above the supercell.

With `lspinorb` the closure fails for every axis, because a spin-orbit Hamiltonian's eigenstates are not eigenstates of $\boldsymbol{\sigma} \cdot \hat{\mathbf{n}}$ for any $\hat{\mathbf{n}}$. The pitch is then not protected and it is not supposed to be — the anisotropy makes different pitches inequivalent, so a texture that relaxes is physics rather than error — but the two cannot be separated by reading the pitch, and the comparison to make is against a seeded supercell at the same N. **The iteration count above may also not be charged there**, because spin-orbit coupling *gaps* the flat manifold and leaves nothing to wander along: a 15-cell PAW NiBr2 helix seeded about z with its reference along x — the expensive arrangement — converged in 54 iterations rather than anything like 290. One cell is not a scaling, so read the warning as advice and not as a forecast.

The fix for the frame, the cone and the lean is a basis closed under time reversal. All three are the basis preferring the reference's own direction: its majority orbitals carry spin along $+\hat{\mathbf{e}}_0$ only, so turning a moment away needs a majority orbital with the other spinor, which the basis does not have. `kramers_pairs`, **on by default for a magnetized noncollinear reference**, adds each frozen state's Kramers partner, the states of the reference with its magnetization reversed, from a second fixed-density solve, and diagonalises the reference Hamiltonian in the union once, before the loop; `kramers_pairs=False` asks for the reference's states alone, which is what every noncollinear number before it was. Without spin-orbit coupling that span is the orbitals times both spinors, which prefers no direction at all. On the same four cells of hydrogen, at the same 16 states per folded k-point:

basis	reference	steps	cone	uniform moment	E above the supercell
`nbnd = 16`	along z	$90.0001°$	$10.64°$	0.0628	$+4.37 \times 10^{-4}$
`nbnd = 16`	along x	$94°$, $74°$, $84°$	0	0.069	$+4.99 \times 10^{-4}$
Kramers, `nbnd = 8`	along z	$90.0000°$	0	0	$+1.3 \times 10^{-6}$
Kramers, `nbnd = 8`	along x	$90.0000°$	0	0	$+1.3 \times 10^{-6}$

in Ry per cell, 10 iterations for both closed runs against 14 and 23, and at twice the size the old basis reaches only $3.81°$ and $+1.56 \times 10^{-4}$. The two sets are nearly dependent, since a band with little exchange splitting has nearly the same orbital in both spin channels: directions whose overlap eigenvalue falls below 10^{-8} are dropped, and `kramers_ranks` says how many each folded k-point kept. Those small directions are the ones that matter, the differences between the majority and the minority orbitals, and a floor of 10^{-2} keeps the closure and loses the accuracy ($+1.96 \times 10^{-4}$). It costs a second frozen solve and a basis of up to twice `nbnd`, so four times the matrix, so `nbnd` buys twice the states it does without it. With spin-orbit coupling the partners remove only the preference between $+\hat{\mathbf{e}}_0$ and $-\hat{\mathbf{e}}_0$, and on NiBr2 that is enough: a three-cell PAW helix at `nbnd = 40` keeps a uniform moment of 1.3×10^{-4} of its length in both frames, the order of Elk's 9×10^{-5}, where the old basis keeps 0.415 with the reference along the axis and 0.300 in the plane, and still 0.288 and 0.188 at twice the bands; the two frames agree to 10^{-8} Ry and lie 2.2 mRy below the old basis at the same size.

```
from defuqumat import Calculator
import numpy as np

cal = Calculator.from_file("h-noncolin-z.in")  # the moment along z, free to turn

# 90 degrees per cell, turning about the direction the moment already points in
def helix(x):
    phase = 2 * np.pi * x[..., 0] / 4
```

```
    return np.stack([np.cos(phase), np.sin(phase),
                     np.zeros_like(phase)], axis=-1)

ulr = cal.get_ultracell(supercell=(4, 1, 1), kgrid=(1, 2, 2), nbnd=24,
                        seed_magnetization=helix, mixing_beta=0.3,
                        kramers_pairs=False)   # the reference's states alone

m = ulr.cell_moments()
print(ulr.iterations)                                  # 14
print(np.degrees(np.arctan2(m[:, 1], m[:, 0])))    # 35.32 125.32 -144.68 -54.68
print(np.linalg.norm(m, axis=1))                       # 0.33687 on all four
print(m.mean(axis=0))                                  # [0, 0, 0.036926]: the remnant

# the default, a basis closed under time reversal, at a third of the size
closed = cal.get_ultracell(supercell=(4, 1, 1), kgrid=(1, 2, 2), nbnd=8,
                           seed_magnetization=helix, mixing_beta=0.3)
m = closed.cell_moments()
print(closed.iterations)                               # 11
print(np.degrees(np.diff(np.unwrap(np.arctan2(m[:, 1], m[:, 0])))))  # 90 90 90
print(m.mean(axis=0))                                  # [0, 0, 0] to 1e-7: no remnant
print(closed.total_energy - ulr.total_energy)      # -2.52e-4 Ry per cell
print(closed.kramers_ranks)                            # 16 at every folded k-point
```

The four directions are 90.0000 degrees apart and the offset is not a defect: the seed is evaluated pointwise, so the helix turns continuously across a cell and a cell's own moment sits at the average of the phases inside it. The *differences* are the pitch.

`kramers_pairs=True` on a collinear cell, whose one channel's partner is the other channel and is already in the basis, and on a nonmagnetic spinor cell, whose states already come in Kramers pairs. A reference whose k-set is not the unshifted, unreduced `supercell` $\times$ `kgrid` Monkhorst-Pack grid. The tiled unit-cell density is a fixed point of the loop only when the unit cell integrated the zone over exactly the points the ultracell folds onto, so on any other set the loop converges to a different sampling and reports the difference as a modulation: 0.635 Ry per cell with nothing applied, on silicon converged at `4 2 2` and folded onto `2 1 1`. The check is on the points, so an explicit list that is that grid passes, and a symmetry-reduced wedge is refused by name. Left out, `kgrid` in `get_ultracell` is the reference's grid divided by `supercell`, which is the one value that passes. Occupations other than `'fixed'` and `'smearing'`: a tetrahedron weight interpolates a band across a mesh, and an ultracell state at one $\mathbf{k}_0$ is a mixture across the N folded points, so it is not a band of any mesh (Elk's `occupyulr` has only a smeared branch), and `'from_input'` has no band left to hold its prescribed occupation. Both used to reach the smeared branch at `degauss = 0` and return NaN. Gradient-corrected functionals: the gradient of the ultracell density carries the envelope's own gradient, which a per-cell evaluation drops (Elk takes that approximation silently; it is refused here). The *whole-cell* exit region of a transmission (`run_ultracell_transport(exit_region="volume")`) on an augmented dataset, which is the one Gram matrix that is $\langle\psi|S|\psi\rangle$ — the transmission itself runs, as do the image and the spectrum, because all three sample the state in the *vacuum*, where a pseudo-wavefunction is the true one, and all three refuse a plane inside an augmentation sphere as the unit cell's own versions do. Meta-GGA. A `seed_magnetization` of $|s| > 1$, which asks for more magnetization than the cell has and makes a channel density negative — it is a fraction of the reference's own moment, with `starting_magnetization`'s range. A seed on a reference with **no moment**, since scaling zero leaves every iter-

ation a no-op and returns the unpolarized state as a converged answer; and a *vector* seed on a **compensated** reference, an antiferromagnetic unit cell whose net moment is a residue of cancellation whose direction is round-off, so the texture has no direction to be turned rigidly from — turning one sublattice by sublattice is not written. An applied `magnetic_field` on a noncollinear cell that **carries no magnetization** — a spin-orbit run on a nonmagnetic crystal, where the density and the potential have one component apiece and a field has nothing to be added to; give the unit cell a small `starting_magnetization` so that it carries four, which it may then converge back to nearly zero, and what the run measures is the Pauli susceptibility. A ground state converged **under a magnetic field or a constrained moment**, because the frozen eigenvalues would carry the field the SCF ended with — which `reducebf` and the fixed-spin-moment scheme both change from the input — while δV is rebuilt here from the density alone; apply the field through `magnetic_field` instead. DFT+U, the occupation matrix being per atom where an ultracell has N copies of each. Spin spirals, as a second modulation of the same states. Gamma-only storage, the basis being built at N points none of which is Γ. A double grid is *not* refused, and what remains of the refusal that used to stand here is a warning firing the other way: `ecutrho` defaults to four times `ecutwfc` here as it does in `pw.x`, so an augmented input that simply leaves it out runs with the augmentation charge on the wavefunction grid, which is the one thing such a dataset exists not to do. Symmetry is not used at all: a modulation breaks the crystal's point group, so the run needs `nosym` and the whole k-grid. And a non-integer ultracell is refused, where Elk's `avecu` is an independent input with no consistency check against its Q-grid. An **unconverged** unit-cell ground state, because its density is the one thing an ultracell holds fixed and there is no later iteration in which a bad seed could work itself out.

Two things are *not* refused and are limits on a claim rather than on a run. The total energy's monotone bound rests on the free energy being minimised, which Methfessel-Paxton smearing breaks — its occupation function is non-monotone, so the generalised entropy is not concave and the self-consistent point is not a minimum; gaussian, Fermi-Dirac and Marzari-Vanderbilt are safe, and the energy itself is computed either way. And a cell with more than one self-consistent solution has no ladder at all: the energy then *ranks* solutions at one `nbnd` and does not certify a sequence across several, since nothing says the same solution is found at each.

14 What a modulation looks like to a tip

A charge or spin density wave, a screened impurity, a domain wall: what all of these are actually seen with is a scanning-tunnelling microscope, so an ultracell that cannot be imaged is a calculation of something nobody measures directly. This section is the join between the ultracell and the two things a tip measures, the Tersoff-Hamann image and the vertical tunnelling transmission. The third, the *spectrum* at one place over many biases, is the same relabelling read along an energy axis and has its own subsection under *Tunnelling spectra* — on a modulation it is usually the one to reach for, since a wave moves a band edge and an image at one bias cannot tell a level that moved from a level that gained weight.

Nothing new is computed, and the reason is one observation about the basis. An ultracell state is

$$\Psi_j(\mathbf{r}) = \sum_{\mathbf{Q},n} a^j_{\mathbf{Q}n}\, \psi_{\mathbf{k}_0+\mathbf{Q},n}(\mathbf{r}),$$

and in a plane-wave basis that sum is a relabelling rather than a transform: a plane wave

with unit-cell Miller index $\mathbf{G}$ belonging to $\mathbf{Q}$ is the ultracell plane wave $\mathbf{h} = n\mathbf{G} + \mathbf{q}$, and no two of them collide, because reducing $n(\mathbf{G} - \mathbf{G}') + (\mathbf{q} - \mathbf{q}') = 0$ modulo n forces $\mathbf{q} = \mathbf{q}'$ and the sphere sits inside the dense grid. So the union over $\mathbf{Q}$ of the N unit-cell spheres *is* the ultracell's own sphere at $\mathbf{k}_0$, the band index contracts away, and what is left is an ordinary coefficient vector that every routine written for a unit-cell wavefunction takes unchanged once it is handed the ultracell's Miller indices, its cell and $\mathbf{k}_0$ in its reciprocal coordinates. On a modulated two-cell silicon ultracell the state sampled from that vector agrees with the box transform the density is built from to 1.7×10^{-16} against a peak of 0.14, and it integrates to 1 over the ultracell.

The image is then the ultracell density with a smeared delta at the tip energy in place of the occupations, which is what a Tersoff-Hamann image is in Elk's `wfplot.f90` and in QE's `stm.f90`, and the transmission is the same contraction `run_vertical_transport` uses with the ultracell's geometry in it. The entry points are `run_ultracell_stm` and `run_ultracell_transport`, or `Calculator.get_ultracell_stm` and `Calculator.get_ultracell_transport`.

```
from defuqumat import Calculator
import numpy as np

cal = Calculator.from_file("si.in")        # the two-atom unit cell

# a potential of one ultracell period along a_1, in Ry
cal.get_ultracell(supercell=(8, 1, 1), kgrid=(1, 2, 2), nbnd=24,
                  external=lambda x: 0.05 * np.cos(2 * np.pi * x[..., 0] / 8))

img = cal.get_ultracell_stm(height=0.35, axis=2, shape=(96, 12),
                            bias=-1.5, width=0.01)
print(img.values.shape)        # (96, 12): the plane spans all eight cells
print(img.integral)            # 8.0039: the electrons in the window, per cell
```

Two conventions, and the second is the one to know. The plane is given in *unit-cell* crystal coordinates running over $[0, n_i)$ across the ultracell, which is what `external` already takes, so `height = 0.35` means what it means in a unit-cell run and a map spans the whole modulation by default. And the image is a tunnelling density of states *per unit cell*, in the same units a unit-cell image reports, so the two are directly comparable: with nothing applied, the ultracell image is the unit cell's image tiled, to 1.6×10^{-7} of the peak at $N = 2$. The window above covers the whole valence band, so the integral comes back as silicon's eight valence electrons; a run with a Fermi level puts the tip at it instead, and one with fixed occupations warns and takes the middle of the gap, which is `stm.f90`'s own rule.

A magnetic tip is what a spin density wave needs. A collinear system is invariant under flipping every spin together with the sign of B, so the charge cannot respond at first order and a wave driven by `magnetic_field` is almost invisible in the charge *density*: on an eight-cell silicon ultracell carrying 0.12 Bohr magnetons the total density varies by 1.9×10^{-4} of its mean from cell to cell. What an unpolarized tip sees is neither that small nor the wave itself, but the wave *squared*, two periods where the wave has one, at 37 per cent of its mean against a polarized tip's 82, since the leading charge response is at twice the wavevector and the tip energy is a smeared band edge rather than a count of electrons. `spin` and `polarization` project the channel, exactly as they do for a unit-cell image.

The number. The reference is the one the ultracell itself is checked against: a real N-cell supercell under the same applied potential, run through this package's own SCF and imaged the same way. The error on the induced corrugation falls with `nbnd`, which is the

whole content of "a variational truncation to `nbnd` bands per folded k-point": 22.2, 5.2 and 2.0 per cent at `nbnd` $= 12$, 24 and 48 on a two-cell silicon ultracell, where the induced corrugation is itself 0.6 per cent of the image. Widening the exit region of the transmission to the whole ultracell must give the image back exactly, with no factor between them, and on a norm-conserving dataset does so to 10^{-11}; that identity is what holds the two normalisations against each other, since the image carries its N in the weights and the transmission carries it in the volume the states are normalised over.

An ultrasoft or PAW dataset needs nothing extra, and where the planes are is the reason. A tip point and an exit plane both sit in the vacuum, where the augmentation charge is zero and a pseudo-wavefunction is the true one, so the sampled amplitude and the plane's Gram matrix are exact as they stand — which is the same argument a unit-cell junction rests on, and it has always run on such a dataset. The exception is the *whole-cell* exit region: there the Gram matrix is $\langle\Psi|S|\Psi\rangle$ rather than an integral over a plane, and S for an ultracell state spans N unit-cell spheres with their own projectors, so `exit_region = "volume"` is refused by name. It is refused rather than returned, and on purpose: the frozen basis is exactly S-orthonormal across $\mathbf{Q}$, so that Gram matrix *is* the identity by construction, and handing the assembly the identity would make the diagnostic agree without looking at the coefficients it exists to check.

What such a run needs is a cell with vacuum in it, which a bulk crystal is not. Silicon's PAW dataset reaches 2.647 bohr where the interplanar spacing along the stacking axis is 5.889 bohr with an atom every quarter of it, so the widest clearance any plane can have is 2.208 bohr and the guard refuses every height — correctly, since a substrate and a tip belong either side of a *sheet*. On the square silicon sheet committed as `ultracell-sheet-paw.in`, with the two planes at 0.15 and 0.85 and the atom at 0.5, the unmodulated two-cell ultracell reproduces the unit cell's map tiled to 6.2×10^{-7} (ultrasoft) and 9.7×10^{-7} (PAW) of the peak, and under a $0.05\cos(\pi x_1)$ Ry modulation the map converges to a real two-cell supercell's own transmission: 1.09×10^{-2} at `nbnd = 12` and 1.96×10^{-3} at 24. The modulated rung is the one that discriminates, because an unmodulated folded state is a single basis function whose normalisation is the unit cell's, while a state that mixes $\mathbf{Q}$ makes $\sum_{\mathbf{Q}} |a_{\mathbf{Q}}|^2 = 1$ the S-orthonormality claim itself.

```
from defuqumat import Calculator

cal = Calculator.from_file("ultracell-sheet-paw.in")   # a PAW sheet in vacuum

cal.get_ultracell(supercell=(2, 1, 1), kgrid=(2, 4, 1), nbnd=12)

t = cal.get_ultracell_transport(exit_height=0.15, height=0.85, axis=2,
                                shape=(12, 4), broadening=0.05)
print(t.values.shape)             # (12, 4): the map spans both cells
print(float(t.values.max()))      # 4.1856e-05, in the transmission's own units
print(t.notes["hermiticity"])     # 0.0 -- the Gram matrices, not imposed
```

Sampling the plane is exact and its set is the ultracell's own dense sphere, $|\mathbf{G} + \mathbf{Q}|^2 \leq$ `ecutrho`, rather than Elk's choice of the unit cell's dense sphere at every $\mathbf{Q}$. Read back at the box's own grid points, the sphere reproduces a modulated silicon density to 3×10^{-16} where the other set is out by 5×10^{-9} at $N = 2$ and 2×10^{-8} at $N = 3$. The box itself aliases its outermost shell from about $N = 5$, at 10^{-11}, which is the ordinary `ecutrho` $= 4$ `ecutwfc` corner aliasing one level out and is a property of the ultracell's grid rather than of the sampling.

Everything the ultracell itself refuses, since the run has to exist first. A result whose states were dropped (`keep_states = False`), which cannot be rebuilt, the diagonalisation on the folded k-set being the expensive step of the whole method. For the transmission, an ultracell that is more than one cell deep along the stacking axis: the exit plane would then sit between two of the stacked slabs, so the electron leaves through the material rather than out of it, and two **Q** differing only along that axis share every in-plane index the exit integral has to tell apart. The same axis must carry one k-point, for the reason a unit-cell junction needs one. A denser k-set for the image, the way `grid` re-solves the bands for a unit-cell image: on an ultracell that is not post-processing but another run of the whole loop, so it is asked for with `kgrid` in `get_ultracell` instead. And a constrained `tot_magnetization`, whose two channels have their own Fermi levels where a tip sees one energy. On an ultrasoft or PAW dataset: a plane inside an **augmentation sphere**, which the guard reports the distance for, and the transmission's **whole-cell** exit region, for the reason given above — the transmission itself, the image and the spectrum all run. An applied `magnetic_field` is **not** refused here, where a unit-cell image refuses it: there the field is something the ground state was converged under and the reported total does not carry, while here it is the modulation's own driver and the eigenvalues the tip energy is read from are the eigenvalues under it.

15 Continuing an all-electron ground state

Elk is an all-electron LAPW code and this is a pseudopotential plane-wave one, so their converged densities are different objects: Elk's carries the core electrons and the nuclear cusp, and this one carries a smooth pseudo valence density normalised to the valence electron count. What crosses between them is a *starting guess*, and the run's own SCF still happens on top of it.

Inside each muffin tin Elk stores the density as a real-spherical-harmonic expansion on a logarithmic radial mesh, and everywhere else as a periodic function on a uniform grid,

$$\rho(\mathbf{r}) = \begin{cases} \sum_{lm} f_{lm}(r)\, R_{lm}(\hat{\mathbf{r}}), & |\mathbf{r} - \mathbf{R}_a| < r_a, \\ \sum_{\mathbf{G}} \tilde{\rho}(\mathbf{G})\, e^{i\mathbf{G}\cdot\mathbf{r}}, & \text{otherwise,} \end{cases}$$

so putting it on a plane-wave grid means deciding which region each grid point is in, then interpolating radially or summing the Fourier series. The muffin-tin part is written in pointwise and is never Fourier truncated, which is why the nuclear cusp transfers exactly and why nothing is lost to aliasing.

```
from defuqumat import Calculator

calc = Calculator.from_file("scf.in", pseudo_dir="pseudo")
seed = calc.get_elk_seed("elk_run/")     # STATE.OUT beside GEOMETRY.OUT
result = calc.get_scf(starting_density=seed)
```

The argument is an Elk *run directory* rather than a file: `STATE.OUT` carries neither the lattice vectors nor the atomic positions, so `GEOMETRY.OUT` has to be beside it. And `elk.in` is not a substitute, because Elk's default `tshift` moves the origin onto the inversion centre and only `GEOMETRY.OUT` is written in that shifted frame. Pass `report=[]` to get back a `SeedReport`, which is what the reconstruction measured on the way: what it integrated to before

renormalisation, how many of Elk's reciprocal lattice vectors this run's density cutoff kept, and what fraction of the norm sat outside it.

`ElkState` is the reader on its own, for looking at the state rather than seeding with it. `read_elk_state` is the same thing as a function and `read_elk_geometry` reads `GEOMETRY.OUT` alone, returning the `ElkGeometry` that `state.geometry` also holds; all four are exported from `defuqumat.io`.

```
from defuqumat.io import ElkState

state = ElkState.read("elk_run/")
print(state.rmt, state.efermi)            # muffin-tin radii; e_F in Rydberg
print(state.muffin_tin_charges())         # the charge inside each sphere
print(state.interstitial_charge())        # and the charge outside all of them
rho = state.evaluate_at(points)           # the density at arbitrary points
```

The pieces of the reconstruction are exported too, from `defuqumat.io.elk_density`, because each is a separately checkable part of somebody else's convention rather than an internal: `real_spherical_harmonics` (the sign convention is Elk's and is not the textbook one), `poly4` (the 4-point Lagrange radial window), `spline_weights` (the quadrature the radial integrals use, where Simpson's rule is 9e-6 away), `characteristic_function` (the Fourier-truncated step that separates the two regions) and `interstitial_coefficients` (`rhoir` as a G-vector list). `density_on` is what `get_elk_seed` calls.

What it is worth, measured. On simple-cubic hydrogen at $a = 3.0$ bohr, the reconstruction agrees with Elk's own evaluation of its own density at every one of the 4096 points Elk printed, to 4.5×10^{-11}, which is the precision the file was printed at rather than any error in the transfer. The charge inside the sphere is 0.6125761996 and outside it 0.3874238004, both exact to every digit Elk printed. And a run started from that density and a run started from a superposition of atomic charges converge to the same state: -1.080181442650 Ry either way, agreeing to 6×10^{-14} Ry with converged densities 9×10^{-8} apart.

It buys no iterations, and that is the honest result. Both runs take four. At a lower cutoff the seeded one takes *one more*. For a hydrogen atom a superposition of atomic charges is already almost the answer, and the all-electron cusp inside the pseudisation radius is a place where Elk's density is further from the pseudo one than the atomic guess is. The reader is worth having because it bridges to an all-electron ground state, not because it is faster.

Refuses. A **spin-polarized** Elk state, and a **spin spiral** with it — a magnetization is a second field with a transfer rule of its own, and a seed that carries the charge while silently dropping the moment would start a magnetic run from a non-magnetic guess. A **DFT+U** or fixed-tensor-moment state, whose occupation matrices are trailing records nothing here reads. A state written by Elk older than **2.0.0**, which Elk's own reader refuses. A directory with no `GEOMETRY.OUT`, or one whose geometry is from a different run. A calculation on a **different cell** — a density is transferable between identical lattices and there is no interpolation here that would make anything else mean something. A calculation with more than one spin channel. And an element with a **core**: the muffin-tin density includes the core states and nothing in `STATE.OUT` separates them from the valence charge, so a run whose valence electron count is far from what the state integrates to is refused by name rather than scaled to fit. Hydrogen, with no core at all, is what this path is for.
Passing an Elk density as a **fixed** density, with no SCF above it, is not offered: the two

codes' converged densities are different functions wherever there is a core, and no splice of the two would be a solution of anything.

16 Performance: dials, memory and GPUs

One switch in front of the dials: `memory_mode`. The four dials below are the mechanism; the question a run on a card actually has is *does it fit, and if it fits, how fast*, and `memory_mode` answers it once. It takes two values, and each sets the four dials:

mode on a GPU	`k_batch`	`band_batch`	`projectors`	`wfc_store`
`'speed'`	all	all	`store`	`device`
`'memory'`	what fits	all, if it fits	`rebuild`	`stream`, if a chunk

`'speed'` puts every k-point on the card at once. `'memory'` puts on it the largest chunk of k-points that fits (`k_batch='fit'` below), forms the projectors when they are used, and, *when that chunk is smaller than the mesh*, keeps the wavefunctions in host RAM for the whole run and *streams* them through the card a chunk at a time — for the starting guess, for each Davidson call, and for the density, which is summed over k-points as they pass and finished once. Where the whole mesh fits, or there is one k-point, the store stays on the card, since the states being solved are there either way. With one k-point a call this is Quantum ESPRESSO's own loop with its buffer in host memory, which `k_batch=1` still asks for. A band structure, an NSCF run and a density of states, which want energies only, solve each k-point from scratch and keep none of its states on the card. The physics is the same and so are the numbers, to round-off.

What it changes is how the peak grows with the k-mesh. Measured on a GTX 1060 (6 GB), eight-atom silicon at 20 Ry, `nosym`, per SCF:

k-points	`memory` peak	`speed` peak	`memory` time	`speed` time
1	33 MB	34 MB	0.34 s	0.35 s
27	38 MB	877 MB	5.4 s	3.5 s
64	45 MB	2080 MB	12.3 s	7.7 s
125	56 MB	4055 MB	24.6 s	18.6 s
216	91 MB	does not fit	44.4 s	—

Each peak is one SCF in a fresh process with the compiled kernels already cached; each time is the second of two runs. `speed` grows by 33 MB per k-point on this cell; `memory` by about 0.2 MB, which is what stays resident per k-point — $|k+\mathbf{G}|^2$, the FFT index and mask, and the projector core, $n_{\mathrm{pwx}}(3+2n_{\mathrm{cs}})$ numbers against the wavefunctions' $2n_{\mathrm{bnd}}n_{\mathrm{pol}}n_{\mathrm{pwx}}$. At 216 k-points the 91 MB is the setup's own peak, building that core; the SCF stays below it. The price is time, 1.3–1.6x here, and it is the per-k-point Davidson solve rather than the transfers (0.23 s of the 12.3): a larger chunk buys most of it back, `k_batch=4` giving 9.7 s at 141 MB. On an RTX A2000, with the Davidson steps equal and after the small subspace solve and the stick fill stopped being loops on the card, the same cell at 27 k-points takes 372 ms an iteration in `'memory'` against 153 in `'speed'`, 2.4x, and 167 with `k_batch='fit'` below.

The defaults. On a GPU the default is `'memory'`. `'speed'` is *checked* before it runs: the peak is estimated from the input (`Calculator.estimate`) and compared with 60% of what the card has free, and a run that would not fit falls back to `'memory'` with a warning instead

of dying in its first Davidson call. The 60% is the estimate's floor showing — on this card the measured peaks were 1.57–1.61x the estimate. On a CPU the default is `'speed'`, which there is Quantum ESPRESSO's loop already and is what every validated number in this document was measured with; `'memory'` on a CPU changes one thing, the projector storage, since host and device are one memory. An explicit dial beats the mode, and `DEFUQUMAT_MEMORY_MODE` sets it for a whole process:

```
from defuqumat import Calculator

calc = Calculator.from_file("scf.in", memory_mode="speed")
result = calc.get_scf()
print(calc.calculation.memory_mode, calc.calculation.k_batch)
```

```
DEFUQUMAT_MEMORY_MODE=speed python3 run.py
```

Nothing is refused; what the mode does not reach is worth knowing. **It bounds what grows with the k-mesh and nothing else**: a peak set by the augmentation tables, the dense grid or one k-point's band block is the same in both modes — on seven-k spin-orbit bismuthene and eight-k PAW platinum the two peaks agree within 1.5%. **A streamed result's wavefunctions are a host array**, and most of what reads them walks them a chunk at a time too: the force and the stress (each chunk's share of the gradient pulled back through the whole-cell terms, so the force adds nothing to the SCF's peak), $dE/d\mathbf{q}$, the density and projector occupations, a projected DOS, an STM image, the structure factors, the site moments, the Berry curvature and the orbital magnetization. **So do the dielectric tensor and the Born charges**: the bare perturbations, the first-order states and the Sternheimer solves go a block of k-points at a time with the first-order states in host memory, in `'memory'` on a card always, even when `k_batch='fit'` takes the whole mesh in one block. What they add then does not grow with the mesh: nothing on norm-conserving silicon, and on ultrasoft silicon a fixed 153–155 MB at one k-point a block and nothing with the whole mesh in one, where the old route's Born charges had reached 7.4 times the SCF's peak. **So does the dynamical matrix, at Γ and at $\mathbf{q}$**: its $3\,n_{\mathrm{at}}$ bare perturbations and first-order states are held in host memory and each column of the force constants is assembled a block of k-points at a time, where before a streamed store did not run at all at Γ and went back to the card whole at $\mathbf{q}$; at $\mathbf{q}$ the states on the second sphere, at $\mathbf{k}+\mathbf{q}$, are written into host memory a block at a time as well. On a run without symmetry the phonon's $3\,n_{\mathrm{at}}$ induced potentials and response densities, grids with no k index that grow with the number of atoms displaced rather than with the mesh, are kept in host memory as well and go to the card one mode at a time; with symmetry they stay on the card, because the displacement average acts on all of them together. That average, like every symmetrisation here, walks the group's operations one at a time once holding them all at once would pass 64 MB, rather than gathering `nsym` copies of the field: on eight-atom silicon with 24 operations the Γ phonon of every atom reads 89 MB on a card where it read 503, against an SCF of 39. `DEFUQUMAT_GATHER_BUDGET` sets that threshold in MB (`0` walks always); the result moves only in its last digits either way. **So do the piezoelectric tensor, the strain response and the elastic constants**: the piezoelectric tensor's derivative of the stress along the field's response is assembled a block of k-points at a time from the field's stores in host memory, as the Born charges are, and on two-atom ultrasoft AlAs it no longer grows with the mesh (769 MB at 64 k-points where it read 1350, the SCF's being 178); the strain response

keeps its six bare perturbations and first-order states in host memory, and the elastic constants differentiate the walked stress once more along them. **So do the two third derivatives**, the electrostriction's $\partial\epsilon/\partial\varepsilon$ and the Raman tensors: the variational second-order energy is a sum over k plus terms of the whole sums, so its derivative along each strain or displacement is taken a block of k-points at a time, with the three further Sternheimer solves per direction on the block; and the vibrational spectrum assembles its dynamical matrix from the walked displacement response the Raman tensors hand over. **What still moves them to the card whole** is the transcribed piezoelectric route `method="zstar_eu"` and the sum-over-states spectra. **The host must hold the set** ($n_{\text{spin}} n_k n_{\text{bnd}} n_{\text{pwx}} n_{\text{pol}}$ complex), which is the point but is not free. And **the fall-back from `'speed'` fires only where the preset is what would run**: with a `k_batch` or `projectors` of your own, or `DEFUQUMAT_K_BATCH`, the estimate would describe another calculation, so nothing is checked. **A mode given to a `Calculator` configures its own setup**, and the calculator's post-SCF methods — bands, NSCF, DOS, PDOS, STM, transport, structure factors, the spiral scan, the topological invariants, the optical and magnetic spectra — run on that same setup, moved to their own k-points where they need others, so they follow its mode. **A solve at more bands than the SCF is re-checked for its own count**: the mode and the band batch were sized for the SCF's bands, so a spectrum's sixty bands on a cell whose SCF has four can fall back from `'speed'` to `'memory'`, or run fewer bands at a time, for that solve alone, with a warning. The magnetic-anisotropy force theorem and torque build a setup of their own, because they rotate the moments; they resolve the mode from `DEFUQUMAT_MEMORY_MODE` and the platform.

A k-chunk sized to the card: `k_batch='fit'`, `'memory'`'s default on a card. One k-point per call is Quantum ESPRESSO's loop, and on a card it pays a fixed cost per call — one dispatch per kernel of a Davidson solve sized for one small k-point — that a batch over k amortises. `k_batch='fit'` (or `DEFUQUMAT_K_BATCH=fit`) chooses the largest chunk whose estimated peak fits 60% of what the card has free, from the same estimate that sizes the band batch: the whole mesh whenever it fits, otherwise the fewest calls and then the least padding. On an RTX A2000, eight-atom silicon at 20 Ry with `nosym`, Davidson steps equal, it takes memory mode from 555 to 282 ms per SCF iteration at 27 k-points and from 1280 to 657 at 64, against 275 and 628 in `'speed'`, at speed mode's peak. On sixteen atoms at 30 Ry the call is bound by arithmetic rather than by its fixed cost, and the batch is a wash: 4476 against 4310 ms at 22 k-points a call on a 64-point mesh, where `'speed'` would not fit at all. The width ladder of the Davidson solve, which sizes it by the live subspace, is kept inside a batch at the batch's widest rung, which is what made that a wash rather than a 28% loss. On an H100 the same 27-point mesh takes 184.8 ms an iteration at one k-point and 31.4 at `'fit'`, against 20.6 in `'speed'`, Davidson steps and energy equal, which is why it is the default.

```
from defuqumat import Calculator

calc = Calculator.from_file("scf.in", k_batch="fit")
print(calc.calculation.k_batch, calc.estimate().k_batch)
```

It is `'memory'`'s default on a card (since 2026-10-02), and a `'speed'` run that does not fit falls back to it; `k_batch=1` or `DEFUQUMAT_K_BATCH=1` asks for one k-point a call. **On a CPU it is the platform's default**, there being no card to fill and a batch over k measured slower there. **A chunk is grown only from a whole band block**: a run whose band block had to be split to fit stays at one k-point. **It is sized for the SCF's k-set**; a

band path or an NSCF set reuses the chunk. **A function that is handed a `k_batch` without a system to size refuses `'fit'` by name** (an NSCF given one explicitly, for instance); the environment form is silent to it. **A batch over k runs each Davidson step until its slowest k-point has converged**, which is a pure speed lever while the steps are equal across k-points, as they are on every cell measured.

The band side in single precision: **`band_precision.`** `'single'` (or `DEFUQUMAT_BAND_PRECISION=single`) runs $H\psi$, the Davidson work arrays and the wavefunction store in float32, and keeps the density, the potential, the mixer and the energies in float64; the small subspace solves run in double whatever the bands do. It is a performance mode and no correctness claim is made in it. On an RTX A2000, a float32 part, the 64-atom silicon SCF to `conv_thr=1e-7` takes 2.3 s against 9.5 s in double in memory mode and 1.9 against 8.8 in speed mode, at half the device peak (1.65 against 3.15 GiB), and its energy is 2.8×10^{-5} Ry from double's at the same threshold; on sixteen atoms it is 1.7x, and on a CPU an iteration is 1.6–1.8x. `'mixed'` starts in single and switches the band side to double inside the same run, at the first iteration whose threshold single cannot deliver, keeping the mixer's history, so the state it converges to is a double one (within 10^{-10} Ry of a double run on every cell tried). It is measured *not* to pay on most of them: 1.03–1.21x on sixteen-atom silicon, 0.72–0.75x on 64, because the 10^{-7} perturbation of single's first iterations costs the double phase the superlinear mixing step that a cell like that converges by.

```
from defuqumat import Calculator

calc = Calculator.from_file("scf.in", band_precision="mixed")
result = calc.get_scf()
print([entry["band_precision"] for entry in result.history])
```

```
DEFUQUMAT_BAND_PRECISION=single python3 run.py
```

Single runs the SCF of a collinear norm-conserving calculation, one or two spin channels, and nothing else: an ultrasoft or PAW dataset, a spinor, a spiral, DFT+U and a meta-GGA are refused by name. **Every derivative of the energy is refused** on a calculation whose band side is single — forces, stress, the response stack — because a gradient through a float32 $H\psi$ carries its round-off as a plausible force; converge in double for those, from the single result if you like (`run_scf(..., starting_from=result)`). A calculator built with `'mixed'` keeps its single-precision setup, so its derivatives are refused too. **`conv_thr` below 10^{-7} is refused in single**: the energy does not follow it there (10^{-6} to 8×10^{-6} Ry off whatever is asked). **Davidson's threshold is floored at** $4\,\epsilon_{32}\,E_{\text{cut}}$, where Quantum ESPRESSO's 10^{-13} would stall every call at the budget and let the run report itself converged with the energy wrong. **On an accelerator the float32 matrix products are pinned to full precision** against TF32's 10-bit mantissa, for the process. And **a mover that rebuilds the projector core** — a new geometry, a band path — builds it in double, so the moved calculation is double throughout rather than half single.

The card's pool. Importing `defuqumat` lets the device allocator take 90% of the card instead of JAX's own 75% (on a 6 GB card, 5.45 GB rather than 4.76), by setting `XLA_CLIENT_MEM_FRACTION` before the backend starts. A value you set yourself, in either spelling of that variable, is left alone; `DEFUQUMAT_MEM_FRACTION` takes a number in $(0, 1]$, or `off` for JAX's default — which is what to use when another program shares the card.

```
DEFUQUMAT_MEM_FRACTION=off python3 run.py
```

Two batching dials control how much is in flight. The band dial follows the platform: on a CPU, Quantum ESPRESSO's loop — one band at a time, which is what a *cache* wants — and on a GPU the whole block, which is what a card wants. **In `'memory'` mode on a GPU the whole block is checked first**: once the k axis streams, one k-point's band loop through the grid is the largest thing left on a large cell, so the setup estimates the peak at every band batch and, if the whole block would not fit, runs the fewest blocks that do and says so in a warning. The answer is identical bit for bit, since chunking the band axis is a map rather than a sum; what it costs is time. The chosen value is `calc.calculation.band_batch`, and an explicit `band_batch` or `DEFUQUMAT_BAND_BATCH` is taken as given:

```
calc = Calculator.from_file("scf.in", band_batch=16)
print(calc.calculation.band_batch, calc.estimate().band_batch)
```

The estimate counts what `'memory'` actually puts on the card — one streamed chunk of the wavefunctions rather than the whole set — and the starting guess, $\max(n_{\mathrm{atomwfc}}, n_{\mathrm{bnd}})$ vectors with H and S applied, which on a large cell is where the peak is. The k dial follows the memory mode above: one k-point on a CPU in either mode and in `'memory'` on a GPU, all of them in `'speed'`. Either end is reachable everywhere:

```
DEFUQUMAT_K_BATCH=all DEFUQUMAT_BAND_BATCH=all python3 run.py
```

or per call, `run_scf(..., k_batch=...)`. The chunk size changes the order contributions are summed in and nothing else — round-off, ∼1e-15 Ry.

The three field directions. The dielectric tensor and the Born charges solve one Sternheimer equation per Cartesian direction of the field, and Quantum ESPRESSO's `solve_e` solves them one after another. On a GPU they are solved as one batch, one conjugate-gradient loop per k-point for all three, because the loop is short and its cost there is the launches and the reads of its stopping test, which the batch divides by three; each band of each direction still stops on its own threshold, so the tensor is the serial one to round-off (1e-14 on ultrasoft AlAs) with the same iteration counts. The price is that every direction runs as many steps as the slowest one, and on a CPU, where there is nothing to amortise, the serial loop stays the default. In the walked route the batch holds three directions' states where one was, so the k-chunk is a third of the SCF's. `DEFUQUMAT_FIELD_BATCH` sets it for a whole process:

```
DEFUQUMAT_FIELD_BATCH=0 python3 run.py    # one direction at a time, as ph.x
```

The floor under Davidson's threshold. The SCF tightens the threshold `ethr` with its own error, as `electrons.f90` does, and floors it at 10^{-13} Ry on every platform; `DEFUQUMAT_ETHR_MIN` sets the floor for a whole process. That is a lever and not a default, and the history of why is worth knowing because it looked like a floor problem. Davidson stops on the *change* in each eigenvalue between two steps, and on a GPU one call at that floor could take between 3 and 100 steps from round-off alone: at `conv_thr=1e-10` the 64-atom SCF in the default memory mode took 132 s against 14.5 s. The cause was not the threshold. The subspace solve parked the directions it is not using at $1000\times$ the largest diagonal element of H so that they never entered the lowest roots, which gave the matrix the dense eigensolver diagonalises a norm of about 3×10^4 against a live block of 6 to 9 Ry, and the card's

`eigh` is accurate only to machine epsilon times that norm (3.4×10^{-12} Ry on the stalled call's own matrices), where a CPU's is not affected by it at all. They now sit one above a bound of the reduced live block's spectrum (`solvers.subspace.generalised_eigh`'s `parked`), where the card's error is at its floor; every arm measured takes the same steps, sixteen atoms the CPU's in every iteration at Quantum ESPRESSO's floor, and the CPU's own numbers did not move.

```
from defuqumat.batching import resolve_ethr_floor

print(resolve_ethr_floor(1e-13))   # QE's floor, on every platform
```

```
DEFUQUMAT_ETHR_MIN=3e-12 python3 run.py
```

Nothing is refused; what is worth knowing. **Raising the floor costs accuracy**: on a displaced sixteen-atom cell at `conv_thr=1e-10`, a floor of 3×10^{-12} doubled the force error (3.4×10^{-6} to 6.5×10^{-6} Ry/bohr) and took the energy error from 1.8×10^{-11} to 7.6×10^{-11} Ry. **A call that uses the whole 100-step budget and leaves bands unsettled gives a `RuntimeWarning`, once per run**, naming the iteration, the threshold and the unsettled count; the steps each iteration paid are `history[i]["davidson_iterations"]`, and a time on a card is read against them. **The parking follows the live block**, not the kinetic energy at the cutoff, so its margin does not shrink as `ecutwfc` grows; sixteen atoms were measured at 30, 60 and 90 Ry. And **the card measurements are on a float32 part**: the component that differs is the device `eigh` on the Davidson matrix, and whether another card's has the same error on it was not measured.

The wavefunction transforms have two layouts. The default, on every platform, is Quantum ESPRESSO's: the z transform over only the columns the wavefunction sphere occupies, then one contiguous xy pass. `DEFUQUMAT_FFT_LAYOUT=box` takes the sphere through the whole box with one fused three-dimensional transform instead. They are the same transform summed in a different order, so $H\psi$ agrees to 10^{-16} relative and an SCF reaches the same energy in the same number of iterations. Which is faster depends on the cell and the card. On an RTX A2000, a float32 part, with Davidson's steps equal in every arm, the sticks are faster at sixteen atoms (63.9 against 70.3 ms an iteration) and on a 27-point mesh of eight atoms (153 against 173), equal at 64 atoms and slower at eight atoms with one k-point (17.7 against 16.6); on a CPU they are equal at eight atoms and 9.5 per cent faster at sixteen. The sticks had been losing on a card to their own fill of the box, a scatter the card ran as a loop over sticks, which is now a gather. No default follows the platform until a float64 card has been measured.

The band dial is worth twice as much in a spinor run, because a band in flight is then *two* real-space boxes rather than one: a noncollinear calculation with 403 bands on a $240\times54\times200$ grid puts $403\times2\times2{,}592{,}000$ complex numbers — 33 GB — into one transform if the whole block goes at once, and several such arrays are live inside a single application of H. `DEFUQUMAT_BAND_BATCH=16` makes that 1.3 GB. The answer is unchanged bit for bit, not merely to round-off, because chunking the band axis reorders nothing: it is a map, not a sum.

Why the band default is per platform. A GPU has no cache to fit in and inverts the conclusion: on a ten-atom metal, `k=1,b=1` cost **4.5x** what `k=all,b=all` does on the same card (measured before the stick layout's fill of the box became a gather, a loop the card had paid once per band: on sixteen-atom silicon on an RTX A2000 one band at a time is now 71.0 against 64.0 ms an iteration, where it was 196.6), sixteen-atom silicon runs at **0.20x**

— an outright loss against four CPU cores — and at thirty-two atoms `b=1` did not finish at all. So a run on a device batches its bands unless it says otherwise. The plausible half-measure `k=all,b=1` is in neither mode: it is the *worst* setting available, because it buys the batched mode's memory with the looped mode's kernel launches. The other off-diagonal, `k=1,b=all`, is `'memory'`'s, and it is measured rather than assumed: 8.95 s against 7.4 s for `all/all` and 39.5 s for `1/1` on the 64-k silicon above. An explicit argument beats the environment variables, which beat the mode.

A third dial says where the wavefunctions are *kept* (`wfc_store`, `DEFUQUMAT_WFC_STORE`), which is a different question from how many are in flight. The store is $n_{\mathrm{spin}} n_k n_{\mathrm{bnd}} n_{\mathrm{pwx}} n_{\mathrm{pol}}$ complex — 11.3 GB on a 45-atom slab at six k-points, 45 GB at twenty-four — and it stays resident all through the SCF because it is the next iteration's starting guess. Quantum ESPRESSO does not keep it on the device: `c_bands.f90:get_buffer` and its `save_buffer` partner hold one k-point's worth and `io_level` says where the rest lives. `wfc_store='host'` is that buffer.

What it buys, and what it does not. Parked, the store is off the accelerator while the energy terms, the potential on the dense grid, the mixer's history, the PAW one-centre terms and the checkpoint write are running — which is where the other large allocations are. It is *on* the device for the diagonalisation, for `becsum` and for the density, because it is an input to all three, so this does **not** lower a peak that sits inside the eigensolver. The default follows the platform for a measured reason: on a CPU the host *is* the device, and a round trip of the store runs at 1.3 GB/s here, which at slab scale would be 18.6 s per iteration for nothing. On an accelerator the same transfer is PCIe, around a second. So `device` on a CPU, `host` on anything else, and the answer is bit-for-bit identical either way — a transfer is not arithmetic.

The third place, `wfc_store='stream'`, is `'memory'`'s and is the one that moves the peak: the store never comes back whole. It stays in host RAM, and the solve, `becsum`, the density, τ and DFT+U's occupations each walk it a chunk of k-points at a time with the same compiled kernels the whole-set path runs, so the card holds one chunk's states where `'host'` holds the whole set twice. **It is taken only where a chunk is smaller than the mesh**: with every k-point in flight it saved 0.04 GB of the peak with one k-point and 0.01 GB with a 27-point mesh in one batch on an H100, and cost 1.9x and 1.5x an iteration there, so `'memory'` then keeps the store on the card; `wfc_store='stream'` or `DEFUQUMAT_WFC_STORE=stream` still asks for it. It was checked against the whole-set store on norm-conserving, ultrasoft and PAW silicon, the three spiral chains, `tb09`, spin-orbit bismuthene, PAW spin-orbit platinum, DFT+U on FeO and on noncollinear BN, and gamma-only storage: total energies agree to 5e-12 Ry or better, most of them to the last printed digit. `'host'` is no mode's default any more.

A fourth dial says whether the *projectors* are kept (`projectors`, `DEFUQUMAT_PROJECTORS`), and it is the one that has been measured to move a peak rather than a stage. $\langle k+G|\beta\rangle$ is $n_k n_{\mathrm{pwx}} n_{\mathrm{kb}}$ complex and resident for the whole run: on the 45-atom slab above that is **13.96 GB**, larger than the wavefunctions themselves. Quantum ESPRESSO does not hold it either — `c_bands.f90:111` calls `init_us_2` *inside* its k-loop — and `projectors='rebuild'` is that: only the phase-free core is kept, one column per *species* channel where the full array has one per *atom* channel, and each k-point's is formed on demand. The expression is the same one either way, so the result is bit-for-bit identical.

Why it moves a peak where the store dial does not. The other three dials bound what is *in flight* or move what is held between two points; this one removes what is *held*, so it sits underneath every stage. Measured on that slab, two runs on the same card differing only in this flag: every stage bracket falls by 13.96 GB, and the peak with them — **79.14 GB to 65.18 GB**, with the converged energies identical to the last printed digit. **The saving**

is the whole of $\langle k+G|\beta\rangle$, not that array minus a core: the core is built unconditionally and both routes pay for it, so what the dial chooses between is the whole-k array and one rebuilt chunk. The saving is therefore n_k/k_{batch} times the chunk — it grows with how many k-points are *not* in flight, and at a single k-point it is exactly neutral, buying nothing and costing nothing. It is the ratio $n_{\text{kb}}/n_{\text{cs}}$, atoms per species, that decides how much smaller the core is than the array, which is why the core stays cheap on a large cell. Measured cost: $n_{\text{at}}n_{\text{pwx}}$ complex exponentials each time a k-point's projectors are used, which on that slab is **+1.5% per SCF iteration** (46.1 s against 45.4 s) and 10 s more to compile. The default is `store` on a CPU and in `'speed'`, and `rebuild` in `'memory'`.

> Nothing is refused, but two things are worth knowing before switching. Every validated number in this document was measured with `store`. And a size estimate follows the dial only from `433abd7` onward: an older `defuqumat size` reports the stored array for a rebuilt run and so overstates the floor by the largest single line in its own table.

Measured GPU speedups (one H200 against one CPU core, per SCF iteration): 2x on a cheap metal, 3–5x on norm-conserving silicon, 9–21x with an augmentation charge or a magnetization, and **83–115x on spin-orbit**. Energies agree to ∼1e-9 Ry — the same agreements the CPU already has.

Memory. Spin-orbit is where it bites: a 20-atom bismuth cell with a relativistic ultrasoft dataset peaks at **34.7 GB**, which fits a 141 GB card and does not fit a 32 GB one. Everything else in that sweep sits under 2 GB.

A response costs memory according to how it is differentiated, which is worth knowing before starting one. A *forward* response — the Sternheimer solves behind ϵ and Z^* — costs 0.7–1.0 GB over the SCF that produced the state, and a *forward-over-reverse* one — the dynamical matrix, the Raman tensors — 1–2 GB. The *stress* is the exception and it is a *reverse* derivative through the radial transforms: **10.5 GB** on eight-atom ultrasoft silicon, ten times any of the others, and an order more than the SCF it came from. Measure your own with `tools/gpu/phase5.py <case> --property <name>`.

The augmentation charge is the largest object in an ultrasoft or PAW run on a large cell, and there are two ways of keeping it. Below `DEFUQUMAT_AUG_MAX_BYTES` (default 2 GB) $Q_{ij}(\mathbf{G})$ is built on the whole G sphere once and kept, which is what every number in the accuracy table below was measured with. Above it, Quantum ESPRESSO's scheme takes over: the *radial* transforms

$$Q^L_{nm}(q) \;=\; \frac{4\pi}{\Omega}\int_0^{r_{\text{kkbeta}}} \mathrm{d}r\; j_L(qr)\,\left[r^2 Q^L_{nm}(r)\right]$$

are tabulated on a grid in $|q|$ at $\mathrm{d}q = 0.01$ and $Q_{ij}(\mathbf{G})$ is rebuilt from them a block of **G** at a time, every iteration, exactly as `addusdens` and `newd` do it. On a 45-atom slab with 21 Å of vacuum that is **8.4 MB** against **65 GB** for one relativistic nickel dataset.

```
DEFUQUMAT_AUG_MAX_BYTES=off python3 run.py   # never tabulate
DEFUQUMAT_AUG_MAX_BYTES=2G  python3 run.py   # the default
DEFUQUMAT_AUG_CHUNK=8192    python3 run.py   # G vectors per block
```

`DEFUQUMAT_AUG_CHUNK` sets how many **G** vectors the rebuild takes at a time, and defaults to putting an (n_h, n_h, chunk) block near 16 MB (16384 **G** vectors for silicon, never fewer than 1024); like the batching dials it is a loop bound over an exact sum and is invisible in a result. The rebuild no longer forms that block: `becsum` is contracted with the angular coefficients in the basis of the radial functions first, once and with no **G** index, so a block of **G** holds the radial table and the harmonics, and a strain's second derivative on ultrasoft

AlAs compiles to 68 MB where the block form took 176. The default chunk is therefore conservative. The two schemes agree to **4e-10 Ry** on a total energy and exactly at $\mathbf{G} = 0$. On eight-atom ultrasoft silicon the two contractions cost about **4.5x** more from the table, and the whole run comes out level, because the table is cheaper to build than the stored array is.

Every radial transform — $Q^L_{nm}(q)$, the local potential, the core and atomic charges, the projectors — is integrated a block of $|q|$ values at a time, the block sized so that one $(\text{block}, \text{mesh})$ integrand takes 8 MB (1246 values on an 841-point mesh, never more than 4096). **Its derivatives in $|q|$ are transforms of the same kind**, not derivatives taken through the integrand: every one of these is $T(q) = \sum_m h_m\, j_l(q r_m)$ with the quadrature weights in h, and

$$\frac{dT}{dq} = \sum_m h_m\, r_m\, j_l'(q r_m),$$

meaning that the derivative of a transform is another transform with one more power of r and one more derivative of the Bessel function, which is the rule the code hands to the differentiation. So a stress, an elastic constant or a third derivative holds one block of one transform at a time whatever its order, and the derivatives of j_l come from a series below $x = 2$ and the closed form above, within 5.5×10^{-14} of the exact functions to the fourth order (differentiating QE's own `sph_bes` form, which switches at $x = 0.05$, lost up to 3.5×10^{-8} in j_0'' there). The values themselves are QE's. On ultrasoft AlAs at 200 Ry the strain's second derivative of the local potential compiles to 16 MB of temporaries where it took 136, and the stress on a card peaks at the SCF's own 169 MB (652 MB with the old fixed 4096 values before the block was sized at all), and the strained calls run 2 to 17% faster than when the derivative was taken through the integrand, because nothing has to be recomputed for a backward pass. `DEFUQUMAT_RADIAL_CHUNK` sets the count; like the other chunk dials it bounds a loop over an exact sum and moves a result only in its last digits.

Past the end of the table is NaN, not an extrapolation. The table reaches $2\sqrt{\texttt{ecutrho}}$ — Quantum ESPRESSO's own `cell_factor` — so that a strained cell, whose G vectors move while the G *set* does not, still lands on it. A strain large enough to take $|\mathbf{G}|$ past that gives NaN rather than a clamped value, because a clamped $Q^L(q)$ would be smooth, of the right order and wrong, and would show up only as a bad stress.

A PAW run's other large object is its one-centre quadrature, and what it costs is a *derivative* rather than a run. Each atom's sphere carries $(n_{\text{spin}}, n_x, \text{mesh})$ radial fields, and inside a force or a stress every atom's reverse pass is live at once: **272 MB per atom** for a relativistic nickel dataset with a magnetization, so 4.1 GB for a fifteen-atom sublattice and no ceiling. `DEFUQUMAT_PAW_ATOM_BATCH` is how many spheres are in flight together — `all` for one `vmap`, an integer for `paw_onecenter.f90`'s own loop over atoms:

```
DEFUQUMAT_PAW_ATOM_BATCH=1   python3 run.py   # one sphere at a time
DEFUQUMAT_PAW_ATOM_BATCH=all python3 run.py   # the whole sublattice at once
```

Left alone it chunks above three atoms per species and not below, because that is where the crossover was measured: chunked, the tape stops growing with the sublattice altogether (4.1 GB → 0.80 GB at fifteen nickel atoms) and the run gets *faster* — a ten-atom PAW cell's force is 0.505 s against 0.418 s and its SCF iteration 1.40 s against 1.28 s. On two atoms it is the other way, 0.761 s against 1.286 s, which is what the threshold is for. Like the other batching dials it is a loop bound over an exact sum and is invisible in a result beyond round-off.

One calculation on many cores: k-point pools. What does a core buy when it is given its own k-points instead of a share of one k-point's threads? Inside one k-point the work is a chain of operations each the size of one band's transform or a subspace product over a few dozen bands, and threads pay little for it: on six identical desktop cores a k-point gains 1.33x from four threads at sixteen atoms and 2.16x from six at sixty-four. So the cores go to the k-points, which is `pw.x -nk`: `DEFUQUMAT_POOLS` runs one calculation as that many processes, each holding and diagonalising its own share of the k-points, and the pools meet where Quantum ESPRESSO's do — the raw sums over k (the density, `becsum`, τ, n_s) are all-reduced before they are finished, the eigenvalues are gathered so that every pool evaluates the occupations on the whole set, and rank 0's accuracy, convergence decision and mixed density are broadcast so that no pool can decide differently. The numbers are those of one process to round-off; the pool count changes only the order the k contributions are added in. A run without symmetry — a magnet with spin-orbit coupling, a texture no crystal operation keeps — has the whole grid as its k-points, which is where pools pay most. Launched by hand on one machine, each pool with its rank, one coordinator address and its own core:

```
export DEFUQUMAT_POOLS=2 DEFUQUMAT_COORDINATOR=localhost:23456 DEFUQUMAT_THREADS=1
    JAX_PLATFORMS=cpu
DEFUQUMAT_POOL_RANK=0 taskset -c 0 python3 run.py &
DEFUQUMAT_POOL_RANK=1 taskset -c 2 python3 run.py
```

with `run.py` an ordinary script, which every pool runs whole:

```
from defuqumat import Calculator
from defuqumat.parallel import current_pools

calc = Calculator.from_file("scf.in")
result = calc.get_scf()
forces = calc.get_forces()
if current_pools().rank == 0:
    print(result.total_energy, forces.max_force)
```

Under Slurm the rank and the coordinator come from the job, so `export DEFUQUMAT_POOLS=$SLURM_NTASKS DEFUQUMAT_THREADS=$SLURM_CPUS_PER_TASK` and `srun python3 run.py` is the whole launch. `get_scf`, `get_forces`, `get_stress` and `get_relax` run under pools: the force and the stress walk each pool's rows and reduce, and a relaxation steps every pool's optimizer from rank 0's energy, forces and stress. On displaced silicon at eight k-points the forces agree with one process to 10^{-12} Ry/bohr and the stress to 10^{-10}, and a relaxation takes the same steps.

The regimes a production run is in go through unchanged: a spin spiral, whose two components on their two spheres the kernels already address from one k row; a magnetic field or a fixed-spin-moment constraint, whose controller steps on rank 0's output and hands its field to every pool; the moments turned inside the SCF (`rotate_moments`), whose torque is rank 0's before the stepper sees it; and a seed or a checkpoint. A checkpoint is every pool's rows in a file stamped with the iteration, written before rank 0 writes the state and the mixer, so a kill at any moment leaves one whole checkpoint, and a restart reads the rows it needs from whichever files hold them — at the same pool count, at another, or in one process. The k-points are dealt out longest first on their plane-wave count, the Γ point counted twice, so that the pool holding the most expensive one takes fewer others. Only rank 0 mixes, and the others hold no history.

Stopping a pooled job. A pooled process does not die of SIGTERM: JAX's preemption service catches it, tells every other pool, and the pools agree on an iteration at which all

of them stop and write their checkpoint. A Slurm job asks for the warning early enough for that, at least two SCF iterations and one checkpoint write before the wall clock:

```
#SBATCH --signal=TERM@600
srun python3 run.py            # every pool stops at the same iteration, checkpointed
```

A pool that dies ends the job rather than hanging it: across nodes `srun` cancels the step at once, and on one machine the others are aborted when the coordination service misses the dead pool's heartbeat, `DEFUQUMAT_POOL_HEARTBEAT` seconds later (100 by default). Between nodes the collectives go over whatever address the node's name resolves to, which on Triton is its Ethernet interface at about 100 MB/s; `DEFUQUMAT_POOL_INTERFACE=ib0` puts them on another interface, and a name the node does not have stops the run at start-up.

What pools measured against `pw.x -nk`. Six identical cores of an i5-12600K, every defuqumat process and every `pw.x` MPI rank pinned to its own core, cells without symmetry, speedup over the same code on one core:

cell, k-points	1 process × 6 threads	6 pools × 1 core	`pw.x -nk 6`	share of Quantum E
si16, 12	1.35x	about 4.1x	5.09x	about 80%
si32, 6	1.70x	4.59x	5.06x	91%
si16 magnetic spinor, 6	1.33x	3.19x	4.92x	65%

On a server node (AMD EPYC 7713, Triton's `batch-milan`) the 40-atom silicon cell without symmetry, 27 k-points, runs 20.3x faster on 54 cores as 27 pools of two cores, against `pw.x -nk 27`'s 26.8x on 54 ranks, and 14.0x on 27 one-core pools against 16.5x. Pools wider than two cores lose to Quantum ESPRESSO's plane-wave distribution (9 pools of six cores: 9.8x against 24.2x), because threads on one k-point stop at about 2x even on a production-sized box. What the pool layer itself costs is 5–7% of an iteration (the collectives about 1%, the rest waiting for the slowest pool); the spinor is held back by its Γ point needing twice the Davidson steps of the other k-points — `pw.x` shows the same, run alone — and by the k-independent work every pool repeats. `PERFORMANCE.md` has the timings, the per-rank breakdown and the one A/B that a noisy single run needed.

The local term and the density a chunk of planes at a time. Six processes sharing a memory system slowed `h_psi` 1.5x where the dense algebra did not notice them, because each band's box was streamed through the shared cache several times. Between the two z transforms, which run on the sticks, everything is plane-local, so on a CPU the local term and the density take a chunk of z planes through the xy transforms and the product (or $w|\psi|^2$) while it sits in a core's own cache: on si32's box the local term of 64 bands went from 111 ms alone and 232 ms six at once to 99 ms both ways, with the same answer. The chunk is sized from a 384 KB budget that counts the spinor components and the potential's; `DEFUQUMAT_PLANE_CHUNK` takes a plane count, or `off` for the whole box, which is what a GPU keeps.

Under `DEFUQUMAT_POOLS` greater than one, refused by name before anything runs: more pools than k-points; a residual `scf_solver` (it differentiates the whole set in one gradient); the analytic force and stress (they read every k-point's states); and every `Calculator` method outside `get_scf`, `get_forces`, `get_stress` and `get_relax`. A seed whose states were held by a different pool layout is refused rather than read from the wrong k-point; seed from its density, or run with its pool count.

Precision. Every dtype comes from `config.Precision`; x64 is enabled before any array exists, and all validation is float64. float32 is a performance mode and **never one a correctness claim is made in**.

17 Accuracy summary

Against Quantum ESPRESSO on the same input:

quantity	**agreement**
total energy, norm-conserving LDA	$\sim$1e-9 Ry
ultrasoft, PAW	$\leq$3e-9 Ry
PBE / revPBE / PBEsol	$\leq$6e-9 Ry
metals, every smearing and tetrahedra	2.5e-8 Ry
collinear spin	1.2e-9 Ry
noncollinear magnetism	2.8e-9 Ry
spin-orbit coupling	$\leq$1.3e-8 Ry
DFT+U	$\leq$6.7e-9 Ry
band structure	0.0002 eV
forces	$\leq$2e-5 Ry/bohr
stress (13 cases)	$\leq$2.7e-7 Ry/bohr^3
relaxed geometry	1e-6 bohr
dielectric constant (NC, US, PAW)	$\leq$1.2e-4
Born effective charges (NC)	every printed digit
Born effective charges (US, PAW)	$\leq$4.9e-5
phonons at Γ, silicon	0.05 cm^{-1}
phonons at Γ, silicon (US, PAW)	0.02–0.03 cm^{-1}
phonons at Γ, aluminium (metal)	0.002 cm^{-1}
Raman/IR spectra vs `dynmat.x`	every printed digit

One case does not close, and is recorded rather than explained. `bi10-soc` — ten bismuth atoms with a fully-relativistic dataset — sits **1.9e-4 Ry** from QE on a total of -1477.737, which is 1.3e-7 relative. Both codes choose the same symmetry operations, the same k-point, the same grid and the same G-vectors, and both converge. It is in the test suite at that tolerance, not hidden.

Where to go next

- `README.md` — the short tour and a first calculation
- `notebooks/` — 26 worked examples on concrete systems, executed and committed, each with a `.md` export beside it
- `PERFORMANCE.md` — the running performance log, CPU and GPU
- `PLAN.md` — the architecture, the phase breakdown, the trap catalogue
- `notes/GPU.md` — the GPU roadmap